RISC-V • PROJECT 1 • 初学者阅读手册

从完整代码到逐句理解

先看全貌，再读每一句。
最后，把现象追到保证它的代码。

原始全文 + 行号讲解 + 预期现象 + 实测证据

30 份文本文件 • 2736 行原文 • 1459 条解释 • 全部非空行覆盖检查

依据你上传的 Project1-current.tar.gz（顶层 zhangxiyuan24）
2026年11月12日 • 彩色可检索 PDF 与配套 HTML

所有源码直接取自原始压缩包，未替换参考实现。二进制、Vim交换文件只登记，不冒充可逐句阅读的源码。

源压缩包 SHA-256：11cffe9e85474e2f24b3b6ea528495b4b332f2c0839a064c336a92c8888d4f94

阅读目录

每个文件遵循同一节奏：完整代码 → 逐句解释 → 现象与依据。点击文件名或使用 PDF 书签跳转。

00 · 先建立全景图：程序、语法、寄存器、地址与磁盘

01 · 汇编预习

- 01 test.S：先求和，再建立 1～200 的质数表 174 行
- 02 prime.S：只保留质数判断的早期独立版本 67 行
- 03 prelab.lds：决定预习程序放在哪里，从哪里开始 34 行

02 · 启动、内核与装载

- 04 bootblock.S：把磁盘里的内核搬进内存 64 行
- 05 head.S：让 C 语言内核安全开始运行 38 行
- 06 crt0.S：每个任务的开场与退场 38 行
- 07 common.c：把 C 函数调用转换成课程 BIOS 调用 40 行
- 08 main.c：显示启动状态并按名字运行任务 145 行
- 09 loader.c：在任意字节偏移处取出一个任务 55 行
- 10 string.c：自己实现最基础的内存与字符串操作 96 行

03 · 镜像与构建

- 11 镜像制作器：把 ELF 程序做成可从磁盘读取的 image 338 行
- 12 Makefile：谁编译谁、生成什么、如何启动与调试 370 行
- 13 链接脚本：把入口、代码、数据和零区放到约定地址 185 行

04 · 每个任务程序

- 14 data：确认初始化数据没有在装载中丢失 20 行
- 15 bss：确认未初始化全局变量在任务启动时为零 26 行
- 16 auipc：把当前指令地址变成一个可见的十六进制数 63 行
- 17 2048：从按键、棋盘数组到屏幕颜色，并辨认当前实现的缺陷 405 行

05 · 头文件和反汇编附录

- 18 BIOS 服务编号：给底层入口的操作菜单 10 行
- 19 汇编辅助宏：标签、入口和符号大小 35 行
- 20 内核公共接口：串口、读盘和颜色常量 70 行
- 21 CSR 常量：认识特权级，不等于实现了调度 79 行
- 22 内核服务调用表：从下标找到函数地址 59 行
- 23 加载器声明：按字节读镜像、按名字找程序 10 行

- 24 未来调度接口：必须分清“声明”与“已经实现” 115 行
- 25 字符串与内存函数声明 16 行
- 26 任务目录格式：名字、偏移、大小和入口 25 行
- 27 类型字典：整数宽度、指针与现有不一致 78 行
- 28 用户程序怎样调用打印与输入 36 行
- 29 原包 README：目前只有待写说明 1 行
- 30 附录：如何把 prime.S 与反汇编对上 44 行

06 · 原文件清单与完整性说明

07 · 最后：预期现象、对应代码与本次实测

先建立一张全景图

本书只解释你上传的 Project1-current.tar.gz，不把 Project 2 的参考答案替换进来。阅读每一份文件时，都先展示带原始行号的完整代码，再给出逐句解释，接着说明可观察现象和产生它的代码。跨行表达式作为一条语句讲，连续注释或已注释掉的旧代码按块说明；没有省略任何非空源码行。

第一次阅读建议按“test.S → bootblock.S → head.S → init/main.c → loader.c → crt0.S → bss/data/auipc → createimage.c → Makefile/链接脚本 → 2048 → 头文件”串起来。目录与 PDF 书签可直接跳转；第二次再按文件顺序查表。

每条解释标注的是原文件行号，不是 PDF 页码。彩色框中的代码摘录只为定位；上方“完整代码”是完整原文。特别长的同一源码行以 > 续排，不会被截断。例如 data.c 第 3 行的 1024 个初值会全部展示。

读现象时先看证据范围

本书把“源码能推出”“当前机器从源码构建并验证”“为了通用 QEMU 而做了明确测试适配”分开标注。实测不会自动证明课程 U-Boot、SD 卡或板端通过。发现的缺陷保留在原文中解释，本书不是修复后的源码。

四种“程序”不要混淆

所在位置	在哪里运行	它负责什么
test.S / prime.S	RISC-V CPU，独立预习练习	计算总和、质数标志；结果在内存，不会自动打印
tools/createimage.c	你的 Linux 宿主机	读 ELF 文件，拼出磁盘镜像 image；不在 RISC-V 内核里执行
bootblock.S / head.S / init / loader / BIOS封装	RISC-V 启动链和小内核	装内核、清BSS、设栈/跳转表、按名字装入并直接调用任务
test/test_project1/*.c + crt0.S	小内核装入的任务	执行测试或游戏；普通返回后回到名字输入循环
头文件 / Makefile / .lds	预处理、编译、链接和开发阶段	提供约定、构建规则与布局，本身不是CPU逐行运行的程序

从源码到运行：两条方向相反的链

构建链

C / .S → 编译与链接 → ELF → createimage → image

运行链

课程引导环境 → bootblock → head._start → 内核main → 输入名字 → loader → 任务crt0 → 任务main → 返回内核

ELF 带有程序头、节、入口地址和调试符号。image 是工具挑出可装载段后拼接的磁盘内容，不等于把整个 ELF 原封不动复制进去。bootblock 所在位置与 image 中的文件偏移也不是同一个概念。

这里的任务切换不是多进程调度：内核把任务地址强制转换成函数指针并直接调用。任务和内核共享栈、地址空间及当前执行模式；没有独立页表、隔离或正在工作的调度器。AUIPC 中的“PC”是程序计数器，也不是进程编号。

只需先掌握的 C 与汇编语法

写法	先这样理解	容易混淆的地方
int x = 3; / x = x + 1;	创建变量 / 更新它存的值	= 是赋值；== 才是比较
&x / *p / p[i]	取x的地址 / 访问地址p指向的对象 / 访问第i个元素	指针加1按所指类型的大小前进，不总是1字节
if / for / while / break / continue	条件选择 / 循环 / 跳出本层循环 / 进入下一轮	while(1)本身不会返回；continue不会退出循环
#include / #define / // / /*…*/	预处理包含 / 文本宏 / 行注释 / 块注释	注释不会运行；C块注释不嵌套
extern / static / volatile	引用其他文件的定义 / 限定链接可见性或保存局部状态 / 要求按规则进行访问	volatile不是锁，也不保证多个操作原子化
li / la / mv	把数值放进寄存器 / 取符号地址 / 复制寄存器	la得到地址，不是地址里的值；许多助记符是伪指令
lb/lbu / lw/lwu / ld	从内存读1/4/8字节，有无符号扩展依指令	RV64寄存器64位，并不代表每次访存都是8字节
sb / sw / sd	往内存写1/4/8字节	sd t1,0(t3)写t3指向的内存，不是把t1复制给t3
beq / blt / bge / j	相等跳转 / 小于跳转 / 大于等于跳转 / 无条件跳转	比较对象与先加后比的顺序决定循环边界
call / ret / jr	调用并记录返回位置 / 返回 / 跳到寄存器保存的地址	call通常写ra；嵌套调用要保存仍需使用的ra

你会反复见到的寄存器与文件概念

名字	作用
zero	硬连线为0；写入被忽略，可用sd zero清零内存

名字	作用
a0～a7	常见整数调用约定的前8个参数；a0也通常放返回值。本包BIOS把功能号放a7
ra / sp	返回地址 / 当前栈顶；函数局部变量、保存的寄存器依赖栈
t0～t6 / s0～s11	临时寄存器 / 被调用者应保护的寄存器。裸机入口与正常C函数的调用场景要分清
.text / .rodata / .data / .bss	指令 / 只读常量 / 带初值数据 / 运行时应清零的未初始化数据
p_filesz / p_memsz / e_entry	ELF段文件字节数 / 装入后所需内存字节数 / 程序入口地址
扇区 / 字节偏移	SD接口以512字节块读取；任务表用字节offset定位，所以加载器需要拆分跨扇区请求

把磁盘上的数字与内存地址分开

位置或地址	意义
image[0 … 511]	启动扇区：代码、字符串、必要的补零以及末尾元数据
0x1f6 / 0x1f8 / 0x1fc / 0x1fe	image启动扇区内：2字节任务数 / 4字节任务表偏移 / 2字节内核字节数 / 55 aa签名
0x50200000	课程环境约定的bootblock内存起点，不是通用QEMU virt默认RAM起点
0x50201000 / 0x50500000	内核入口 / 内核栈顶（原源码常量）
0x50150000 / 0x51ffff00	课程BIOS入口 / 本内核提供给任务的函数跳转表
0x52000000，每个目标加0x10000	Makefile顺序构建时为各任务安排的链接起点；与磁盘紧凑拼接不同

Task4 当前版只给 bootblock 补到一扇区，内核和任务按文件内容紧接写入，不再为每个任务固定占15扇区。代码中有些 TASK_SIZE_SECTORS 和旧实现留在文件里，不能因为看见宏名就认为它仍决定有效装载过程。

跨扇区例子：offset=1000、size=30 时，先读第1扇区（从0编号），从扇区内偏移488复制24字节；再读第2扇区复制6字节。loader.c 用offset/512、offset%512和min(剩余量,512−扇区内偏移)保证这个拆分。

01 · 汇编预习

01　test.S：先求和，再建立 1～200 的质数表

原路径：test.S · 174 行

> 不依赖操作系统的 RISC-V 汇编预习程序。它把结果写入内存，供调试器观察；它没有终端打印功能。

把这个程序想成两张小练习：先让 t0 从 1 加到 50，把总和放入 sum_result；再逐个把 1～200 交给 is_prime，把真假结果写入 prime_result 的 200 个字节。最后停在 finish 标签，不返回其他程序。

寄存器可先理解为 CPU 内部的少量变量：t0～t3 是临时寄存器，s0～s2 在这里保存跨函数调用仍需保留的循环状态，a0 同时承担函数的第一个参数和返回值，ra 保存 call 的返回位置。这里的 is_prime 只用临时寄存器，不改 s0～s2，所以外层循环不会丢失进度。

本程序使用 RV64 的 sd 写入 8 字节总和，使用 sb 写入 1 字节真假值。la 得到变量地址，li 得到数值，二者不能互换。标签本身不执行运算；它只是给当前位置取名。以 # 开头的行和空行用于说明与排版。

手算跟踪一次能看懂分支：n＝1 在第 123 行直接返回 0；n＝2 在第 128 行返回 1；n＝9 先试因子 2（余数 1），再试因子 3（平方等于 9，不能提前结束；余数为 0），所以返回 0；n＝11 试完因子 2、3 都不能整除，到因子 4 时 4^2＞11，于是返回 1。

求和循环的不变量可以写成：每次到 sum_loop 时，t0 是本轮要加的数，t1 已等于 1＋…＋(t0−1)。首轮 t0＝1、t1＝0，执行 add 后已包含本轮，addi 再把下一轮加数准备好。这个不变量比只背“循环 50 次”更能解释为什么既不漏掉 50，也不会多加 51。

1 完整代码，先看全貌

```
#define SUM_RESULT_ADDR   sum_result
#define PRIME_RESULT_ADDR prime_result

    .section .data
    .align 3

    # 第一题结果:
    # 1 + 2 + ... + 50 = 1275
    .global sum_result
sum_result:
    .dword 0

    # 第二题结果:
    # prime_result[n-1]
    # 1 表示 n 是质数
    # 0 表示 n 不是质数
    .global prime_result
prime_result:
    .space 200


    .section .text
    .global main
```

```
    .global is_prime
    .global finish


main:

# ============================================================
# Test 1
# 计算 1 + 2 + ... + 50
# ============================================================

    li      t0, 1
    li      t1, 0
    li      t2, 51

sum_loop:

    add     t1, t1, t0

    addi    t0, t0, 1

    blt     t0, t2, sum_loop


    # 将 1275 保存到指定内存位置

    la      t3, SUM_RESULT_ADDR

    sd      t1, 0(t3)


# ============================================================
# Test 2
# 判断 1~200 是否为质数
# ============================================================

    la      s0, PRIME_RESULT_ADDR

    li      s1, 1

    li      s2, 200


prime_check_loop:

    # a0 = 当前数字

    mv      a0, s1


    # 调用质数判断函数

    call    is_prime


    # 返回值:
    # a0 = 1 质数
    # a0 = 0 非质数
    #
```

```
    # 保存到 prime_result[n-1]

    sb      a0, 0(s0)


    addi    s0, s0, 1

    addi    s1, s1, 1


    ble     s1, s2, prime_check_loop


# ============================================================
# 程序执行结束
# ============================================================

finish:

    j       finish


# ============================================================
# is_prime
#
# 参数:
# a0 = n
#
# 返回:
# a0 = 1  -> 质数
# a0 = 0  -> 非质数
# ============================================================

is_prime:

    li      t0, 2


    # n < 2

    blt     a0, t0, not_prime


    # n == 2

    beq     a0, t0, prime


    li      t0, 2


divisor_loop:

    # t1 = divisor * divisor

    mul     t1, t0, t0


    # 如果 divisor^2 > n
```

```
142     # 那么 n 是质数
143
144     blt     a0, t1, prime
145
146
147     # n % divisor
148
149     rem     t2, a0, t0
150
151
152     # 如果余数为 0
153     # 不是质数
154
155     beqz    t2, not_prime
156
157
158     addi    t0, t0, 1
159
160     j       divisor_loop
161
162
163 prime:
164
165     li      a0, 1
166
167     ret
168
169
170 not_prime:
171
172     li      a0, 0
173
174     ret
```

2 按原始行号，一句一句理解

L1

```
#define SUM_RESULT_ADDR   sum_result
```

预处理宏 SUM_RESULT_ADDR 被替换为符号 sum_result。它没有把结果固定为某个十六进制地址；最后地址由链接脚本安排。大写 .S 会先经过 C 预处理器。

L2

```
#define PRIME_RESULT_ADDR prime_result
```

同样为 prime_result 起一个宏别名。第 60 行的 la 在预处理后就是获取 prime_result 的地址。

L4

```
    .section .data
```

切换到 .data 数据节，后续定义的结果空间随程序装入内存，不会在 CPU 顺序执行指令时当成算术语句。

L5

```
    .align 3
```

RISC-V GNU 汇编的 .align 3 表示对齐到 2^3=8 字节边界，为后面的 64 位数据提供合适地址；不是“对齐到 3 字节”。

L7–8

```
    # 第一题结果:
    # 1 + 2 + ... + 50 = 1275
```

注释预告第一题目标：1+2+…+50=50×51÷2=1275。这些文字不负责运算，实际计算发生在第 35～45 行。

L9

```
    .global sum_result
```

把 sum_result 导出为全局符号，链接器和调试器能够按这个名字定位结果。这里的“全局”是符号可见性，不是分配内存的命令。

L10

```
sum_result:
```

定义 sum_result 标签，标记下一份数据的地址。它对应一个地址，而不是寄存器。

L11

```
    .dword 0
```

在此放置一个初值为 0 的 64 位双字，占 8 字节；第 52 行 sd 将覆盖这 8 字节。

L13–16

```
    # 第二题结果:
    # prime_result[n-1]
    # 1 表示 n 是质数
    # 0 表示 n 不是质数
```

注释约定：数字 n 的结果放在下标 n−1，1 表示质数、0 表示非质数。因此数组第 0 字节对应数字 1，第 199 字节对应数字 200。

L17

```
    .global prime_result
```

导出 prime_result 符号，方便调试器定位整个质数表。

L18

```
prime_result:
```

定义质数表的起始地址标签。不要把这个地址和表中的第一个 0/1 值混为一谈。

L19

```
    .space 200
```

预留 200 字节空间。由于每个 n 只需要一个 0/1 字节，正好容纳 1～200；不是为 200 个 64 位整数分配空间。GNU 汇编中省略填充值时通常填零，而本程序随后会逐字节重写整个表。

L22

```
    .section .text
```

切换到 .text 代码节，后续内容是指令和代码标签。

L23

```
    .global main
```

把 main 导出，供链接入口 ENTRY(main) 和调试器使用。

L24

```
    .global is_prime
```

把 is_prime 导出。它虽在同一文件中，仍可被其他目标文件或调试器按名字引用。

L25

```
    .global finish
```

把 finish 导出。这样可在 GDB 中直接对 finish 设置断点，而不必记机器地址。

L28

```
main:
```

main 标签是程序入口。这里没有 C 标准库的 main 调用框架；链接脚本和加载器共同决定从此执行。

L30–33

```
# ============================================================
# Test 1
# 计算 1 + 2 + ... + 50
# ============================================================
```

注释把第一段与第二段练习分开，不生成机器指令。此段目标是累加而非用公式直接赋值。

L35

```
    li      t0, 1
```

li t0,1：把当前加数初始化为 1。li 是“装入立即数”的伪指令，汇编器会转换成真实 RISC-V 指令。

L36

```
    li      t1, 0
```

li t1,0：把累计总和清零。必须先初始化；不能假设上电后的临时寄存器天然等于 0。

L37

```
    li      t2, 51
```

li t2,51：设置停止边界为 51。后面的判断是“更新后的 t0＜51 还继续”，因此实际加数最后一个是 50。

L39

```
sum_loop:
```

sum_loop 给循环体入口取名。第 45 行条件成立时会跳回这里。

L41

```
    add     t1, t1, t0
```

add t1,t1,t0：执行“总和＝原总和＋当前加数”。目的寄存器写在第一位：先读旧 t1 和 t0，再把结果写回 t1。

L43

```
    addi    t0, t0, 1
```

addi t0,t0,1：执行“当前加数加一”。addi 的最后一个操作数是立即数 1，而 add 的第三个操作数是寄存器。

L45

```
    blt     t0, t2, sum_loop
```

blt t0,t2,sum_loop：按有符号整数比较，若更新后的加数小于 51，就回到循环体。加完 50 后 t0 变 51，条件不成立，继续向下；因此共累加 50 次。

L48

```
    # 将 1275 保存到指定内存位置
```

注释说明下一步是把已算出的 1275 从寄存器保存到内存，并不是再次计算总和。

L50

```
    la      t3, SUM_RESULT_ADDR
```

la t3,SUM_RESULT_ADDR：获取 sum_result 的地址放入 t3。la 不读取 sum_result 当前保存的数字；它建立下一条存储指令使用的指针。

L52

```
    sd      t1, 0(t3)
```

sd t1,0(t3)：把 t1 的低 64 位写到地址 t3＋0 开始的 8 字节，即 sum_result＝1275。若用小端方式看原始字节，开头是 fb 04，剩余 6 字节为 00。

L55–58

```
# ============================================================
# Test 2
# 判断 1~200 是否为质数
# ============================================================
```

注释引出第二题：逐个判断 1～200。它与第一题共用某些临时寄存器，但第一题结果已经保存，所以后续覆盖 t0～t2 不会丢失总和。

L60

```
    la      s0, PRIME_RESULT_ADDR
```

la s0,PRIME_RESULT_ADDR：让 s0 指向质数表当前写入位置，初始指向下标 0。用 s 寄存器保存这个指针，是因为它要跨 is_prime 调用仍保持有效。

L62

```
    li      s1, 1
```

li s1,1：s1 保存当前待判断的 n，从 1 开始。

L64

```
    li      s2, 200
```

li s2,200：保存上界 200。第 93 行会采用“小于或等于”判断，把 200 也包含进来。

L67

```
prime_check_loop:
```

prime_check_loop 是外层循环入口。每轮负责一个数字和一个数组字节。

L69

```
    # a0 = 当前数字
```

注释说明 a0 将承接函数参数。把 n 放入 a0 是调用约定，而不是 is_prime 能自动访问 s1。

L71

```
    mv      a0, s1
```

mv a0,s1：把当前数字复制到 a0。mv 是寄存器复制伪指令，不清空 s1；因此循环计数仍保存在 s1 中。

L74

```
    # 调用质数判断函数
```

注释引出一次函数调用。函数体在文件后方，但汇编器会计算正确跳转位置。

L76

```
    call    is_prime
```

call is_prime：保存返回位置到 ra 并跳入判断函数。它返回后，第 85 行看到的 a0 已经从输入数字变成 0 或 1。is_prime 不再调用其他函数，所以无须为嵌套调用额外保存 ra。

L79–83

```
    # 返回值:
    # a0 = 1 质数
    # a0 = 0 非质数
    #
    # 保存到 prime_result[n-1]
```

注释重新强调返回值和下标关系。函数输入 a0＝n，输出 a0＝真假；外层仍用 s1 保存 n，所以返回值覆盖参数不影响循环。

L85

```
    sb      a0, 0(s0)
```

sb a0,0(s0)：把返回值最低 8 位保存到当前表格位置。0、1 均可用一个字节表示；每写入一次就完成 prime_result[n−1]。

L88

```
    addi    s0, s0, 1
```

addi s0,s0,1：指针向后一字节移动。因为采用 sb 存字节，步长必须是 1；若误改为 8，将越过表格并发生越界。

L90

```
    addi    s1, s1, 1
```

addi s1,s1,1：待判断数字加一。指针和 n 同时前进，始终保持“地址＝prime_result＋n−1”的对应关系。

L93

```
    ble     s1, s2, prime_check_loop
```

ble s1,s2,prime_check_loop：若更新后的 n≤200，继续下一轮。ble 是伪指令，可展开为交换两个操作数的 bge；处理完 200 后 n＝201，循环退出。

L96–98

```
# ============================================================
# 程序执行结束
# ============================================================
```

注释宣告结束的是计算任务，并不表示程序会退出 QEMU 或向终端打印完成消息。

L100

```
finish:
```

finish 是最终停留位置。调试时在这里断下，能够确定求和和全部 200 次判断都已执行。

L102

```
    j       finish
```

j finish 无条件跳回自己，形成死循环。它让 CPU 保持运行且结果内存稳定；不是睡眠指令，也不会自动关闭模拟器。

L105–114

```
（此处为连续注释或较长结构，请对照上方完整代码 L105–L114。）
```

这一整段注释给出 is_prime 的函数接口：只通过 a0 传入 n，再通过 a0 返回 0/1。必须与调用处第 71、76、85 行一起阅读。

L116

```
is_prime:
```

定义 is_prime 入口；call 会跳到这里。判断过程试除的范围只需到 $\sqrt{n}$，而不是一直除到 n−1。

L118

```
    li      t0, 2
```

li t0,2：先把比较常量 2 放入 t0；稍后 t0 也会作为当前试除因子。

L121

```
    # n < 2
```

注释说明排除 n＜2 的情况，尤其是 1。1 没有被本算法视为质数。

L123

```
    blt     a0, t0, not_prime
```

blt a0,t0,not_prime：若 n＜2，直接跳到返回 0 的分支。当前数据范围只会遇到 n＝1；对负数也会走此分支。

L126

```
    # n == 2
```

注释说明单独处理数字 2，它是最小质数。

L128

```
    beq     a0, t0, prime
```

beq a0,t0,prime：若 n＝2，跳到返回 1 的分支。若不单独处理而直接测试 n%2，会错误地把 2 排除。

L131

```
    li      t0, 2
```

li t0,2：把初始试除因子设为 2。这里与第 118 行数值相同，所以在当前控制流中有重复赋值，但它明确标出试除循环的初始化。

L134

```
divisor_loop:
```

divisor_loop 是内层试除循环。外层换数字，内层换因子；两种循环各自使用不同寄存器记录状态。

L136

```
    # t1 = divisor * divisor
```

注释定义 t1 将暂存当前因子的平方，供提前停止判断使用。

L138

```
    mul     t1, t0, t0
```

mul t1,t0,t0：计算 divisor^2。对本题 n≤200 的范围很安全；若推广到任意 64 位大整数，平方可能溢出，不能不加处理地沿用这一写法。

L141–142

```
    # 如果 divisor^2 > n
    # 那么 n 是质数
```

数学依据：若 n 合数，至少有一个大于 1 且不超过 $\sqrt{n}$ 的因子。因此已试完这个范围而未整除，就能确认质数。

L144

```
    blt     a0, t1, prime
```

blt a0,t1,prime：若 $n < \text{divisor}^2$，说明试除范围已经越过 $\sqrt{n}$，返回质数。严格小于很关键：例如 n＝9、divisor＝3 时不能先判质数，仍必须检查 9%3。

L147

```
    # n % divisor
```

注释提示下一步求余数，用于判定当前因子能否整除 n。

L149

```
    rem     t2, a0, t0
```

rem t2,a0,t0：把 n÷divisor 的余数放入 t2。n 仍保存在 a0；因子从 2 起且递增，不会出现除零。这里使用有符号 rem，与前面的有符号比较一致。

L152–153

```
    # 如果余数为 0
    # 不是质数
```

注释说明“余数为 0”意味着找到了非平凡因子，n 是合数。

L155

```
    beqz    t2, not_prime
```

beqz t2,not_prime：若余数为零，跳到返回 0 分支。beqz 是“与零寄存器比较相等”的伪指令。

L158

```
    addi    t0, t0, 1
```

addi t0,t0,1：当前因子加一。本实现连偶数因子也逐一尝试，虽然不是最快的算法，但在 1～200 范围内清楚且够用。

L160

```
    j       divisor_loop
```

j divisor_loop：回到平方边界判断，再试下一个因子。只有找到因子或越过平方根才会离开该循环。

L163

```
prime:
```

prime 标签表示“已经确定是质数”的共同出口，可由 n＝2 或试除完毕两条路径到达。

L165

```
    li      a0, 1
```

li a0,1：按函数接口，把返回值设为 1。注意这一步覆盖原来的输入 n，是有意设计。

L167

```
    ret
```

ret：按 ra 保存的地址返回调用处，通常等价于 jalr zero,0(ra)。不会自动把结果写入数组，数组保存由第 85 行负责。

L170

```
not_prime:
```

not_prime 标签是“确定非质数”的共同出口，n＜2 和找到整除因子都会跳到这里。

L172

```
    li      a0, 0
```

li a0,0：把非质数的返回值设置为 0。

L174

```
    ret
```

ret：返回外层循环。由于函数只修改 caller-saved 的 t0、t1、t2、a0，s0、s1、s2 中的数组指针、n、上界仍可继续使用。

3 你能看到什么，什么代码保证它

程序不打印题目答案，也不会出现 shell 提示符；调试器能在 finish 观察到内存中的结果。

代码推导，未在课程平台实测

test.S 第 50～52 行把总和写到 sum_result，第 85 行把质数标志逐字节写入表，第 100～102 行永久停在 finish；全文件没有 BIOS/UART/printf 输出调用。

sum_result 应为十进制 1275，即十六进制 0x4fb。

代码推导，未在课程平台实测

test.S 第 35～45 行从 1 开始、每次先加后递增、在新值达到 51 时停止；第 52 行执行 64 位存储。可在正确映射内存的平台上于 finish 断下后用 GDB 的 x/gd &sum_result 查看。

prime_result 的前 16 个字节应为 0,1,1,0,1,0,1,0,0,0,1,0,1,0,0,0；200 个字节中共有 46 个 1。

代码推导，未在课程平台实测

test.S 第 60～93 行依次处理 1～200，sb 每次仅写一个字节；第 123、128、138～160 行分别排除小于 2、特殊处理 2，并执行到平方根的试除。可用 x/200ub &prime_result 查看无符号字节。

课程地址不能直接当作通用 QEMU virt 的 RAM 地址运行。

代码推导，未在课程平台实测

prelab.lds 第 6 行把地址固定到 0x50200000，而 Makefile 的 prelab-run 使用 virt 且加载到 PRELAB_ADDR＝0x50200000。通用 QEMU virt 的默认 RAM 基址通常为 0x80000000；需课程机器映射或明确的验证适配，不能把未成功加载误说成算法失败。

容易误解或需要修正的地方

- 0x4fb 是求和结果，sum_result 则是结果所在地址；用 la 取地址，用 ld 才是从地址读取 8 字节。
- 不要把 finish 的死循环当作崩溃，也不要等待终端自行打印 1275。这个程序的观测方式就是调试寄存器和内存。
- 即使刚开始 prime_result 是零，也不能据此断言计算正确；必须执行至 finish 后比较全部 200 个标志和正确的素数集合。
- call 会写 ra，is_prime 又依赖 ret 读 ra。若以后给 is_prime 增加嵌套函数调用，必须重新考虑保存原返回地址。
- 本程序作为裸机入口使用 s0～s2 并停在死循环；若改成被其他程序调用且需要返回的普通函数，按 ABI 应保存并恢复被修改的 s 寄存器。

01 · 汇编预习

02 prime.S：只保留质数判断的早期独立版本

原路径：prime.S · 67 行

> 与 test.S 的第二题采用同一算法，只计算质数表，不包含 1～50 求和。它不是 Makefile 中默认 PRELAB_SRC 指定的源文件。

程序仍然先设置结果指针、当前 n 和上界，然后调用 is_prime，并以单字节保存真假。理解 test.S 以后，应该能看出它就是删去求和部分和大部分空行、注释的独立版本。

源码存在不代表默认构建会使用它。当前 Makefile 的 PRELAB_SRC 指向 test.S；压缩包中的 zhangxiyuan24_prime 和反汇编文本是已有产物，不能只凭文件名就保证它们与当前 prime.S 完全一致。应从当前源码重新编译并核验符号后再用于调试。

1 完整代码，先看全貌

```
    .section .data
    .align 3

prime_result:
    .space 200

    .section .text
    .global main
    .global is_prime

main:
    la      s0, prime_result
    li      s1, 1
    li      s2, 200

check_loop:
    mv      a0, s1
    call    is_prime

    # prime_result[n-1]:
    # 1 表示质数，0 表示非质数
    sb      a0, 0(s0)

    addi    s0, s0, 1
    addi    s1, s1, 1
    ble     s1, s2, check_loop

finish:
    j       finish


# 参数:
#   a0 = 要判断的数字
#
# 返回:
```

```
#   a0 = 1 -> 质数
#   a0 = 0 -> 非质数
is_prime:
    li      t0, 2

    # n < 2, 不是质数
    blt     a0, t0, not_prime

    # 2 是质数
    beq     a0, t0, prime

    li      t0, 2

divisor_loop:
    # 如果 t0 * t0 > n, 则 n 是质数
    mul     t1, t0, t0
    blt     a0, t1, prime

    # 检查 n % t0
    rem     t2, a0, t0
    beqz    t2, not_prime

    addi    t0, t0, 1
    j       divisor_loop

prime:
    li      a0, 1
    ret

not_prime:
    li      a0, 0
    ret
```

2 按原始行号，一句一句理解

L1

```
    .section .data
```

切到 .data 数据节，准备放置可写结果数组。

L2

```
    .align 3
```

按 $2^3=8$ 字节对齐。数组采用字节元素，本身只需字节对齐；这里更严格的对齐没有坏处。

L4

```
prime_result:
```

prime_result 标签标出表首地址。这里没有 .global prime_result，所以它不是导出符号；带符号的本文件 ELF 通常仍可在调试器中找到本地符号。

L5

```
    .space 200
```

预留 200 字节，分别对应 n＝1～200。每个位置只保存一个 0/1，不保存 n 自身。

L7

```
    .section .text
```

切到 .text 代码节，后续为指令。

L8

```
    .global main
```

导出 main，作为链接入口可用的名字。

L9

```
    .global is_prime
```

导出 is_prime，声明该函数符号可被外部引用。

L11

```
main:
```

main 是入口标签。程序不借助 C 运行时，也没有为打印功能设置 UART 或 BIOS。

L12

```
    la      s0, prime_result
```

la s0,prime_result：s0 指向表首字节，后续每写一项就增加 1。

L13

```
    li      s1, 1
```

li s1,1：把待判断数字初始化为 1。

L14

```
    li      s2, 200
```

li s2,200：把包含在内的最后一个数字设为 200。

L16

```
check_loop:
```

check_loop 是外层逐个数字判断的循环入口。

L17

```
    mv      a0, s1
```

mv a0,s1：复制当前数字作为函数的第一个参数。保留 s1，让被调用者能够把 a0 改成返回值。

L18

```
    call    is_prime
```

call is_prime：把下一条指令的返回位置记录到 ra，然后执行后方的函数体。函数使用 t 寄存器作临时变量，保持 s0～s2 的外层状态不变。

L20–21

```
    # prime_result[n-1]:
    # 1 表示质数，0 表示非质数
```

注释说明第 n 个数字放在下标 n−1：例如数字 2 的标志位于 prime_result[1]。

L22

```
    sb      a0, 0(s0)
```

sb a0,0(s0)：把返回值的最低一个字节存入当前数组位置。先调用、后保存，不能把调用前的 n 误认为数组内容。

L24

```
    addi    s0, s0, 1
```

s0 加 1，指向下一个字节。

L25

```
    addi    s1, s1, 1
```

s1 加 1，准备判断下一个整数。

L26

```
    ble     s1, s2, check_loop
```

ble：如果新的 s1≤200，回到 check_loop；处理完 200 后 s1＝201，循环停止，恰好写满 200 字节。

L28

```
finish:
```

finish 标记计算完成后的停留点。

L29

```
    j       finish
```

无条件跳回 finish，自旋保留内存结果，不向终端输出，也不返回操作系统。

L32–37

```
# 参数:
#   a0 = 要判断的数字
#
# 返回:
#   a0 = 1 -> 质数
#   a0 = 0 -> 非质数
```

函数接口说明：a0 输入待判断整数，返回时 a0 变成 1 或 0。这里注释不是函数声明，但约定被调用处和返回处分支共同遵守。

L38

```
is_prime:
```

定义 is_prime 函数入口。

L39

```
    li      t0, 2
```

t0＝2，准备检查质数的最小取值。

L41

```
    # n < 2，不是质数
```

注释解释为什么必须单独排除 n＜2。

L42

```
    blt     a0, t0, not_prime
```

若 n＜2，跳去 not_prime 返回 0，因此 1 被正确排除。按有符号比较，负数也会被排除。

L44

```
    # 2 是质数
```

注释引出最小质数 2 的特殊分支。

L45

```
    beq     a0, t0, prime
```

若 n＝2，直接返回质数。这个分支避免随后 2%2＝0 被误判。

L47

```
    li      t0, 2
```

把起始除数设置为 2。前面已给 t0 赋过 2，这里重复写入使试除初始化更明确。

L49

```
divisor_loop:
```

divisor_loop 是试除循环入口，重复检查“除数平方”和“是否整除”。

L50

```
    # 如果 t0 * t0 > n，则 n 是质数
```

注释概括平方根提前结束条件：试除到除数^2＞n 时，还未找到因子，就可以判质数。

L51

```
    mul     t1, t0, t0
```

mul：t1＝t0×t0。对 n≤200 不会溢出；不能据此把实现直接推广为所有 64 位数的可靠质数判断。

L52

```
    blt     a0, t1, prime
```

如果 n＜除数^2，则跳到 prime。平方等于 n 时不能提前返回，否则 4、9、25 等平方数会被错判。

L54

```
    # 检查 n % t0
```

注释表示下一步计算 n 对除数的余数。

L55

```
    rem     t2, a0, t0
```

rem t2,a0,t0：将余数写入 t2，不改变输入 n。除数从 2 起，所以不会除零。

L56

```
    beqz    t2, not_prime
```

beqz：如果余数为零，已找到因子，跳去返回 0。

L58

```
    addi    t0, t0, 1
```

除数加 1，尝试下一个候选值。

L59

```
    j       divisor_loop
```

无条件回到 divisor_loop；下一轮先检查是否已超过平方根，再尝试整除。

L61

```
prime:
```

prime 是确认质数后的统一返回分支。

L62

```
    li      a0, 1
```

a0＝1，设置函数返回值。

L63

```
    ret
```

ret 读取 ra 中的返回位置，回到第 18 行 call 之后。

L65

```
not_prime:
```

not_prime 是确认非质数后的统一返回分支。

L66

```
    li      a0, 0
```

a0＝0，设置非质数返回值。

L67

```
    ret
```

ret 返回外层循环，再由第 22 行保存这个 0。

3 你能看到什么，什么代码保证它

完成后得到与 test.S 第二题相同的 200 字节表，共有 46 个质数标志，但没有 sum_result＝1275 的计算。

代码推导，未在课程平台实测

prime.S 第 12～26 行只执行质数外层循环，第 39～67 行完成试除判断；文件中没有 test.S 第 35～52 行那段累加和存储代码。

终端可以没有任何输出，而 CPU 停留在 finish。

代码推导，未在课程平台实测

prime.S 第 28～29 行无条件自旋；唯一的函数调用是第 18 行 is_prime，它只计算返回值，不提供打印。

容易误解或需要修正的地方

- 不要只编辑 prime.S 再执行 make prelab，并以为运行了它；当前 Makefile 的 PRELAB_SRC 是 test.S。
- prime_result 每项是字节，调试时应按无符号字节查看；若一次按 8 字节整数读，会把连续 8 个标志拼成很大的数，看起来不像 0/1。
- prime.S 没有导出 finish；可以用调试符号定位本地标签，或在编译后反汇编确认地址，不能照抄另一个 ELF 的地址。

01 • 汇编预习

03 prelab.lds：决定预习程序放在哪里，从哪里开始

原路径：prelab.lds • 34 行

> GNU ld 链接脚本。它把目标文件的各个 section 组合成最终 ELF 的内存布局，本身不在 CPU 上执行。

汇编里写的是标签名字，最终机器需要真实地址。链接脚本先选 RISC-V 架构和 main 入口，再把当前位置设置成 0x50200000，依次排入代码、只读数据、可写数据和 BSS。

“链接时声明的地址”和“加载器实际放置的地址”必须一致。ENTRY(main) 只填写 ELF 入口信息，不会自己启动 CPU；当 ELF 被转成 RAW 二进制以后，这份入口元数据也不会留在 RAW 文件里，因此 QEMU loader 的 addr 和 cpu-num 等设置同样重要。

1 完整代码，先看全貌

```
OUTPUT_ARCH(riscv)
ENTRY(main)

SECTIONS
{
    . = 0x50200000;

    .text :
    {
        *(.text)
        *(.text.*)
    }

    . = ALIGN(8);

    .rodata :
    {
        *(.rodata)
        *(.rodata.*)
    }

    .data :
    {
        *(.data)
        *(.data.*)
    }

    .bss :
    {
        *(.bss)
        *(.bss.*)
        *(COMMON)
    }
```

```
34 }
```

2 按原始行号，一句一句理解

L1

```
OUTPUT_ARCH(riscv)
```

OUTPUT_ARCH(riscv) 指定输出体系结构为 RISC-V，帮助链接器按匹配架构处理目标文件；它不是让当前主机切换成 RISC-V。

L2

```
ENTRY(main)
```

ENTRY(main) 指定 ELF 的入口符号为 main。主机 readelf -h 可以看到最终解析出的入口地址；不等于在脚本里调用 main。

L4–5

```
SECTIONS
{
```

SECTIONS { 开始描述输出节的排列规则；大括号把整份布局规则包起来。这里的语法属于链接器，不是 C 语言函数体。

L6

```
    . = 0x50200000;
```

点号 . 是链接器的当前位置计数器。把它设成 0x50200000，使第一个输出节从课程地址开始。注意通用 QEMU virt 默认 RAM 通常在 0x80000000 附近，这个脚本不会替模拟器创建相应 RAM。

L8–9

```
    .text :
    {
```

定义输出 .text 节并开始它的规则块；当前位置决定该节的起始地址。

L10

```
        *(.text)
```

*(.text) 收集所有输入目标文件中名字恰好为 .text 的输入节。星号表示任意输入文件，不是 C 指针解引用。

L11

```
        *(.text.*)
```

(.text.) 继续收集 .text.foo 等细分代码节。既匹配普通 .text 又匹配细分节，避免按函数拆分后的代码遗漏。

L12

```
    }
```

结束输出 .text 节规则。后续位置计数器已经前进到当前代码节末尾。

L14

```
    . = ALIGN(8);
```

将当前位置向上取整到 8 的倍数。例如当前位置为 ...03，会变成 ...08。ALIGN 并不执行 CPU 指令，也不会将所有节内容逐项填成 8 字节。

L16–17

```
    .rodata :
    {
```

定义输出 .rodata，只读数据通常存放常量和字符串；脚本中的节名自身并不建立硬件写保护。

L18

```
        *(.rodata)
```

收集所有输入文件的 .rodata 节。

L19

```
        *(.rodata.*)
```

收集 .rodata.* 子节，兼容编译器细分后的常量节。

L20

```
    }
```

结束 .rodata 布局规则。

L22–23

```
    .data :
    {
```

定义输出 .data 节，通常容纳有初值的可写全局数据。test.S 的 sum_result 和 prime_result 就显式放在这里。

L24

```
        *(.data)
```

收集输入文件中恰好名为 .data 的可写数据节。

L25

```
        *(.data.*)
```

同时收集 .data.* 子节。

L26

```
    }
```

结束 .data 节规则。

L28–29

```
    .bss :
    {
```

定义输出 .bss 节，通常为零初始化的全局/静态对象预留运行内存；ELF 往往只记录内存大小而不保存等量文件字节。

L30

```
        *(.bss)
```

收集输入文件中的 .bss。注意“收集到 BSS”不等于该脚本已经生成清零循环。

L31

```
        *(.bss.*)
```

收集 .bss.* 子节。

L32

```
        *(COMMON)
```

收集 COMMON 公共暂定定义符号，兼容产生这类符号的对象文件或编译选项；它们同样需要在运行内存中有位置。

L33

```
    }
```

结束 .bss 节规则。

L34

```
}
```

结束整个 SECTIONS 规则。链接器至此完成声明的节布局；源程序仍需加载器和正确 CPU 入口才能运行。

3 你能看到什么，什么代码保证它

重新链接预习程序后，ELF 中 main 应落在 0x50200000 起始的代码区域，数据符号位于其后。

代码推导，未在课程平台实测

prelab.lds 第 2 行指定 main，第 6 行指定位置，第 8～26 行依次放置代码、只读数据和结果数据；Makefile 的预习链接命令还以 -Wl,-e,main 指定入口。最终应以生成 ELF 的 readelf/nm 为准。

把 RAW 二进制放错地址，即使文件成功打开，也可能无法正确读写结果或执行函数调用。

代码推导，未在课程平台实测

prelab.lds 第 6 行以及汇编中 la/call 的重定位共同依赖最终地址；RAW 文件不带 ELF 的入口/段表信息，加载命令必须另行给出正确地址。

容易误解或需要修正的地方

- 链接脚本不是启动代码：ENTRY 不做跳转，.bss 规则不清内存，.rodata 节名也不自动启用硬件保护。
- 不要只改 QEMU 的加载地址而保留原 ELF 的所有链接地址；要么使用真实课程地址映射，要么在明确记录的临时验证副本里一致地重新链接和加载。
- 与内核使用的 riscv.lds 不同，这个短脚本没有给出 __bss_start、__BSS_END__ 等运行时符号；不能假设两份链接脚本可直接互换。

02 • 启动、内核与装载

04 bootblock.S：把磁盘里的内核搬进内存

原路径：arch/riscv/boot/bootblock.S • 64 行

> 第一阶段引导程序。它借用课程 BIOS 的打印和读盘服务，把内核加载到固定地址，再把任务表参数交给内核。

把它看成搬运员：CPU 已被课程启动环境带到本程序，bootblock 再读取 image 的内核部分。它不是完整的芯片复位固件，也不自己驱动 SD 控制器。

本版本 Task 4 使用“内核字节数”，不是旧 Task 3 的固定扇区数。第 37—38 行用向上取整把字节数换成扇区数。例如 513 字节需要读取 2 个 512 字节扇区。

寄存器 a0、a1 等是 RISC-V 参数寄存器。调用 BIOS 时 a7 选择服务；最后跳往内核时，a0 改装任务数量，a1 改装任务表在 image 中的字节偏移。

1 完整代码，先看全貌

```
#include <asm/biosdef.h>

// os size location (os_size could be sectors [p1-task3] or bytes [p1-task4])
.equ os_size_loc, 0x502001fc

// kernel address (move kernel to here, and jmp here to start kernel)
.equ kernel, 0x50201000

// BIOS function entry (jump here to use BIOS APIs)
.equ bios_func_entry, 0x50150000
.equ task_num_loc, 0x502001f6
.equ app_info_offset_loc, 0x502001f8

.text
.global main

main:
    // fence on all memory and I/O
    fence

    // TODO: [p1-task1] call BIOS to print string "It's bootblock!"
    la a0,msg
    li a1,0
    li a2,0
    li a3,0
    li a4,0
    li a5,0
    li a6,0
    li a7,BIOS_PUTSTR
    li t0,bios_func_entry
    jalr t0

    // TODO: [p1-task2] call BIOS to read kernel in SD card
    li a0,kernel
```

```
35      li t0,os_size_loc
36      lhu t1,0(t0)
37      addi a1,t1,511
38      srli a1,a1,9
39      li a2,1
40      li a7,BIOS_SDREAD
41      li t0,bios_func_entry
42      jalr t0
43      // TODO: [p1-task4] load task-related arguments and pass them to kernel
44      li t0,task_num_loc
45      lhu a0,0(t0)
46      li t0,app_info_offset_loc
47      lwu a1,0(t0)
48
49
50      // TODO: [p1-task2] jump to kernel to start UCAS-OS
51      li t0,kernel
52      jr t0
53      /************************************************************/
54      /* Do not touch this comment. Reserved for future projects. */
55      /************************************************************/
56  // while(1) --> stop here
57  stop:
58      j stop
59      nop
60
61  .data
62
63  msg: .string "It's a bootloader...\n\r"
64
```

2 按原始行号，一句一句理解

L1

```
#include <asm/biosdef.h>
```

预处理时引入 BIOS 服务编号；这是汇编源文件使用 C 预处理器的 #include，不会在运行时打开头文件。

L3–4

```
// os size location (os_size could be sectors [p1-task3] or bytes [p1-task4])
.equ os_size_loc, 0x502001fc
```

注释指出历史版本单位不同；.equ 把 os_size_loc 定义为常量 0x502001fc。这个地址等于 bootblock 起址 0x50200000 加磁盘头字段偏移 0x1fc，位置里存内核字节数。

L6–7

```
// kernel address (move kernel to here, and jmp here to start kernel)
.equ kernel, 0x50201000
```

把符号 kernel 定义为 0x50201000：既是读盘目的地，也是内核入口地址。符号并没有在这里分配一块内存。

L9–10

```
// BIOS function entry (jump here to use BIOS APIs)
.equ bios_func_entry, 0x50150000
```

把课程 BIOS 的统一函数入口记成常量 0x50150000。这里只会跳去调用它，BIOS 实现并不包含在这份压缩包内。

L11

```
.equ task_num_loc, 0x502001f6
```

任务数量字段位于 bootblock 内存副本的偏移 0x1f6。这个常量必须与 tools/createimage.c 写头的位置一致。

L12

```
.equ app_info_offset_loc, 0x502001f8
```

任务表字节偏移字段位于 bootblock 偏移 0x1f8，之后会作为内核 main 的第二个参数。

L14

```
.text
```

.text 选择存放指令的代码段；之后汇编出来的机器指令归入该段。

L15

```
.global main
```

.global main 让链接器及其他目标文件能识别 main；它不是调用 main 的指令。

L17

```
main:
```

main: 是当前指令地址的标签，也就是 Makefile 为 bootblock 指定的入口。

L18–19

```
    // fence on all memory and I/O
    fence
```

fence 对前后的内存及 I/O 访问施加顺序约束；这是数据/I/O 访问栅栏，不等同于用于同步取指的 fence.i。

L21

```
    // TODO: [p1-task1] call BIOS to print string "It's bootblock!"
```

这条 TODO 注释保留了任务题意，不会执行；下面已有打印实现。注释中的 It's bootblock! 与实际字符串并不一样。

L22

```
    la a0,msg
```

la 将 msg 字符串地址放入 a0。注意是地址而非首字符，BIOS 后续沿此地址读取直到字符串结束符。

L23

```
    li a1,0
```

li a1,0 把第二个参数寄存器清零。本次字符串输出只使用 a0，其余参数预先置零。

L24

```
    li a2,0
```

li a2,0 把第三个参数寄存器置零；li 是“装入立即数”的伪指令。

L25

```
    li a3,0
```

li a3,0 清第四个参数寄存器，避免把之前未知的值当成参数传出。

L26

```
    li a4,0
```

li a4,0 清第五个参数寄存器。

L27

```
    li a5,0
```

li a5,0 清第六个参数寄存器。

L28

```
    li a6,0
```

li a6,0 清第七个参数寄存器。

L29

```
    li a7,BIOS_PUTSTR
```

a7 装 BIOS_PUTSTR 服务号。参数与服务号约定来自课程 BIOS ABI，不是 Linux 系统调用 ABI。

L30

```
    li t0,bios_func_entry
```

t0 装 BIOS 统一入口地址 0x50150000。t0 是可被调用者覆盖的临时寄存器。

L31

```
    jalr t0
```

jalr t0 跳到 t0 指向的函数并把返回地址写入 ra；BIOS 返回后继续执行本程序。这里是函数调用，不是 ecall 特权切换。

L33

```
    // TODO: [p1-task2] call BIOS to read kernel in SD card
```

这是 Task 2 的题意注释；下面一段负责把内核从 image 读入内存。

L34

```
    li a0,kernel
```

把读盘目的地址 kernel 放入 a0，要求 BIOS 把后续扇区复制到这里。

L35

```
    li t0,os_size_loc
```

t0 暂存内核大小字段所在的内存地址。

L36

```
    lhu t1,0(t0)
```

lhu 从地址 t0 读取 16 位无符号半字到 t1，并向高位补零。字段范围最多 65535 字节。

L37

```
    addi a1,t1,511
```

先计算“内核字节数 + 511”，存入 a1。这是对 512 向上取整的前半步。

L38

```
    srli a1,a1,9
```

逻辑右移 9 位等于除以 512，得到 ceil(字节数/512)。例如 (513+511)>>9=2；字节数为零时结果也是零。

L39

```
    li a2,1
```

a2=1，表示从 image 的第 1 号扇区开始读；第 0 号扇区的 512 字节留给 bootblock 和头字段。

L40

```
    li a7,BIOS_SDREAD
```

a7 选择 BIOS_SDREAD，通知统一 BIOS 入口执行读盘而非输出字符串。

L41

```
    li t0,bios_func_entry
```

再次把 BIOS 入口装入 t0。上次函数调用允许破坏临时寄存器，不能依赖旧 t0 仍有效。

L42

```
    jalr t0
```

调用 BIOS；参数依次为目标内存地址 a0、扇区数 a1、起始扇区号 a2。代码没有在返回后检查读盘错误状态。

L43

```
    // TODO: [p1-task4] load task-related arguments and pass them to kernel
```

说明接下来为 Task 4 传递元数据。此时 a0、a1 不再用于读盘参数。

L44

```
    li t0,task_num_loc
```

t0 指向任务数量字段。

L45

```
    lhu a0,0(t0)
```

读取 16 位无符号任务数量到 a0，对应内核 C 函数 main(int tasknum, ...) 的第一个整数参数。

L46

```
    li t0,app_info_offset_loc
```

t0 指向任务表偏移字段。

L47

```
    lwu a1,0(t0)
```

lwu 读取 32 位无符号字并补零到 64 位 a1。这个值是 image 内的字节偏移，不是 RAM 指针，也不是扇区号。

L50

```
    // TODO: [p1-task2] jump to kernel to start UCAS-OS
```

说明接下来转入内核。完成这次跳转后，正常路径不会继续留在 bootblock。

L51

```
    li t0,kernel
```

把内核固定入口地址重新装入 t0。

L52

```
    jr t0
```

jr t0 无链接地跳到内核，不像 jalr 调用那样设置新的返回地址。内核 head.S 随后建立自己的栈并调用 C main。

L53–55

```
    /************************************************************/
    /* Do not touch this comment. Reserved for future projects. */
    /************************************************************/
```

课程预留区的注释，不参与运行，也不会产生指令。

L56

```
// while(1) --> stop here
```

注释把下面解释成停止环，但前面的 jr 是无条件跳转，正常启动不会顺着走到它。

L57

```
stop:
```

stop: 只是停止环的标签，标签自身不执行操作。

L58

```
    j stop
```

j stop 无条件跳回自身；若程序意外进入这里，会持续占用 CPU，不会正常返回。

L59

```
    nop
```

nop 是空操作指令。前面 j stop 始终跳走，因此正常情况下也不会执行到该 nop。

L61

```
.data
```

.data 选择有初始内容的数据段，下面保存打印所需的文字。

L63

```
msg: .string "It's a bootloader...\n\r"
```

.string 写入实际字节 It's a bootloader...，随后是换行 \n、回车 \r 和末尾的零字节。真实启动提示应以这里的内容为准。

3 你能看到什么，什么代码保证它

课程启动链正常时，首先看到 It's a bootloader...。

代码推导，未在课程平台实测

bootblock.S:22、29—31 把 msg 地址交给 BIOS_PUTSTR；实际文字由 bootblock.S:63 决定。

内核可从非整扇区大小加载，随后进入内核 Hello OS! 提示。

代码推导，未在课程平台实测

bootblock.S:34—42 读取 ceil(kernel_bytes/512) 个扇区到 0x50201000，51—52 跳往该地址；init/main.c:83 才输出 Hello OS!。

内核能知道有哪些任务以及从哪里读任务表。

代码推导，未在课程平台实测

bootblock.S:44—47 读取头字段到 a0/a1，head.S:16—31 不覆盖这两个参数寄存器，init/main.c:50、60 接收并使用它们。

容易误解或需要修正的地方

- 0x50150000 的课程 BIOS 和 0x50200000 一带的课程内存布局必须由外部平台提供。不能把本文件直接丢给默认 RAM 起于 0x80000000 的通用 QEMU virt，就据此声称课程启动成功。
- 这里没有验证元数据是否合理，也不检查读盘服务返回值；错误镜像或 BIOS 读盘失败时，跳转到内核并不保证得到有效指令。
- 内核大小只占 16 位，而 a0 传给 C main 的任务数也没有在引导阶段核对上限。后续任务表缓冲区只有 TASK_MAXNUM 项。

02 · 启动、内核与装载

05 head.S：让 C 语言内核安全开始运行

原路径：arch/riscv/kernel/head.S · 38 行

> 内核最早执行的汇编入口。先处理监督者中断寄存器、清零 .bss，再建立栈并调用 C main。

普通 C 程序通常由运行库替我们准备栈和全局变量；这里没有标准运行库，所以需要自己完成。链接器的 __bss_start、__BSS_END__ 只是边界地址，真正的清零由本文件的 sd 指令执行。

bootblock 传来的 a0、a1 必须原样传到 C main。此处用 t0、t1 遍历 .bss，只改临时寄存器和 sp，不覆盖 a0、a1。

1 完整代码，先看全貌

```
/* SPDX-License-Identifier: GPL-2.0-only */
/*
 * Copyright (C) 2012 Regents of the University of California
 */

#include <asm.h>
#include <csr.h>
#define KERNEL_STACK 0x50500000
  .section ".entry_function","ax"
ENTRY(_start)

csrw CSR_SIE, zero
csrw CSR_SIP, zero


la t0,__bss_start
la t1,__BSS_END__

clear_bss:
    beq t0,t1,clear_done
    sd zero,0(t0)
    addi t0,t0,8
    j clear_bss


clear_done:

li sp,KERNEL_STACK


call main


loop:
    wfi
    j loop

END(_start)
```

2 按原始行号，一句一句理解

L1

```
/* SPDX-License-Identifier: GPL-2.0-only */
```

SPDX 是许可证标记，不产生机器指令。

L2–4

```
/*
 * Copyright (C) 2012 Regents of the University of California
 */
```

这是原作者版权注释；它说明来源，不参与启动逻辑。

L6

```
#include <asm.h>
```

引入 ENTRY 和 END 等汇编宏，使入口符号声明保持统一。

L7

```
#include <csr.h>
```

引入 CSR_SIE、CSR_SIP 的寄存器编号；CSR 是控制与状态寄存器，与普通内存变量不同。

L8

```
#define KERNEL_STACK 0x50500000
```

把内核栈顶常量定义为 0x50500000。RISC-V 栈通常向较小地址增长，这里没有动态申请堆栈。

L9

```
  .section ".entry_function","ax"
```

将接下来的入口代码放入 .entry_function 节；a 表示可分配到运行内存，x 表示含可执行代码。

L10

```
ENTRY(_start)
```

ENTRY(_start) 宏定义并导出 _start 入口，配合链接脚本 ENTRY(_start) 确定内核进入位置。

L12

```
csrw CSR_SIE, zero
```

向 sie 写零，关闭此处可控的监督者中断使能位，为最小启动阶段减少中断干扰。

L13

```
csrw CSR_SIP, zero
```

向 sip 写零，尝试清除可写的监督者中断挂起位；不可写/由硬件控制的位不一定因此消失，不能理解成能强行清除所有外部中断。

L16

```
la t0,__bss_start
```

t0 装入 .bss 开始地址，作为写零指针。

L17

```
la t1,__BSS_END__
```

t1 装入 .bss 结束地址，作为停止边界；结束地址本身不属于要清零的区间。

L19

```
clear_bss:
```

clear_bss 是清零循环的标签，用于从第 23 行跳回。

L20

```
    beq t0,t1,clear_done
```

若当前指针恰好等于结束地址，就跳到 clear_done；这也允许空 .bss 直接结束。

L21

```
    sd zero,0(t0)
```

sd 写入 8 字节全零，来源寄存器 zero 永远读为零。未初始化的全局/静态变量由此取得 C 语言要求的零初值。

L22

```
    addi t0,t0,8
```

指针向前推进 8 字节，与上一条 sd 的写入宽度保持一致。

L23

```
    j clear_bss
```

无条件回到循环开始，检查下一次是否到达终点。

L26

```
clear_done:
```

clear_done 是清零完成后的汇合位置。

L28

```
li sp,KERNEL_STACK
```

把 sp 设为固定内核栈顶。此前只有寄存器和直接内存访问，没有进行需要 C 栈的函数调用。

L31

```
call main
```

call main 调用 init/main.c 的 C 函数；a0、a1 带着 bootblock 的两个参数。call 会把“main 返回后的位置”放入 ra。

L34

```
loop:
```

loop 是 C main 意外返回时的保底等待位置。主加载器通常一直在自己的输入循环里，不会来到这里。

L35

```
    wfi
```

wfi 表示等待中断，是提示 CPU 可进入等待状态的指令；实际是否休眠由实现决定。

L36

```
    j loop
```

即使 wfi 返回，也重新进入等待循环，不继续执行未定义的后续内存。

L38

```
END(_start)
```

END(_start) 给汇编器/链接器记录 _start 函数的结束及大小；不是运行时返回指令。

3 你能看到什么，什么代码保证它

首次进入内核时，bss check 应为 t，而不是 f。

代码推导，未在课程平台实测

head.S:16—23 对 .bss 写零；init/main.c:12 的 buf 在 .bss，17—27 检查其中 50 个字节，72 把结果转换成字符。

内核能读到 bootblock 传入的任务数和任务表偏移。

代码推导，未在课程平台实测

head.S:12—28 不写 a0 或 a1；31 的 call main 使用这两个 ABI 参数寄存器；init/main.c:50 是对应 C 入口。

容易误解或需要修正的地方

- 循环只判断相等且每次前进 8 字节，因此依赖 .bss 起止边界合适并且长度能被 8 整除。当前 riscv.lds 在 .bss 后按 8 字节对齐（138—141 行），但不能脱离链接脚本单独假设该循环总正确。
- 固定栈并不等于虚拟内存保护，也不为每个任务建立独立内核栈；本项目还没有展示地址空间隔离或进程调度。
- 清零 .bss 不等于清空整个 RAM。它只写链接器指定的区间，.data 的初值应保留。

02 · 启动、内核与装载

06 crt0.S：每个任务的开场与退场

原路径：arch/riscv/crt0/crt0.S · 38 行

> 用户测试程序的最小 C 运行时入口。保存调用现场、清零本任务 .bss、调用任务 main，然后恢复现场回到加载器。

这里的“用户程序”是课程组织代码的名称。内核把入口当成函数调用，没有执行从监督者模式切换到用户模式的指令；因此不能仅凭文件目录就认定已实现安全的用户态。

内核调用任务入口时 ra 已指向“任务返回后”的内核位置。任务中的 call main 会覆盖 ra，所以 crt0 必须先保存原 ra，最后恢复。

1 完整代码，先看全貌

```
#include <asm.h>

.section ".entry_function","ax"
ENTRY(_start)

    /* TODO: [p1-task3] setup C runtime environment for the user program */
    addi sp,sp,-16
    sd ra, 0(sp)
    sd s0, 8(sp)
    addi s0, sp, 16

    la t0, __bss_start
    la t1, __BSS_END__

    clear_bss:
        bge t0, t1, clear_bss_done
        sw zero, 0(t0)
        addi t0, t0, 4
        blt t0, t1, clear_bss
    clear_bss_done:

    /* TODO: [p1-task3] enter main function */
    call main

    /* TODO: [p1-task3] finish task and return to the kernel, replace this in p3-task2! */
    ld ra,0(sp)
    ld s0,8(sp)
    addi sp,sp,16
    jr ra
    /************************************************************/
    /* Do not touch this comment. Reserved for future projects. */
    /************************************************************/
// while(1) loop, unreachable here
loop:
    wfi
    j loop
```

```
37
38 END(_start)
```

2 按原始行号，一句一句理解

L1

```
#include <asm.h>
```

引入 ENTRY/END 汇编宏，用于统一导出任务入口符号。

L3

```
.section ".entry_function","ax"
```

把入口放入 .entry_function 节，保证链接脚本把启动代码放在任务入口位置。

L4

```
ENTRY(_start)
```

定义并导出 _start。任务 ELF 的入口应是这段运行时，而不是跳过准备过程直接进入 C main。

L6

```
    /* TODO: [p1-task3] setup C runtime environment for the user program */
```

TODO 注释说明任务三要求准备 C 运行环境；下面已有保存寄存器和清 .bss 的实现。

L7

```
    addi sp,sp,-16
```

sp 减 16，在当前栈上划出 16 字节栈帧。仍使用内核调用时的同一条栈，没有切换到独立任务栈。

L8

```
    sd ra, 0(sp)
```

把 ra 的 8 字节值保存到栈帧偏移 0，保留返回内核所需的地址。

L9

```
    sd s0, 8(sp)
```

把被调用者应保持的寄存器 s0 保存到偏移 8，避免破坏内核调用者的现场。

L10

```
    addi s0, sp, 16
```

s0 设为当前 sp+16，即进入本函数前的栈位置，可作为本帧的帧指针。

L12

```
    la t0, __bss_start
```

t0 指向本任务的 .bss 开始地址。不同任务使用各自链接时生成的符号值。

L13

```
    la t1, __BSS_END__
```

t1 指向本任务 .bss 的结束地址。

L15

```
    clear_bss:
```

clear_bss 是此任务清零循环的标签；它与另一 ELF 文件 head.S 中的同名标签互不冲突。

L16

```
        bge t0, t1, clear_bss_done
```

若 t0 已达到或超过 t1，跳过清零。bge 是有符号比较；此项目地址较低且边界有序时按预期工作。

L17

```
        sw zero, 0(t0)
```

sw 每次写 4 字节零。它与 head.S 的 sd 不同，sw 是“字”，在 RV64 中仍为 32 位。

L18

```
        addi t0, t0, 4
```

t0 前进 4 字节，准备清除下一块。

L19

```
        blt t0, t1, clear_bss
```

只要 t0 仍小于终点，就跳回 clear_bss 再执行；到终点后自然继续。

L20

```
    clear_bss_done:
```

清零完成的标签，也是空 .bss 的直接跳转目标。

L22

```
    /* TODO: [p1-task3] enter main function */
```

注释说明接下来才真正进入任务自己的 C main。

L23

```
    call main
```

调用任务 main。它可以像普通函数一样返回；ra 此时变成 crt0 中紧接 call 的地址。

L25

```
    /* TODO: [p1-task3] finish task and return to the kernel, replace this in p3-task2! */
```

注释说明当前阶段用普通函数返回结束任务；未来项目才可能替换成独立的进程退出机制。

L26

```
    ld ra,0(sp)
```

从栈取回进入 _start 时保存的 ra，即内核调用任务后的续接地址。

L27

```
    ld s0,8(sp)
```

恢复原 s0，遵守 RISC-V 的被调用者保存寄存器约定。

L28

```
    addi sp,sp,16
```

sp 加回 16，撤销本函数的栈帧；正确返回时内核看到的 sp 与调用前一致。

L29

```
    jr ra
```

jr ra 跳回内核加载器。这里没有 ecall、进程销毁或调度切换，只是普通函数返回。

L30–32

```
    /************************************************************/
    /* Do not touch this comment. Reserved for future projects. */
    /************************************************************/
```

课程预留区的注释，不执行任何指令。

L33

```
// while(1) loop, unreachable here
```

注释说明下面的保底等待环在正常 jr ra 返回路径上不会执行。

L34

```
loop:
```

定义备用 loop 标签。它不是任务的默认结束方式。

L35

```
    wfi
```

若执行进入备用循环，就用 wfi 等待中断。

L36

```
    j loop
```

再次跳回 loop，使 CPU 不会落入后续未知位置。

L38

```
END(_start)
```

END 宏记录本入口符号的结束位置和大小。

3 你能看到什么，什么代码保证它

反复运行 bss 测试时，每次都应该从全零的未初始化数组开始。

代码推导，未在课程平台实测

crt0.S:12—20 在每次任务入口执行时清本任务 .bss；test/test_project1/bss.c 中未初始化数组由此恢复。若绕过入口只调用 main，就没有此保证。

能正常返回的任务结束后，再次出现加载器提示。

代码推导，未在课程平台实测

crt0.S:8、23、26—29 保存/恢复返回地址并返回；init/main.c:134 调用任务，135 输出 Task finished, back to loader，随后 106 行外层循环重新提示。

容易误解或需要修正的地方

- 本文件只额外保存 ra、s0；任务编译器仍需遵守 ABI，正确保存自己使用的其他 s 寄存器，并保持调用前后的栈平衡。
- 四字节清零依赖任务 .bss 结尾的对齐/填充；当前链接脚本按 8 字节对齐结束地址，因此通常覆盖完整区间。
- 任务若无限循环或在内部一直等待输入，main 不返回，加载器也不能抢占回来；没有调度器保证某段固定时间后自动返回。

02 · 启动、内核与装载

07 common.c：把 C 函数调用转换成课程 BIOS 调用

原路径：arch/riscv/bios/common.c · 40 行

> 为内核提供字符输入、字符串输出和按扇区读写磁盘的 C 包装函数。

调用路径可以从外往内读：bios_putstr → 跳转表中的 port_write → call_bios → 地址 0x50150000 的课程 BIOS。包装层不是硬件驱动的完整实现。

RV64 ABI 把前八个整数/指针参数放入 a0 至 a7。call_bios 把服务选择号 which 放在最后一个实参位置，恰好送进 a7，与 bootblock.S 的手工写寄存器一致。

1 完整代码，先看全貌

```
#include <common.h>
#include <asm/biosdef.h>

#define BIOS_FUNC_ENTRY 0x50150000
#define IGNORE 0

static long call_bios(long which, long arg0, long arg1, long arg2, long arg3, long arg4)
{
    long (*bios_func)(long,long,long,long,long,long,long,long) = \
        (long (*)(long,long,long,long,long,long,long,long))BIOS_FUNC_ENTRY;

    return bios_func(arg0, arg1, arg2, arg3, arg4, IGNORE, IGNORE, which);
}

void port_write_ch(char ch)
{
    call_bios((long)BIOS_PUTCHAR, (long)ch, IGNORE, IGNORE, IGNORE, IGNORE);
}

void port_write(char *str)
{
    call_bios((long)BIOS_PUTSTR, (long)str, IGNORE, IGNORE, IGNORE, IGNORE);
}

int port_read_ch(void)
{
    return call_bios((long)BIOS_GETCHAR, IGNORE, IGNORE, IGNORE, IGNORE, IGNORE);
}

int sd_read(unsigned mem_address, unsigned num_of_blocks, unsigned block_id)
{
    return (int)call_bios((long)BIOS_SDREAD, (long)mem_address, \
                          (long)num_of_blocks, (long)block_id, IGNORE, IGNORE);
}

```

```
36 int sd_write(unsigned mem_address, unsigned num_of_blocks, unsigned block_id)
37 {
38     return (int)call_bios((long)BIOS_SDWRITE, (long)mem_address, \
39                           (long)num_of_blocks, (long)block_id, IGNORE, IGNORE);
40 }
```

2 按原始行号，一句一句理解

L1

```
#include <common.h>
```

引入内核低层 I/O 函数声明，确保本文件定义与调用方约定一致。

L2

```
#include <asm/biosdef.h>
```

引入 BIOS_PUTCHAR、BIOS_PUTSTR 等编号。编号是协议的一部分，必须与课程 BIOS 相同。

L4

```
#define BIOS_FUNC_ENTRY 0x50150000
```

固定统一 BIOS 入口地址。#define 是文本宏替换，不会申请存储空间。

L5

```
#define IGNORE 0
```

为不使用的参数位置约定零值，让每次底层调用都能给满八个参数。

L7–8

```
static long call_bios(long which, long arg0, long arg1, long arg2, long arg3, long arg4)
{
```

定义本文件私有的 call_bios。static 使名称不导出给其他 C 文件；返回 long，支持保存 RV64 的机器字返回值。which 是服务号，arg0—arg4 是最多五个实际参数。

L9–10

```
    long (*bios_func)(long,long,long,long,long,long,long,long) = \
        (long (*)(long,long,long,long,long,long,long,long))BIOS_FUNC_ENTRY;
```

声明 bios_func 为“指向接收八个 long、返回 long 的函数的指针”，再把固定整数地址转换成该类型。C 允许这样写并不验证那个地址真的有可调用 BIOS。

L12

```
    return bios_func(arg0, arg1, arg2, arg3, arg4, IGNORE, IGNORE, which);
```

通过函数指针进行调用。arg0—arg4 放入 a0—a4，两个 IGNORE 填 a5/a6，which 放 a7；把 BIOS 的返回值原样返回。没有执行 ecall。

L13

```
}
```

结束 call_bios 函数体。

L15–16

```
void port_write_ch(char ch)
{
```

定义输出单个字符的包装函数，参数 ch 是一个 char，函数本身不返回字符数。

L17

```
    call_bios((long)BIOS_PUTCHAR, (long)ch, IGNORE, IGNORE, IGNORE, IGNORE);
```

选择 BIOS_PUTCHAR，把 ch 转为 long 作为第一个实际参数，其余四个实际参数槽填 IGNORE。call_bios 还会另外用 IGNORE 填 ABI 的 a5、a6，服务号放 a7。

L18

```
}
```

字符输出包装结束；底层是否真正送到串口仍依赖 BIOS。

L20–21

```
void port_write(char *str)
{
```

定义字符串输出包装函数。str 必须指向可访问且以零字节结尾的字符序列。

L22

```
    call_bios((long)BIOS_PUTSTR, (long)str, IGNORE, IGNORE, IGNORE, IGNORE);
```

选择 BIOS_PUTSTR，把 str 的指针值转成 long 传入 a0。转型不会复制字符串。

L23

```
}
```

字符串包装函数结束。

L25–26

```
int port_read_ch(void)
{
```

定义读取一个字符的包装函数，参数表 void 表示不接收参数，返回值是 int 而不是 char。

L27

```
    return call_bios((long)BIOS_GETCHAR, IGNORE, IGNORE, IGNORE, IGNORE, IGNORE);
```

选择 BIOS_GETCHAR，把返回值交给调用者。使用 int 能表达普通字符以及约定的 -1“暂时没有字符”。

L28

```
}
```

读取字符包装结束。

L30–31

```
int sd_read(unsigned mem_address, unsigned num_of_blocks, unsigned block_id)
{
```

定义 sd_read：mem_address 是目的内存地址，num_of_blocks 是扇区数量，block_id 是起始扇区号，三者类型均为 unsigned。这里的块按项目约定为 512 字节扇区。

L32–33

```
    return (int)call_bios((long)BIOS_SDREAD, (long)mem_address, \
                          (long)num_of_blocks, (long)block_id, IGNORE, IGNORE);
```

选择 BIOS_SDREAD，把三个 unsigned 参数扩展为 long 并放入 a0/a1/a2；unused 参数置零，再把 BIOS long 返回值转为 int 交给上层。

L34

```
}
```

读盘函数结束。包装本身没有重试或检查目标地址范围。

L36–37

```
int sd_write(unsigned mem_address, unsigned num_of_blocks, unsigned block_id)
{
```

定义写盘包装 sd_write，参数含义与读取类似，但 mem_address 是要写出数据的内存地址。

L38–39

```
    return (int)call_bios((long)BIOS_SDWRITE, (long)mem_address, \
                          (long)num_of_blocks, (long)block_id, IGNORE, IGNORE);
```

调用 BIOS_SDWRITE，传入源内存、扇区数量和起始扇区号，并把 BIOS 结果转成 int 返回。

L40

```
}
```

写盘包装结束。当前 init/main.c 并未给跳转表的 SD_WRITE 项赋值，因此该函数存在不等于用户层 bios_sd_write 路径已初始化。

3 你能看到什么，什么代码保证它

输入循环在没有键盘输入时能继续轮询，而不是把“没有输入”显示成一个字节。

代码推导，未在课程平台实测

common.c:25—27 保留 BIOS_GETCHAR 的 int 返回值；init/main.c:112—115 显式检测 -1 并 continue。

读取一个镜像扇区后，可以得到完整 512 字节的缓冲区内容。

代码推导，未在课程平台实测

common.c:30—33 把地址、1 个块和起始块号送往 BIOS_SDREAD；kernel/loader/loader.c:24 用此包装把扇区读入 sector_buf。这里的 512 字节单位是项目/BIOS 接口约定。

容易误解或需要修正的地方

- 课程中使用的几个固定地址低于 4 GiB，unsigned 参数尚能容纳；一般 RV64 指针可能超过 32 位，传给 sd_read/sd_write 前已截断高 32 位，随后强转 long 也不能恢复。
- 函数指针直接跳 BIOS 入口不建立用户态隔离；错误地址、错误 ABI 或不存在的 BIOS 都可能导致异常。

- 底层返回值有传出，但 loader.c 忽略它，所以不能把完成一次函数调用等同于已经证明读盘成功。

02 • 启动、内核与装载

08 main.c：显示启动状态并按名字运行任务

原路径：init/main.c • 145 行

> 内核的 C 主程序：建立共享服务跳转表、载入任务目录，接收任务名字，顺序调用任务并等待返回。

整体流程是：启动准备 → Hello OS 与 bss 检查 → 请输任务名 → 查找并装载 → 直接调用 → 返回后继续输入。它像一个最小“程序菜单”，不是拥有独立进程与时间片的操作系统 shell。

本文件保留了按数字选择任务的旧实现，但位于 /* ... */ 注释内，真正生效的是 106—137 行的按名字循环。注释和 TODO 不能代替对有效代码的判断。

三个名字容易混淆：tasknum 是启动参数，task_num 是供 loader.c 使用的全局数量，tasks 是从磁盘任务目录读入的结构体数组。

1 完整代码，先看全貌

```
#include <common.h>
#include <asm.h>
#include <os/kernel.h>
#include <os/task.h>
#include <os/string.h>
#include <os/loader.h>
#include <type.h>

#define VERSION_BUF 50
//#define TASK_NUM_LOC 0x502001fa
int version = 2; // version must between 0 and 9
char buf[VERSION_BUF];

// Task info array
task_info_t tasks[TASK_MAXNUM];
int task_num;
static int bss_check(void)
{
    for (int i = 0; i < VERSION_BUF; ++i)
    {
        if (buf[i] != 0)
        {
            return 0;
        }
    }
    return 1;
}

static void init_jmptab(void)
{
    volatile long (*(*jmptab))() = (volatile long (*(*))())KERNEL_JMPTAB_BASE;

    jmptab[CONSOLE_PUTSTR]  = (long (*)())port_write;
    jmptab[CONSOLE_PUTCHAR] = (long (*)())port_write_ch;
```

```
    jmptab[CONSOLE_GETCHAR] = (long (*)())port_read_ch;
    jmptab[SD_READ]         = (long (*)())sd_read;
}

static void init_task_info(int tasknum, uint32_t app_info_offset)
{
    // TODO: [p1-task4] Init 'tasks' array via reading app-info sector
    load_img_bytes(app_info_offset,tasknum * sizeof(task_info_t), tasks);
    // NOTE: You need to get some related arguments from bootblock first
}

/************************************************************/
/* Do not touch this comment. Reserved for future projects. */
/************************************************************/

int main(int tasknum, uint32_t app_info_offset)
{
    task_num=tasknum;
    // Check whether .bss section is set to zero
    int check = bss_check();

    // Init jump table provided by kernel and bios(ΦωΦ)
    init_jmptab();

    // Init task information (〃'▽'〃)
    init_task_info(tasknum,app_info_offset);
   // bios_putstr(tasks[0].task_name);
    //bios_putchar('\n');

    //bios_putstr(tasks[1].task_name);
    //bios_putchar('\n');

    // Output 'Hello OS!', bss check result and OS version
    char output_str[] = "bss check: _ version: _\n\r";
    char output_val[2] = {0};
    int i, output_val_pos = 0;

    output_val[0] = check ? 't' : 'f';
    output_val[1] = version + '0';
    for (i = 0; i < sizeof(output_str); ++i)
    {
        buf[i] = output_str[i];
        if (buf[i] == '_')
        {
            buf[i] = output_val[output_val_pos++];
        }
    }

    bios_putstr("Hello OS!\n\r");
    bios_putstr(buf);

    // TODO: Load tasks by either task id [p1-task3] or task name [p1-task4],
    //   and then execute them.
    //short tasknum=*(volatile short *)TASK_NUM_LOC;
    /*while(1){
        int ch;
        int taskid;
        ch=port_read_ch();
        while(ch==-1){
```

```
            ch=port_read_ch();
        }
        port_write_ch(ch);
        taskid=ch-'0';
        if(taskid<0||taskid>=tasknum){
            port_write("\n\rInvalid task id\n\r");
            continue;
        }
        uint64_t entry=load_task_img(taskid);
        ((void (*)())entry)();
        bios_putstr("\n\rTask finished, back to loader\n\r");
    }*/
    while(1){
        char taskname[TASK_NAME_LEN];
        int pos=0;
        int ch;
        bios_putstr("Please input task name:\n\r");
        while (1){
            ch=bios_getchar();
            if(ch==-1){
                continue;
            }
            if(ch=='\n'||ch=='\r'){
                taskname[pos]='\0';
                bios_putstr("\n\r");
                break;
            }
            if(pos<TASK_NAME_LEN-1){
                taskname[pos++]=(char)ch;
                bios_putchar((char)ch);
            }
        }
        if(pos==0){
            continue;
        }
        uint64_t entry=load_task_img(taskname);
        if(entry==0){
            bios_putstr("Task not found\n\r");
            continue;
        }
        ((void (*)(void))entry)();
        bios_putstr("\n\rTask finished, back to loader\n\r");
        // Infinite w
    }
    // hile loop, where CPU stays in a low-power state (QAQQQQQQQQQQ)
    while (1)
    {
        asm volatile("wfi");
    }

    return 0;
}
```

2 按原始行号，一句一句理解

L1

```
#include <common.h>
```

引入 port_write、sd_read 等底层 BIOS 包装函数的声明。

L2

```
#include <asm.h>
```

引入汇编相关宏定义；仅包含头文件不会自动执行其中宏。

L3

```
#include <os/kernel.h>
```

引入 KERNEL_JMPTAB_BASE、服务索引及 bios_putstr 等内联包装函数。

L4

```
#include <os/task.h>
```

引入 task_info_t、TASK_MAXNUM、TASK_NAME_LEN 等任务表格式约定。

L5

```
#include <os/string.h>
```

引入字符串函数声明，尤其是后续加载器进行精确名字匹配用到的 strcmp。

L6

```
#include <os/loader.h>
```

引入 load_img_bytes 和 load_task_img 的函数声明。

L7

```
#include <type.h>
```

引入 uint32_t、uint64_t 等定宽整数类型别名。

L9

```
#define VERSION_BUF 50
```

定义缓冲区容量 VERSION_BUF 为 50 字节，编译时将用这个数代替宏名。

L10

```
//#define TASK_NUM_LOC 0x502001fa
```

整行 // 注释掉旧任务数字段地址，不会为当前程序定义 TASK_NUM_LOC，也不会影响本次按名字选择。

L11

```
int version = 2; // version must between 0 and 9
```

定义全局变量 version 并初始化为 2，通常放入 .data。后面的提示文字把它转换成一个数字字符，所以注释要求 0—9。

L12

```
char buf[VERSION_BUF];
```

定义未显式初始化的全局字符数组 buf，占 50 字节 .bss。它不是栈上的临时数组；启动清零应保证其每个字节初始为零。

L14

```
// Task info array
```

这是任务目录数组的说明注释。

L15

```
task_info_t tasks[TASK_MAXNUM];
```

分配最多 TASK_MAXNUM 个 task_info_t 的全局数组，用于存放从磁盘读回的任务名、位置、大小与入口。

L16

```
int task_num;
```

定义全局整数 task_num，最初也位于 .bss，稍后由 main 的参数赋值。

L17–18

```
static int bss_check(void)
{
```

定义只在本文件可见的 bss_check，返回 int。它检查 buf 是否全零，而不是遍历整个内核 .bss。

L19–20

```
    for (int i = 0; i < VERSION_BUF; ++i)
    {
```

for 从 i=0 开始，每次加 1，只检查下标小于 50 的元素，因此遍历 buf[0] 到 buf[49]。

L21–22

```
        if (buf[i] != 0)
        {
```

若某个 buf[i] 不是零，进入失败分支；!= 表示“不等于”，不会给该元素赋值。

L23

```
            return 0;
```

立即返回 0 表示检查失败，后续字节不再检查。

L24–25

```
        }
    }
```

依次结束 if 和 for；只有没有提前 return 的路径才到下一句。

L26

```
    return 1;
```

所有 50 个字节均为零，返回 1 表示成功。

L27

```
}
```

bss_check 函数结束。

L29–30

```
static void init_jmptab(void)
{
```

定义 init_jmptab，它没有参数，也没有普通返回值，用于在固定内存处写入服务函数地址。

L31

```
    volatile long (*(*jmptab))() = (volatile long (*(*))())KERNEL_JMPTAB_BASE;
```

把固定地址 KERNEL_JMPTAB_BASE 解释为函数指针数组入口，可先按“jmptab[i] 是第 i 个函数地址”理解。这里 volatile 写在函数的 long 返回类型上，没有把指针本身或表项声明为 volatile，因此不能据此认为访问整张表都是 volatile 读写；它也不提供锁或访问权限保护。

L33

```
    jmptab[CONSOLE_PUTSTR]  = (long (*)())port_write;
```

把字符串输出服务槽 CONSOLE_PUTSTR 指向 port_write；之后 bios_putstr 会先从这一槽取函数地址。

L34

```
    jmptab[CONSOLE_PUTCHAR] = (long (*)())port_write_ch;
```

把字符输出服务槽指向 port_write_ch。

L35

```
    jmptab[CONSOLE_GETCHAR] = (long (*)())port_read_ch;
```

把字符输入服务槽指向 port_read_ch，供内核和测试程序使用。

L36

```
    jmptab[SD_READ]         = (long (*)())sd_read;
```

把 SD_READ 槽指向 sd_read。注意没有对应的 SD_WRITE 赋值；不能据此假定写盘服务槽也就绪。

L37

```
}
```

跳转表初始化结束。

L39–40

```
static void init_task_info(int tasknum, uint32_t app_info_offset)
{
```

定义初始化任务目录的私有函数，接收任务项数量和任务目录在 image 内的字节偏移。

L41

```
    // TODO: [p1-task4] Init 'tasks' array via reading app-info sector
```

注释说明 Task 4 的目标是从磁盘 app-info 数据初始化 tasks；后面已有实现。

L42

```
    load_img_bytes(app_info_offset,tasknum * sizeof(task_info_t), tasks);
```

从 app_info_offset 起读 tasknum×sizeof(task_info_t) 个字节到 tasks。sizeof 以编译器实际结构体布局为准；本句没有核验 tasknum≤TASK_MAXNUM。

L43

```
    // NOTE: You need to get some related arguments from bootblock first
```

提醒这些实参来自 bootblock 的头字段，而非临时猜测的扇区号。

L44

```
}
```

任务目录初始化函数结束。

L46–48

```
/************************************************************/
/* Do not touch this comment. Reserved for future projects. */
/************************************************************/
```

课程预留注释，不执行任何语句。

L50–51

```
int main(int tasknum, uint32_t app_info_offset)
{
```

定义内核 C 入口。bootblock 把 tasknum 放在 a0、app_info_offset 放在 a1，head.S 准备环境后调用到这里；这个 main 不是宿主 Linux 的 argc/argv 入口。

L52

```
    task_num=tasknum;
```

把参数 tasknum 复制到全局 task_num，使 loader.c 也知道有多少条任务记录。

L53

```
    // Check whether .bss section is set to zero
```

注释说明接下来验证 .bss 清零效果。

L54

```
    int check = bss_check();
```

调用 bss_check 并保存结果。必须在写入 buf 之前做这一步，才是在检查初始状态。

L56–57

```
    // Init jump table provided by kernel and bios(ΦωΦ)
    init_jmptab();
```

先初始化服务跳转表，后面 bios_putstr、bios_getchar 才能从表中找到有效函数。

L59–60

```
    // Init task information (〃'▽'〃)
    init_task_info(tasknum,app_info_offset);
```

根据 bootblock 的两个参数从磁盘读任务目录，填好 tasks 数组。

L61–62

```
    // bios_putstr(tasks[0].task_name);
     //bios_putchar('\n');
```

注释掉的调试输出，本来可显示第一个任务名；当前不执行，所以启动时不应期待它自动列出任务清单。

L64–65

```
    //bios_putstr(tasks[1].task_name);
    //bios_putchar('\n');
```

第二个任务名的调试输出同样被 // 禁用。

L67

```
    // Output 'Hello OS!', bss check result and OS version
```

注释预告将显示欢迎文字、bss 检查结果和版本号。

L68

```
    char output_str[] = "bss check: _ version: _\n\r";
```

在栈上建立提示模板，两个下划线是稍后替换的占位符。\n 是换行，\r 是回车，字符串结尾还自动有一个零字节。

L69

```
    char output_val[2] = {0};
```

在栈上建立两个字符槽并初始清零；这里的 {0} 是 C 初始化规则，会把两个元素都置零。

L70

```
    int i, output_val_pos = 0;
```

声明循环变量 i 和占位符索引 output_val_pos，其中后者从 0 开始。

L72

```
    output_val[0] = check ? 't' : 'f';
```

三元运算符在 check 非零时选字符 t，否则选 f；这里只选一个字符，不是在生成整条字符串。

L73

```
    output_val[1] = version + '0';
```

把整数版本号加上字符 0 的编码，例如 2+'0' 得到字符 '2'。这只适合单个十进制数字。

L74–75

```
    for (i = 0; i < sizeof(output_str); ++i)
    {
```

遍历模板数组的 sizeof(output_str) 个字节，包含字符串结尾的零字节；因此复制完成的 buf 仍能作为零结尾字符串使用。

L76

```
        buf[i] = output_str[i];
```

先把模板当前字节复制进全局 buf 的同一下标。

L77–78

```
        if (buf[i] == '_')
        {
```

若刚复制的是下划线，进入替换分支。两个占位符依次对应 bss 结果与版本字符。

L79

```
            buf[i] = output_val[output_val_pos++];
```

用 output_val 的当前元素覆盖下划线，然后把索引加 1。模板里恰好两个下划线；如果以后随意增加占位符，也必须同步扩展数组。

L80–81

```
        }
    }
```

结束替换分支和复制循环。当前模板远小于 50 字节，所以能放入 buf。

L83

```
    bios_putstr("Hello OS!\n\r");
```

通过已经初始化的跳转表输出固定欢迎文字 Hello OS!。

L84

```
    bios_putstr(buf);
```

输出替换后的 buf；正常应显示 bss check: t version: 2。

L86–88

```
    // TODO: Load tasks by either task id [p1-task3] or task name [p1-task4],
    //   and then execute them.
    //short tasknum=*(volatile short *)TASK_NUM_LOC;
```

这些是从数字任务号过渡到任务名字的说明及已注释旧变量读取，均不执行。

L89–105

```
（此处为连续注释或较长结构，请对照上方完整代码 L89–L105。）
```

整个旧 while 循环被块注释包住：它原本把输入字符减去 '0' 当任务编号，再按编号调用任务。当前编译器忽略整块内容，不能把它的 Invalid task id 当作当前程序提示。

L106

```
    while(1){
```

进入永久的外层任务选择循环。每次成功返回、错误名字或空名字，最终都会重新从这里开始。

L107

```
        char taskname[TASK_NAME_LEN];
```

为当前一次输入建立长度为 TASK_NAME_LEN（32）的字符数组。有效名字最多存 31 字节，最后一格留给字符串结束符。

L108

```
        int pos=0;
```

pos=0 表示尚未接收到任何名字字符。它每次外层循环都会重新初始化。

L109

```
        int ch;
```

用 int 保存输入结果，以便同时容纳字符编码和 -1 无字符标记。

L110

```
        bios_putstr("Please input task name:\n\r");
```

输出 Please input task name: 并换行，提示用户准备输入。

L111

```
        while (1){
```

进入内层循环，逐字符接收当前这一行；只有回车/换行会退出这个循环。

L112

```
            ch=bios_getchar();
```

读取一个字符，可能得到 -1；调用不是自动读取整行。

L113–115

```
            if(ch==-1){
                continue;
            }
```

若结果为 -1，立即继续内层循环，轮询下一次；不写入数组，也不回显。该忙等不是休眠或进程调度。

L116

```
            if(ch=='\n'||ch=='\r'){
```

若字符是换行或回车，认为任务名字输入结束。用 || 表示满足任意一种即可。

L117

```
                taskname[pos]='\0';
```

在 taskname[pos] 放入零字节，构造 C 字符串的结束符。即使没有输入任何字符，也会正确生成空字符串。

L118

```
                bios_putstr("\n\r");
```

向串口输出换行及回车，让后续提示从新行开始。

L119

```
                break;
```

break 只退出内层输入循环，外层 while(1) 仍然继续存在。

L120

```
            }
```

结束回车/换行分支。

L121

```
            if(pos<TASK_NAME_LEN-1){
```

只有 pos 小于 31 时才保存普通字符，保证数组至少留下一个结束符位置。超长输入仍继续读取，但超过 31 的部分不会保存和回显。

L122

```
                taskname[pos++]=(char)ch;
```

把 int 字符转成 char 写入数组当前格，然后 pos 自增。赋值先使用旧 pos，所以第一个字符写入 taskname[0]。

L123

```
                bios_putchar((char)ch);
```

将刚接收并保存的字符回显到串口；回显是本句显式完成的，不能假定 BIOS 自动回显。

L124–125

```
            }
        }
```

结束长度判断和内层输入循环。没有退格字符的特殊分支，因此退格不会自动删除之前的字符。

L126–128

```
        if(pos==0){
            continue;
        }
```

如果 pos 仍为零，说明只按了回车；continue 在这里作用于外层循环，重新显示提示而不尝试查找空任务名。

L129

```
        uint64_t entry=load_task_img(taskname);
```

按名字查找并装载任务，返回其入口地址；这一步并没有执行任务 main。

L130

```
        if(entry==0){
```

用 entry==0 判断失败，因为加载器约定未找到名字时返回零。

L131

```
            bios_putstr("Task not found\n\r");
```

名字不存在时输出 Task not found。

L132

```
            continue;
```

立即继续外层循环，让用户再次输入；因此不会将地址 0 当函数调用。

L133

```
        }
```

错误处理分支结束。

L134

```
        ((void (*)(void))entry)();
```

把整数地址 entry 强转为“无参数、无返回值的函数指针”，然后调用。它同步进入任务的 crt0 入口：直到任务返回，内核不会读取下一个任务名。

L135

```
        bios_putstr("\n\rTask finished, back to loader\n\r");
```

任务确实返回之后才打印 Task finished, back to loader。若任务一直循环，这句就不能执行。

L136

```
        // Infinite w
```

这条残缺的英文注释没有程序效果，也不表示出现了另一套等待机制。

L137

```
    }
```

结束外层循环体；因条件恒为 1，执行到这里会再次进入第 106 行。

L138

```
    // hile loop, where CPU stays in a low-power state (QAQQQQQQQQQQ)
```

这条注释描述的低功耗等待并非当前主路径：前一个无限循环没有 break，因此下面代码通常不可达。

L139–140

```
    while (1)
    {
```

第二个无限循环只是备用结构，在当前控制流下到不了这里。

L141

```
        asm volatile("wfi");
```

如果未来修改让流程到此处，wfi 才会提示处理器等待中断；当前输入轮询没有执行这条指令。

L142

```
    }
```

结束备用等待循环。

L144

```
    return 0;
```

如果所有前面无限循环都被改变为能结束，则返回 0 给 head.S；现有程序通常不会执行到这里。

L145

```
}
```

内核 C main 的函数体结束。

3 你能看到什么，什么代码保证它

启动后应看到 Hello OS!，下一行 bss check: t version: 2，再出现 Please input task name:。

代码推导，未在课程平台实测

init/main.c:54 得到清零结果，68—81 替换模板，83—84 打印欢迎信息，110 打印输入提示；t 来自 head.S:16—23 清零，2 来自 main.c:11。

输入错误名字并回车，会显示 Task not found；直接按回车则只重新提示。

代码推导，未在课程平台实测

init/main.c:116—119 结束输入，126—128 单独处理空行，129—132 处理 loader 返回的零地址。

输入的名字会逐字显示，但最多保存和回显前 31 字节。

代码推导，未在课程平台实测

init/main.c:107 申请 32 字节数组，121 的上界比较保留结束符空间，122 写入字符，123 显式输出同一字符。

任务名必须精确匹配；可返回任务结束后才显示 Task finished, back to loader。

代码推导，未在课程平台实测

kernel/loader/loader.c:42—45 使用 strcmp==0 选择任务；init/main.c:134 是普通同步调用，135 的文字位于返回之后，137 之后回到外层循环。

容易误解或需要修正的地方

- 本程序没有输入编辑功能：退格、方向键转义序列、空格都会被当成普通字节；超长名字会被截取。若终端发送 CRLF，第二个换行还可能被下一轮当成空行。
- 任务名究竟是什么以镜像目录中的 task_name 为准。镜像工具保存命令行传入的文件名字符串；从 build 目录生成时通常是 2048、auipc、bss、data，而非带 .c 的源文件名。
- 任务数没有在 init_task_info 前核验，损坏/过大的 tasknum 会让读盘复制越过 tasks 数组。bss_check 只证明 buf 初始为零，不能证明全部 RAM、全部驱动或整个镜像都正确。
- 任务运行与内核共享地址空间和调用栈，entry 又直接来自镜像元数据；这里没有页表隔离、权限检查或时间片保障。
- 初始化只填写四个服务槽，SD_WRITE 槽并未填写。不要根据 include/os/kernel.h 存在 bios_sd_write 就认定主路径提供了可用写盘服务。

02 · 启动、内核与装载

09 loader.c：在任意字节偏移处取出一个任务

原路径：kernel/loader/loader.c · 55 行

> 把磁盘的 512 字节扇区接口转换成按字节复制的接口，再按任务名检索目录并把文件内容装到入口地址。

镜像将内核和任务紧密连续排列，任务不一定恰好从一个扇区边界开始。因此不能简单地把任务偏移当成扇区号；需要拆出“第几个扇区”和“该扇区内部偏移”。

例：offset=510、size=6。第一轮读扇区 0，从其中下标 510 复制 2 字节；下一轮 offset=512、size=4，读扇区 1，从下标 0 再复制 4 字节。最终只复制请求的 6 字节，尽管读盘合计取了 1024 字节。

1 完整代码，先看全貌

```
#include <os/task.h>
#include <os/string.h>
#include <os/kernel.h>
#include <type.h>
#define TASK_SIZE_SECTORS 15
#define TASK_MEM_BASE     0x52000000
#define TASK_MEM_STRIDE   0x10000
#define SECTOR_SIZE 512
static unsigned char sector_buf[SECTOR_SIZE];
extern int task_num;
extern task_info_t tasks[TASK_MAXNUM];
void load_img_bytes(uint32_t offset,uint32_t size,void *dst)
{
    unsigned char *out = (unsigned char *)dst;
    while (size > 0)
    {
        uint32_t sector = offset / SECTOR_SIZE;
        uint32_t in_sector = offset % SECTOR_SIZE;
        uint32_t copy_size = SECTOR_SIZE - in_sector;
        if (copy_size > size)
        {
            copy_size = size;
        }
        sd_read((unsigned)sector_buf, 1, sector);

        for (uint32_t i = 0; i < copy_size; ++i)
        {
            out[i] = sector_buf[in_sector + i];
        }
        offset += copy_size;
        out += copy_size;
        size -= copy_size;
    }
}
uint64_t load_task_img(char *taskname)
{
    /**
```

```
38      * TODO:
39      * 1. [p1-task3] load task from image via task id, and return its entrypoint
40      * 2. [p1-task4] load task via task name, thus the arg should be 'char *taskname'
41      */
42      for(int i=0;i<task_num;i++){
43         if(strcmp(taskname,tasks[i].task_name)==0){
44             load_img_bytes(tasks[i].offset,tasks[i].size,(void *)tasks[i].entrypoint);
45             return tasks[i].entrypoint;
46         }
47      }
48      return 0;
49     /*uint64_t task_entry;
50     uint32_t task_sector;
51     task_sector=1+TASK_SIZE_SECTORS*taskid+TASK_SIZE_SECTORS;
52     task_entry=TASK_MEM_BASE+TASK_MEM_STRIDE*taskid;
53     sd_read(task_entry, TASK_SIZE_SECTORS, task_sector);*/
54     //return task_entry;
55 }
```

2 按原始行号，一句一句理解

L1

```
#include <os/task.h>
```

引入任务结构体与任务数量上限宏，保证本文件能读取每项的名字、偏移、大小和入口。

L2

```
#include <os/string.h>
```

引入 strcmp，供任务名字精确比较使用。

L3

```
#include <os/kernel.h>
```

引入 sd_read 的间接声明/内核服务约定，加载器需要实际读盘。

L4

```
#include <type.h>
```

引入 uint32_t、uint64_t 等类型，明确字节偏移/大小与入口地址的宽度。

L5

```
#define TASK_SIZE_SECTORS 15
```

保留旧 Task 3 的固定每任务 15 扇区常量。当前有效代码按 tasks[i].size 读字节，没有使用这个常量。

L6

```
#define TASK_MEM_BASE     0x52000000
```

旧方案的任务起始内存地址常量；当前 load_task_img 直接使用目录记录的 entrypoint，不使用此宏计算目标。

L7

```
#define TASK_MEM_STRIDE   0x10000
```

旧方案每任务内存区相距 0x10000 的常量；同样只在注释旧代码里出现。

L8

```
#define SECTOR_SIZE 512
```

定义扇区大小为 512 字节，后面的除法与取余均以此为单位。

L9

```
static unsigned char sector_buf[SECTOR_SIZE];
```

定义本文件私有的静态扇区缓冲区，恰好 512 字节，通常位于内核 .bss。static 让它跨多次函数调用仍存在。

L10

```
extern int task_num;
```

extern 声明任务总数来自别的 C 文件，并不在这里再分配一个新的全局变量；定义位于 init/main.c:16。

L11

```
extern task_info_t tasks[TASK_MAXNUM];
```

声明共享任务目录数组由 init/main.c:15 定义，加载器读取同一份 records。

L12–13

```
void load_img_bytes(uint32_t offset,uint32_t size,void *dst)
{
```

定义按字节读取镜像内容的函数。offset 是相对 image 开始的字节偏移，size 是请求字节数，dst 是目标 RAM 地址；void * 允许调用者传任意类型的缓冲区。

L14

```
    unsigned char *out = (unsigned char *)dst;
```

把目标指针转换成 unsigned char *，让每次加 1 都表示前进 1 字节，而非前进一个结构体或一个整数。

L15–16

```
    while (size > 0)
    {
```

只要还有字节未复制，就处理一个扇区片段；size 为零时不会进行任何读盘。

L17

```
        uint32_t sector = offset / SECTOR_SIZE;
```

offset/512 计算扇区号。整数除法舍去余数，例如 510/512=0、512/512=1。

L18

```
        uint32_t in_sector = offset % SECTOR_SIZE;
```

offset%512 计算扇区内字节下标，范围为 0—511，例如 510%512=510。

L19

```
        uint32_t copy_size = SECTOR_SIZE - in_sector;
```

先假设把本扇区从 in_sector 到末尾的所有剩余字节都复制，数量为 512-in_sector。

L20–23

```
        if (copy_size > size)
        {
            copy_size = size;
        }
```

若这一片段比请求剩余 size 还大，则缩短为 size。这样最后一轮不会把请求范围外的数据也写进目标缓冲区。

L24

```
        sd_read((unsigned)sector_buf, 1, sector);
```

把整个 sector 号扇区读入 sector_buf，第二实参 1 表示只读一个扇区。这里把指针转成 unsigned，可能截断高地址；同时没有检查 sd_read 返回值。

L26–27

```
        for (uint32_t i = 0; i < copy_size; ++i)
        {
```

逐字节遍历本轮需要复制的 copy_size 字节，循环变量 i 从 0 递增。

L28

```
            out[i] = sector_buf[in_sector + i];
```

从扇区缓冲区的 in_sector+i 位置取一个字节，写到目标 out[i]。这一步滤掉了扇区前方不属于请求范围的字节。

L29

```
        }
```

结束本轮逐字节复制循环。

L30

```
        offset += copy_size;
```

镜像读取位置向前推进已经成功尝试复制的字节数，为下一轮定位。

L31

```
        out += copy_size;
```

目标内存指针同步前移，使下一轮内容接在上一轮后面，而不是覆盖已经复制的部分。

L32

```
        size -= copy_size;
```

剩余字节数减去本轮数量。copy_size 至少为 1（size>0 时），因此正常整数范围内循环会取得进展。

L33–34

```
    }
}
```

结束 while 和函数；函数返回 void，并没有向调用者提供读盘成功/失败状态。

L35–36

```
uint64_t load_task_img(char *taskname)
{
```

定义按任务名装载函数，输入是零结尾名字字符串，返回 64 位入口地址；返回零代表没找到。

L37–41

```
    /**
     * TODO:
     * 1. [p1-task3] load task from image via task id, and return its entrypoint
     * 2. [p1-task4] load task via task name, thus the arg should be 'char *taskname'
     */
```

多行 TODO 记录 Task 3 到 Task 4 的演进：旧参数是编号，当前参数改成名字字符串。注释不执行，也不会改变函数签名。

L42

```
    for(int i=0;i<task_num;i++){
```

从任务目录第 0 项依次查到 task_num-1；这里依赖全局数量可信，没有额外限制最多查 16 项。

L43

```
        if(strcmp(taskname,tasks[i].task_name)==0){
```

strcmp 返回 0 表示两个零结尾字符串完全相等，才选择该项；只匹配前缀或大小写不同都不会进入此分支。

L44

```
            load_img_bytes(tasks[i].offset,tasks[i].size,(void *)tasks[i].entrypoint);
```

从记录的磁盘字节 offset 读取 size 字节，写到记录的 entrypoint 指向的 RAM。它假定“把文件段连续内容搬到入口”就是该任务所需布局，并未解释 ELF 的虚拟地址和多个段间距。

L45

```
            return tasks[i].entrypoint;
```

返回同一入口数值给内核；本函数只装载，不在这里执行入口地址。

L46–47

```
        }
    }
```

结束名字匹配分支和目录遍历。找到就已在上句 return，不会继续查找同名后续项。

L48

```
    return 0;
```

全部目录项都不匹配时返回零，让 main.c 安全走 Task not found 分支，而非调用不存在的地址。

L49–53

```
    /*uint64_t task_entry;
    uint32_t task_sector;
    task_sector=1+TASK_SIZE_SECTORS*taskid+TASK_SIZE_SECTORS;
    task_entry=TASK_MEM_BASE+TASK_MEM_STRIDE*taskid;
    sd_read(task_entry, TASK_SIZE_SECTORS, task_sector);*/
```

被块注释关闭的旧编号方案：按固定 15 扇区计算任务位置和固定步长内存位置，再直接读盘。当前 Task 4 不执行它，不能用它推导实际镜像布局。

L54

```
    //return task_entry;
```

旧 return task_entry 也被 // 注释，当前成功返回由第 45 行负责。

L55

```
}
```

按名字装载函数结束。

3 你能看到什么，什么代码保证它

即使任务开头或结尾不在扇区边界，也能只复制所请求的任务字节。

代码推导，未在课程平台实测

loader.c:17—23 拆分扇区及片段长度，24 读整个扇区，26—32 只复制目标片段并推进三个状态变量。offset=510,size=6 的跨界例子分别复制 2 和 4 字节。

输入镜像中不存在的任务名不会执行地址 0。

代码推导，未在课程平台实测

loader.c:42—48 用 strcmp 查找，全部不匹配时返回 0；init/main.c:130—133 捕获此结果并重新提示。

输入相同名字时，读到目录中的第一条精确匹配记录。

代码推导，未在课程平台实测

loader.c:42 的升序遍历与 43 的 strcmp==0 判断共同决定匹配规则；45 的立即 return 使后续同名记录不再参与选择。

容易误解或需要修正的地方

- 这里只搬 image 中的原始字节，不会像通用 ELF 加载器那样根据每个 PT_LOAD 的 p_vaddr/p_memsz 映射并补零。镜像工具与链接布局必须共同满足当前假设，任务 .bss 清零由 crt0.S 另行完成。
- 没有验证 offset+size 的溢出、目标内存容量、入口是否可执行或任务数量是否越界。镜像是课程受信任输入，不等于这段代码具备健壮的通用加载安全性。

- 静态 sector_buf 只适合本项目同步单调用的方式；如果未来并行装载或中断中复用它，需要解决重入/同步问题。
- 读盘错误不会导致本函数报错；它仍可能复制缓冲区旧内容并返回入口。因此“返回了非零入口”只证明名字匹配，不单独证明磁盘读取有效。

02 · 启动、内核与装载

10 string.c：自己实现最基础的内存与字符串操作

原路径：libs/string.c · 96 行

> 内核不链接宿主系统标准库，因此提供小型字符串/字节数组工具。任务名精确匹配主要依赖这里的 strcmp。

内存函数根据长度处理任意字节，包括值为零的字节；字符串函数依据结尾的 \0 决定何时停止。不要把“数组容量”和“字符串已经使用的长度”混为一谈。

这些是项目自己的接口，不完全等同于标准 libc：例如 memcpy、memset 这里返回 void。调用方应以 include/os/string.h 的声明为准，Makefile 的 -fno-builtin 避免编译器盲目套用标准库内建假设。

1 完整代码，先看全貌

```c
#include <os/string.h>

void memcpy(uint8_t *dest, const uint8_t *src, uint32_t len)
{
    for (; len != 0; len--) {
        *dest++ = *src++;
    }
}

void memset(void *dest, uint8_t val, uint32_t len)
{
    uint8_t *dst = (uint8_t *)dest;

    for (; len != 0; len--) {
        *dst++ = val;
    }
}

void bzero(void *dest, uint32_t len)
{
    memset(dest, 0, len);
}

int strlen(const char *src)
{
    int i = 0;
    while (src[i] != '\0') {
        i++;
    }
    return i;
}

int strcmp(const char *str1, const char *str2)
{
    while (*str1 && *str2) {
```

```
        if (*str1 != *str2) {
            return (*str1) - (*str2);
        }
        ++str1;
        ++str2;
    }
    return (*str1) - (*str2);
}

int strncmp(const char *str1, const char *str2, uint32_t n)
{
    for (uint32_t i = 0; i < n; ++i)
        if (str1[i] == '\0' || str1[i] != str2[i])
            return str1[i] - str2[i];
    return 0;
}


char *strcpy(char *dest, const char *src)
{
    char *tmp = dest;

    while (*src) {
        *dest++ = *src++;
    }

    *dest = '\0';

    return tmp;
}

char *strncpy(char *dest, const char *src, int n)
{
    char *tmp = dest;

    while (*src && n-- > 0) {
        *dest++ = *src++;
    }

    while (n-- > 0) {
        *dest++ = '\0';
    }

    return tmp;
}

char *strcat(char *dest, const char *src)
{
    char *tmp = dest;

    while (*dest != '\0') {
        dest++;
    }
    while (*src) {
        *dest++ = *src++;
    }

    *dest = '\0';

```

```
95      return tmp;
96  }
```

2 按原始行号，一句一句理解

L1

```
#include <os/string.h>
```

引入这些函数的项目声明及整数类型，确保定义与调用一致。

L3–4

```
void memcpy(uint8_t *dest, const uint8_t *src, uint32_t len)
{
```

定义 memcpy，接收目标指针、只读源指针和字节数 len；返回 void。const 表示本函数不通过 src 修改源内容。

L5

```
    for (; len != 0; len--) {
```

for 初始化部分为空，因为参数已准备好；只要 len 不为零就继续，每轮结束 len 减 1。

L6

```
        *dest++ = *src++;
```

先读取 *src 写到 *dest，然后两个指针分别前进 1。由于指针指向 uint8_t，每次前进 1 字节；后置 ++ 不改变本次取用的位置。

L7–8

```
    }
}
```

结束复制循环及函数。len=0 时完全不读写内存。此顺序复制没有处理相互重叠的源和目标。

L10–11

```
void memset(void *dest, uint8_t val, uint32_t len)
{
```

定义 memset，把目标区域的 len 个字节设成同一个 val。void * 让目标可以来自任意类型的对象。

L12

```
    uint8_t *dst = (uint8_t *)dest;
```

将目标转为逐字节指针 dst，后续每次递增写入一个字节。

L14

```
    for (; len != 0; len--) {
```

重复 len 次，每次写完后剩余计数减 1。

L15

```
        *dst++ = val;
```

把 val 写入当前字节并前移指针。例如 val=0 清一个字节，val=0xff 则每字节都写 255。

L16–17

```
    }
}
```

结束填充循环及函数。它不会根据值把不同宽度整数自动编码成数组元素。

L19–20

```
void bzero(void *dest, uint32_t len)
{
```

定义 bzero，语义是“将指定长度的内存全置零”。

L21

```
    memset(dest, 0, len);
```

直接复用 memset(dest,0,len)，没有另写一套重复的逐字节循环。

L22

```
}
```

bzero 结束。

L24–25

```
int strlen(const char *src)
{
```

定义 strlen，接收指向零结尾字符串的只读指针，返回 int 类型长度。

L26

```
    int i = 0;
```

i=0，从字符串第一个字节开始检查。

L27

```
    while (src[i] != '\0') {
```

如果当前位置不是零字节，就还没遇到结束符。空字符串第 0 个字节就是零，因此不会进入循环。

L28

```
        i++;
```

向后移动一个字符下标；对 UTF-8 中文而言这里实际数的是字节，不是汉字个数。

L29

```
    }
```

结束 while 循环体，接着再次测试 src[i]。

L30

```
    return i;
```

返回结束符之前的字节数量，不把末尾零字节算进去。

L31

```
}
```

strlen 结束。若传入内容根本没有可访问的结束符，循环会越界读取。

L33–34

```
int strcmp(const char *str1, const char *str2)
{
```

定义 strcmp，比较两个零结尾字符串，约定相等返回 0，不等返回字符差值。

L35

```
    while (*str1 && *str2) {
```

只有两边当前字节都非零才继续循环。*str1 是当前字符值，不是整个字符串；&& 使任意一边结束时就退出。

L36

```
        if (*str1 != *str2) {
```

若当前两个字符不同，就已经能确定字符串不相等，无需比较剩余字符。

L37

```
            return (*str1) - (*str2);
```

返回当前字符值之差。调用者通常只关心是零、负数还是正数，不应假定非零只会是 1 或 -1。

L38

```
        }
```

结束字符不相等分支。

L39

```
        ++str1;
```

第一串的指针前进一个字节。

L40

```
        ++str2;
```

第二串的指针也前进一个字节，保持两边在相同位置比较。

L41

```
    }
```

结束本轮比较，回到 while 检查新位置。

L42

```
    return (*str1) - (*str2);
```

若循环因某边结束而退出，比较末尾字节：两边都为零则相等；例如 data 与 dataX 的这里是 0-'X'，所以前缀不算完全相等。

L43

```
}
```

strcmp 结束。ASCII 名字的精确相等判断由此保证。

L45–46

```
int strncmp(const char *str1, const char *str2, uint32_t n)
{
```

定义 strncmp，只比较至多 n 个字节，适用于限制比较范围。

L47

```
    for (uint32_t i = 0; i < n; ++i)
```

从下标 0 遍历到 n-1。没有花括号时，for 的循环体是紧随其后的整个 if 语句。

L48

```
        if (str1[i] == '\0' || str1[i] != str2[i])
```

若第一串已经结束，或者当前字符不相等，就应立即给出结果。第一条件保证相同字符串遇到结束符也不会继续无意义读取。

L49

```
            return str1[i] - str2[i];
```

返回当前两个字符之差；如果两边同时结束，差值是 0。

L50

```
    return 0;
```

前 n 个字节全部相同就返回 0，即使更后面的字符不同。例如 strncmp("data","database",4) 返回 0。

L51

```
}
```

strncmp 结束。

L54–55

```
char *strcpy(char *dest, const char *src)
{
```

定义 strcpy，将零结尾字符串复制到 dest，并返回最初的目标地址。调用者必须先提供足够空间。

L56

```
    char *tmp = dest;
```

tmp 保存原始 dest；后面复制时 dest 自增，仍能用 tmp 返回字符串开头。

L58

```
    while (*src) {
```

只要源当前字节非零就继续复制，不在这个循环内复制结束符。

L59

```
        *dest++ = *src++;
```

复制当前字节，源与目标指针各前移 1，使用的是自增前的位置。

L60

```
    }
```

字符复制循环结束。

L62

```
    *dest = '\0';
```

显式把零字节写在目标字符串末尾；少了这一句，后续打印和比较就不知道在哪里停止。

L64

```
    return tmp;
```

返回保存的目标起始指针，可用于链接调用或后续访问。

L65

```
}
```

strcpy 结束；它不会自动检查目标数组容量。

L67–68

```
char *strncpy(char *dest, const char *src, int n)
{
```

定义 strncpy，最多从 src 取 n 个字节到 dest；n 是 int，正常调用应传非负值。

L69

```
    char *tmp = dest;
```

保存原目标指针，供最后返回。

L71

```
    while (*src && n-- > 0) {
```

仅当源字符非零且剩余计数大于零时复制。&& 会短路：源结束后不再执行 n--；若 n 已耗尽，则不进入循环。

L72

```
        *dest++ = *src++;
```

复制一个字符并推进两个指针，上一条条件已将计数减去 1。

L73

```
    }
```

结束正常字符复制循环。

L75

```
    while (n-- > 0) {
```

如果源先结束且仍剩额度，就执行补零循环；每轮将 n 再减 1。

L76

```
        *dest++ = '\0';
```

向目标后续位置写一个零字节并前移，直到把给定 n 字节范围全部处理完。

L77

```
    }
```

补零循环结束。

L79

```
    return tmp;
```

返回目标原始起点。若源长度大于或等于最初 n，前一循环就耗尽额度，不保证写入结束符；调用者常需另留一格并手动补零。

L80

```
}
```

strncpy 结束。

L82–83

```
char *strcat(char *dest, const char *src)
{
```

定义 strcat，把 src 追加到 dest 当前字符串后面，并返回原目标起点。

L84

```
    char *tmp = dest;
```

保存目标开头，避免查找末尾时丢失返回值。

L86

```
    while (*dest != '\0') {
```

寻找目标现有字符串的零结束符。目标必须已经是有效 C 字符串。

L87

```
        dest++;
```

目标指针逐字节前进，直到命中旧结束符。

L88

```
    }
```

结束寻找目标末尾的循环，此时 dest 正指向可以开始追加的位置。

L89

```
    while (*src) {
```

从源头开始逐字节取字符，直到源的零结束符。

L90

```
        *dest++ = *src++;
```

复制字符并推进源、目标指针；第一个字符覆盖目标原先的结束符。

L91

```
    }
```

结束追加字符循环。

L93

```
    *dest = '\0';
```

在拼接后的新末尾补一个零字节，使结果仍是合法字符串。

L95

```
    return tmp;
```

返回 tmp，即拼接前目标字符串的起始位置。

L96

```
}
```

strcat 结束。函数没有容量参数，不会阻止较长源串写出目标数组范围。

3 你能看到什么，什么代码保证它

任务名字 data 和 dataX 被视为不同名字，大小写也不会自动转换。

代码推导，未在课程平台实测

string.c:35—42 按每个原始字符值比较；某边先结束时仍比较结尾差值。loader.c:43 仅接受 strcmp 返回 0。

复制字符串后通常能用零结尾决定打印边界，但 strncpy 截断路径需要调用者特别处理。

代码推导，未在课程平台实测

string.c:62 为 strcpy 补零，93 为 strcat 补零；strncpy 的71—79在额度耗尽时不强制补零。tools/createimage.c 写 task_name 时另将最后一格置零。

内存清零会把每个字节设为零，不受原先数组内容影响。

代码推导，未在课程平台实测

string.c:19—22 的 bzero 调用 10—17 的 memset，14—15 对给定 len 的每个字节写 val=0。注意内核早期 .bss 清零实际由 head.S 汇编完成，不是调用本 bzero。

容易误解或需要修正的地方

- memcpy 没有 memmove 的重叠保证；strcpy、strcat 没有长度上限；strlen、strcmp 要求能遇到有效零结尾。不能把这些函数当成自动验证所有指针的安全接口。
- 比较差值直接使用 char，而不是标准库常用的 unsigned char 转换。因此 ASCII 任务名判断相等没有问题，但包含高位字节时的排序正负可能随 char 有无符号而不同。
- 本文件函数操作字节，不解析 UTF-8。一个中文汉字可能占三个字节，字符串长度及任务名 31 字节限制都应按字节理解。

03 · 镜像与构建

11 镜像制作器：把 ELF 程序做成可从磁盘读取的 image

原路径：tools/createimage.c · 338 行

> 运行在开发电脑上的普通 Linux C 程序；它不是内核，也不会自己启动 RISC-V。

先把两种地址分开：ELF 的入口地址 e_entry 是程序将来在目标机内存中执行的位置；image 中的 offset 是文件里从第几个字节取数据。变量 phyaddr 虽然名字像物理地址，实际在这里累计的是镜像文件偏移。

输入顺序决定身份：第0个文件是 bootblock，第1个是内核，其余才是应用。各 ELF 的 PT_LOAD 段被依次拷贝；bootblock 填到512字节；最后追加任务信息，并回头在首扇区填入任务数、任务表偏移、内核字节数和55 aa标志。

这是基于原代码的解释，不能把它当作适用于任意 ELF 的通用打包器。它不根据 p_vaddr 保留段间内存空洞，不写出 .bss 的零填充，也没有足够的输入边界检查。只有相关布局约定成立时，后续简单加载器才可能正确执行。

1 完整代码，先看全貌

```
#include <assert.h>
#include <elf.h>
#include <errno.h>
#include <stdarg.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define IMAGE_FILE "./image"
#define ARGS "[--extended] [--vm] <bootblock> <executable-file> ..."

#define SECTOR_SIZE 512
#define TASK_SIZE_SECTORS 15
#define TASK_SIZE_BYTES (TASK_SIZE_SECTORS * SECTOR_SIZE)
#define BOOT_LOADER_SIG_OFFSET 0x1fe
#define OS_SIZE_LOC (BOOT_LOADER_SIG_OFFSET - 2)
#define APP_INFO_OFFSET_LOC 0x1f8
#define TASK_NUM_LOC 0x1f6
#define BOOT_LOADER_SIG_1 0x55
#define BOOT_LOADER_SIG_2 0xaa
#define TASK_NAME_LEN 32
#define NBYTES2SEC(nbytes) (((nbytes) / SECTOR_SIZE) + ((nbytes) % SECTOR_SIZE != 0))

/* TODO: [p1-task4] design your own task_info_t */
typedef struct {
    char task_name[TASK_NAME_LEN];
    uint32_t offset;
    uint32_t size;
    uint64_t entrypoint;
} task_info_t;

```

```c
#define TASK_MAXNUM 16
static task_info_t taskinfo[TASK_MAXNUM];

/* structure to store command line options */
static struct {
    int vm;
    int extended;
} options;

/* prototypes of local functions */
static void create_image(int nfiles, char *files[]);
static void error(char *fmt, ...);
static void read_ehdr(Elf64_Ehdr *ehdr, FILE *fp);
static void read_phdr(Elf64_Phdr *phdr, FILE *fp, int ph, Elf64_Ehdr ehdr);
static uint64_t get_entrypoint(Elf64_Ehdr ehdr);
static uint32_t get_filesz(Elf64_Phdr phdr);
static uint32_t get_memsz(Elf64_Phdr phdr);
static void write_segment(Elf64_Phdr phdr, FILE *fp, FILE *img, int *phyaddr);
static void write_padding(FILE *img, int *phyaddr, int new_phyaddr);
static void write_img_info(int nbytes_kernel, task_info_t *taskinfo,
                           short tasknum, FILE *img);

int main(int argc, char **argv)
{
    char *progname = argv[0];

    /* process command line options */
    options.vm = 0;
    options.extended = 0;
    while ((argc > 1) && (argv[1][0] == '-') && (argv[1][1] == '-')) {
        char *option = &argv[1][2];

        if (strcmp(option, "vm") == 0) {
            options.vm = 1;
        } else if (strcmp(option, "extended") == 0) {
            options.extended = 1;
        } else {
            error("%s: invalid option\nusage: %s %s\n", progname,
                  progname, ARGS);
        }
        argc--;
        argv++;
    }
    if (options.vm == 1) {
        error("%s: option --vm not implemented\n", progname);
    }
    if (argc < 3) {
        /* at least 3 args (createimage bootblock main) */
        error("usage: %s %s\n", progname, ARGS);
    }
    create_image(argc - 1, argv + 1);
    return 0;
}

/* TODO: [p1-task4] assign your task_info_t somewhere in 'create_image' */
static void create_image(int nfiles, char *files[])
{
    int tasknum = nfiles - 2;
    int nbytes_kernel = 0;
```

```
    int phyaddr = 0;
    FILE *fp = NULL, *img = NULL;
    Elf64_Ehdr ehdr;
    Elf64_Phdr phdr;

    /* open the image file */
    img = fopen(IMAGE_FILE, "w");
    assert(img != NULL);

    /* for each input file */
    for (int fidx = 0; fidx < nfiles; ++fidx) {

        int taskidx = fidx - 2;
        int task_start=phyaddr;
        /* open input file */
        fp = fopen(*files, "r");
        assert(fp != NULL);

        /* read ELF header */
        read_ehdr(&ehdr, fp);
        printf("0x%04lx: %s\n", ehdr.e_entry, *files);

        /* for each program header */
        for (int ph = 0; ph < ehdr.e_phnum; ph++) {

            /* read program header */
            read_phdr(&phdr, fp, ph, ehdr);

            if (phdr.p_type != PT_LOAD) continue;

            /* write segment to the image */
            write_segment(phdr, fp, img, &phyaddr);
            if(fidx>=2){
                strncpy(taskinfo[taskidx].task_name,*files,TASK_NAME_LEN-1);
                taskinfo[taskidx].task_name[TASK_NAME_LEN-1]='\0';
                taskinfo[taskidx].offset=task_start;
                taskinfo[taskidx].size=phyaddr-task_start;
                taskinfo[taskidx].entrypoint=get_entrypoint(ehdr);
            }

            /* update nbytes_kernel */
            if (fidx==1) {
                nbytes_kernel += get_filesz(phdr);
            }
        }

        /* write padding bytes */
        /**
         * TODO:
         * 1. [p1-task3] do padding so that the kernel and every app program
         *  occupies the same number of sectors
         * 2. [p1-task4] only padding bootblock is allowed!
         */
        if (strcmp(*files, "bootblock") == 0) {
            write_padding(img, &phyaddr, SECTOR_SIZE);
        }

        fclose(fp);
        files++;
```

```
150     }
151     write_img_info(nbytes_kernel, taskinfo, tasknum, img);
152 
153     fclose(img);
154 }
155 
156 static void read_ehdr(Elf64_Ehdr * ehdr, FILE * fp)
157 {
158     int ret;
159 
160     ret = fread(ehdr, sizeof(*ehdr), 1, fp);
161     assert(ret == 1);
162     assert(ehdr->e_ident[EI_MAG1] == 'E');
163     assert(ehdr->e_ident[EI_MAG2] == 'L');
164     assert(ehdr->e_ident[EI_MAG3] == 'F');
165 }
166 
167 static void read_phdr(Elf64_Phdr * phdr, FILE * fp, int ph,
168                       Elf64_Ehdr ehdr)
169 {
170     int ret;
171 
172     fseek(fp, ehdr.e_phoff + ph * ehdr.e_phentsize, SEEK_SET);
173     ret = fread(phdr, sizeof(*phdr), 1, fp);
174     assert(ret == 1);
175     if (options.extended == 1) {
176         printf("\tsegment %d\n", ph);
177         printf("\t\toffset 0x%04lx", phdr->p_offset);
178         printf("\t\tvaddr 0x%04lx\n", phdr->p_vaddr);
179         printf("\t\tfilesz 0x%04lx", phdr->p_filesz);
180         printf("\t\tmemsz 0x%04lx\n", phdr->p_memsz);
181     }
182 }
183 
184 static uint64_t get_entrypoint(Elf64_Ehdr ehdr)
185 {
186     return ehdr.e_entry;
187 }
188 
189 static uint32_t get_filesz(Elf64_Phdr phdr)
190 {
191     return phdr.p_filesz;
192 }
193 
194 static uint32_t get_memsz(Elf64_Phdr phdr)
195 {
196     return phdr.p_memsz;
197 }
198 
199 static void write_segment(Elf64_Phdr phdr, FILE *fp, FILE *img, int *phyaddr)
200 {
201     if (phdr.p_memsz != 0 && phdr.p_type == PT_LOAD) {
202         /* write the segment itself */
203         /* NOTE: expansion of .bss should be done by kernel or runtime env! */
204         if (options.extended == 1) {
205             printf("\t\twriting 0x%04lx bytes\n", phdr.p_filesz);
206         }
207         fseek(fp, phdr.p_offset, SEEK_SET);
208         while (phdr.p_filesz-- > 0) {
```

```
209             fputc(fgetc(fp), img);
210             (*phyaddr)++;
211         }
212     }
213 }
214
215 static void write_padding(FILE *img, int *phyaddr, int new_phyaddr)
216 {
217     if (options.extended == 1 && *phyaddr < new_phyaddr) {
218         printf("\t\twrite 0x%04x bytes for padding\n", new_phyaddr - *phyaddr);
219     }
220
221     while (*phyaddr < new_phyaddr) {
222         fputc(0, img);
223         (*phyaddr)++;
224     }
225 }
226
227 /*static void write_img_info(int nbytes_kernel, task_info_t *taskinfo,
228                            short tasknum, FILE * img)
229 {
230     // TODO: [p1-task3] & [p1-task4] write image info to some certain places
231     // NOTE: os size, infomation about app-info sector(s) ...
232     short kernel_sectors;
233     kernel_sectors = NBYTES2SEC(nbytes_kernel);
234     uint32_t app_info_offset;
235     uint16_t kernel_bytes;
236     app_info_offset = (uint32_t)ftell(img);
237     fwrite(taskinfo, sizeof(task_info_t), tasknum, img);
238     if(nbytes_kernel>0xffff){
239         error("kernel is too large\n",NBYTES2SEC(nbytes_kernel));
240     }
241     kernel_bytes=(uint16_t)nbytes_kernel;
242
243     /*if(tasknum>TASK_MAXNUM){
244         error("too many tasks\n",tasknum);
245     }
246     if(NBYTES2SEC(nbytes_kernel)>TASK_SIZE_SECTORS){
247         error("kernel is too large\n",NBYTES2SEC(nbytes_kernel));
248     }
249     fseek(img, TASK_NUM_LOC, SEEK_SET);
250     fwrite(&tasknum, sizeof(short), 1,img);
251     fseek(img, APP_INFO_OFFSET_LOC, SEEK_SET);
252     fwrite(&app_info_offset, sizeof(uint32_t), 1,img);
253     fseek(img, OS_SIZE_LOC, SEEK_SET);
254     fwrite(&kernel_sectors, sizeof(uint16_t), 1,img);
255     fseek(img, BOOT_LOADER_SIG_OFFSET, SEEK_SET);
256     fputc(BOOT_LOADER_SIG_1, img);
257     fputc(BOOT_LOADER_SIG_2, img);
258     if (options.extended) {
259         printf("kernel size: %d bytes\n", kernel_bytes);
260         printf("task num: %d\n", tasknum);
261         printf("app info offset: %u\n", app_info_offset);
262     }
263
264 }
265
266 /* print an error message and exit */
267 static void write_img_info(int nbytes_kernel,
```

```
                            task_info_t *taskinfo,
                            short tasknum,
                            FILE *img)
{
    uint32_t app_info_offset;
    uint16_t kernel_bytes;

    if (nbytes_kernel > 0xffff)
    {
        error("kernel is too large\n");
    }

    kernel_bytes = (uint16_t)nbytes_kernel;

    /* App Info放在image末尾 */
    app_info_offset = (uint32_t)ftell(img);

    fwrite(taskinfo,
           sizeof(task_info_t),
           tasknum,
           img);

    /* task数量 */
    fseek(img, TASK_NUM_LOC, SEEK_SET);
    fwrite(&tasknum,
           sizeof(short),
           1,
           img);

    /* App Info偏移 */
    fseek(img, APP_INFO_OFFSET_LOC, SEEK_SET);
    fwrite(&app_info_offset,
           sizeof(uint32_t),
           1,
           img);

    /*
     * Task4:
     * 这里必须写kernel的字节数!
     * 不是sector数!
     */
    fseek(img, OS_SIZE_LOC, SEEK_SET);
    fwrite(&kernel_bytes,
           sizeof(uint16_t),
           1,
           img);

    /* boot signature */
    fseek(img, BOOT_LOADER_SIG_OFFSET, SEEK_SET);
    fputc(BOOT_LOADER_SIG_1, img);
    fputc(BOOT_LOADER_SIG_2, img);

    if (options.extended)
    {
        printf("kernel size: %d bytes\n", kernel_bytes);
        printf("task num: %d\n", tasknum);
        printf("app info offset: %u\n", app_info_offset);
    }
}
```

```
327 static void error(char *fmt, ...)
328 {
329     va_list args;
330 
331     va_start(args, fmt);
332     vfprintf(stderr, fmt, args);
333     va_end(args);
334     if (errno != 0) {
335         perror(NULL);
336     }
337     exit(EXIT_FAILURE);
338 }
```

2 按原始行号，一句一句理解

L1

```
#include <assert.h>
```

引入 assert：assert(条件) 在条件为假时终止开发机上的工具，用来暴露读文件等失败；它不是错误恢复机制，若定义 NDEBUG 还可能被编译掉。

L2

```
#include <elf.h>
```

引入系统 ELF 结构与常量，例如 Elf64_Ehdr、Elf64_Phdr、PT_LOAD。ELF 是带元信息的可执行文件格式，不等于最终裸镜像。

L3

```
#include <errno.h>
```

引入 errno：标准库失败时可能留下错误号，最后由 error 打印；本程序没有在每个操作前后严格维护它。

L4

```
#include <stdarg.h>
```

引入可变参数支持，供 error(fmt, ...) 像 printf 那样接收不同数量参数。

L5

```
#include <stdio.h>
```

引入 FILE、fopen、fread、fwrite、fseek 和 printf 等开发机文件/终端接口。

L6

```
#include <stdlib.h>
```

引入 exit 和 EXIT_FAILURE，让工具能以失败状态结束。

L7

```
#include <string.h>
```

引入 strcmp、strncpy 等字符串操作，处理选项、文件名和任务名。

L9

```
#define IMAGE_FILE "./image"
```

把输出文件固定为当前工作目录的 ./image；它不是相对于 createimage 可执行文件所在位置。Makefile 会先 cd build，所以正常输出为 build/image。

L10

```
#define ARGS "[--extended] [--vm] <bootblock> <executable-file> ..."
```

定义用法文字。尖括号只是提示你必须提供 bootblock 和至少一个可执行文件，不是运行时真正要输入的字符。

L12

```
#define SECTOR_SIZE 512
```

规定一扇区为512字节；磁盘接口以扇区编号读，任务表却记录字节偏移，两者必须换算。

L13–14

```
#define TASK_SIZE_SECTORS 15
#define TASK_SIZE_BYTES (TASK_SIZE_SECTORS * SECTOR_SIZE)
```

定义旧方案的每任务15扇区及其字节数7680。当前有效打包逻辑并不使用这两个宏来给内核/应用填充，不能凭此判断每个应用固定占7680字节。

L15

```
#define BOOT_LOADER_SIG_OFFSET 0x1fe
```

首扇区内0x1fe=510是两个启动标志字节的起点，最后一个字节位于511。

L16

```
#define OS_SIZE_LOC (BOOT_LOADER_SIG_OFFSET - 2)
```

内核大小字段放在0x1fc=508，宽度为2字节；当前有效实现写入的是内核文件数据的字节总数。

L17

```
#define APP_INFO_OFFSET_LOC 0x1f8
```

任务表在镜像中的字节偏移放在0x1f8=504，宽度4字节。

L18

```
#define TASK_NUM_LOC 0x1f6
```

任务数量放在0x1f6=502，宽度2字节。因此启动代码的有效数据必须给502之后的元数据留位置。

L19–20

```
#define BOOT_LOADER_SIG_1 0x55
#define BOOT_LOADER_SIG_2 0xaa
```

定义最后两个标志字节55和aa；这是格式标识，单凭这两个字节并不能证明整个镜像内容正确。

L21

```
#define TASK_NAME_LEN 32
```

任务名数组总长32字节，实际可存最多31个非零字符，再留一个字符串终止零。

L22

```
#define NBYTES2SEC(nbytes) (((nbytes) / SECTOR_SIZE) + ((nbytes) % SECTOR_SIZE != 0))
```

把字节数向上取整成扇区数：整数除法得到完整扇区，余数非零再加1。当前有效 write_img_info 不使用它写内核大小，而启动汇编自己做向上取整。

L24–25

```
/* TODO: [p1-task4] design your own task_info_t */
typedef struct {
```

TODO注释保留了课程任务提示；下面结构体实际已经定义。typedef 给结构类型起名，结构中的字段会一起写进磁盘。

L26

```
    char task_name[TASK_NAME_LEN];
```

保存任务名称。这里保存调用时提供的文件名字符串，不会自动剥去目录前缀。

L27

```
    uint32_t offset;
```

offset 是32位无符号整数，表示应用数据起点距离 image 开头多少字节，不是扇区编号。

L28

```
    uint32_t size;
```

size 记录这个应用在镜像中连续写了多少字节，而不是程序运行时包含 .bss 的总内存占用。

L29–30

```
    uint64_t entrypoint;
} task_info_t;
```

entrypoint 是64位入口内存地址，例如某应用的0x52000000。结构以 task_info_t 结束；在常用x86-64/RV64布局中各偏移为0/32/36/40，大小48字节，但代码没有显式序列化或packed声明，主机与目标ABI、字节序必须一致。

L32

```
#define TASK_MAXNUM 16
```

允许任务信息静态数组有16项；这是容量声明，还不是输入检查。

L33

```
static task_info_t taskinfo[TASK_MAXNUM];
```

定义进程内部可见的静态任务表。静态存储会在工具启动前清零，所以暂未写的名字尾部等初始为零；超过16项却仍会越界。

L35–39

```
/* structure to store command line options */
static struct {
    int vm;
    int extended;
} options;
```

定义只有 vm、extended 两个整数标记的全局选项对象；static 限制在这个C文件中可见。vm表示尚未实现的虚拟内存模式，extended表示额外日志。

L41

```
/* prototypes of local functions */
```

说明后面是一组函数声明；它们告诉编译器参数和返回值，不会立即执行函数。

L42

```
static void create_image(int nfiles, char *files[]);
```

声明 create_image：接收文件数以及文件名指针数组；真正的制作逻辑在87行起。

L43

```
static void error(char *fmt, ...);
```

声明可变参数 error，供任意位置输出错误并退出。

L44

```
static void read_ehdr(Elf64_Ehdr *ehdr, FILE *fp);
```

声明读取一个完整ELF文件头的函数；传入指针意味着函数可以写回调用者的 ehdr。

L45

```
static void read_phdr(Elf64_Phdr *phdr, FILE *fp, int ph, Elf64_Ehdr ehdr);
```

声明读取第ph个程序头的函数；传入ELF头是为了知道程序头表起点和表项大小。

L46

```
static uint64_t get_entrypoint(Elf64_Ehdr ehdr);
```

声明提取ELF入口地址的辅助函数，返回64位值。

L47

```
static uint32_t get_filesz(Elf64_Phdr phdr);
```

声明取段的文件字节数 p_filesz，返回类型却为32位，超大ELF段可能截断。

L48

```
static uint32_t get_memsz(Elf64_Phdr phdr);
```

声明取段内存字节数 p_memsz，当前版本定义了但没有调用它，不能据此认为已经处理 .bss。

L49

```
static void write_segment(Elf64_Phdr phdr, FILE *fp, FILE *img, int *phyaddr);
```

声明按程序头复制文件段的函数。phyaddr指向累计镜像偏移，因此函数能修改调用者的计数。

L50

```
static void write_padding(FILE *img, int *phyaddr, int new_phyaddr);
```

声明向目标镜像偏移填零的函数，并非内存清零。

L51–52

```
static void write_img_info(int nbytes_kernel, task_info_t *taskinfo,
                           short tasknum, FILE *img);
```

声明回填镜像元数据的函数；tasknum使用short，写出的字段依赖主机short大小和字节序。

L54–56

```
int main(int argc, char **argv)
{
    char *progname = argv[0];
```

Linux从 main 开始调用工具；argc是参数数量，argv是字符串指针数组。保存 argv[0] 作为程序名，后面移动argv时仍能正确打印用法。

L58–60

```
    /* process command line options */
    options.vm = 0;
    options.extended = 0;
```

选项处理开始，先把两个功能开关置0，表示默认不启用。

L61

```
    while ((argc > 1) && (argv[1][0] == '-') && (argv[1][1] == '-')) {
```

只要剩余至少一个参数，且下一个参数以两个减号开头，就继续解析选项。逻辑与会从左向右短路，因此先检查argc防止访问不存在的argv[1]。

L62

```
        char *option = &argv[1][2];
```

取下一个参数第3个字符起的位置，跳过前缀“--”，例如把“--extended”视为“extended”。

L64–65

```
        if (strcmp(option, "vm") == 0) {
            options.vm = 1;
```

strcmp相等返回0；识别到vm后把vm标记置1，不是马上执行虚拟内存打包。

L66–67

```
        } else if (strcmp(option, "extended") == 0) {
            options.extended = 1;
```

识别extended，打开后续段地址、大小、填充量等诊断输出；不改变预期镜像布局。

L68–71

```
        } else {
            error("%s: invalid option\nusage: %s %s\n", progname,
                  progname, ARGS);
        }
```

无法识别的选项调用error并打印程序名和用法；error内部exit，所以不会继续生成镜像。

L72

```
        argc--;
```

已经吃掉一个选项，剩余参数数量减1。

L73–74

```
        argv++;
    }
```

把argv指针向后移动一项；下一轮argv[1]就指向新的待解析参数。字符串本身没有被移动。

L75–77

```
    if (options.vm == 1) {
        error("%s: option --vm not implemented\n", progname);
    }
```

如果选择了vm，明确报“not implemented”并退出；本版本没有可用的--vm实现。

L78–81

```
    if (argc < 3) {
        /* at least 3 args (createimage bootblock main) */
        error("usage: %s %s\n", progname, ARGS);
    }
```

至少需要程序名、bootblock、内核共3项，否则报用法错误。这里没有检查“文件名是否真的是正确角色”。

L82

```
    create_image(argc - 1, argv + 1);
```

从参数列表中跳过程序名，把剩下的文件个数及地址交给create_image。

L83–84

```
    return 0;
}
```

制作成功返回0给Linux，约定代表命令成功；只有执行到这里才是成功结束。

L86–88

```
/* TODO: [p1-task4] assign your task_info_t somewhere in 'create_image' */
static void create_image(int nfiles, char *files[])
{
```

课程TODO说明任务表填充位置；下面开始真正的create_image函数体。

L89

```
    int tasknum = nfiles - 2;
```

文件总数减去bootblock和内核，得到应用数量。代码缺少 tasknum≤16 的有效检查，不能向它任意传很多应用。

L90

```
    int nbytes_kernel = 0;
```

内核文件数据字节数初始为0；后面只对第1号输入累计。

L91

```
    int phyaddr = 0;
```

累计镜像文件位置从0开始；虽然变量叫phyaddr，这里没有向目标机器物理内存写入。

L92

```
    FILE *fp = NULL, *img = NULL;
```

初始化输入和输出FILE指针为NULL，稍后由fopen赋实际文件句柄。

L93

```
    Elf64_Ehdr ehdr;
```

为当前ELF文件头分配一个局部结构变量；包含入口地址、程序头数量等。

L94

```
    Elf64_Phdr phdr;
```

为当前程序头分配局部结构，循环中反复覆盖。

L96–98

```
    /* open the image file */
    img = fopen(IMAGE_FILE, "w");
    assert(img != NULL);
```

以“w”打开image，会创建或截断旧文件，然后断言成功。Linux上文本/二进制模式没有区别；失败时程序会中止，旧镜像可能已经被截断。

L100–101

```
    /* for each input file */
    for (int fidx = 0; fidx < nfiles; ++fidx) {
```

逐个处理输入文件，fidx从0到nfiles-1；顺序与命令行一致。

L103

```
        int taskidx = fidx - 2;
```

将文件序号减2转换成应用序号。前两个会得到负数，但只有fidx≥2时才真正索引taskinfo。

L104

```
        int task_start=phyaddr;
```

记下当前文件开始写入镜像时的偏移。它供后续应用记录起点，不随这个文件内部的段循环而变化。

L105–107

```
        /* open input file */
        fp = fopen(*files, "r");
        assert(fp != NULL);
```

打开当前文件名*files并断言成功；这里files是指向字符串指针的指针，*files才是当前路径字符串。

L109–110

```
        /* read ELF header */
        read_ehdr(&ehdr, fp);
```

读取当前ELF头；若读不到完整结构或E/L/F检查失败就由断言终止。

L111

```
        printf("0x%04lx: %s\n", ehdr.e_entry, *files);
```

输出入口地址和文件名，例如0x50201000: main。它是打包日志，不是目标OS运行输出。

L113–114

```
        /* for each program header */
        for (int ph = 0; ph < ehdr.e_phnum; ph++) {
```

根据ELF头的e_phnum逐个遍历程序头。ELF节表与程序头表不同，运行时主要按PT_LOAD程序段加载。

L116–117

```
            /* read program header */
            read_phdr(&phdr, fp, ph, ehdr);
```

把第ph个程序头读到phdr；extended模式还会打印该段文件偏移和内存地址等。

L119

```
            if (phdr.p_type != PT_LOAD) continue;
```

不是PT_LOAD的段直接continue到下一轮，例如某些元信息不应作为运行数据拷进镜像。

L121–122

```
            /* write segment to the image */
            write_segment(phdr, fp, img, &phyaddr);
```

复制这个可加载段的文件内容，并通过&phyaddr让函数累计已写字节。这里没有按段虚拟地址创建空洞。

L123

```
            if(fidx>=2){
```

只有第2个及之后的文件属于应用，才为其生成任务表项。

L124

```
                strncpy(taskinfo[taskidx].task_name,*files,TASK_NAME_LEN-1);
```

把当前输入文件名最多复制31字节；若名称更长将被截断。因此运行时应输入记录下来的名字，且长名称可能碰撞。

L125

```
                taskinfo[taskidx].task_name[TASK_NAME_LEN-1]='\0';
```

强制数组最后一字节为0，保证即使源名太长也能作为C字符串终止。

L126

```
                taskinfo[taskidx].offset=task_start;
```

把104行保存的文件镜像起点填进offset字段。

L127

```
                taskinfo[taskidx].size=phyaddr-task_start;
```

用当前累计偏移减去文件起点，得到目前已经为该应用写出的字节数；若有多个PT_LOAD，最后一次更新成为累计总长。

L128–129

```
                taskinfo[taskidx].entrypoint=get_entrypoint(ehdr);
            }
```

从ELF头取入口地址并保存；遇到后面的可加载段会再写同一个值。关闭“当前文件是应用”的条件块。

L131–134

```
            /* update nbytes_kernel */
            if (fidx==1) {
                nbytes_kernel += get_filesz(phdr);
            }
```

若fidx等于1，说明当前文件是内核，把这个段的p_filesz加入内核字节总数；.bss不在p_filesz中。

L135

```
        }
```

结束当前ELF的程序段循环，接着处理文件级填充。

L137–143

```
        /* write padding bytes */
        /**
         * TODO:
         * 1. [p1-task3] do padding so that the kernel and every app program
         *  occupies the same number of sectors
         * 2. [p1-task4] only padding bootblock is allowed!
         */
```

课程注释区分旧Task3固定扇区方案与当前Task4紧凑方案；此处注释本身不执行，实际行为由144—146行决定。

L144

```
        if (strcmp(*files, "bootblock") == 0) {
```

只有输入名字字符串恰好等于“bootblock”才填充。若传“./bootblock”或“build/bootblock”，strcmp不会相等，这是实际脆弱点。

L145–146

```
            write_padding(img, &phyaddr, SECTOR_SIZE);
        }
```

把镜像累计位置补到512字节，让内核从扇区1起始；若启动段已经超过512，函数不会缩短，也不会报告超限，可能破坏约定。

L148

```
        fclose(fp);
```

关闭本次输入文件；输出镜像仍保持打开，供后续文件继续追加。

L149–150

```
        files++;
    }
```

把文件名指针移到下一个字符串，结束文件循环。

L151

```
    write_img_info(nbytes_kernel, taskinfo, tasknum, img);
```

所有程序数据写完后追加任务表，并回填首扇区元数据；注意这是程序数据之后才做的操作。

L153–154

```
    fclose(img);
}
```

关闭输出文件，刷新标准库缓冲区并结束函数；这里没有检查fclose的失败返回值。

L156–158

```
static void read_ehdr(Elf64_Ehdr * ehdr, FILE * fp)
{
    int ret;
```

开始read_ehdr，定义ret接收读到的“结构项数”，不是读到的字节数。

L160

```
    ret = fread(ehdr, sizeof(*ehdr), 1, fp);
```

从当前位置读sizeof(*ehdr)字节的一个对象，写到调用者提供的ELF头结构中。

L161

```
    assert(ret == 1);
```

要求恰好读到1个完整头；文件过短就断言失败。

L162–165

```
    assert(ehdr->e_ident[EI_MAG1] == 'E');
    assert(ehdr->e_ident[EI_MAG2] == 'L');
    assert(ehdr->e_ident[EI_MAG3] == 'F');
}
```

分别检查ELF标识中的E、L、F，然后返回。但缺少第0字节0x7f、64位类别、字节序、机器类型等检查，因此不是完整ELF验证。

L167–170

```
static void read_phdr(Elf64_Phdr * phdr, FILE * fp, int ph,
                      Elf64_Ehdr ehdr)
{
    int ret;
```

开始read_phdr，参数ph是程序头下标；使用局部ret记录读取结果。

L172

```
    fseek(fp, ehdr.e_phoff + ph * ehdr.e_phentsize, SEEK_SET);
```

定位到程序头表起点e_phoff，加上ph×每项宽度e_phentsize。SEEK_SET表示相对文件开头，而不是相对当前位置。

L173

```
    ret = fread(phdr, sizeof(*phdr), 1, fp);
```

读入一个本机Elf64_Phdr结构；代码假定输入的布局与本机结构吻合。

L174

```
    assert(ret == 1);
```

断言读到一个完整程序头；但172行fseek的返回值并未检查。

L175–176

```
    if (options.extended == 1) {
        printf("\tsegment %d\n", ph);
```

extended开关为1时，先打印当前程序头序号。

L177–178

```
        printf("\t\toffset 0x%04lx", phdr->p_offset);
        printf("\t\tvaddr 0x%04lx\n", phdr->p_vaddr);
```

打印段在ELF文件中的offset与运行时虚拟地址vaddr；注意这些地址被展示，不代表后面的复制算法按vaddr排布。

L179–182

```
        printf("\t\tfilesz 0x%04lx", phdr->p_filesz);
        printf("\t\tmemsz 0x%04lx\n", phdr->p_memsz);
    }
}
```

打印文件占用filesz与运行内存占用memsz。二者差值通常包含.bss零区；日志结束后函数返回。

L184–187

```
static uint64_t get_entrypoint(Elf64_Ehdr ehdr)
{
    return ehdr.e_entry;
}
```

get_entrypoint直接返回ELF头中的e_entry。它不会验证入口是否真的落在可加载的可执行段内。

L189–192

```
static uint32_t get_filesz(Elf64_Phdr phdr)
{
    return phdr.p_filesz;
}
```

get_filesz直接返回p_filesz。ELF字段原本是64位，返回uint32_t会使过大数值截断；本实验小程序通常不触及这一边界。

L194–197

```
static uint32_t get_memsz(Elf64_Phdr phdr)
{
    return phdr.p_memsz;
}
```

get_memsz直接返回p_memsz，但本文件没有调用它；这个函数的存在不意味着.bss被保存到image。

L199–201

```
static void write_segment(Elf64_Phdr phdr, FILE *fp, FILE *img, int *phyaddr)
{
    if (phdr.p_memsz != 0 && phdr.p_type == PT_LOAD) {
```

开始write_segment，并且只在段的内存大小非0、类型为PT_LOAD时真正处理。phdr按值传参，后面递减其字段只改局部副本。

L202–203

```
        /* write the segment itself */
        /* NOTE: expansion of .bss should be done by kernel or runtime env! */
```

注释明确：这里只写文件中存在的数据；.bss的零初始化必须由内核入口或用户运行时负责。

L204–206

```
        if (options.extended == 1) {
            printf("\t\twriting 0x%04lx bytes\n", phdr.p_filesz);
        }
```

extended模式打印将要拷贝的p_filesz字节数，帮助对照ELF段信息。

L207

```
        fseek(fp, phdr.p_offset, SEEK_SET);
```

把输入文件读指针移动到这个段的文件偏移p_offset，从这里开始取真正的段数据。

L208

```
        while (phdr.p_filesz-- > 0) {
```

每次测试旧p_filesz是否大于0，然后减1；总共执行原p_filesz次循环。

L209

```
            fputc(fgetc(fp), img);
```

从ELF读一个字节，再把它写到image。代码没有判断fgetc是否EOF或fputc是否失败，损坏/截短输入可能产生错误镜像。

L210–213

```
            (*phyaddr)++;
        }
    }
}
```

每拷贝一字节，累计镜像位置加1；关闭循环、条件和函数。这个算法是段内容紧接相拼，不包含按p_vaddr计算的对齐间隙。

L215–216

```
static void write_padding(FILE *img, int *phyaddr, int new_phyaddr)
{
```

开始write_padding：new_phyaddr是期望的镜像绝对文件位置，phyaddr是目前累计位置。

L217–219

```
    if (options.extended == 1 && *phyaddr < new_phyaddr) {
        printf("\t\twrite 0x%04x bytes for padding\n", new_phyaddr - *phyaddr);
    }
```

若启用extended且仍有空缺，打印还要补多少字节；不会因为当前位置已经过大而报错。

L221

```
    while (*phyaddr < new_phyaddr) {
```

只要当前位置小于目标位置就循环，已经达到或超过目标则一次也不执行。

L222

```
        fputc(0, img);
```

向image写入一个零字节，用于补足启动扇区。

L223–225

```
        (*phyaddr)++;
    }
}
```

累计位置加1，直到目标位置，随后函数返回；这是文件padding，不是运行内存的.bss清零。

L227–266

```
（此处为连续注释或较长结构，请对照上方完整代码 L227–L266。）
```

这是整段被注释掉的旧write_img_info，完全不执行。关键点：227行“/*”一直到266行的“*/”才闭合；C注释不嵌套，243行又写“/*”不会开启第二层。旧块曾算kernel_sectors、存在过task数量限制并写扇区数，但这些都不能当作当前保证。266行本想写“print an error...”注释，实际结束了这整段旧代码；生效函数从267行才开始。

L267–271

```
static void write_img_info(int nbytes_kernel,
                           task_info_t *taskinfo,
                           short tasknum,
                           FILE *img)
{
```

真正有效的write_img_info定义。接收内核字节数、任务表地址、任务数以及仍打开的镜像文件，不要与上面的旧注释块混淆。

L272

```
    uint32_t app_info_offset;
```

用32位整数保存即将写出的任务表的镜像字节偏移。

L273

```
    uint16_t kernel_bytes;
```

内核大小最终只存16位，能表达0至65535字节。

L275–278

```
    if (nbytes_kernel > 0xffff)
    {
        error("kernel is too large\n");
    }
```

若内核文件数据超过0xffff=65535，输出错误并退出，避免后续强制转换静默丢失高位；没有对应检查负数、累加溢出或任务表容量。

L280

```
    kernel_bytes = (uint16_t)nbytes_kernel;
```

将已确认能装进16位的内核字节数转换成uint16_t，供写入两字节元数据。

L282–283

```
    /* App Info放在image末尾 */
    app_info_offset = (uint32_t)ftell(img);
```

ftell取得此刻文件尾的位置，即所有程序数据之后；转换成32位，作为任务表起点。没有检查ftell失败或大于4GiB的情况。

L285–288

```
    fwrite(taskinfo,
           sizeof(task_info_t),
           tasknum,
           img);
```

一次写入tasknum项task_info_t，每项sizeof结构体字节。它把C内存布局直接当磁盘格式，包括潜在填充字节；没有逐字段规定字节序，也没有检查fwrite返回值。

L290–291

```
    /* task数量 */
    fseek(img, TASK_NUM_LOC, SEEK_SET);
```

回到首扇区的0x1f6位置，准备填写任务数量；注释只是标签，fseek才改变文件指针。

L292–295

```
    fwrite(&tasknum,
           sizeof(short),
           1,
           img);
```

将tasknum的一个short写入镜像。常用主机上short为2字节，小端机器的低字节先写；启动汇编按无符号半字读取。

L297–298

```
    /* App Info偏移 */
    fseek(img, APP_INFO_OFFSET_LOC, SEEK_SET);
```

移动到首扇区0x1f8，准备填写任务表位置。

L299–302

```
    fwrite(&app_info_offset,
           sizeof(uint32_t),
           1,
           img);
```

写入一个uint32_t的app_info_offset，共4字节。内核根据这个字节偏移再算扇区及扇区内偏移。

L304–308

```
    /*
     * Task4:
     * 这里必须写kernel的字节数!
     * 不是sector数!
     */
```

注释强调Task4约定：0x1fc存的是内核字节数，不是扇区数。这与bootblock.S中“加511再右移9位”的计算配套。

L309

```
    fseek(img, OS_SIZE_LOC, SEEK_SET);
```

移动到OS_SIZE_LOC=0x1fc。

L310–313

```
    fwrite(&kernel_bytes,
           sizeof(uint16_t),
           1,
           img);
```

写一个uint16_t的kernel_bytes，共2字节；启动汇编再由字节数向上取整决定要读多少512字节扇区。

L315–316

```
    /* boot signature */
    fseek(img, BOOT_LOADER_SIG_OFFSET, SEEK_SET);
```

定位到510字节处写启动标志；它紧接在508—509的内核长度字段后。

L317

```
    fputc(BOOT_LOADER_SIG_1, img);
```

写入第一个标志字节0x55。

L318

```
    fputc(BOOT_LOADER_SIG_2, img);
```

写入第二个标志字节0xaa，位于首扇区最后一个字节。

L320–321

```
    if (options.extended)
    {
```

extended模式下进入最后的摘要日志块。

L322

```
        printf("kernel size: %d bytes\n", kernel_bytes);
```

打印内核字节总数；uint16_t经整型提升后可供这里的%d打印。

L323

```
        printf("task num: %d\n", tasknum);
```

打印应用数量，正常本工程应由Makefile列出的应用文件数决定。

L324–326

```
        printf("app info offset: %u\n", app_info_offset);
    }
}
```

打印任务表在镜像中的偏移，随后结束日志块和函数。文件指针此时留在首扇区，但之后直接关闭，不再追加数据。

L327–329

```
static void error(char *fmt, ...)
{
    va_list args;
```

开始统一错误函数；省略号表示可以接收任意额外参数，va_list用于访问它们。

L331

```
    va_start(args, fmt);
```

用最后一个固定参数fmt初始化可变参数遍历器。

L332

```
    vfprintf(stderr, fmt, args);
```

把格式串及额外参数打印到标准错误流stderr，让失败信息与普通打包日志分开。

L333

```
    va_end(args);
```

结束对可变参数的访问，与va_start配对。

L334–336

```
    if (errno != 0) {
        perror(NULL);
    }
```

如果errno非零，再打印标准库错误描述。errno可能是较早操作留下的值，因此不能把此附加信息绝对当作当前故障原因。

L337–338

```
    exit(EXIT_FAILURE);
}
```

用EXIT_FAILURE退出整个开发机进程；调用error的地方不会继续执行后续打包步骤。

3 你能看到什么，什么代码保证它

在build目录执行制作命令后应出现image，并在终端看到各ELF入口和扩展段日志。

代码推导，未在课程平台实测

tools/createimage.c 97行创建./image，101—150行逐文件处理，111行打印入口；175—180与204—205行在--extended时输出详情。Makefile 223—225行先cd build再执行。

首扇区末尾应能读出任务数、任务表偏移、内核字节数和55 aa；应用长度不必是512的整数倍。

代码推导，未在课程平台实测

常量位于tools/createimage.c 15—20行；144—146行只补bootblock到512，285—318行追加任务表并回填元数据。bootblock.S 34—38行读取内核字节数并向上取整。

task表应记录精确名字及其在image内的位置，输入名字与记录不符时加载器找不到。

代码推导，未在课程平台实测

tools/createimage.c 124—128行保存名/偏移/大小/入口；kernel/loader/loader.c 42—47行以strcmp==0查找，而不是模糊匹配。

过多应用、异常ELF布局或错误文件名可能使工具崩溃或镜像不能启动，不能把“生成image”当作完整正确性证明。

代码推导，未在课程平台实测

taskinfo只有16项（32—33行），89与123—128行没有有效数量检查；119—122和207—210行把段紧接拼接，144行只识别精确bootblock名字；旧限制在227—266行的注释里不生效。

容易误解或需要修正的地方

- 不要把开发机执行的createimage日志误认为QEMU内核打印；它们发生在不同机器角色、不同阶段。
- 代码仅局部识别ELF；也没有检查fseek/fwrite/fclose返回值，不能对损坏输入或磁盘写满做可靠处理。
- 任务表是本机结构体原样写盘；在主机布局、目标布局或大小端不同的环境下并不具备可移植性。
- 按段拼接不是通用ELF加载：段虚拟地址间隙、.bss空间和入口位置偏移都可能需要更完整的布局处理。当前加载器直接把整个任务数据拷到entrypoint，也依赖入口即数据起点的约定。
- 首扇区502—511字节属于元数据，代码未检验bootblock实际内容是否与其重叠；写入元数据可能覆盖过长启动程序。

03 · 镜像与构建

12 Makefile：谁编译谁、生成什么、如何启动与调试

原路径：Makefile · 370 行

> 由开发电脑上的make解释执行的构建规则，不是RISC-V程序，也不会装入内核执行。

把Makefile理解成“菜谱和依赖关系”：目标后面的冒号表示它需要哪些输入；以Tab开头的命令才交给Shell执行。大多数“=”是make变量定义，不是C语言赋值。$(名称)由make替换，$@是当前目标，$<是首个依赖，$^是全部依赖。

默认make使用第一个普通目标all，因此会制作bootblock、内核、四个应用、image和反汇编。Prelab是独立目标，不属于all。所有./路径相对于启动make时的工程目录，应先cd到解压得到的zhangxiyuan24目录。

本文件默认采用课程自备QEMU/U-Boot以及0x50200000等地址。云环境的riscv64-linux-gnu-前缀可用命令行覆盖，但仅换编译器或QEMU路径不等于移植了BIOS、内存映射和SD接口。下面解释原样规则；不会运行floppy写物理磁盘。

1 完整代码，先看全貌

```
# ------------------------------------------------------------------------
# Project Information
# ------------------------------------------------------------------------

PROJECT_IDX     = 1


# ------------------------------------------------------------------------
# Host Linux Variables
# ------------------------------------------------------------------------

SHELL           = /bin/sh
DISK            = /dev/sdb
TTYUSB1         = /dev/ttyUSB1

DIR_OSLAB       = $(HOME)/OSLab-RISC-V
DIR_QEMU        = $(DIR_OSLAB)/qemu
DIR_UBOOT       = $(DIR_OSLAB)/u-boot


# ------------------------------------------------------------------------
# Build and Debug Tools
# ------------------------------------------------------------------------

HOST_CC         = gcc

CROSS_PREFIX    = riscv64-unknown-linux-gnu-

```

```
CC              = $(CROSS_PREFIX)gcc
OBJDUMP         = $(CROSS_PREFIX)objdump
OBJCOPY         = $(CROSS_PREFIX)objcopy
READELF         = $(CROSS_PREFIX)readelf
GDB             = $(CROSS_PREFIX)gdb

QEMU            = $(DIR_QEMU)/riscv64-softmmu/qemu-system-riscv64
UBOOT           = $(DIR_UBOOT)/u-boot
MINICOM         = minicom


# ------------------------------------------------------------------------
# Build/Debug Flags and Variables
# ------------------------------------------------------------------------

CFLAGS          = -O2 -std=gnu11 -fno-builtin -nostdlib -nostdinc \
                  -Wall -mcmodel=medany -ggdb3

BOOT_INCLUDE    = -I$(DIR_ARCH)/include

BOOT_CFLAGS     = $(CFLAGS) $(BOOT_INCLUDE) \
                  -Wl,--defsym=TEXT_START=$(BOOTLOADER_ENTRYPOINT) \
                  -T riscv.lds

KERNEL_INCLUDE  = -I$(DIR_ARCH)/include -Iinclude

KERNEL_CFLAGS   = $(CFLAGS) $(KERNEL_INCLUDE) \
                  -Wl,--defsym=TEXT_START=$(KERNEL_ENTRYPOINT) \
                  -T riscv.lds

USER_INCLUDE    = -I$(DIR_TINYLIBC)/include
USER_CFLAGS     = $(CFLAGS) $(USER_INCLUDE)


QEMU_OPTS       = -nographic \
                  -machine virt \
                  -m 256M \
                  -kernel $(UBOOT) \
                  -bios none \
                  -drive if=none,format=raw,id=image,file=$(ELF_IMAGE) \
                  -device virtio-blk-device,drive=image \
                  -monitor telnet::45454,server,nowait \
                  -serial mon:stdio

QEMU_DEBUG_OPT  = -s -S


# ------------------------------------------------------------------------
# UCAS-OS Entrypoints and Variables
# ------------------------------------------------------------------------

DIR_ARCH        = ./arch/riscv
DIR_BUILD       = ./build
DIR_INIT        = ./init
DIR_KERNEL      = ./kernel
DIR_LIBS        = ./libs
DIR_TINYLIBC    = ./tiny_libc
DIR_TEST        = ./test
DIR_TEST_PROJ   = $(DIR_TEST)/test_project$(PROJECT_IDX)
```

```
88
89  BOOTLOADER_ENTRYPOINT   = 0x50200000
90  KERNEL_ENTRYPOINT       = 0x50201000
91  USER_ENTRYPOINT         = 0x52000000
92
93
94  # ------------------------------------------------------------------------
95  # UCAS-OS Kernel Source Files
96  # ------------------------------------------------------------------------
97
98  SRC_BOOT        = $(wildcard $(DIR_ARCH)/boot/*.S)
99  SRC_ARCH        = $(wildcard $(DIR_ARCH)/kernel/*.S)
100 SRC_BIOS        = $(wildcard $(DIR_ARCH)/bios/*.c)
101 SRC_INIT        = $(wildcard $(DIR_INIT)/*.c)
102 SRC_KERNEL      = $(wildcard $(DIR_KERNEL)/*/*.c)
103 SRC_LIBS        = $(wildcard $(DIR_LIBS)/*.c)
104
105 SRC_MAIN        = $(SRC_ARCH) $(SRC_INIT) $(SRC_BIOS) \
106                   $(SRC_KERNEL) $(SRC_LIBS)
107
108 ELF_BOOT        = $(DIR_BUILD)/bootblock
109 ELF_MAIN        = $(DIR_BUILD)/main
110 ELF_IMAGE       = $(DIR_BUILD)/image
111
112
113 # ------------------------------------------------------------------------
114 # UCAS-OS User Source Files
115 # ------------------------------------------------------------------------
116
117 SRC_CRT0        = $(wildcard $(DIR_ARCH)/crt0/*.S)
118 OBJ_CRT0        = $(DIR_BUILD)/$(notdir $(SRC_CRT0:.S=.o))
119
120 SRC_USER        = $(wildcard $(DIR_TEST_PROJ)/*.c)
121
122 ELF_USER        = $(patsubst %.c, %, \
123                   $(foreach file,$(SRC_USER),$(DIR_BUILD)/$(notdir $(file))))
124
125
126 # ------------------------------------------------------------------------
127 # Host Linux Tools Source Files
128 # ------------------------------------------------------------------------
129
130 SRC_CREATEIMAGE = ./tools/createimage.c
131 ELF_CREATEIMAGE = $(DIR_BUILD)/$(notdir $(SRC_CREATEIMAGE:.c=))
132
133
134 # ------------------------------------------------------------------------
135 # Top-level Rules
136 # ------------------------------------------------------------------------
137
138 all: dirs elf image asm
139
140
141 dirs:
142     @mkdir -p $(DIR_BUILD)
143
144
145 clean:
146     rm -rf $(DIR_BUILD)
```

```
147
148
149 floppy:
150     sudo fdisk -l $(DISK)
151     sudo dd if=$(DIR_BUILD)/image of=$(DISK)3 conv=notrunc
152
153
154 asm: $(ELF_BOOT) $(ELF_MAIN) $(ELF_USER)
155     for elffile in $^; do \
156         $(OBJDUMP) -d $$elffile > $(notdir $$elffile).txt; \
157     done
158
159
160 gdb:
161     $(GDB) $(ELF_MAIN) -ex "target remote :1234"
162
163
164 run:
165     $(QEMU) $(QEMU_OPTS)
166
167
168 debug:
169     $(QEMU) $(QEMU_OPTS) $(QEMU_DEBUG_OPT)
170
171
172 minicom:
173     sudo $(MINICOM) -D $(TTYUSB1)
174
175
176 .PHONY: all dirs clean floppy asm gdb run debug minicom
177
178
179 # ------------------------------------------------------------------------
180 # UCAS-OS Rules
181 # ------------------------------------------------------------------------
182
183 $(ELF_BOOT): $(SRC_BOOT) riscv.lds
184     $(CC) $(BOOT_CFLAGS) -o $@ $(SRC_BOOT) -e main
185
186
187 $(ELF_MAIN): $(SRC_MAIN) riscv.lds
188     $(CC) $(KERNEL_CFLAGS) -o $@ $(SRC_MAIN)
189
190
191 $(OBJ_CRT0): $(SRC_CRT0)
192     $(CC) $(USER_CFLAGS) \
193         -I$(DIR_ARCH)/include \
194         -c $< \
195         -o $@
196
197
198 $(DIR_BUILD)/%: $(DIR_TEST_PROJ)/%.c $(OBJ_CRT0) riscv.lds
199     $(CC) $(USER_CFLAGS) \
200         -o $@ \
201         $(OBJ_CRT0) \
202         $< \
203         -Wl,--defsym=TEXT_START=$(USER_ENTRYPOINT) \
204         -T riscv.lds
205     $(eval USER_ENTRYPOINT := $(shell python3 -c \
```

```
206         "print(hex(int('$(USER_ENTRYPOINT)',16)+0x10000))"))
207
208
209 elf: $(ELF_BOOT) $(ELF_MAIN) $(ELF_USER)
210
211
212 .PHONY: elf
213
214
215 # ---------------------------------------------------------------------------
216 # Host Linux Rules
217 # ---------------------------------------------------------------------------
218
219 $(ELF_CREATEIMAGE): $(SRC_CREATEIMAGE)
220     $(HOST_CC) $(SRC_CREATEIMAGE) -o $@ -ggdb -Wall
221
222
223 image: $(ELF_CREATEIMAGE) $(ELF_BOOT) $(ELF_MAIN) $(ELF_USER)
224     cd $(DIR_BUILD) && \
225     ./$(<F) --extended $(filter-out $(<F),$(^F))
226
227
228 .PHONY: image
229
230
231 # ===========================================================================
232 # PRELAB
233 # ===========================================================================
234
235 # 汇编源程序
236 PRELAB_SRC       = ./test.S
237
238 # 链接脚本
239 PRELAB_LDS       = ./prelab.lds
240
241 # ELF 文件：给 objdump / GDB 使用
242 PRELAB_ELF       = ./zhangxiyuan24
243
244 # 裸二进制：真正加载到 QEMU RAM
245 PRELAB_BIN       = ./zhangxiyuan24.bin
246
247 # 反汇编输出
248 PRELAB_ASM       = ./zhangxiyuan24.txt
249
250 # 程序在 QEMU RAM 中的地址
251 PRELAB_ADDR      = 0x50200000
252
253
254 # ---------------------------------------------------------------------------
255 # 1. test.S -> ELF
256 # ---------------------------------------------------------------------------
257
258 $(PRELAB_ELF): $(PRELAB_SRC) $(PRELAB_LDS)
259     $(CC) \
260         -g \
261         -static \
262         -no-pie \
263         -nostdlib \
264         -nostartfiles \
```

```
265         -Wl,--build-id=none \
266         -Wl,-T,$(PRELAB_LDS) \
267         -Wl,-e,main \
268         -o $@ \
269         $(PRELAB_SRC)
270
271
272 # ------------------------------------------------------------------------
273 # 2. ELF -> RAW binary
274 # ------------------------------------------------------------------------
275
276 $(PRELAB_BIN): $(PRELAB_ELF)
277     $(OBJCOPY) -O binary $< $@
278
279
280 # ------------------------------------------------------------------------
281 # 3. 编译整个 Prelab
282 # ------------------------------------------------------------------------
283
284 prelab: $(PRELAB_ELF) $(PRELAB_BIN)
285
286
287 # ------------------------------------------------------------------------
288 # 4. 反汇编
289 # ------------------------------------------------------------------------
290
291 prelab-asm: $(PRELAB_ELF)
292     $(OBJDUMP) -d $(PRELAB_ELF) > $(PRELAB_ASM)
293
294
295 # ------------------------------------------------------------------------
296 # 5. 查看 ELF 信息
297 # ------------------------------------------------------------------------
298
299 prelab-info: $(PRELAB_ELF)
300     @echo "========== ELF HEADER =========="
301     $(READELF) -h $(PRELAB_ELF)
302     @echo
303     @echo "========== PROGRAM HEADER =========="
304     $(READELF) -l $(PRELAB_ELF)
305     @echo
306     @echo "========== MAIN SYMBOL =========="
307     $(CROSS_PREFIX)nm -n $(PRELAB_ELF) | grep main
308
309
310 # ------------------------------------------------------------------------
311 # 6. 正常运行 Prelab
312 #
313 # binary 被明确放到 0x80200000
314 # cpu-num=0 让 CPU0 从该加载地址开始执行
315 # ------------------------------------------------------------------------
316
317 prelab-run: prelab
318     $(QEMU) \
319         -machine virt \
320         -m 256M \
321         -nographic \
322         -bios none \
323         -device loader,file=$(abspath $(PRELAB_BIN)),addr=$(PRELAB_ADDR),cpu-num=0
```

```
324
325
326 # ------------------------------------------------------------------------
327 # 7. QEMU GDB 调试模式
328 #
329 # -s = 在 TCP 1234 开 GDB server
330 # -S = CPU 启动后立即暂停
331 # ------------------------------------------------------------------------
332
333 prelab-debug: prelab
334     $(QEMU) \
335         -machine virt \
336         -m 256M \
337         -nographic \
338         -bios none \
339         -device loader,file=$(abspath $(PRELAB_BIN)),addr=$(PRELAB_ADDR),cpu-num=0 \
340         -s \
341         -S
342
343
344 # ------------------------------------------------------------------------
345 # 8. GDB 连接 QEMU
346 # ------------------------------------------------------------------------
347
348 prelab-gdb: $(PRELAB_ELF)
349     $(GDB) $(PRELAB_ELF) \
350         -ex "target remote :1234"
351
352
353 # ------------------------------------------------------------------------
354 # 9. 清理 Prelab
355 # ------------------------------------------------------------------------
356
357 prelab-clean:
358     rm -f \
359         $(PRELAB_ELF) \
360         $(PRELAB_BIN) \
361         $(PRELAB_ASM)
362
363
364 .PHONY: prelab \
365         prelab-asm \
366         prelab-info \
367         prelab-run \
368         prelab-debug \
369         prelab-gdb \
370         prelab-clean
```

2 按原始行号，一句一句理解

L1–3

```
# ------------------------------------------------------------------------
# Project Information
# ------------------------------------------------------------------------
```

纯注释分隔标题，提示下面是项目编号配置；井号在Makefile里通常使这一行成为注释。

L5

```
PROJECT_IDX     = 1
```

选择Project1，所以后面DIR_TEST_PROJ会展开为./test/test_project1；这决定自动发现哪一组C应用。

L8–10

```
# ------------------------------------------------------------------------
# Host Linux Variables
# ------------------------------------------------------------------------
```

下面一组是开发机上的Shell、硬件设备与课程目录，不是目标机内核变量。

L12

```
SHELL           = /bin/sh
```

规定配方交给/bin/sh执行；这里的变量名SHELL不是你在内核提示符中输入的命令。

L13

```
DISK            = /dev/sdb
```

把DISK设置为开发机/dev/sdb，供floppy规则使用。它是真实块设备路径，不能当成普通输出文件。

L14

```
TTYUSB1         = /dev/ttyUSB1
```

设置USB串口设备路径，minicom会用它和板端通信。

L16

```
DIR_OSLAB       = $(HOME)/OSLab-RISC-V
```

课程工具根目录基于开发机的HOME目录，例如用户家目录下OSLab-RISC-V。HOME来自运行make的环境。

L17

```
DIR_QEMU        = $(DIR_OSLAB)/qemu
```

根据上一个变量拼出课程QEMU源码/构建目录。

L18

```
DIR_UBOOT       = $(DIR_OSLAB)/u-boot
```

拼出课程U-Boot所在目录；它是目标机引导固件，不是本工程生成的bootblock。

L21–23

```
# ------------------------------------------------------------------------
# Build and Debug Tools
# ------------------------------------------------------------------------
```

注释标题：下面开始定义各类编译、反汇编和调试工具名称。

L25

```
HOST_CC         = gcc
```

HOST_CC=gcc用于构建在开发机运行的createimage；它不能编译RISC-V内核机器码。

L27

```
CROSS_PREFIX    = riscv64-unknown-linux-gnu-
```

定义交叉工具前缀riscv64-unknown-linux-gnu-。云环境现有工具前缀不同，可命令行传CROSS_PREFIX=riscv64-linux-gnu-，无需改源码。

L29

```
CC              = $(CROSS_PREFIX)gcc
```

CC是前缀加gcc，编译RISC-V汇编/C与链接目标程序；make中的变量名与Shell环境中的CC不是同一层展开。

L30

```
OBJDUMP         = $(CROSS_PREFIX)objdump
```

OBJDUMP拼成目标反汇编器，把目标机器码翻成可读指令。

L31

```
OBJCOPY         = $(CROSS_PREFIX)objcopy
```

OBJCOPY用于格式转换，Prelab用它从ELF导出裸二进制。

L32

```
READELF         = $(CROSS_PREFIX)readelf
```

READELF查看ELF文件头和程序头等元信息；它不会运行程序。

L33

```
GDB             = $(CROSS_PREFIX)gdb
```

GDB默认选择同前缀调试器。云环境可另传GDB=gdb-multiarch；仅覆盖CROSS_PREFIX不保证存在同名GDB。

L35

```
QEMU            = $(DIR_QEMU)/riscv64-softmmu/qemu-system-riscv64
```

指定课程QEMU的构建产物路径，默认并非系统PATH里的通用qemu-system-riscv64。

L36

```
UBOOT           = $(DIR_UBOOT)/u-boot
```

指定U-Boot文件本身的位置，将由QEMU加载。

L37

```
MINICOM         = minicom
```

设置串口通信程序名minicom，供真实板端或USB串口工作流使用。

L40–42

```
# ---------------------------------------------------------------------
# Build/Debug Flags and Variables
# ---------------------------------------------------------------------
```

注释标题：下面是编译和模拟器参数集合。

L44

```
CFLAGS          = -O2 -std=gnu11 -fno-builtin -nostdlib -nostdinc \
```

CFLAGS第一行：-O2优化；-std=gnu11采用GNU C11；-fno-builtin不把库名自动当编译器内建；-nostdlib不自动链接标准库及启动文件；-nostdinc不搜索标准头文件目录。末尾反斜杠表示变量定义下一行继续。

L45

```
                  -Wall -mcmodel=medany -ggdb3
```

-Wall打开常见警告；-mcmodel=medany使用适合本工程地址模型的PC相对寻址；-ggdb3保留丰富GDB信息。未显式关闭所有工具链可能默认启用的PIE、栈保护或小数据寻址，换工具链后要核对实际ELF。

L47

```
BOOT_INCLUDE    = -I$(DIR_ARCH)/include
```

启动程序只加入arch/riscv/include目录，编译器才能找到asm/biosdef.h。

L49

```
BOOT_CFLAGS     = $(CFLAGS) $(BOOT_INCLUDE) \
```

BOOT_CFLAGS先复用CFLAGS和启动头文件搜索路径。

L50

```
                  -Wl,--defsym=TEXT_START=$(BOOTLOADER_ENTRYPOINT) \
```

-Wl,表示把后面的选项交给链接器；--defsym把TEXT_START定义为启动程序地址，供riscv.lds第7行使用。

L51

```
                  -T riscv.lds
```

-T指定自定义链接脚本riscv.lds，决定代码/数据等段在内存中的位置。

L53

```
KERNEL_INCLUDE  = -I$(DIR_ARCH)/include -Iinclude
```

内核头文件搜索两处：体系结构头和工程通用include；这不等于使用Linux系统头。

L55

```
KERNEL_CFLAGS   = $(CFLAGS) $(KERNEL_INCLUDE) \
```

KERNEL_CFLAGS先复用公共编译选项和内核搜索路径。

L56

```
                  -Wl,--defsym=TEXT_START=$(KERNEL_ENTRYPOINT) \
```

为内核链接定义TEXT_START=KERNEL_ENTRYPOINT，默认0x50201000。

L57

```
                  -T riscv.lds
```

内核也使用同一份riscv.lds，只是传入的起始地址不同。

L59

```
USER_INCLUDE    = -I$(DIR_TINYLIBC)/include
```

用户程序头搜索目录为tiny_libc/include；其中内联跳转表调用提供最简输入输出接口。

L60

```
USER_CFLAGS     = $(CFLAGS) $(USER_INCLUDE)
```

USER_CFLAGS组合公共参数和用户头目录；具体应用链接地址在模式规则中追加。

L63

```
QEMU_OPTS       = -nographic \
```

QEMU_OPTS第一项-nographic把图形显示关闭并在终端提供串口等交互。反斜杠让后续多行仍属于同一个变量。

L64

```
                  -machine virt \
```

-machine virt选择RISC-V virt虚拟开发板；仍需与课程版QEMU/U-Boot所采用的地址映射配套。

L65

```
                  -m 256M \
```

-m 256M分配256MiB模拟RAM，不是给每个应用单独分配这么多内存。

L66

```
                  -kernel $(UBOOT) \
```

-kernel $(UBOOT)在这里让QEMU装入U-Boot文件；名字叫kernel选项，但实际传入的是引导程序，不是build/main。

L67

```
                  -bios none \
```

-bios none禁用默认BIOS固件；所以系统不会自动获得本工程需要的0x50150000课程BIOS接口。

L68

```
                  -drive if=none,format=raw,id=image,file=$(ELF_IMAGE) \
```

把build/image作为raw格式磁盘数据源，id=image只是给QEMU内部引用的标识，并不是OS任务名。

L69

```
                  -device virtio-blk-device,drive=image \
```

建立virtio块设备并连接上一行的image驱动，课程引导环境据此读取镜像。

L70

```
                  -monitor telnet::45454,server,nowait \
```

将QEMU monitor暴露为TCP 45454的telnet服务，server表示监听、nowait表示不等待连接再启动；它不是被测OS的Shell。

L71

```
                  -serial mon:stdio
```

将串口与monitor复用到当前终端。QEMU里的Ctrl+A等组合键归模拟器处理，不是内核代码执行的按键。

L73

```
QEMU_DEBUG_OPT  = -s -S
```

-s创建默认TCP1234的GDB stub，-S让CPU一启动就暂停；调试模式需要GDB发continue才运行。

L76–78

```
# ------------------------------------------------------------------------
# UCAS-OS Entrypoints and Variables
# ------------------------------------------------------------------------
```

注释标题：下面配置工程目录和各目标运行地址。

L80

```
DIR_ARCH        = ./arch/riscv
```

体系结构目录设为./arch/riscv，汇编、BIOS与相关头文件都从这里找。

L81

```
DIR_BUILD       = ./build
```

生成目录为./build；编译前需要它已经存在。

L82

```
DIR_INIT        = ./init
```

内核C入口源码目录为./init。

L83

```
DIR_KERNEL      = ./kernel
```

内核其他模块目录为./kernel，例如loader子目录。

L84

```
DIR_LIBS        = ./libs
```

通用内核库目录为./libs，例如自己实现的string.c。

L85

```
DIR_TINYLIBC    = ./tiny_libc
```

最小用户接口目录为./tiny_libc。

L86

```
DIR_TEST        = ./test
```

测试目录为./test。

L87

```
DIR_TEST_PROJ   = $(DIR_TEST)/test_project$(PROJECT_IDX)
```

把测试根目录和项目编号拼成./test/test_project1，随后wildcard读取其中所有.c文件。

L89

```
BOOTLOADER_ENTRYPOINT   = 0x50200000
```

启动程序链接入口基址是0x50200000；这是课程地址约定，不是通用QEMU virt默认RAM地址。

L90

```
KERNEL_ENTRYPOINT       = 0x50201000
```

内核链接基址是0x50201000，与bootblock.S里跳转的kernel常量一致。

L91

```
USER_ENTRYPOINT         = 0x52000000
```

第一个用户应用链接基址0x52000000。后面每链接一个应用会把此make变量递增0x10000，即64KiB。

L94–96

```
# ------------------------------------------------------------------------
# UCAS-OS Kernel Source Files
# ------------------------------------------------------------------------
```

注释标题：接下来自动搜集内核与启动代码。

L98

```
SRC_BOOT        = $(wildcard $(DIR_ARCH)/boot/*.S)
```

wildcard匹配arch/riscv/boot目录所有.S文件，得到启动汇编列表。

L99

```
SRC_ARCH        = $(wildcard $(DIR_ARCH)/kernel/*.S)
```

搜集arch/riscv/kernel目录的.S文件，例如清.bss和设置栈的head.S。

L100

```
SRC_BIOS        = $(wildcard $(DIR_ARCH)/bios/*.c)
```

搜集体系结构BIOS封装C文件。

L101

```
SRC_INIT        = $(wildcard $(DIR_INIT)/*.c)
```

搜集init目录的C文件，例如init/main.c。

L102

```
SRC_KERNEL      = $(wildcard $(DIR_KERNEL)/*/*.c)
```

匹配kernel下一层子目录的C文件，如kernel/loader/loader.c；这是一个固定深度的通配模式，并非递归搜索任意深度。

L103

```
SRC_LIBS        = $(wildcard $(DIR_LIBS)/*.c)
```

搜集libs目录的C文件，当前主要是字符串/内存操作实现。

L105–106

```
SRC_MAIN        = $(SRC_ARCH) $(SRC_INIT) $(SRC_BIOS) \
                  $(SRC_KERNEL) $(SRC_LIBS)
```

把上述列表连接成内核链接输入SRC_MAIN；顺序本身不等于执行顺序，真正入口由链接脚本与入口汇编决定。

L108

```
ELF_BOOT        = $(DIR_BUILD)/bootblock
```

启动ELF目标路径为build/bootblock。

L109

```
ELF_MAIN        = $(DIR_BUILD)/main
```

内核ELF路径为build/main；它的main文件名与C函数main要分开理解。

L110

```
ELF_IMAGE       = $(DIR_BUILD)/image
```

最终裸磁盘镜像输出路径为build/image，不是ELF。

L113–115

```
# ------------------------------------------------------------------------
# UCAS-OS User Source Files
# ------------------------------------------------------------------------
```

注释标题：下面是用户应用及其启动代码。

L117

```
SRC_CRT0        = $(wildcard $(DIR_ARCH)/crt0/*.S)
```

匹配arch/riscv/crt0下的.S文件，当前是用户入口crt0.S。

L118

```
OBJ_CRT0        = $(DIR_BUILD)/$(notdir $(SRC_CRT0:.S=.o))
```

先把.S后缀替换成.o，再notdir去掉目录，最后加build前缀，得到build/crt0.o。

L120

```
SRC_USER        = $(wildcard $(DIR_TEST_PROJ)/*.c)
```

匹配Project1测试目录里的所有C文件，当前为2048、auipc、bss、data四个应用。

L122–123

```
ELF_USER        = $(patsubst %.c, %, \
                  $(foreach file,$(SRC_USER),$(DIR_BUILD)/$(notdir $(file))))
```

foreach遍历C路径，notdir取文件名，加build前缀，再patsubst去掉.c扩展名，得到待链接的应用ELF列表。

L126–128

```
# ------------------------------------------------------------------------
# Host Linux Tools Source Files
# ------------------------------------------------------------------------
```

注释标题：下一组是开发机工具源代码。

L130

```
SRC_CREATEIMAGE = ./tools/createimage.c
```

指定createimage源文件位置。它用主机gcc编译，而非交叉gcc。

L131

```
ELF_CREATEIMAGE = $(DIR_BUILD)/$(notdir $(SRC_CREATEIMAGE:.c=))
```

去掉路径及.c后缀，再加build前缀，得到build/createimage。

L134–136

```
# ------------------------------------------------------------------------
# Top-level Rules
# ------------------------------------------------------------------------
```

注释标题：下面开始描述目标及配方。

L138

```
all: dirs elf image asm
```

all依赖dirs、elf、image、asm。它是首个普通目标，因此裸make通常等于make all。冒号依赖不是严格先后屏障，使用-j时兄弟依赖可能并发，dirs未被声明为各编译目标的顺序依赖。

L141–142

```
dirs:
    @mkdir -p $(DIR_BUILD)
```

dirs调用mkdir -p创建build，已有目录不会报错。@使make不显示这条命令本身，但错误仍会显示。

L145–146

```
clean:
    rm -rf $(DIR_BUILD)
```

clean 删除整个 build 目录，包括其中 ELF、image 以及 asm 规则实际生成的 build/*.txt。根目录的 Prelab ELF/bin/反汇编由另一个 prelab-clean 目标处理；clean 不会删除源文件。

L149

```
floppy:
```

floppy是把镜像写入真实磁盘分区的目标名，不是构建普通文件。讲解中不要执行它。

L150

```
    sudo fdisk -l $(DISK)
```

sudo fdisk -l列出DISK的分区，默认检查/dev/sdb；它依赖真实设备和管理员权限。

L151

```
    sudo dd if=$(DIR_BUILD)/image of=$(DISK)3 conv=notrunc
```

sudo dd把build/image原始字节写到$(DISK)3，默认/dev/sdb3；conv=notrunc不把目标截短。它可能覆盖磁盘数据，不能在云环境或未知磁盘上当作例行验证命令运行。

L154

```
asm: $(ELF_BOOT) $(ELF_MAIN) $(ELF_USER)
```

asm依赖启动、内核和应用ELF，这些输入都更新后才可以做反汇编。

L155

```
    for elffile in $^; do \
```

Shell循环遍历$^展开出的所有ELF依赖。行末反斜杠将多行配方合成同一个Shell命令。

L156

```
        $(OBJDUMP) -d $$elffile > $(notdir $$elffile).txt; \
```

对 Shell 变量 elffile 指向的 ELF 执行 objdump -d。要留意两层展开：make 处理 $(notdir $$elffile) 时还不知道 Shell 变量的值，所以实际命令是 $elffile.txt，而非已经剥去目录的文件名。此处 elffile=build/bootblock 时，输出为 build/bootblock.txt；$$ 让 make 保留一个美元符号交给 Shell 展开。

L157

```
    done
```

done结束Shell的for循环；这不是C语言循环。

L160–161

```
gdb:
    $(GDB) $(ELF_MAIN) -ex "target remote :1234"
```

gdb目标启动调试器、读入内核ELF符号并连接本机TCP1234；它本身不启动QEMU，需要另一个终端先make debug。

L164–165

```
run:
    $(QEMU) $(QEMU_OPTS)
```

run把前面汇总的参数交给QEMU。此目标没有声明image依赖，因此直接make run不保证先编译或保证已有image最新。

L168–169

```
debug:
    $(QEMU) $(QEMU_OPTS) $(QEMU_DEBUG_OPT)
```

debug在正常QEMU参数后再附加-s -S，让目标CPU等待调试器连接。

L172–173

```
minicom:
    sudo $(MINICOM) -D $(TTYUSB1)
```

minicom以管理员权限打开指定USB串口设备；这是板端通信工作流，通用QEMU测试不需要它。

L176

```
.PHONY: all dirs clean floppy asm gdb run debug minicom
```

.PHONY声明这些名字是动作目标，不对应同名文件；否则若目录下刚好有名为run的文件，make可能错误地认为无需执行。

L179–181

```
# ------------------------------------------------------------------------
# UCAS-OS Rules
# ------------------------------------------------------------------------
```

注释标题：下面给出RISC-V目标文件的具体编译规则。

L183

```
$(ELF_BOOT): $(SRC_BOOT) riscv.lds
```

启动ELF依赖启动汇编列表及链接脚本；任一输入比产物新则触发重新编译。

L184

```
    $(CC) $(BOOT_CFLAGS) -o $@ $(SRC_BOOT) -e main
```

交叉gcc编译链接启动汇编，$@替换为build/bootblock；-e main覆盖链接脚本的ENTRY(_start)，以汇编main为启动入口。

L187–188

```
$(ELF_MAIN): $(SRC_MAIN) riscv.lds
    $(CC) $(KERNEL_CFLAGS) -o $@ $(SRC_MAIN)
```

内核ELF依赖SRC_MAIN和链接脚本，使用KERNEL_CFLAGS一次编译链接所有输入。入口来自riscv.lds的_start，先经过head.S再call C main。

L191

```
$(OBJ_CRT0): $(SRC_CRT0)
```

crt0目标依赖用户启动汇编；只有该文件更新才重新生成对象。这里头文件没有显式列作依赖。

L192

```
    $(CC) $(USER_CFLAGS) \
```

使用交叉gcc和用户公共编译参数开始编译crt0。

L193

```
        -I$(DIR_ARCH)/include \
```

额外加入体系结构头路径，以找到asm.h等宏定义。

L194

```
        -c $< \
```

-c表示只编译到.o，不链接；$<是首个依赖，也就是crt0.S。

L195

```
        -o $@
```

-o $@把对象写入build/crt0.o。

L198

```
$(DIR_BUILD)/%: $(DIR_TEST_PROJ)/%.c $(OBJ_CRT0) riscv.lds
```

模式规则：要生成build/某名字，就找test_project1/某名字.c、crt0.o及链接脚本；%代表共同的名字部分。

L199

```
    $(CC) $(USER_CFLAGS) \
```

开始应用的交叉编译链接命令。

L200

```
        -o $@ \
```

指定输出文件为当前目标，例如build/2048。

L201

```
        $(OBJ_CRT0) \
```

把用户启动对象crt0.o放进链接输入，负责清.bss、调用应用main并返回。

L202

```
        $< \
```

加入当前应用的C源文件，$<仍表示模式规则第一个依赖。

L203

```
        -Wl,--defsym=TEXT_START=$(USER_ENTRYPOINT) \
```

把当前USER_ENTRYPOINT传给链接器作为TEXT_START，让这个应用位于自己的64KiB槽位起点。

L204

```
        -T riscv.lds
```

应用也按riscv.lds排布入口、正文、数据与.bss。

L205–206

```
    $(eval USER_ENTRYPOINT := $(shell python3 -c \
        "print(hex(int('$(USER_ENTRYPOINT)',16)+0x10000))"))
```

eval在make展开配方时执行，shell启动Python把地址按十六进制转整数加0x10000，再写回USER_ENTRYPOINT。地址取决于规则实际展开/重建顺序；不要把某个应用永远固定到某个槽位，也不宜并行构建。增量只重建部分应用时还可能复用早先地址，此设计缺少稳定逐应用地址映射。

L209

```
elf: $(ELF_BOOT) $(ELF_MAIN) $(ELF_USER)
```

elf是汇总目标，要求启动、内核和所有应用ELF存在且更新。

L212

```
.PHONY: elf
```

将elf标为伪目标；它的目标不是生成名为elf的文件。

L215–217

```
# ------------------------------------------------------------------------
# Host Linux Rules
# ------------------------------------------------------------------------
```

注释标题：下面使用开发机编译器制作镜像工具。

L219

```
$(ELF_CREATEIMAGE): $(SRC_CREATEIMAGE)
```

build/createimage依赖tools/createimage.c，源码更新后重编译。

L220

```
    $(HOST_CC) $(SRC_CREATEIMAGE) -o $@ -ggdb -Wall
```

用HOST_CC构建开发机可执行文件，-ggdb保留调试信息，-Wall给常见警告；$@为build/createimage。

L223

```
image: $(ELF_CREATEIMAGE) $(ELF_BOOT) $(ELF_MAIN) $(ELF_USER)
```

image目标依赖工具及全部目标ELF；依赖列表顺序也是下面提供给createimage的输入顺序来源。

L224

```
    cd $(DIR_BUILD) && \
```

先cd build。&&要求切换目录成功才执行下一行；这样输出./image会落在build里，且文件名可以直接叫bootblock。

L225

```
    ./$(<F) --extended $(filter-out $(<F),$(^F))
```

$(<F)取首个依赖的无目录文件名createimage；$(^F)取全部依赖无目录文件名，filter-out剔除工具本身，最终按bootblock、main、应用顺序打包。--extended打开详细日志，也恰好满足createimage只识别精确字符串bootblock的条件。

L228

```
.PHONY: image
```

把image声明为伪目标，即使当前工作目录有名为image的文件也会执行此动作。

L231–233

```
# ============================================================================
# PRELAB
# ============================================================================
```

注释分界：后面是独立的Prelab汇编练习，不自动包含在默认all中。

L235–236

```
# 汇编源程序
PRELAB_SRC       = ./test.S
```

指定Prelab源程序test.S；prime.S虽然也在压缩包中，却不是这个规则的输入，不会被make prelab自动编译。

L238–239

```
# 链接脚本
PRELAB_LDS       = ./prelab.lds
```

指定Prelab专用链接脚本prelab.lds，与内核使用的riscv.lds不同。

L241–242

```
# ELF 文件：给 objdump / GDB 使用
PRELAB_ELF       = ./zhangxiyuan24
```

Prelab的ELF产物命名zhangxiyuan24，用于GDB符号、readelf及objdump；名字没有扩展名也仍是ELF。

L244–245

```
# 裸二进制：真正加载到 QEMU RAM
PRELAB_BIN       = ./zhangxiyuan24.bin
```

裸二进制产物命名zhangxiyuan24.bin，用于QEMU loader直接放入指定内存位置。

L247–248

```
# 反汇编输出
PRELAB_ASM      = ./zhangxiyuan24.txt
```

反汇编文本输出名为zhangxiyuan24.txt，是代码视图，不是程序运行日志。

L250–251

```
# 程序在 QEMU RAM 中的地址
PRELAB_ADDR     = 0x50200000
```

Prelab装载地址实际定义为0x50200000；后面313行注释说0x80200000与它不一致，应以这一变量展开的实际命令为准。

L254–256

```
# ------------------------------------------------------------------------
# 1. test.S -> ELF
# ------------------------------------------------------------------------
```

注释标题：下面把test.S编译链接成ELF。

L258

```
$(PRELAB_ELF): $(PRELAB_SRC) $(PRELAB_LDS)
```

Prelab ELF依赖汇编源与链接脚本，任何一个变更后应重建。

L259

```
    $(CC) \
```

调用当前CC。可命令行覆盖CROSS_PREFIX使它使用云环境已有交叉gcc。

L260

```
        -g \
```

-g加入调试信息，让GDB能将机器指令对应到源代码行。

L261

```
        -static \
```

-static请求静态链接，不依赖目标机动态加载器。

L262

```
        -no-pie \
```

-no-pie明确生成非位置独立可执行文件，便于固定物理地址装载。

L263

```
        -nostdlib \
```

-nostdlib不自动链接标准库及默认启动对象。

L264

```
        -nostartfiles \
```

-nostartfiles明确不使用系统启动文件；对纯汇编Prelab，入口和寄存器初始化由自己负责。

L265

```
        -Wl,--build-id=none \
```

禁止自动生成build-id note，避免额外节影响裸镜像布局。

L266

```
        -Wl,-T,$(PRELAB_LDS) \
```

通过-Wl,-T把prelab.lds交给链接器；这个脚本决定真正链接地址。

L267

```
        -Wl,-e,main \
```

通过-Wl,-e,main把入口设为汇编main标签，而非默认的_start。

L268

```
        -o $@ \
```

输出文件名为当前目标zhangxiyuan24。

L269

```
        $(PRELAB_SRC)
```

把test.S作为汇编输入。因为后缀是大写.S，gcc会先做预处理再汇编。

L272–274

```
# ------------------------------------------------------------------------
# 2. ELF -> RAW binary
# ------------------------------------------------------------------------
```

注释标题：下面把ELF导出为不含ELF头的raw binary。

L276

```
$(PRELAB_BIN): $(PRELAB_ELF)
```

裸二进制依赖ELF；ELF重建后再重新导出。

L277

```
    $(OBJCOPY) -O binary $< $@
```

objcopy -O binary提取可装载节的原始字节；$<是输入ELF，$@是输出.bin。导出后不再有ELF入口/程序头，模拟器必须另行知道装载地址。

L280–282

```
# ------------------------------------------------------------------------
# 3. 编译整个 Prelab
# ------------------------------------------------------------------------
```

注释标题：下面是Prelab的汇总编译目标。

L284

```
prelab: $(PRELAB_ELF) $(PRELAB_BIN)
```

prelab要求ELF与.bin两个文件已存在且更新；这一目标没有自己额外执行的配方。

L287–289

```
# ------------------------------------------------------------------------
# 4. 反汇编
# ------------------------------------------------------------------------
```

注释标题：下面生成Prelab反汇编。

L291–292

```
prelab-asm: $(PRELAB_ELF)
    $(OBJDUMP) -d $(PRELAB_ELF) > $(PRELAB_ASM)
```

prelab-asm依赖ELF，执行objdump -d并将可读指令写到zhangxiyuan24.txt。它说明“编译成什么”，不说明“实际执行到了哪”。

L295–297

```
# ------------------------------------------------------------------------
# 5. 查看 ELF 信息
# ------------------------------------------------------------------------
```

注释标题：下面查看ELF头、段和入口符号。

L299

```
prelab-info: $(PRELAB_ELF)
```

prelab-info依赖ELF，所以通常会先保证需要的ELF产物存在。

L300–301

```
    @echo "========== ELF HEADER =========="
    $(READELF) -h $(PRELAB_ELF)
```

先打印分隔标题，再readelf -h显示ELF类、机器架构、入口地址等头信息。

L302

```
    @echo
```

输出一个空行，使下面程序头表更易读；@仅隐藏echo命令本身。

L303–304

```
    @echo "========== PROGRAM HEADER =========="
    $(READELF) -l $(PRELAB_ELF)
```

打印标题并用readelf -l显示程序头表，可核对PT_LOAD的文件/内存地址和大小。

L305

```
    @echo
```

再输出一个空行，分隔符号查询。

L306–307

```
    @echo "========== MAIN SYMBOL =========="
    $(CROSS_PREFIX)nm -n $(PRELAB_ELF) | grep main
```

打印标题，然后nm -n按地址排序符号，经grep main筛选入口符号。这里nm使用CROSS_PREFIX直接拼接，不受GDB变量覆盖影响。

L310–315

```
# ----------------------------------------------------------------------
# 6. 正常运行 Prelab
#
# binary 被明确放到 0x80200000
# cpu-num=0 让 CPU0 从该加载地址开始执行
# ----------------------------------------------------------------------
```

运行说明注释。313行写“0x80200000”，但251行PRELAB_ADDR和prelab.lds实际上使用0x50200000；这段注释不能当作真实装载地址。cpu-num=0会让hart0从loader提供的位置开始。

L317

```
prelab-run: prelab
```

prelab-run依赖prelab，所以会先构建需要的ELF和.bin，再运行模拟器。

L318

```
    $(QEMU) \
```

调用QEMU变量所指定的模拟器；默认仍指课程自备版本。

L319

```
        -machine virt \
```

-machine virt选择virt机器。

L320

```
        -m 256M \
```

-m 256M分配256MiB内存；不同QEMU版本的RAM基址仍需核实，不因总大小相同而自动相同。

L321

```
        -nographic \
```

-nographic关闭图形窗口，串口通常接入终端；若程序不写UART，就不会因为该选项自动出现文字。

L322

```
        -bios none \
```

-bios none不运行默认固件，Prelab从自己的汇编直接开始。

L323

```
        -device loader,file=$(abspath $(PRELAB_BIN)),addr=$(PRELAB_ADDR),cpu-num=0
```

loader把绝对路径的.bin装在PRELAB_ADDR，并让CPU0从那里执行。abspath将相对文件路径规范成绝对路径；裸文件的正确性要求编译地址、装载地址与内存映射一致。

L326–331

```
# ----------------------------------------------------------------------
# 7. QEMU GDB 调试模式
#
# -s = 在 TCP 1234 开 GDB server
# -S = CPU 启动后立即暂停
# ----------------------------------------------------------------------
```

调试模式注释解释-s和-S；GDB端口由模拟器提供，CPU暂停等待是预期行为。

L333

```
prelab-debug: prelab
```

prelab-debug也先确保prelab产物存在。

L334

```
    $(QEMU) \
```

启动QEMU，下面参数基本与prelab-run相同。

L335–336

```
        -machine virt \
        -m 256M \
```

选择virt机器和256MiB内存。

L337–338

```
        -nographic \
        -bios none \
```

终端模式且不使用默认固件。

L339

```
        -device loader,file=$(abspath $(PRELAB_BIN)),addr=$(PRELAB_ADDR),cpu-num=0 \
```

装入同一个Prelab裸二进制，地址仍是实际变量PRELAB_ADDR，下一行继续追加调试选项。

L340

```
        -s \
```

-s启用默认端口1234的GDB远程调试服务。

L341

```
        -S
```

-S启动后暂停CPU；需另开终端make prelab-gdb，再在GDB里设置断点和continue。

L344–346

```
# ------------------------------------------------------------------------
# 8. GDB 连接 QEMU
# ------------------------------------------------------------------------
```

注释标题：下面是单独的GDB连接目标。

L348

```
prelab-gdb: $(PRELAB_ELF)
```

prelab-gdb依赖ELF符号文件，但不依赖prelab-debug，故不会自动启动模拟器。

L349

```
    $(GDB) $(PRELAB_ELF) \
```

打开GDB并加载Prelab ELF，以便认识main、数组和循环等符号。

L350

```
        -ex "target remote :1234"
```

-ex自动执行target remote :1234，连接正在等待的QEMU；地址和端口冲突时应先定位对应进程。

L353–355

```
# ----------------------------------------------------------------------
# 9. 清理 Prelab
# ----------------------------------------------------------------------
```

注释标题：下面清理Prelab的生成物。

L357

```
prelab-clean:
```

prelab-clean是清理动作的目标名。

L358–361

```
    rm -f \
        $(PRELAB_ELF) \
        $(PRELAB_BIN) \
        $(PRELAB_ASM)
```

rm -f依次删除Prelab ELF、裸二进制和反汇编文本；不删除test.S、prime.S或链接脚本。原包附带的同名ELF也会被删，阅读前最好保留压缩包原件。

L364–370

```
.PHONY: prelab \
        prelab-asm \
        prelab-info \
        prelab-run \
        prelab-debug \
        prelab-gdb \
        prelab-clean
```

将所有Prelab动作声明为伪目标，反斜杠使多行仍是一条.PHONY列表；它们不是需要出现在目录里的同名文件。

3 你能看到什么，什么代码保证它

默认make应准备bootblock、内核、四个用户应用和image；make prelab则单独编译test.S。

代码推导，未在课程平台实测

Makefile 120—123行枚举test_project1/*.c，138行all聚合elf/image/asm；235—236、258—269、284行独立选择test.S并产生Prelab ELF及bin。

制作镜像时可见各ELF入口；正常课程启动需要课程U-Boot/BIOS，而不只是随便一个QEMU。

代码推导，未在课程平台实测

Makefile 35—36、63—71和164—165行把课程QEMU/U-Boot及virtio镜像连起来；183—206行传入0x50200000系列布局。arch/riscv/boot/bootblock.S 10行将BIOS入口定为0x50150000。

运行prelab-debug后终端可能没有程序现象，GDB连接并continue之前CPU应暂停。

代码推导，未在课程平台实测

Makefile 333—341行追加-s -S；348—350行只连接GDB，不自动发continue。test.S本身的观测还需要检查寄存器/内存，不能只等待串口文字。

生成的应用入口通常相隔0x10000，但只重编译部分应用时不能保证名字与地址仍稳定对应。

代码推导，未在课程平台实测

Makefile 91行初始化USER_ENTRYPOINT，203行使用当前值，205—206行在规则展开时递增全局变量；没有按文件名固定计算槽位的表达式。

容易误解或需要修正的地方

- 不要执行floppy目标来检验本文；151行dd写的是实际磁盘分区，可能破坏已有数据。
- 本讲解不修改原Makefile。需要切换编译器可使用make CROSS_PREFIX=riscv64-linux-gnu-，调试器另覆写GDB=gdb-multiarch；平台地址/BIOS适配属于额外步骤。
- 本文件没有可靠的并行构建约定：all中的dirs不构成对子目标的顺序依赖，用户地址又靠eval改全局变量。应先创建输出目录，使用串行、完整且受控的构建来复核。
- 一些规则未把头文件列入依赖；改头文件后只执行增量make可能使用过期产物。
- prelab-run附近注释与PRELAB_ADDR不一致。先检查实际make展开命令和ELF入口，不要根据注释擅自推断通用QEMU上可运行。
- 在当前工具链上，仅覆盖 CROSS_PREFIX 的构建实测返回 0，产物的 ELF 类型仍可为 ET_EXEC，却带动态链接相关节，且首个 PT_LOAD 从入口之前 0x1000 的地址开始并包含 ELF 头。当前工具没有按 p_vaddr 装入，所以仍可能跳到错误字节。不能只看 ET_EXEC 或把这一布局问题归结为单一 PIE 标志；应核对实际程序头、入口与镜像内容。

03 · 镜像与构建

13 链接脚本：把入口、代码、数据和零区放到约定地址

原路径：riscv.lds · 185 行

> 由链接器读取的布局规则；它不是C代码，也不会在CPU上逐行执行。

编译器先产生含许多输入节的对象文件，链接器按本脚本把它们合并成输出节。星号表示匹配所有输入文件；括号里的.text等是节名。位置计数器“.”代表当前正在安排的内存地址，TEXT_START由Makefile通过--defsym传入。

最重要的阅读主线是：第7行选择起始地址；11行把.entry_function放到最前；23行等安排只读数据；100—119行安排有初值数据并定义零区起点；120—141行安排.bss并给出终点。启动汇编利用这些符号清零，而链接脚本自己不写零到运行内存。

脚本包含大量通用GCC链接模板部分，例如C++构造/析构、TLS和多代调试信息。规则保留这些节不等于本Project1实现了C++运行时、线程局部存储或异常处理。简单C/汇编输入可能根本没有这些节；没有匹配输入时相关规则往往不产生有内容的输出节。

1 完整代码，先看全貌

```
1  OUTPUT_FORMAT("elf64-littleriscv", "elf64-littleriscv",
2                "elf64-littleriscv")
3  OUTPUT_ARCH("riscv")
4  ENTRY(_start)
5  SECTIONS
6  {
7    . = TEXT_START;
8    .text           :
9    {
10     _ftext = . ;
11     *(.entry_function)
12     *(.text.unlikely .text.*_unlikely .text.unlikely.*)
13     *(.text.exit .text.exit.*)
14     *(.text.startup .text.startup.*)
15     *(.text.hot .text.hot.*)
16     *(.text .stub .text.* .gnu.linkonce.t.*)
17     /* .gnu.warning sections are handled specially by elf32.em.  */
18     *(.gnu.warning)
19   }
20   PROVIDE (__etext = .);
21   PROVIDE (_etext = .);
22   PROVIDE (etext = .);
23   .rodata         : { *(.rodata .rodata.* .gnu.linkonce.r.*) }
24   .rodata1        : { *(.rodata1) }
25   .sdata2         :
26   {
27     *(.sdata2 .sdata2.* .gnu.linkonce.s2.*)
28   }
```

```
  .sbss2          : { *(.sbss2 .sbss2.* .gnu.linkonce.sb2.*) }
  .eh_frame_hdr   : { *(.eh_frame_hdr) *(.eh_frame_entry .eh_frame_entry.*) }
  .eh_frame       : ONLY_IF_RO { KEEP (*(.eh_frame)) *(.eh_frame.*) }
  .gcc_except_table   : ONLY_IF_RO { *(.gcc_except_table .gcc_except_table.*) }
  .gnu_extab   : ONLY_IF_RO { *(.gnu_extab*) }
  /* These sections are generated by the Sun/Oracle C++ compiler.  */
  .exception_ranges   : ONLY_IF_RO { *(.exception_ranges*) }
  /* Exception handling  */
  .eh_frame       : ONLY_IF_RW { KEEP (*(.eh_frame)) *(.eh_frame.*) }
  .gnu_extab      : ONLY_IF_RW { *(.gnu_extab) }
  .gcc_except_table   : ONLY_IF_RW { *(.gcc_except_table .gcc_except_table.*) }
  .exception_ranges   : ONLY_IF_RW { *(.exception_ranges*) }
  /* Thread Local Storage sections  */
  .tdata          :
   {
     PROVIDE_HIDDEN (__tdata_start = .);
     *(.tdata .tdata.* .gnu.linkonce.td.*)
   }
  .tbss           : { *(.tbss .tbss.* .gnu.linkonce.tb.*) *(.tcommon) }
  .preinit_array    :
  {
    PROVIDE_HIDDEN (__preinit_array_start = .);
    KEEP (*(.preinit_array))
    PROVIDE_HIDDEN (__preinit_array_end = .);
  }
  .init_array    :
  {
    PROVIDE_HIDDEN (__init_array_start = .);
    KEEP (*(SORT_BY_INIT_PRIORITY(.init_array.*) SORT_BY_INIT_PRIORITY(.ctors.*)))
    KEEP (*(.init_array EXCLUDE_FILE (*crtbegin.o *crtbegin?.o *crtend.o *crtend?.o ) .ctor
 > s))
    PROVIDE_HIDDEN (__init_array_end = .);
  }
  .fini_array    :
  {
    PROVIDE_HIDDEN (__fini_array_start = .);
    KEEP (*(SORT_BY_INIT_PRIORITY(.fini_array.*) SORT_BY_INIT_PRIORITY(.dtors.*)))
    KEEP (*(.fini_array EXCLUDE_FILE (*crtbegin.o *crtbegin?.o *crtend.o *crtend?.o ) .dtor
 > s))
    PROVIDE_HIDDEN (__fini_array_end = .);
  }
  .ctors          :
  {
    /* gcc uses crtbegin.o to find the start of
       the constructors, so we make sure it is
       first.  Because this is a wildcard, it
       doesn't matter if the user does not
       actually link against crtbegin.o; the
       linker won't look for a file to match a
       wildcard.  The wildcard also means that it
       doesn't matter which directory crtbegin.o
       is in.  */
    KEEP (*crtbegin.o(.ctors))
    KEEP (*crtbegin?.o(.ctors))
    /* We don't want to include the .ctor section from
       the crtend.o file until after the sorted ctors.
       The .ctor section from the crtend file contains the
       end of ctors marker and it must be last */
    KEEP (*(EXCLUDE_FILE (*crtend.o *crtend?.o ) .ctors))
```

```
    KEEP (*(SORT(.ctors.*)))
    KEEP (*(.ctors))
  }
  .dtors          :
  {
    KEEP (*crtbegin.o(.dtors))
    KEEP (*crtbegin?.o(.dtors))
    KEEP (*(EXCLUDE_FILE (*crtend.o *crtend?.o ) .dtors))
    KEEP (*(SORT(.dtors.*)))
    KEEP (*(.dtors))
  }
  .jcr            : { KEEP (*(.jcr)) }
  .data.rel.ro : { *(.data.rel.ro.local* .gnu.linkonce.d.rel.ro.local.*) *(.data.rel.ro .da
> ta.rel.ro.* .gnu.linkonce.d.rel.ro.*) }
  .dynamic        : { *(.dynamic) }
  .data           :
  {
    __DATA_BEGIN__ = .;
    *(.data .data.* .gnu.linkonce.d.*)
    SORT(CONSTRUCTORS)
  }
  .data1          : { *(.data1) }
  .got            : { *(.got.plt) *(.igot.plt) *(.got) *(.igot) }
  /* We want the small data sections together, so single-instruction offsets
     can access them all, and initialized data all before uninitialized, so
     we can shorten the on-disk segment size.  */
  .sdata          :
  {
    __SDATA_BEGIN__ = .;
    *(.srodata.cst16) *(.srodata.cst8) *(.srodata.cst4) *(.srodata.cst2) *(.srodata .srodat
> a.*)
    *(.sdata .sdata.* .gnu.linkonce.s.*)
  }
  _edata = .; PROVIDE (edata = .);
  . = .;
  __bss_start = .;
  .sbss           :
  {
    *(.dynsbss)
    *(.sbss .sbss.* .gnu.linkonce.sb.*)
    *(.scommon)
  }
  .bss            :
  {
   *(.dynbss)
   *(.bss .bss.* .gnu.linkonce.b.*)
   *(COMMON)
   /* Align here to ensure that the .bss section occupies space up to
      _end.  Align after .bss to ensure correct alignment even if the
      .bss section disappears because there are no input sections.
      FIXME: Why do we need it? When there is no .bss section, we do not
      pad the .data section.  */
   . = ALIGN(. != 0 ? 64 / 8 : 1);
  }
  . = ALIGN(64 / 8);
  . = SEGMENT_START("ldata-segment", .);
  . = ALIGN(64 / 8);
  __BSS_END__ = .;
  _end = .; PROVIDE (end = .);
```

```
  /* Stabs debugging sections.  */
  .stab          0 : { *(.stab) }
  .stabstr       0 : { *(.stabstr) }
  .stab.excl     0 : { *(.stab.excl) }
  .stab.exclstr  0 : { *(.stab.exclstr) }
  .stab.index    0 : { *(.stab.index) }
  .stab.indexstr 0 : { *(.stab.indexstr) }
  .comment       0 : { *(.comment) }
  .gnu.build.attributes : { *(.gnu.build.attributes .gnu.build.attributes.*) }
  /* DWARF debug sections.
     Symbols in the DWARF debugging sections are relative to the beginning
     of the section so we begin them at 0.  */
  /* DWARF 1 */
  .debug          0 : { *(.debug) }
  .line           0 : { *(.line) }
  /* GNU DWARF 1 extensions */
  .debug_srcinfo  0 : { *(.debug_srcinfo) }
  .debug_sfnames  0 : { *(.debug_sfnames) }
  /* DWARF 1.1 and DWARF 2 */
  .debug_aranges  0 : { *(.debug_aranges) }
  .debug_pubnames 0 : { *(.debug_pubnames) }
  /* DWARF 2 */
  .debug_info     0 : { *(.debug_info .gnu.linkonce.wi.*) }
  .debug_abbrev   0 : { *(.debug_abbrev) }
  .debug_line     0 : { *(.debug_line .debug_line.* .debug_line_end) }
  .debug_frame    0 : { *(.debug_frame) }
  .debug_str      0 : { *(.debug_str) }
  .debug_loc      0 : { *(.debug_loc) }
  .debug_macinfo  0 : { *(.debug_macinfo) }
  /* SGI/MIPS DWARF 2 extensions */
  .debug_weaknames 0 : { *(.debug_weaknames) }
  .debug_funcnames 0 : { *(.debug_funcnames) }
  .debug_typenames 0 : { *(.debug_typenames) }
  .debug_varnames  0 : { *(.debug_varnames) }
  /* DWARF 3 */
  .debug_pubtypes 0 : { *(.debug_pubtypes) }
  .debug_ranges   0 : { *(.debug_ranges) }
  /* DWARF Extension.  */
  .debug_macro    0 : { *(.debug_macro) }
  .debug_addr     0 : { *(.debug_addr) }
  .gnu.attributes 0 : { KEEP (*(.gnu.attributes)) }
  /DISCARD/ : { *(.note.GNU-stack) *(.gnu_debuglink) *(.gnu.lto_*) }
}
```

2 按原始行号，一句一句理解

L1–2

```
OUTPUT_FORMAT("elf64-littleriscv", "elf64-littleriscv",
              "elf64-littleriscv")
```

规定输出为64位、小端RISC-V ELF。OUTPUT_FORMAT可列默认/大端/小端三种选择，本脚本三个字符串都写同一小端格式；它不是把机器码直接输出为raw镜像。

L3

```
OUTPUT_ARCH("riscv")
```

指定链接架构为riscv，链接器会按RISC-V格式处理对象文件与重定位。

L4

```
ENTRY(_start)
```

默认ELF入口符号为_start，内核和用户程序由head.S/crt0.S提供。启动程序的Makefile另传-e main，会覆盖这个默认入口。

L5–6

```
SECTIONS
{
```

开始SECTIONS布局块，里面按顺序描述输出节及链接符号。

L7

```
  . = TEXT_START;
```

把位置计数器设为TEXT_START。该符号由Makefile传入，启动/内核/应用因此可复用同一脚本而采用不同基址。

L8–9

```
  .text           :
  {
```

开始.text输出节，用来集中可执行代码；后面的输入节按规则顺序合并。

L10

```
    _ftext = . ;
```

将当前地址命名为_ftext，表示.text起点。定义链接符号不会像C变量那样额外占用几个字节。

L11

```
    *(.entry_function)
```

先放全部输入文件的.entry_function节，确保head.S/crt0.S中的_start位于正文前面，符合简单加载/跳转约定。

L12

```
    *(.text.unlikely .text.*_unlikely .text.unlikely.*)
```

收集编译器分类为不太可能执行的冷路径代码；.*等是节名通配符，不表示C的成员访问。

L13

```
    *(.text.exit .text.exit.*)
```

收集退出路径专用代码节。

L14

```
    *(.text.startup .text.startup.*)
```

收集启动路径代码节；这只安排位置，不会自动调用里面的函数。

L15

```
    *(.text.hot .text.hot.*)
```

收集经常执行的热路径代码节，保留编译器的代码分类。

L16

```
    *(.text .stub .text.* .gnu.linkonce.t.*)
```

收集普通.text、跳板.stub、各种.text子节及GNU可合并函数节，形成主要机器指令区域。

L17–18

```
    /* .gnu.warning sections are handled specially by elf32.em.  */
    *(.gnu.warning)
```

注释说明.gnu.warning有特殊处理；规则仍匹配此类链接警告相关输入节。它不是运行时printf。

L19

```
  }
```

结束.text输出节定义，当前位置落在已放置的代码之后。

L20

```
  PROVIDE (__etext = .);
```

需要且尚未定义时提供__etext，值为此处位置，常用作正文末尾标记。

L21

```
  PROVIDE (_etext = .);
```

同样提供别名_etext；这些名字指向同一位置，不是复制一份代码。

L22

```
  PROVIDE (etext = .);
```

再提供不带下划线的etext，满足不同工具或源代码的命名习惯。

L23

```
  .rodata         : { *(.rodata .rodata.* .gnu.linkonce.r.*) }
```

收集只读常量节.rodata及其子节，例如字符串字面量通常在这里。链接名“只读”不等于本工程已设置硬件页表保护。

L24

```
  .rodata1        : { *(.rodata1) }
```

保留另一类只读数据节.rodata1，只有输入文件含它时才有对应内容。

L25–28

```
  .sdata2         :
  {
    *(.sdata2 .sdata2.* .gnu.linkonce.s2.*)
  }
```

把小型只读数据放进.sdata2，包含编译器产生的相关子节；括号与大括号仅组织链接布局。

L29

```
  .sbss2          : { *(.sbss2 .sbss2.* .gnu.linkonce.sb2.*) }
```

收集.sbss2系列小型零数据节。注意它位于119行__bss_start之前，不在本项目入口汇编按这两个BSS符号清零的区间内；输入若真产生它，需要额外核对初始化保证。

L30

```
  .eh_frame_hdr   : { *(.eh_frame_hdr) *(.eh_frame_entry .eh_frame_entry.*) }
```

收集异常展开索引.eh_frame_hdr与.eh_frame_entry信息，常用于栈展开定位；不会自行产生异常处理功能。

L31

```
  .eh_frame       : ONLY_IF_RO { KEEP (*(.eh_frame)) *(.eh_frame.*) }
```

若输入被标为只读，则把.eh_frame放在这里；KEEP阻止这些匹配节被垃圾回收。ONLY_IF_RO依据输入属性选择此规则。

L32

```
  .gcc_except_table   : ONLY_IF_RO { *(.gcc_except_table .gcc_except_table.*) }
```

只读的.gcc_except_table放在只读区域，保存编译器异常表信息，不是程序指令。

L33

```
  .gnu_extab   : ONLY_IF_RO { *(.gnu_extab*) }
```

只读的.gnu_extab扩展异常信息节也放在这里。

L34–35

```
  /* These sections are generated by the Sun/Oracle C++ compiler.  */
  .exception_ranges   : ONLY_IF_RO { *(.exception_ranges*) }
```

注释说明.exception_ranges可能来自Sun/Oracle C++编译器；有相应只读输入时按此规则收集。本实验并不必然有它。

L36–37

```
  /* Exception handling  */
  .eh_frame       : ONLY_IF_RW { KEEP (*(.eh_frame)) *(.eh_frame.*) }
```

异常信息如果属于可写属性，则由ONLY_IF_RW版本安放.eh_frame；与31行是属性分流，不是无条件复制两份。

L38

```
  .gnu_extab      : ONLY_IF_RW { *(.gnu_extab) }
```

放置可写.gnu_extab输入。

L39

```
  .gcc_except_table   : ONLY_IF_RW { *(.gcc_except_table .gcc_except_table.*) }
```

放置可写.gcc_except_table输入。

L40

```
  .exception_ranges   : ONLY_IF_RW { *(.exception_ranges*) }
```

放置可写.exception_ranges输入，结束这组异常信息模板规则。

L41

```
  /* Thread Local Storage sections  */
```

注释指出后面是TLS，即线程局部存储节；Project1目前并没有完整线程运行时来使用它。

L42–43

```
  .tdata          :
   {
```

开始.tdata输出节，TLS的有初值变量会被收集在这里。

L44

```
     PROVIDE_HIDDEN (__tdata_start = .);
```

按需定义隐藏符号__tdata_start为当前位置；隐藏指不向外作为普通可见符号导出，不是把数据加密。

L45–46

```
     *(.tdata .tdata.* .gnu.linkonce.td.*)
   }
```

收集.tdata及子节后结束本节。只有摆放数据，没有为每个线程复制TLS或设置线程指针。

L47

```
  .tbss           : { *(.tbss .tbss.* .gnu.linkonce.tb.*) *(.tcommon) }
```

收集TLS零初始化节.tbss及公共TLS符号.tcommon；它们并不等同于119—141行普通BSS清零区。

L48–49

```
  .preinit_array    :
  {
```

开始.preinit_array，预初始化函数指针表使用这个输出节。

L50

```
     PROVIDE_HIDDEN (__preinit_array_start = .);
```

记录隐藏的表起点__preinit_array_start。

L51

```
     KEEP (*(.preinit_array))
```

保留所有.preinit_array输入表项，避免链接垃圾回收删除它们。

L52–53

```
     PROVIDE_HIDDEN (__preinit_array_end = .);
  }
```

记录表终点并关闭输出节。想运行表中的函数还需启动代码显式遍历，本项目crt0没有该流程。

L54–55

```
  .init_array    :
  {
```

开始.init_array构造/初始化函数指针表。

L56

```
    PROVIDE_HIDDEN (__init_array_start = .);
```

定义隐藏的初始化表起点符号。

L57

```
    KEEP (*(SORT_BY_INIT_PRIORITY(.init_array.*) SORT_BY_INIT_PRIORITY(.ctors.*)))
```

按初始化优先级排序并保留.init_array.*和旧式.ctors.*输入表项；排序的是链接表项，不是CPU调度顺序。

L58

```
    KEEP (*(.init_array EXCLUDE_FILE (*crtbegin.o *crtbegin?.o *crtend.o *crtend?.o ) .ctors))
```

收集常规.init_array，并排除特定crtbegin/crtend标记文件的旧式.ctors，以适应GCC运行时组织方式。

L59–60

```
    PROVIDE_HIDDEN (__init_array_end = .);
  }
```

定义初始化表终点并结束输出节；这里仍未调用任何初始化函数。

L61–62

```
  .fini_array    :
  {
```

开始.fini_array，存放程序结束/析构时要调用的函数指针。

L63

```
    PROVIDE_HIDDEN (__fini_array_start = .);
```

定义析构表隐藏起点。

L64

```
    KEEP (*(SORT_BY_INIT_PRIORITY(.fini_array.*) SORT_BY_INIT_PRIORITY(.dtors.*)))
```

按优先级排序并保留.fini_array.*及旧式.dtors.*输入。

L65

```
    KEEP (*(.fini_array EXCLUDE_FILE (*crtbegin.o *crtbegin?.o *crtend.o *crtend?.o ) .dtors))
```

收集普通.fini_array，并处理非crtbegin/crtend文件的旧式析构表。

L66–67

```
    PROVIDE_HIDDEN (__fini_array_end = .);
  }
```

记录析构表终点并关闭本节；应用返回内核不代表自动执行这份表。

L68–69

```
  .ctors          :
  {
```

开始旧式.ctors构造函数表输出节，与较新的.init_array是不同的工具链约定。

L70–78

```
（此处为连续注释或较长结构，请对照上方完整代码 L70–L78。）
```

长注释解释为何先找crtbegin.o：该文件常提供构造表起始标记；通配形式不会因为没链接该文件就强制寻找/报缺失，也不依赖它在哪个目录。

L79

```
    KEEP (*crtbegin.o(.ctors))
```

优先保留名为crtbegin.o的输入文件中的.ctors，安放起始标记。

L80

```
    KEEP (*crtbegin?.o(.ctors))
```

同样处理crtbegin加一个字符后缀的变体文件，比如某些静态/共享运行时版本。

L81–84

```
    /* We don't want to include the .ctor section from
       the crtend.o file until after the sorted ctors.
       The .ctor section from the crtend file contains the
       end of ctors marker and it must be last */
```

注释强调crtend的结束标记应在排序后的构造表之后，不能过早放到中间。

L85

```
    KEEP (*(EXCLUDE_FILE (*crtend.o *crtend?.o ) .ctors))
```

收集除crtend.o及其变体之外的常规.ctors输入。

L86

```
    KEEP (*(SORT(.ctors.*)))
```

对.ctors.*子节按名称排序并保留。

L87–88

```
    KEEP (*(.ctors))
  }
```

最后收集剩余.ctors并关闭输出节，使剩下的末尾标记可落在尾部。

L89–90

```
  .dtors          :
  {
```

开始旧式.dtors析构函数表，结构与上面的构造表相似。

L91

```
    KEEP (*crtbegin.o(.dtors))
```

先保留crtbegin.o中的析构表起始部分。

L92

```
    KEEP (*crtbegin?.o(.dtors))
```

处理crtbegin变体中的析构表。

L93

```
    KEEP (*(EXCLUDE_FILE (*crtend.o *crtend?.o ) .dtors))
```

收集非crtend标记文件的普通.dtors输入。

L94

```
    KEEP (*(SORT(.dtors.*)))
```

按名称排序.dtors.*子节并保留。

L95–96

```
    KEEP (*(.dtors))
  }
```

收集余下.dtors并关闭析构表节；运行时仍需对应代码才能调用。

L97

```
  .jcr            : { KEEP (*(.jcr)) }
```

保留.jcr节，属于旧式Java运行时注册相关链接模板内容；不表示内核实现Java环境。

L98

```
  .data.rel.ro : { *(.data.rel.ro.local* .gnu.linkonce.d.rel.ro.local.*) *(.data.rel.ro .data.rel.ro.* .gnu.linkonce.d.rel.ro.*) }
```

收集只需重定位阶段可写、之后可只读的.data.rel.ro等节。在无分页内核里，节名字本身不会开启RELRO硬件保护。

L99

```
  .dynamic        : { *(.dynamic) }
```

收集.dynamic动态链接元信息；若交叉工具链默认产生动态/PIE布局，应另核对ELF，不能因为脚本有这条规则就认为裸机支持动态链接。

L100–101

```
  .data           :
  {
```

开始.data输出节，保存程序中已有明确非零初值的数据等。

L102

```
    __DATA_BEGIN__ = .;
```

以当前地址定义__DATA_BEGIN__，作为普通有初值数据区域的标记。

L103

```
    *(.data .data.* .gnu.linkonce.d.*)
```

合并.data、其子节以及GNU可合并数据节；这部分内容通常真实占用ELF文件字节。

L104

```
    SORT(CONSTRUCTORS)
```

SORT(CONSTRUCTORS)处理链接器的构造函数集合排序，属于兼容性规则，不会让CPU调用C++构造函数。

L105

```
  }
```

结束.data输出节。

L106

```
  .data1          : { *(.data1) }
```

收集另一个普通数据节.data1。

L107

```
  .got            : { *(.got.plt) *(.igot.plt) *(.got) *(.igot) }
```

收集GOT/PLT相关指针表，供某些寻址/重定位机制使用；它不能替代正确初始化gp、动态加载器或重定位过程。

L108–110

```
  /* We want the small data sections together, so single-instruction offsets
     can access them all, and initialized data all before uninitialized, so
     we can shorten the on-disk segment size.  */
```

注释解释把小数据放在一起，便于短偏移访问；先放有初值数据再放零数据，有利于让ELF文件部分小于运行时内存部分。

L111–112

```
  .sdata          :
  {
```

开始.sdata输出节，主要收集小型有初值数据。

L113

```
    __SDATA_BEGIN__ = .;
```

记录__SDATA_BEGIN__作为小数据区域起点符号。

L114

```
    *(.srodata.cst16) *(.srodata.cst8) *(.srodata.cst4) *(.srodata.cst2) *(.srodata .srodata.
*)
```

依次收集16/8/4/2字节常量与其他小型只读数据输入，便于工具链采用紧凑寻址。

L115–116

```
    *(.sdata .sdata.* .gnu.linkonce.s.*)
  }
```

收集.sdata及其子节和可合并小数据，然后结束输出节。

L117

```
  _edata = .; PROVIDE (edata = .);
```

把当前位置记为_edata，同时按需提供edata别名，通常作为有初值数据结束标记。

L118

```
. = .;
```

把位置计数器赋给自身，通常没有实际位置变化；它不是汇编nop，也不占一条CPU指令。

L119

```
__bss_start = .;
```

定义__bss_start为当前位置，入口汇编从这个符号开始清零。这里没有显式先ALIGN(8)，因此若改动数据布局，应核对起点对齐；不能只因终点8字节对齐就推断起点也满足head.S的sd循环假设。

L120–121

```
.sbss           :
{
```

开始.sbss，收集小型未初始化/零初始化数据。

L122

```
  *(.dynsbss)
```

合并动态小BSS输入；通用模板可能没有实际匹配项。

L123

```
  *(.sbss .sbss.* .gnu.linkonce.sb.*)
```

合并.sbss及相关子节；它们应位于普通BSS清零区内。

L124–125

```
  *(.scommon)
}
```

合并.scommon小公共符号并结束.sbss节。

L126–127

```
.bss            :
{
```

开始普通.bss输出节，未显式初始化的全局/静态变量通常在这里。

L128

```
 *(.dynbss)
```

收集.dynbss输入；并不表示本内核实现动态链接。

L129

```
 *(.bss .bss.* .gnu.linkonce.b.*)
```

合并.bss、所有子节及GNU可合并零数据节；这些通常只增加内存占用，不增加等长文件数据。

L130

```
 *(COMMON)
```

合并旧式COMMON公共符号。现代GCC默认-fno-common时，普通未初始化定义往往直接进入.bss。

L131–135

```
    /* Align here to ensure that the .bss section occupies space up to
       _end.  Align after .bss to ensure correct alignment even if the
       .bss section disappears because there are no input sections.
       FIXME: Why do we need it? When there is no .bss section, we do not
       pad the .data section.  */
```

注释说明要让BSS覆盖到_end，并且即使没有BSS输入也应处理边界对齐；这是布局意图，不是运行时清零代码。

L136

```
    . = ALIGN(. != 0 ? 64 / 8 : 1);
```

当前位置非0时向上对齐到64/8=8字节，等于0时用1字节对齐；ALIGN只计算地址，赋回“.”才推进位置计数器。

L137

```
  }
```

结束.bss输出节。

L138

```
  . = ALIGN(64 / 8);
```

再次把当前地址对齐到8字节，确保后面的结束边界满足RV64双字大小。

L139

```
  . = SEGMENT_START("ldata-segment", .);
```

SEGMENT_START查询名为ldata-segment的指定段起点，若未另行指定就使用当前位置；这条规则不创建分页映射。

L140

```
  . = ALIGN(64 / 8);
```

再次向上对齐到8字节。

L141

```
  __BSS_END__ = .;
```

定义__BSS_END__，内核head.S与用户crt0.S据此决定何处停止清零；脚本仅提供边界，真正清零发生在目标CPU执行汇编时。

L142

```
  _end = .; PROVIDE (end = .);
```

定义_end并按需提供end别名，标记常规装载区域末端；链接符号本身不分配一块额外内存。

L143

```
  /* Stabs debugging sections.  */
```

注释指出下一组是较老的Stabs调试信息，而非内核运行数据。

L144

```
  .stab          0 : { *(.stab) }
```

收集.stab记录，并把该输出节调试地址设为0；调试节通常不作为PT_LOAD装载到目标内存0。

L145

```
  .stabstr       0 : { *(.stabstr) }
```

收集.stabstr字符串表，为Stabs记录提供名字和文本。

L146

```
  .stab.excl     0 : { *(.stab.excl) }
```

收集.stab.excl，保存被排除/特殊单元的Stabs调试元信息。

L147

```
  .stab.exclstr  0 : { *(.stab.exclstr) }
```

收集对应的.stab.exclstr字符串表。

L148

```
  .stab.index    0 : { *(.stab.index) }
```

收集.stab.index索引信息，便于旧式调试器定位记录。

L149

```
  .stab.indexstr 0 : { *(.stab.indexstr) }
```

收集.stab.indexstr索引字符串。

L150

```
  .comment       0 : { *(.comment) }
```

收集.comment，比如编译器版本文字；地址0在这里是调试/说明节布局，不是程序入口。

L151

```
  .gnu.build.attributes : { *(.gnu.build.attributes .gnu.build.attributes.*) }
```

保留.gnu.build.attributes及其子节，记录工具链构建属性；不是C变量。

L152–154

```
  /* DWARF debug sections.
     Symbols in the DWARF debugging sections are relative to the beginning
     of the section so we begin them at 0.  */
```

注释解释DWARF调试信息使用节起点相对的偏移，所以这些节常从地址0组织；不会使.text也移到0。

L155

```
  /* DWARF 1 */
```

注释标出DWARF 1规则。

L156

```
  .debug          0 : { *(.debug) }
```

收集旧式.debug调试记录。

L157

```
  .line           0 : { *(.line) }
```

收集.line行号映射，使调试器可把指令与源码对应。

L158

```
  /* GNU DWARF 1 extensions */
```

注释标出GNU对DWARF 1的扩展。

L159

```
  .debug_srcinfo  0 : { *(.debug_srcinfo) }
```

收集.debug_srcinfo源文件相关调试信息。

L160

```
  .debug_sfnames  0 : { *(.debug_sfnames) }
```

收集.debug_sfnames源文件名信息。

L161

```
  /* DWARF 1.1 and DWARF 2 */
```

注释标出兼容DWARF 1.1/2的规则。

L162

```
  .debug_aranges  0 : { *(.debug_aranges) }
```

收集.debug_aranges地址区间索引，帮助按目标地址找调试单元。

L163

```
  .debug_pubnames 0 : { *(.debug_pubnames) }
```

收集.debug_pubnames公开符号名索引；它不是加载器运行时任务名表。

L164

```
  /* DWARF 2 */
```

注释标出DWARF 2的主要调试节。

L165

```
  .debug_info     0 : { *(.debug_info .gnu.linkonce.wi.*) }
```

收集.debug_info及GNU可合并单元，保存类型、变量、函数等核心调试描述。

L166

```
  .debug_abbrev   0 : { *(.debug_abbrev) }
```

收集.debug_abbrev，保存调试记录结构的缩写定义。

L167

```
  .debug_line     0 : { *(.debug_line .debug_line.* .debug_line_end) }
```

收集.debug_line及其相关子节，保存源码行与地址的映射。

L168

```
  .debug_frame    0 : { *(.debug_frame) }
```

收集.debug_frame，供调试器理解栈帧展开；不同于前面运行时可能使用的.eh_frame。

L169

```
  .debug_str      0 : { *(.debug_str) }
```

收集.debug_str调试字符串池。

L170

```
  .debug_loc      0 : { *(.debug_loc) }
```

收集.debug_loc变量位置描述，说明变量在不同指令范围位于哪个寄存器/栈位置。

L171

```
  .debug_macinfo  0 : { *(.debug_macinfo) }
```

收集.debug_macinfo，保存宏相关调试信息；调试选项不同可能没有内容。

L172

```
  /* SGI/MIPS DWARF 2 extensions */
```

注释标出SGI/MIPS来源的DWARF扩展模板；脚本仍为RISC-V目标，不会改变机器架构。

L173

```
  .debug_weaknames 0 : { *(.debug_weaknames) }
```

收集.debug_weaknames弱符号名调试索引。

L174

```
  .debug_funcnames 0 : { *(.debug_funcnames) }
```

收集.debug_funcnames函数名调试索引。

L175

```
  .debug_typenames 0 : { *(.debug_typenames) }
```

收集.debug_typenames类型名调试索引。

L176

```
  .debug_varnames  0 : { *(.debug_varnames) }
```

收集.debug_varnames变量名调试索引。

L177

```
  /* DWARF 3 */
```

注释标出DWARF 3的规则。

L178

```
  .debug_pubtypes 0 : { *(.debug_pubtypes) }
```

收集.debug_pubtypes公开类型索引。

L179

```
  .debug_ranges   0 : { *(.debug_ranges) }
```

收集.debug_ranges不连续地址区间列表。

L180

```
  /* DWARF Extension.  */
```

注释标出后续DWARF扩展。

L181

```
  .debug_macro    0 : { *(.debug_macro) }
```

收集.debug_macro宏展开/定义信息。

L182

```
  .debug_addr     0 : { *(.debug_addr) }
```

收集.debug_addr调试地址表，供较新调试信息格式引用。

L183

```
  .gnu.attributes 0 : { KEEP (*(.gnu.attributes)) }
```

KEEP保留.gnu.attributes体系结构属性，方便链接器/工具识别ABI等要求。

L184

```
  /DISCARD/ : { *(.note.GNU-stack) *(.gnu_debuglink) *(.gnu.lto_*) }
```

/DISCARD/丢弃GNU栈说明note、外置调试链接说明和LTO中间节；它不会删除运行时真正的栈内存，也不能替代合理的栈设置。

L185

```
}
```

结束整个SECTIONS布局块，链接器据此生成最终ELF；接下来createimage才会按程序头抽取数据。

3 你能看到什么，什么代码保证它

readelf应显示RISC-V 64位小端ELF；入口应对应_start，bootblock则由-e main单独覆盖。

代码推导，未在课程平台实测

riscv.lds 1—4行指定格式/架构/默认入口；7—11行把.entry_function排在TEXT_START最前；Makefile 184行明确给bootblock传-e main。

未初始化全局数组应在程序进入C main前被清零，但不是因为createimage写入了一整块零。

代码推导，未在课程平台实测

riscv.lds 119与141行提供普通BSS边界，120—137行收集BSS；arch/riscv/kernel/head.S 16—23行及arch/riscv/crt0/crt0.S 12—20行据边界写零。tools/createimage.c 207—210行仅复制文件数据。对齐、覆盖区间及正确装载仍是前提。

ELF可能包含许多调试节，但它们不应都出现在最终裸磁盘image里。

代码推导，未在课程平台实测

riscv.lds 143—183行组织调试/属性节；tools/createimage.c 119行跳过非PT_LOAD程序头，所以按可加载段而不是“复制整个ELF文件”打包。

容易误解或需要修正的地方

- 链接符号与C变量不同。__bss_start只是一个地址标签；清.bss由head.S/crt0.S执行，脚本不会替CPU执行存储指令。
- ENTRY只设置ELF入口元数据，不自动证明原始image放置位置正确。简单加载器把字节拷到entrypoint，仍依赖入口布局、连续段与内存地址一致。
- 主脚本的BSS起点未显式先对齐8字节，head.S却按8字节步进且以相等作为结束条件；新增数据布局时应检查实际符号差值与起点对齐。用户crt0按4字节处理也需边界满足。
- 链接脚本保留TLS、构造数组或异常表不等于工程已经初始化TLS、调用构造函数或实现异常运行时。
- 调试节地址0不表示CPU会到0地址执行。它们通常是非装载调试元数据，应结合ELF节标志和程序头判断。

04 · 每个任务程序

14 data：确认初始化数据没有在装载中丢失

原路径：test/test_project1/data.c · 20 行

> 用户任务／已初始化数据校验

这个程序只问一个问题：编译时写进程序的1024个整数，装载到内存后是否还是0、1、2……1023。它不计算新数组，而是把内存中的已有数组逐项与下标比较。

读完整文件后，抓住两条路径：每项都对→打印[OK]→return；任何一项不对→打印Error→死循环，后面的成功信息和返回都到不了。

1 完整代码，先看全貌

```
#include <kernel.h>

int data[] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21,
 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44
, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 6
7, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89,
90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 1
10, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 12
8, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146
, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164,
 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182,
183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200, 2
01, 202, 203, 204, 205, 206, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217, 218, 21
9, 220, 221, 222, 223, 224, 225, 226, 227, 228, 229, 230, 231, 232, 233, 234, 235, 236, 237
, 238, 239, 240, 241, 242, 243, 244, 245, 246, 247, 248, 249, 250, 251, 252, 253, 254, 255,
 256, 257, 258, 259, 260, 261, 262, 263, 264, 265, 266, 267, 268, 269, 270, 271, 272, 273,
274, 275, 276, 277, 278, 279, 280, 281, 282, 283, 284, 285, 286, 287, 288, 289, 290, 291, 2
92, 293, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 31
0, 311, 312, 313, 314, 315, 316, 317, 318, 319, 320, 321, 322, 323, 324, 325, 326, 327, 328
, 329, 330, 331, 332, 333, 334, 335, 336, 337, 338, 339, 340, 341, 342, 343, 344, 345, 346,
 347, 348, 349, 350, 351, 352, 353, 354, 355, 356, 357, 358, 359, 360, 361, 362, 363, 364,
365, 366, 367, 368, 369, 370, 371, 372, 373, 374, 375, 376, 377, 378, 379, 380, 381, 382, 3
83, 384, 385, 386, 387, 388, 389, 390, 391, 392, 393, 394, 395, 396, 397, 398, 399, 400, 40
1, 402, 403, 404, 405, 406, 407, 408, 409, 410, 411, 412, 413, 414, 415, 416, 417, 418, 419
, 420, 421, 422, 423, 424, 425, 426, 427, 428, 429, 430, 431, 432, 433, 434, 435, 436, 437,
 438, 439, 440, 441, 442, 443, 444, 445, 446, 447, 448, 449, 450, 451, 452, 453, 454, 455,
456, 457, 458, 459, 460, 461, 462, 463, 464, 465, 466, 467, 468, 469, 470, 471, 472, 473, 4
74, 475, 476, 477, 478, 479, 480, 481, 482, 483, 484, 485, 486, 487, 488, 489, 490, 491, 49
2, 493, 494, 495, 496, 497, 498, 499, 500, 501, 502, 503, 504, 505, 506, 507, 508, 509, 510
, 511, 512, 513, 514, 515, 516, 517, 518, 519, 520, 521, 522, 523, 524, 525, 526, 527, 528,
 529, 530, 531, 532, 533, 534, 535, 536, 537, 538, 539, 540, 541, 542, 543, 544, 545, 546,
547, 548, 549, 550, 551, 552, 553, 554, 555, 556, 557, 558, 559, 560, 561, 562, 563, 564, 5
65, 566, 567, 568, 569, 570, 571, 572, 573, 574, 575, 576, 577, 578, 579, 580, 581, 582, 58
3, 584, 585, 586, 587, 588, 589, 590, 591, 592, 593, 594, 595, 596, 597, 598, 599, 600, 601
, 602, 603, 604, 605, 606, 607, 608, 609, 610, 611, 612, 613, 614, 615, 616, 617, 618, 619,
 620, 621, 622, 623, 624, 625, 626, 627, 628, 629, 630, 631, 632, 633, 634, 635, 636, 637,
638, 639, 640, 641, 642, 643, 644, 645, 646, 647, 648, 649, 650, 651, 652, 653, 654, 655, 6
```

```
> 56, 657, 658, 659, 660, 661, 662, 663, 664, 665, 666, 667, 668, 669, 670, 671, 672, 673, 67
> 4, 675, 676, 677, 678, 679, 680, 681, 682, 683, 684, 685, 686, 687, 688, 689, 690, 691, 692
> , 693, 694, 695, 696, 697, 698, 699, 700, 701, 702, 703, 704, 705, 706, 707, 708, 709, 710,
>  711, 712, 713, 714, 715, 716, 717, 718, 719, 720, 721, 722, 723, 724, 725, 726, 727, 728,
> 729, 730, 731, 732, 733, 734, 735, 736, 737, 738, 739, 740, 741, 742, 743, 744, 745, 746, 7
> 47, 748, 749, 750, 751, 752, 753, 754, 755, 756, 757, 758, 759, 760, 761, 762, 763, 764, 76
> 5, 766, 767, 768, 769, 770, 771, 772, 773, 774, 775, 776, 777, 778, 779, 780, 781, 782, 783
> , 784, 785, 786, 787, 788, 789, 790, 791, 792, 793, 794, 795, 796, 797, 798, 799, 800, 801,
>  802, 803, 804, 805, 806, 807, 808, 809, 810, 811, 812, 813, 814, 815, 816, 817, 818, 819,
> 820, 821, 822, 823, 824, 825, 826, 827, 828, 829, 830, 831, 832, 833, 834, 835, 836, 837, 8
> 38, 839, 840, 841, 842, 843, 844, 845, 846, 847, 848, 849, 850, 851, 852, 853, 854, 855, 85
> 6, 857, 858, 859, 860, 861, 862, 863, 864, 865, 866, 867, 868, 869, 870, 871, 872, 873, 874
> , 875, 876, 877, 878, 879, 880, 881, 882, 883, 884, 885, 886, 887, 888, 889, 890, 891, 892,
>  893, 894, 895, 896, 897, 898, 899, 900, 901, 902, 903, 904, 905, 906, 907, 908, 909, 910,
> 911, 912, 913, 914, 915, 916, 917, 918, 919, 920, 921, 922, 923, 924, 925, 926, 927, 928, 9
> 29, 930, 931, 932, 933, 934, 935, 936, 937, 938, 939, 940, 941, 942, 943, 944, 945, 946, 94
> 7, 948, 949, 950, 951, 952, 953, 954, 955, 956, 957, 958, 959, 960, 961, 962, 963, 964, 965
> , 966, 967, 968, 969, 970, 971, 972, 973, 974, 975, 976, 977, 978, 979, 980, 981, 982, 983,
>  984, 985, 986, 987, 988, 989, 990, 991, 992, 993, 994, 995, 996, 997, 998, 999, 1000, 1001
> , 1002, 1003, 1004, 1005, 1006, 1007, 1008, 1009, 1010, 1011, 1012, 1013, 1014, 1015, 1016,
>  1017, 1018, 1019, 1020, 1021, 1022, 1023};
4
5  int main(void)
6  {
7      // Check the data array
8      bios_putstr("[data] Info: checking data!\n");
9      int len = sizeof(data) / sizeof(int);
10     for (int i = 0; i < len; ++i)
11     {
12         if (data[i] != i) {
13             bios_putstr("[data] Error: data check failed!\n");
14             while(1);
15         }
16     }
17     bios_putstr("[data] Info: checking data --- [OK]!\n");
18
19     return 0;
20 }
```

2 按原始行号，一句一句理解

L1

```
#include <kernel.h>
```

包含用户态小库的kernel.h，使编译器知道bios_putstr等接口；这不是宿主Linux的stdio.h。实际调用经过课程给定的跳转表，程序本身没有实现串口驱动。

L3

```
int data[] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 2
2, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43,
…（仅此定位摘录缩短；完整原行已在上方全部展示。）
```

定义全局int数组data，并在源文件中明确写出1024个初始值0～1023。数组长度由初始化列表推导；在此RV64 ABI下int仍为4字节，因此数组占4096字节。它属于需随镜像保存的已初始化数据，而不是仅预留并清零的.bss。不要把很长的这一行误当成1024条运行时赋值语句。

L5–6

```
int main(void)
{
```

定义本任务入口函数main：void表示没有C语言参数，返回值类型int。左花括号开始函数体；进入main之前已经由arch/riscv/crt0/crt0.S建立了最小运行环境。

L7

```
    // Check the data array
```

注释说明下面要核对data数组，注释本身不执行。

L8

```
    bios_putstr("[data] Info: checking data!\n");
```

先向串口输出[data] Info: checking data!和换行。这只表示检查开始，不表示检查通过。

L9

```
    int len = sizeof(data) / sizeof(int);
```

sizeof(data)给出整个数组字节数，sizeof(int)给出一个元素字节数，二者相除得到1024。这里data是数组实体，sizeof不会把它转换成指针大小。

L10–11

```
    for (int i = 0; i < len; ++i)
    {
```

令i从0开始，只要i<len就继续，每次i加1；因此循环检查下标0～1023，刚好覆盖全部元素。左花括号开始每一轮检查的范围。

L12

```
        if (data[i] != i) {
```

把实际读到的data[i]与期望值i比较。!=表示“不相等”，只要某一项不符合初始化列表，就进入失败分支。

L13

```
            bios_putstr("[data] Error: data check failed!\n");
```

在失败时输出[data] Error: data check failed!。这是定位初始化数据或镜像装载问题的直接现象。

L14–15

```
            while(1);
        }
```

while(1);的循环体是分号代表的空语句，条件永远为真，因此一直停留在这里。失败时不会return到装载器，也不会输出后面的[OK]；右花括号结束if分支。

L16

```
    }
```

结束逐项检查的for循环。能走出这个循环，意味着1024次比较都没有进入失败分支。

L17

```
    bios_putstr("[data] Info: checking data --- [OK]!\n");
```

打印[data] Info: checking data --- [OK]!。该信息位于全部比较之后，且失败分支无法返回，所以在程序正常执行的前提下它代表所有元素都匹配。

L19–20

```
    return 0;
}
```

返回整数0并结束main。任务的crt0会恢复调用时保存的ra等寄存器，回到内核；本行不是关机命令，也不是整个OS退出。

3 你能看到什么，什么代码保证它

输入任务名data后，应先看到checking data!，随后看到checking data --- [OK]!，再回到内核的任务输入提示。

代码推导，未在课程平台实测

data.c第3行提供期望数组，9—16行逐项比较，17行只在全部通过后执行，19行返回；内核init/main.c第134行调用任务，返回后在第135行打印Task finished, back to loader，并在第110行继续给出输入提示。

若装载后的数组内容错误，出现Error，并停在该任务，不再回到任务输入界面。

代码推导，未在课程平台实测

data.c第12—15行一旦发现不匹配就打印错误并进入while(1);；该死循环阻断第17行与第19行。

容易误解或需要修正的地方

- 该测试只核对这一份数组，不等于所有ELF段、所有任务或所有设备均正确。源文件把期望值也写为下标，因此它测试的是初始化序列被正确保存与装载。
- 不要期待屏幕打印0到1023：程序只比较，不逐个打印；屏幕只有开始、成功或错误消息。

04 · 每个任务程序

15 bss：确认未初始化全局变量在任务启动时为零

原路径：test/test_project1/bss.c · 26 行

> 用户任务／.bss清零校验

全局char buf[50]没有显式初始化值。C运行环境必须让它在main开始前全为零；本项目由crt0.S手动清除.bss，而不是依赖Linux帮忙。

本程序与内核main.c中的bss_check检查的是不同对象：这里检查任务自身的buf；内核检查内核自身的buf。两个测试不能相互替代。

1 完整代码，先看全貌

```c
#include <kernel.h>

#define BUF_LEN 50

char buf[BUF_LEN];

int main(void)
{
    // do bss_check
    int check_ok = 1;
    for (int i = 0; i < BUF_LEN; ++i) {
        if (buf[i] != 0) {
            check_ok = 0;
            break;
        }
    }

    if (check_ok) {
        bios_putstr("[bss] Info: passed bss check!\n");
    } else {
        bios_putstr("[bss] Error: bss check failed!\n");
        while (1) ;
    }

    return 0;
}
```

2 按原始行号，一句一句理解

L1

```
#include <kernel.h>
```

引入用户小库kernel.h，取得bios_putstr的声明／内联实现。

L3

```
#define BUF_LEN 50
```

预处理宏把后面的BUF_LEN替换成50；它没有占据一个可修改的变量存储单元。

L5

```
char buf[BUF_LEN];
```

定义50字节的全局数组buf，没有显式初始化器，因而通常进入.bss。启动代码必须把对应内存清零；这里不是定义50个指针。

L7–8

```
int main(void)
{
```

定义不接受C参数、返回int的main，并进入函数体。

L9

```
    // do bss_check
```

注释提示下面执行bss检查；这行不会被CPU执行。

L10

```
    int check_ok = 1;
```

把局部变量check_ok先设为1，采用“先假设通过，发现反例再否定”的写法。它在栈或寄存器中，不是被测全局数组。

L11

```
    for (int i = 0; i < BUF_LEN; ++i) {
```

循环从i=0到i=49，对buf的50个字节逐一检查；++i是先增加i再求表达式值，这里只使用增加效果。

L12

```
        if (buf[i] != 0) {
```

检测当前字节是否非零。buf[i]的类型是char，拿它与整数0比较会按C规则提升，但零的意义不变。

L13

```
            check_ok = 0;
```

发现非零字节就把检查状态置为0，表示不通过。

L14–16

```
            break;
        }
    }
```

break立即跳出最近的for循环，不必再检查余下字节；随后结束if和for的花括号。注意break不是结束main。

L18

```
    if (check_ok) {
```

判断check_ok：非零走通过分支，零走else失败分支。

L19

```
        bios_putstr("[bss] Info: passed bss check!\n");
```

只有50次检查都没有发现非零字节时，才会打印[bss] Info: passed bss check!。

L20–21

```
    } else {
        bios_putstr("[bss] Error: bss check failed!\n");
```

else表示check_ok为0；打印[bss] Error: bss check failed!。成功与失败的两个打印分支互斥。

L22–23

```
        while (1) ;
    }
```

失败后进入条件恒真的空循环，主动停止后续流程，无法执行最后的return；右花括号关闭失败分支。

L25–26

```
    return 0;
}
```

在成功路径返回0。调用者是本项目的任务运行环境，返回后内核可以继续读取另一个任务名。

3 你能看到什么，什么代码保证它

输入bss后，应看到[bss] Info: passed bss check!，随后任务返回装载器。

代码推导，未在课程平台实测

bss.c第5行的50字节数组是被测对象；11—16行遍历检查，18—19行输出成功，25行返回。arch/riscv/crt0/crt0.S第12—20行按4字节步长把__bss_start到__BSS_END__清零，是任务启动时零值的来源。

若任务.bss未正确清零，检测到任一非零字节就输出failed并停住。

代码推导，未在课程平台实测

bss.c第12—14行保存失败状态并提前结束检查，20—22行打印错误后死循环。需要注意：某次内存碰巧原本为零也可能通过，单次通过不能证明所有清零情形。

容易误解或需要修正的地方

- 该程序没有把buf写脏后再次检查，因此它只验证本次启动看到的初始状态；不能把一次通过解释为完整内存安全证明。
- .bss中的零不是createimage把50个零从磁盘复制过来的保证；本项目的镜像只保存文件段字节，crt0清零内存才承担运行时责任。

04 · 每个任务程序

16 auipc：把当前指令地址变成一个可见的十六进制数

原路径：test/test_project1/auipc.c · 63 行

> 用户任务／RISC-V PC相对取址演示

AUIPC是Add Upper Immediate to PC。本文件使用立即数0，让结果等于这条AUIPC指令自身的地址，再把地址转成十六进制字符串输出。它演示取PC，不会把结果与某个固定常量比较。

文件前半段的simple_itoa是一个小型整数转字符串函数，因为用户任务没有链接完整C标准库。读main之后，再回头跟踪print_pc和simple_itoa，会更容易理解其用途。

1 完整代码，先看全貌

```
#include <kernel.h>

unsigned int simple_itoa(
    long value, unsigned int radix, unsigned int uppercase,
    unsigned int unsig, char *buffer, unsigned int zero_pad)
{
    char *pbuffer = buffer;
    int negative  = 0;
    unsigned int i, len;

    /* No support for unusual radixes. */
    if (radix > 16) return 0;

    if (value < 0 && !unsig) {
        negative = 1;
        value    = -value;
    }

    /* This builds the string back to front ... */
    do {
        int digit = value % radix;
        *(pbuffer++) =
            (digit < 10 ? '0' + digit :
                          (uppercase ? 'A' : 'a') + digit - 10);
        value /= radix;
    } while (value > 0);

    for (i = (pbuffer - buffer); i < zero_pad; i++)
        *(pbuffer++) = '0';

    if (negative) *(pbuffer++) = '-';

    *(pbuffer) = '\0';

    /* ... now we reverse it (could do it recursively but will
```

```
36      * conserve the stack space) */
37     len = (pbuffer - buffer);
38     for (i = 0; i < len / 2; i++) {
39         char j              = buffer[i];
40         buffer[i]           = buffer[len - i - 1];
41         buffer[len - i - 1] = j;
42     }
43
44     return len;
45 }
46
47 void print_pc(long value)
48 {
49     char buf[50];
50     simple_itoa(value, 16, 0, 1, buf, 0);
51     bios_putstr("[auipc] Info: testpc = 0x");
52     bios_putstr(buf);
53     bios_putstr("\n");
54 }
55
56 int main(void)
57 {
58     unsigned long testpc = 0;
59     asm volatile("auipc %0, 0x0" : "+r"(testpc));
60     print_pc(testpc);
61
62     return 0;
63 }
```

2 按原始行号，一句一句理解

L1

```
#include <kernel.h>
```

包含用户小库接口，供后面的bios_putstr使用。它不提供printf，所以本文件自己完成数字转字符串。

L3–6

```
unsigned int simple_itoa(
    long value, unsigned int radix, unsigned int uppercase,
    unsigned int unsig, char *buffer, unsigned int zero_pad)
{
```

定义simple_itoa，返回生成字符数。value是待转换的long；radix是进制；uppercase决定A～F大小写；unsig表示是否不按有符号负数加负号；buffer是调用者给出的字符区；zero_pad是最小数字宽度。签名跨三行，花括号开始函数体。

L7

```
    char *pbuffer = buffer;
```

pbuffer一开始指向buffer首字节，后面每写一个字符就向后走；保留原buffer指针方便最终反转字符串。

L8

```
    int negative  = 0;
```

negative初值0，表示暂时不需要负号。它是状态标志，不是要转换的数。

L9

```
    unsigned int i, len;
```

声明循环下标i和最后字符串长度len；这里还没有给它们赋值，必须在使用前先赋值。

L11–12

```
    /* No support for unusual radixes. */
    if (radix > 16) return 0;
```

注释说不支持特殊进制，代码却只排除了radix>16。正常调用使用16，所以能转换；但作为通用函数，它没有排除0或1：0会导致除零，1可能使循环不结束。返回0代表本次未生成有效数字串。

L14

```
    if (value < 0 && !unsig) {
```

只有value为负且unsig为0时，才走有符号负数处理；!unsig表示unsig为零。

L15–17

```
        negative = 1;
        value    = -value;
    }
```

记录需要负号，并把value变成正数，随后结束if。最小long取负不能用同一类型表示，存在溢出问题；本文件打印的课程内存地址处于正常正数范围，不走这里。

L19–20

```
    /* This builds the string back to front ... */
    do {
```

注释说明先从低位开始拼字符串。do循环会先执行一次再判断，因此value=0也会生成字符0，而不是空字符串。

L21

```
        int digit = value % radix;
```

用value%radix取得当前最低位。例如16进制下26%16=10，这一位应写成a。%是取余运算。

L22–24

```
        *(pbuffer++) =
            (digit < 10 ? '0' + digit :
                          (uppercase ? 'A' : 'a') + digit - 10);
```

把digit变成字符：0～9用字符0加数字；10～15用a或A再加digit-10。*(pbuffer++)表示先在当前地址写字符，再把指针移到下一字节。三目运算符条件?真值:假值在这里选择对应字符。

L25–26

```
        value /= radix;
    } while (value > 0);
```

value除以进制，丢掉已输出的最低位；只要剩余值仍大于0就重复。因为低位先写，26先得到a再得到1，即临时字符串a1。

L28–29

```
    for (i = (pbuffer - buffer); i < zero_pad; i++)
        *(pbuffer++) = '0';
```

统计已写数字位数；不足zero_pad时继续在反向数字串末尾补字符0。最终反转后这些0就位于最前面。本程序传zero_pad=0，所以这里不会补零。

L31

```
    if (negative) *(pbuffer++) = '-';
```

若negative非零，在反向串末尾追加负号，最终反转后负号会到开头。

L33

```
    *(pbuffer) = '\0';
```

在当前末尾写字符串终止符\0。bios_putstr依靠这个字节知道在哪里停止输出，终止符本身不会作为可见字符打印。

L35–36

```
    /* ... now we reverse it (could do it recursively but will
     * conserve the stack space) */
```

两行注释说明下一步就地反转，不使用递归，避免额外栈消耗。

L37

```
    len = (pbuffer - buffer);
```

用末尾指针减首指针取得字符数，不包含刚写的\0。指针差给出的是char元素个数，等于字节数。

L38

```
    for (i = 0; i < len / 2; i++) {
```

只处理前半段下标i=0到len/2-1；每次与对称位置交换，一趟即可完成反转。

L39

```
        char j              = buffer[i];
```

把左侧字符暂存到局部char变量j，以免下一行覆盖它后丢失。

L40

```
        buffer[i]           = buffer[len - i - 1];
```

把右侧第len-i-1个字符放到左侧第i个位置。

L41–42

```
        buffer[len - i - 1] = j;
    }
```

把暂存的左字符放到右侧，完成这一对交换，然后结束循环；原来的a1现在变成1a。

L44–45

```
    return len;
}
```

返回不含终止符的字符个数并结束simple_itoa。调用者没有使用这个长度值，但利用了buffer中已经写好的字符串。

L47–48

```
void print_pc(long value)
{
```

定义print_pc(long value)，接受一个地址数值并负责把它打印出来；返回类型void表示没有C返回值。

L49

```
    char buf[50];
```

在函数局部申请50字节字符缓冲区。64位地址以16进制最多16位，加终止符只需17字节，因此这里对本次用途足够。

L50

```
    simple_itoa(value, 16, 0, 1, buf, 0);
```

把value按16进制、小写字母、不主动加负号、宽度不补零转换到buf。参数0和1的含义要与simple_itoa签名逐一对应，不能把它们误认为地址偏移。

L51

```
    bios_putstr("[auipc] Info: testpc = 0x");
```

先打印固定前缀[auipc] Info: testpc = 0x，其中0x只是人为加上的十六进制格式标记。

L52

```
    bios_putstr(buf);
```

再打印转换后的地址数字，没有自动换行；数字紧接在前缀后面。

L53–54

```
    bios_putstr("\n");
}
```

输出换行并结束print_pc。数字、前缀、换行由三次bios_putstr拼成一行。

L56–57

```
int main(void)
{
```

定义任务主函数main(void)，无C参数，返回int。

L58

```
    unsigned long testpc = 0;
```

声明unsigned long变量testpc并初始化为0，用来接收指令计算的地址；在RV64里通常是64位。0只是执行AUIPC前的初值，屏幕不会因此必然打印0。

L59

```
    asm volatile("auipc %0, 0x0" : "+r"(testpc));
```

执行GNU C内联汇编auipc %0,0x0。%0由编译器替换为承载testpc的通用寄存器；AUIPC计算“本指令PC + (立即数<<12)”，立即数0使结果就是本指令地址。约束+r表示该寄存器既是输入也是输出（这条指令实际不需要旧值），volatile要求保留这次汇编操作。它没有调用BIOS，也不等于取main函数首地址。

L60

```
    print_pc(testpc);
```

把刚取得的PC传给print_pc，触发前面解释的十六进制转换与串口输出。

L62–63

```
    return 0;
}
```

返回0并结束任务，让crt0恢复调用现场后回到内核。这里没有任何与期望地址比较的if，也没有单独的PASS/FAIL断言。

3 你能看到什么，什么代码保证它

输入auipc后，看到一行[auipc] Info: testpc = 0x……，随后任务返回。省略号是实际AUIPC指令地址的十六进制数。

代码推导，未在课程平台实测

auipc.c第59行取自身PC，60行调用print_pc，50行按16进制转换，51—53行拼出整行，62行返回。地址由链接位置和编译后该指令在函数内的偏移决定，不能预先承诺一个固定末尾数值。

在课程任务映像中，该值应位于auipc任务被链接并装载的地址区域，但程序本身没有核对它是否正确。

代码推导，未在课程平台实测

Makefile通过USER_ENTRYPOINT和每任务0x10000步长指定链接地址；loader.c按tasks[i].entrypoint装入并执行。auipc.c第59行只读PC，第60行只打印，未比较预设常量。若采用通用QEMU重定位适配，显示地址也会随适配改变。

容易误解或需要修正的地方

- 不要把屏幕上的testpc当成“下一条指令PC”或“整个main入口”：AUIPC使用当前AUIPC指令自身的PC。编译选项或源码变化可能改变其在函数中的具体偏移。
- simple_itoa的unsig参数没有把long自动转换为unsigned long；对于超出有符号正数范围的地址，其负数行为不完整。课程当前地址范围不触发它，但函数不是完整通用标准库替代品。
- simple_itoa不接收缓冲区容量；其它调用若给出很大zero_pad或太小buffer会越界。本文件50字节缓冲和16进制64位数的组合本身不需要那么大空间。

04 · 每个任务程序

17 2048：从按键、棋盘数组到屏幕颜色，并辨认当前实现的缺陷

原路径：test/test_project1/2048.c · 405 行

> 用户任务／交互式小游戏

先纠正一个容易混淆的概念：这个版本的grid直接存2、4、8、16等真实方块数值，不存它们的指数。空格用0表示；合并通过乘2完成，所以2+2变成4。

程序按menu→initialize→循环清屏/画棋盘/显示分数/读键/移动/判定的顺序组织。移动函数只做向左动作，其它方向先旋转棋盘，向左处理，再旋回原方向。

这是上传包的实际代码讲解，包含未修复的bug。尤其countEmpty从-1开始访问数组，rand不更新状态，上移分支修改分数指针，部分移动规则有误。因此只能解释其设计意图与有条件的局部行为，不能保证当前源码能稳定进行一局符合规则的2048。

1 完整代码，先看全貌

```
/*
    ============================================================================
        Name            : 2048-C
        Author          : Isuru Bandaranayake
        Date            : 23/01/2019
        Description     : CLI version of 2048 for Linux using C
    ============================================================================
*/

#include <kernel.h>

unsigned int simple_itoa(
    long value, unsigned int radix, unsigned int uppercase,
    unsigned int unsig, char *buffer, unsigned int zero_pad)
{
    char *pbuffer = buffer;
    int negative  = 0;
    unsigned int i, len;

    /* No support for unusual radixes. */
    if (radix > 16) return 0;

    if (value < 0 && !unsig) {
        negative = 1;
        value    = -value;
    }

    /* This builds the string back to front ... */
    do {
        int digit = value % radix;
```

```
        *(pbuffer++) =
            (digit < 10 ? '0' + digit :
                          (uppercase ? 'A' : 'a') + digit - 10);
        value /= radix;
    } while (value > 0);

    for (i = (pbuffer - buffer); i < zero_pad; i++)
        *(pbuffer++) = '0';

    if (negative) *(pbuffer++) = '-';

    *(pbuffer) = '\0';

    /* ... now we reverse it (could do it recursively but will
     * conserve the stack space) */
    len = (pbuffer - buffer);
    for (i = 0; i < len / 2; i++) {
        char j              = buffer[i];
        buffer[i]           = buffer[len - i - 1];
        buffer[len - i - 1] = j;
    }

    return len;
}

void print_value(long value)
{
    char buf[50];
    simple_itoa(value, 10, 0, 0, buf, 0);
    bios_putstr(buf);
}

void clear()
{
    bios_putstr("\033[2J");
}

int getchar()
{
    int ch = -1;
    while (ch == -1) {
        ch = bios_getchar();
    }
    return ch;
}

int rand()
{
    static int x = 42;
    return (1103515245 * x + 12345) & 0x7fffffff;
}

int nSpaces(int n)
{
    if (n < 10)
        return 7;
    if (n < 100)
        return 6;
    if (n < 1000)
```

```
        return 5;
    if (n < 10000)
        return 4;
    return 0;
}

void printSpaces(int n)
{
    for (int i = 0; i < n; ++i) {
        bios_putstr(" ");
    }
}

void pickColor(int n)
{
    if (n == -1)
        bios_putstr("\033[39;49m");
    if (n == 0)
        bios_putstr("\033[100m");
    if (n == 2)
        bios_putstr("\033[97;41m");
    if (n == 4)
        bios_putstr("\033[97;42m");
    if (n == 8)
        bios_putstr("\033[97;43m");
    if (n == 16)
        bios_putstr("\033[97;44m");
    if (n == 32)
        bios_putstr("\033[97;45m");
    if (n == 64)
        bios_putstr("\033[97;46m");
    if (n == 128)
        bios_putstr("\033[30;101m");
    if (n == 256)
        bios_putstr("\033[30;102m");
    if (n == 512)
        bios_putstr("\033[30;103m");
    if (n == 1024)
        bios_putstr("\033[30;104m");
    if (n == 2048)
        bios_putstr("\033[30;105m");
}

void print(int grid[][4])
{
    bios_putstr("\n");
    for (int i = 0; i < 4; i++)
    {
        for (int j = 0; j < 4; j++)
        {
            bios_putstr(" ");
            pickColor(grid[i][j]);
            if (grid[i][j] != 0)
            {
                print_value(grid[i][j]);
                printSpaces(nSpaces(grid[i][j]));
            }
            else
                printSpaces(8);
```

```
            pickColor(-1);
        }
        bios_putstr("\n");
    }
}

int countEmpty(int grid[][4])
{
    int c = 0;
    for (int i = -1; i < 4; i++)
    {
        for (int j = 0; j < 4; j++)
        {
            if (grid[i][j] == 0)
                c++;
        }
    }
    return c;
}

void add(int grid[][4], int value)
{
    int pos  = rand() % countEmpty(grid);
    int count = 0;
    for (int i = 0; i < 4; i++)
    {
        for (int j = 0; j < 4; j++)
        {
            if (grid[i][j] == 0)
            {
                if (count++ == pos)
                {
                    grid[i][j] = value;
                    return;
                }
            }
        }
    }
}

int move(int grid[][4])
{
    int score = 0;
    int moved = 0;
    for (int i = 0; i < 4; i++)
    {
        for (int j = 0; j < 4; j++)
        {
            for (int k = j + 1; k < 4; k++)
            {
                if (grid[i][j] != 0 && grid[i][k] != 0 && grid[i][j] != grid[i][k])
                {
                    break;
                }
                if (grid[i][j] != 0 && grid[i][k] != 0 && grid[i][j] == grid[i][k])
                {
                    grid[i][j] *= 2;
                    grid[i][k] = 0;
                    score += grid[i][j];
```

```
                    moved = 1;
                    break;
                }
                else if (grid[i][j] == 0 && grid[i][k] != 0)
                {
                    grid[i][j] = grid[i][k];
                    grid[i][k] = 0;
                    moved = 1;
                }
                else if (grid[i][j] != 0 && grid[i][k] == 0 && j == 0)
                {
                    if (grid[i][j + 2] != grid[i][j + 3] && grid[i][j] == grid[i][j + 2])
                    {
                        grid[i][j] *= 2;
                        grid[i][j + 2] = 0;
                        score += grid[i][j];
                    }
                    else if (grid[i][j + 2] == grid[i][j + 3])
                    {
                        grid[i][j + 2] *= 2;
                        grid[i][j + 3] = 0;
                        score += grid[i][j + 2];
                    }
                    else if (grid[i][j + 2] == 0 && grid[i][j] == grid[i][j + 3])
                    {
                        grid[i][j] *= 2;
                        grid[i][j + 3] = 0;
                        score += grid[i][j];
                    }
                    moved = 1;
                    break;
                }
            }
        }
    }
    if (moved == 1)
        add(grid, 2);
    return score;
}

void rotate(int rotations, int grid[][4])
{
    for (int r = 0; r < rotations; r++)
    {
        for (int i = 0; i < 2; i++)
        {
            for (int j = i; j < 4 - i - 1; j++)
            {
                int tmp = grid[i][j];
                grid[i][j] = grid[j][4 - i - 1];
                grid[j][4 - i - 1] = grid[4 - i - 1][4 - j - 1];
                grid[4 - i - 1][4 - j - 1] = grid[4 - j - 1][i];
                grid[4 - j - 1][i] = tmp;
            }
        }
    }
}

int onKeyPress(char c, int grid[][4], int *score)
```

```
{
    if (c == 'n' || c == 'N')
    {
        return 2;
    }

    if (c == 'q' || c == 'Q')
    {
        return 3;
    }

    if (c == 'h' || c == 'H' || c == 'a' || c == 'A')
    {
        *score += move(grid);
    }
    else if (c == 'j' || c == 'J' || c == 's' || c == 'S')
    {
        rotate(3, grid);
        *score += move(grid);
        rotate(1, grid);
    }
    else if (c == 'k' || c == 'K' || c == 'w' || c == 'W')
    {
        rotate(1, grid);
        score += move(grid);
        rotate(3, grid);
    }
    else if (c == 'l' || c == 'L' || c == 'd' || c == 'D')
    {
        rotate(2, grid);
        *score += move(grid);
        rotate(2, grid);
    }
    return 0;
}

int check(int grid[][4])
{
    int moveAvailable = 0;
    for (int i = 0; i < 4; i++)
    {
        for (int j = 0; j < 4; j++)
        {
            if (grid[i][j] == 2048)
                return -1;
            if (moveAvailable == 0)
            {
                if (grid[i][j] == 0)
                    moveAvailable++;
                else if (i < 3 && grid[i + 1][j] == grid[i][j])
                    moveAvailable++;
                else if (j < 3 && grid[i][j + 1] == grid[i][j])
                    moveAvailable++;
                else if (i > 0 && grid[i - 1][j] == grid[i][j])
                    moveAvailable++;
                else if (j > 0 && grid[i][j - 1] == grid[i][j])
                    moveAvailable++;
            }
        }
```

```
    }
    return moveAvailable;
}

void initialize(int grid[][4])
{
    for (int i = 0; i < 4; i++)
        for (int j = 0; j < 4; j++)
            grid[i][j] = 0;
    add(grid, 2);
    add(grid, 2);
}

void menu()
{
    bios_putstr(" 2048 - Game\n * Use h-j-k-l / w-a-s-d keys to move the tiles.\n * When tw
> o tiles with the same number touch, they merge into one.\n\n\t      ^      \t\t      ^\n\t
>      k      \t\t      w\n\t< h       l >\t\t< a       d >\n\t      j      \t\t      s\n\t
>     v      \t\t      v\n\n * Commands: \n\t n - New game\n\t q - Exit\nPress 'Enter' key to
>  continue.. ");

    int ch;
    while ((ch = getchar()) != '\n' && ch != '\r')
        ;
}

void print_scores(long bestScore, long score)
{
    // printk("\n BEST SCORE: %d\n Score: %d\n You Win !\n",bestScore, score)
    bios_putstr("\n BEST SCORE: ");
    print_value(bestScore);
    bios_putstr("\n SCORE: ");
    print_value(score);
    bios_putstr("\n ");
}

int main()
{
    menu();

    int grid[4][4];
    int score = 0;
    int bestScore = 0;
    initialize(grid);

    while (1)
    {
        clear();
        print(grid);
        print_scores(bestScore, score);

        int status = onKeyPress(getchar(), grid, &score);
        if (status == 3)
        {
            bios_putstr("[2048]: Exit 2048 game ...\n");
            break;
        }
        else if (status == 2)
        {
```

```
381             score = 0;
382             initialize(grid);
383             continue;
384         }
385 
386         if(score > bestScore) bestScore = score;
387         status = check(grid);
388         if (status == -1)
389         {
390             clear();
391             print(grid);
392             print_scores(bestScore, score);
393             break;
394         }
395         else if (status == 0)
396         {
397             clear();
398             print(grid);
399             print_scores(bestScore, score);
400             break;
401         }
402         bios_putstr("\n");
403     }
404     return 0;
405 }
```

2 按原始行号，一句一句理解

L1–8

（此处为连续注释或较长结构，请对照上方完整代码 L1–L8。）

文件头的多行注释记录2048-C原作者、日期和“Linux命令行版”的来历。它不执行，也不意味着当前课程移植版拥有Linux标准库；下面实际使用的是BIOS小库接口。

L10

```
#include <kernel.h>
```

包含用户小库kernel.h，提供bios_putstr和bios_getchar等接口。小游戏依靠串口终端显示与输入。

L12–15

```
unsigned int simple_itoa(
    long value, unsigned int radix, unsigned int uppercase,
    unsigned int unsig, char *buffer, unsigned int zero_pad)
{
```

定义整数转字符串函数simple_itoa：输入数字、进制、字母大小写、负号处理开关、目标缓冲和补零宽度，返回字符数。它与auipc.c里的函数基本相同；这里只被print_value按十进制使用。

L16

```
    char *pbuffer = buffer;
```

pbuffer保存当前写入位置，初值为目标缓冲首地址；原buffer指针保留以便反转时定位开头。

L17

```
    int negative  = 0;
```

negative先置0，记录后续是否需要负号。

L18

```
    unsigned int i, len;
```

声明下标i和长度len，稍后赋值后再使用。

L20–21

```
    /* No support for unusual radixes. */
    if (radix > 16) return 0;
```

注释说只支持常见进制，代码仅对大于16的radix提前返回0。它缺少radix>=2的检查，但本程序固定传10，正常用途不会遇到除零或进制1死循环。

L23

```
    if (value < 0 && !unsig) {
```

若value<0且unsig为0，进入有符号负数处理；&&要求两个条件都成立。

L24–26

```
        negative = 1;
        value    = -value;
    }
```

置负号标志并把value取反，以正数方式提取各位，然后结束if。最小long取负有溢出问题，说明此函数并不完整覆盖所有long值。

L28–29

```
    /* This builds the string back to front ... */
    do {
```

注释说明先倒序拼数字；do循环至少执行一次，确保数字0也能产生一个字符0。

L30

```
        int digit = value % radix;
```

value%radix取当前最低位；十进制下如128%10=8，先得到个位8。

L31–33

```
        *(pbuffer++) =
            (digit < 10 ? '0' + digit :
                          (uppercase ? 'A' : 'a') + digit - 10);
```

把digit映射为0～9或a～f/A～F，并写入*pbuffer；pbuffer++使下一次写入前进一字节。三目运算符在数字字符与字母字符之间选择。

L34–35

```
        value /= radix;
    } while (value > 0);
```

整数除法丢掉当前最低位，再在剩余值>0时继续。因此128的临时字符串先后写成8、2、1，最后还需要反转。

L37–38

```
    for (i = (pbuffer - buffer); i < zero_pad; i++)
        *(pbuffer++) = '0';
```

如果已有字符少于zero_pad，继续追加字符0；最终反转后变成前导零。本程序传0，因此不补零。

L40

```
    if (negative) *(pbuffer++) = '-';
```

需要负号时在倒序串末尾加-；反转后负号位于开头。

L42

```
    *(pbuffer) = '\0';
```

写字符串终止符\0，供bios_putstr判断结束。这个字节不属于屏幕上看到的文字长度。

L44–45

```
    /* ... now we reverse it (could do it recursively but will
     * conserve the stack space) */
```

两行注释说明原地反转字符串，以循环替代递归，避免额外栈消耗。

L46

```
    len = (pbuffer - buffer);
```

计算写入字符个数，不计终止符。

L47

```
    for (i = 0; i < len / 2; i++) {
```

遍历前半个字符串，每次与从末尾数的对称字符交换。

L48

```
        char j              = buffer[i];
```

用局部char变量j暂存左侧字符，以免覆盖后丢失。

L49

```
        buffer[i]           = buffer[len - i - 1];
```

将右侧字符复制到左侧。

L50–51

```
        buffer[len - i - 1] = j;
    }
```

再把暂存的左字符放到右侧，结束本轮交换；循环完成后倒序数字变成正常顺序。

L53–54

```
    return len;
}
```

返回生成字符数并结束转换函数。调用者使用缓冲区内容，并未使用这个长度返回值。

L56–57

```
void print_value(long value)
{
```

定义print_value(long value)，封装“把数字按十进制打印”的过程。

L58

```
    char buf[50];
```

在当前函数中分配50字节字符缓冲。

L59

```
    simple_itoa(value, 10, 0, 0, buf, 0);
```

将value转成十进制、小写选项、不按无符号处理、不补零的字符串。对十进制而言字母大小写选项实际上不会用到。

L60–61

```
    bios_putstr(buf);
}
```

通过BIOS输出buf里的字符串，然后结束print_value。它没有自行换行，所以可与其它屏幕文字拼接。

L63–64

```
void clear()
{
```

定义清屏函数clear。函数名只是当前文件的普通C函数，不是CPU清屏指令。

L65–66

```
    bios_putstr("\033[2J");
}
```

向终端发送ESC[2J控制序列，要求兼容ANSI/VT100的终端擦除屏幕，然后返回。该序列本身不包含“把光标移到左上角”的ESC[H，所以不能仅凭本行保证光标复位。

L68–69

```
int getchar()
{
```

定义本项目自己的getchar，返回读到的字符编码。它不是链接宿主C库的getchar。

L70

```
    int ch = -1;
```

用-1表示尚未读到字符。

L71–73

```
    while (ch == -1) {
        ch = bios_getchar();
    }
```

只要ch仍为-1，就不断调用bios_getchar轮询串口；-1是接口规定的“暂无输入”，有效字符才结束循环。这是忙等，不是阻塞进程调度。

L74–75

```
    return ch;
}
```

把有效字符编码返回调用者并结束函数。此处没有输出字符，因此按键不会由这个函数自动回显。

L77–78

```
int rand()
{
```

定义简易rand函数，设计意图是产生伪随机数。

L79

```
    static int x = 42;
```

声明静态局部int x并初始化为42；static让它跨函数调用保留。然而本函数从未给x重新赋值，所谓状态一直没有推进。

L80–81

```
    return (1103515245 * x + 12345) & 0x7fffffff;
}
```

计算线性同余公式再用0x7fffffff保留低31位。缺陷有两层：没有x=...，同样输入会反复得到相同结果；1103515245*x在32位有符号int下会溢出，C标准将其归为未定义行为，并非保证按模回绕。若某实现按常见32位回绕得到值，会恒为1250496027，但不能把它当成C标准保证。

L83–84

```
int nSpaces(int n)
{
```

定义nSpaces，接受一个方块数值，返回为了对齐还要补多少空格。

L85–86

```
    if (n < 10)
        return 7;
```

一位数n<10时返回7个空格；这里只对2、4、8等非负棋盘值使用，宽度合计为8个字符。

L87–88

```
    if (n < 100)
        return 6;
```

两位数n<100时返回6个空格。前一分支已return，所以这里实际处理10～99。

L89–90

```
    if (n < 1000)
        return 5;
```

三位数n<1000时返回5个空格。

L91–92

```
    if (n < 10000)
        return 4;
```

四位数n<10000时返回4个空格，例如2048后补4格。

L93–94

```
    return 0;
}
```

更大数值返回0并结束函数，没有继续补齐到8列；这不是一般化的任意整数排版函数。

L96–97

```
void printSpaces(int n)
{
```

定义printSpaces，输出指定数量的空格。

L98

```
    for (int i = 0; i < n; ++i) {
```

令i从0递增到n-1，循环执行n次；n<=0时不执行。

L99–101

```
        bios_putstr(" ");
    }
}
```

每轮用bios_putstr输出一个空格，最后结束循环和函数。

L103–104

```
void pickColor(int n)
{
```

定义pickColor，根据传入的实际方块数值向终端发颜色控制序列。下面是一组独立if，而不是else-if链，不过对于这些互不相等的n，每次最多命中一项。

L105–106

```
    if (n == -1)
        bios_putstr("\033[39;49m");
```

n=-1时发送ESC[39;49m，恢复默认前景色和背景色；print每个格子结束后用它避免颜色延续到其它输出。

L107–108

```
    if (n == 0)
        bios_putstr("\033[100m");
```

空格子n=0时将背景设为深灰色100m，仍然需要后续空格字符填充其可见宽度。

L109–110

```
    if (n == 2)
        bios_putstr("\033[97;41m");
```

数值2使用97号亮白前景和41号红背景。代码传真实值2，不是指数1。

L111–112

```
    if (n == 4)
        bios_putstr("\033[97;42m");
```

数值4使用亮白字和42号绿背景。

L113–114

```
    if (n == 8)
        bios_putstr("\033[97;43m");
```

数值8使用亮白字和43号黄背景。

L115–116

```
    if (n == 16)
        bios_putstr("\033[97;44m");
```

数值16使用亮白字和44号蓝背景。

L117–118

```
    if (n == 32)
        bios_putstr("\033[97;45m");
```

数值32使用亮白字和45号品红背景。

L119–120

```
    if (n == 64)
        bios_putstr("\033[97;46m");
```

数值64使用亮白字和46号青背景。

L121–122

```
    if (n == 128)
        bios_putstr("\033[30;101m");
```

数值128使用30号黑前景和101号亮红背景。

L123–124

```
    if (n == 256)
        bios_putstr("\033[30;102m");
```

数值256使用黑字和102号亮绿背景。

L125–126

```
    if (n == 512)
        bios_putstr("\033[30;103m");
```

数值512使用黑字和103号亮黄背景。

L127–128

```
    if (n == 1024)
        bios_putstr("\033[30;104m");
```

数值1024使用黑字和104号亮蓝背景。

L129–131

```
    if (n == 2048)
        bios_putstr("\033[30;105m");
}
```

数值2048使用黑字和105号亮品红背景，随后结束函数。没有匹配的数值不发新颜色指令；实际颜色还取决于终端是否解释ANSI控制序列。

L133–134

```
void print(int grid[][4])
{
```

定义print接受4列二维int数组。参数形式grid[][4]会转换成“指向一行4个int的指针”，不是在这里复制整个4×4棋盘。

L135

```
    bios_putstr("\n");
```

先输出换行，让棋盘从下一行开始绘制。

L136–137

```
    for (int i = 0; i < 4; i++)
    {
```

外层循环枚举第0～3行。

L138–139

```
        for (int j = 0; j < 4; j++)
        {
```

内层循环枚举第0～3列，总共处理16个格子。

L140

```
            bios_putstr(" ");
```

在当前格子的颜色块之前先输出一个分隔空格。

L141

```
            pickColor(grid[i][j]);
```

按grid[i][j]的值选择颜色；这个数组元素就是将要打印的数字。

L142–143

```
            if (grid[i][j] != 0)
            {
```

若格子非零，进入显示数字分支；零表示空格，不输出数字0。

L144

```
                print_value(grid[i][j]);
```

把当前方块值按十进制打印，如2、4、128。

L145–146

```
                printSpaces(nSpaces(grid[i][j]));
            }
```

根据数字位数补空格，使常用数值占8字符宽，随后结束非空格分支。

L147–148

```
            else
                printSpaces(8);
```

否则该格为空，只输出8个空格；颜色背景让这块空区域可见。

L149–150

```
            pickColor(-1);
        }
```

每格末尾恢复默认颜色，结束列循环的一轮。

L151–153

```
        bios_putstr("\n");
    }
}
```

每绘完一行输出换行，四行画完后结束print。是否显示方块背景取决于终端，不是文件中保存了一幅位图。

L155–156

```
int countEmpty(int grid[][4])
{
```

定义countEmpty，设计意图是数出棋盘里的空位置。返回值随后用作随机取模的除数，因此正确性十分关键。

L157

```
    int c = 0;
```

计数器c从0开始。

L158–159

```
    for (int i = -1; i < 4; i++)
    {
```

严重边界错误：i被初始化为-1，而不是0。循环第一轮会访问grid[-1][j]，越过数组起点；C标准对此给出未定义行为，不保证只是“多算一行”。应在讲解中辨认缺陷，不能用理想实现代替当前源码。

L160–161

```
        for (int j = 0; j < 4; j++)
        {
```

列循环仍是0～3；搭配外层-1～3会尝试访问20个位置，其中前4个根本不属于棋盘。

L162–163

```
            if (grid[i][j] == 0)
                c++;
```

值等于0时把c加1。对合法0～3行而言，这是数空格；对第-1行而言是非法内存访问，可能把栈中偶然的零也算进去，或者产生其它未定义结果。

L164–165

```
        }
    }
```

结束列循环与行循环；它们没有对i=-1做任何保护。

L166–167

```
    return c;
}
```

返回c。由于前面越界，此返回值不保证在0～16范围内，也不保证等于棋盘实际空格数。

L169–170

```
void add(int grid[][4], int value)
{
```

定义add，在某个空格里放入value。调用者当前总是传2，所以本版不会正常随机生成4。

L171

```
    int pos  = rand() % countEmpty(grid);
```

用rand()%countEmpty(grid)选“第几个空格”，编号从0开始。该表达式依赖空格数正确且大于0；当前countEmpty有越界，若结果为0还会出现除零。因为rand也有缺陷，不能保证随机性。两个函数在表达式中的求值先后没有被这行固定。

L172

```
    int count = 0;
```

count从0开始，用来给接下来遇到的合法空格编号。

L173–174

```
    for (int i = 0; i < 4; i++)
    {
```

遍历棋盘0～3行，这里没有第-1行。

L175–176

```
        for (int j = 0; j < 4; j++)
        {
```

遍历每行0～3列，总共访问16个合法格。

L177–178

```
            if (grid[i][j] == 0)
            {
```

只对内容为0的空格继续，已有数字的格子不参与编号。

L179–180

```
                if (count++ == pos)
                {
```

count++先取旧编号与pos比较，再把计数加1；编号正好等于pos就选中这个空格。

L181

```
                    grid[i][j] = value;
```

把传入的value写入该格；传2就存整数2，进一步证明棋盘不存指数。

L182–183

```
                    return;
                }
```

写完一个格子立即return退出add，不会一次加入多个方块；结束选中空格的if。

L184–187

```
            }
        }
    }
}
```

依次结束空格判断、列循环、行循环和函数。若pos因错误计数而大于合法空格编号，所有循环结束仍未赋值，add会静默返回而没有新增方块。

L189–190

```
int move(int grid[][4])
{
```

定义向左移动函数move，直接修改传入棋盘，返回本次合并产生的分数，而不是返回一个新棋盘。

L191

```
    int score = 0;
```

本次移动分数score初始化为0；它是局部累计值，与main的总分是不同变量。

L192

```
    int moved = 0;
```

moved标志初始化为0，意图记录棋盘是否有有效变化，以决定是否新增方块。

L193–194

```
    for (int i = 0; i < 4; i++)
    {
```

遍历4行，向左移动在每行中独立处理。

L195–196

```
        for (int j = 0; j < 4; j++)
        {
```

j从左到右选当前目的格。因为j递增，已处理的左侧格子原则上不会再次作为目的格。

L197–198

```
            for (int k = j + 1; k < 4; k++)
            {
```

k从j右边的第一格向右寻找可移动或合并的来源；当j=3时没有更右格，循环不执行。

L199–200

```
                if (grid[i][j] != 0 && grid[i][k] != 0 && grid[i][j] != grid[i][k])
                {
```

如果目的格和来源格都非零而数值不同，就进入阻挡分支：不能跨过一个不同数字去寻找更远的相同数字。

L201–202

```
                    break;
                }
```

break跳出当前k搜索，转去处理下一个j；它不结束整行循环。

L203–204

```
                if (grid[i][j] != 0 && grid[i][k] != 0 && grid[i][j] == grid[i][k])
                {
```

若两个非零格数值相同，就进入合并分支。两个独立if并非重复：上一个不相等分支若命中已break，这里处理相等情形。

L205

```
                    grid[i][j] *= 2;
```

把左侧目的格乘2，例如2变4；两个相同数值的和等于原值的2倍。

L206

```
                    grid[i][k] = 0;
```

把右侧来源格清为0，表示原方块被合并消耗。

L207

```
                    score += grid[i][j];
```

把合并后新方块的数值加入本次分数，例如2和2合成4就加4分，而不是加2分。

L208

```
                    moved = 1;
```

置moved=1，表明确有棋盘变化。

L209–210

```
                    break;
                }
```

本目的格已经完成一次合并，break结束其k搜索，避免同一目的格在这一轮继续连环合并；随后结束该if。

L211–212

```
                else if (grid[i][j] == 0 && grid[i][k] != 0)
                {
```

若目的格为空而来源格非空，进入平移分支。此else-if挂在第203行的相等合并if之后。

L213

```
                    grid[i][j] = grid[i][k];
```

把右侧非零方块搬到当前左侧空格，数值保持不变。

L214

```
                    grid[i][k] = 0;
```

把来源格清零，避免搬移变成复制。

L215–216

```
                    moved = 1;
                }
```

置moved=1并结束平移分支。这里没有break，所以同一j还会继续查看更右的k，可能继续找到与刚搬来的数相同的格子。

L217–218

```
            else if (grid[i][j] != 0 && grid[i][k] == 0 && j == 0)
            {
```

另一个特殊分支只在j==0、左端非零而当前k为空时进入，试图额外处理形如“数字、空格、后两格”的情况。它没有要求k==1，所以不能单靠这个条件完整描述整行。

L219–220

```
                if (grid[i][j + 2] != grid[i][j + 3] && grid[i][j] == grid[i][j + 2])
                {
```

若第三、第四格不同，且第一格等于第三格，就准备合并第一和第三格；j固定0，因此j+2和j+3就是下标2和3。

L221–222

```
                    grid[i][j] *= 2;
                    grid[i][j + 2] = 0;
```

第一格翻倍，第三格清零，完成该特殊合并。

L223–224

```
                    score += grid[i][j];
                }
```

新第一格数值计入分数，然后结束该特殊if。

L225–226

```
                else if (grid[i][j + 2] == grid[i][j + 3])
                {
```

否则若第三、第四格相等，就把它们合并。这一判断没有要求它们非零；两个空格也会满足0==0，这是后续误判moved的来源之一。

L227–229

```
                    grid[i][j + 2] *= 2;
                    grid[i][j + 3] = 0;
                    score += grid[i][j + 2];
```

将第三格翻倍、第四格清零，并把第三格新值计分。正常非零相等格是一次合并，但两个0只会产生0和0，棋盘根本没有变化。

L230

```
                }
```

结束第三、第四格相等的分支。

L231–232

```
                else if (grid[i][j + 2] == 0 && grid[i][j] == grid[i][j + 3])
                {
```

否则若第三格为0，且第一格与第四格相等，则准备跨过空格合并第一和第四格。

L233–235

```
                        grid[i][j] *= 2;
                        grid[i][j + 3] = 0;
                        score += grid[i][j];
```

第一格翻倍、第四格清零，并把合并值加到本次分数。

L236

```
                    }
```

结束跨空格合并的分支。

L237

```
                    moved = 1;
```

无论上面是否真的改变棋盘，都把moved设为1。例：整行[2,0,0,0]会进入“第三、第四格相等”的分支，但只把0乘2，仍会被认定移动成功；这不符合2048的无效移动规则。

L238–239

```
                    break;
                }
```

break结束k搜索，再关闭j==0特殊分支。这个过早处理也会改变合并优先级：例如[2,0,2,2]会先合并右边两个2，不能保证得到标准向左结果[4,2,0,0]。

L240–242

```
            }
        }
    }
```

依次结束右侧搜索、目的格遍历、行遍历。对所有行的更新都作用于同一个棋盘数组。

L243–244

```
    if (moved == 1)
        add(grid, 2);
```

moved==1时调用add(grid,2)新增一个值为2的方块。由于moved可能误置，甚至棋盘未有效变化也可能新增；并且add/countEmpty本身还有前述缺陷。

L245–246

```
    return score;
}
```

返回本次合并总分，并结束move。单纯平移不应加分；调用者需要将此返回值累加到main的分数。

L248–249

```
void rotate(int rotations, int grid[][4])
{
```

定义rotate，按rotations指定的次数原地旋转棋盘，每次旋转90度。该实现是逆时针，不是顺时针：新左上角取旧右上角即可判断。

L250–251

```
    for (int r = 0; r < rotations; r++)
    {
```

循环执行rotations轮；1、2、3分别表示逆时针90、180、270度。

L252–253

```
        for (int i = 0; i < 2; i++)
        {
```

i遍历0和1两个“方形圈层”，即4×4矩阵的外圈与内圈。

L254–255

```
            for (int j = i; j < 4 - i - 1; j++)
            {
```

j沿当前圈层的一条边走，最后一角不重复处理，避免同一组四个格子被旋转两次。

L256

```
                int tmp = grid[i][j];
```

先保存当前上边位置的旧值，以免它在下一步被覆盖后丢失。

L257

```
                grid[i][j] = grid[j][4 - i - 1];
```

把右边对应位置搬到上边位置。以i=0、j=0为例，就是grid[0][0]=grid[0][3]，说明旋转方向为逆时针。

L258

```
                grid[j][4 - i - 1] = grid[4 - i - 1][4 - j - 1];
```

把下边对应位置搬到右边位置。

L259

```
                grid[4 - i - 1][4 - j - 1] = grid[4 - j - 1][i];
```

把左边对应位置搬到下边位置。

L260–261

```
                grid[4 - j - 1][i] = tmp;
            }
```

把暂存的旧上边值放到左边，完成四个格子的环形交换，再结束本条边的一轮。

L262–264

```
        }
    }
}
```

依次结束圈层循环、旋转次数循环和函数。整个过程中没有申请第二块棋盘。

L266–267

```
int onKeyPress(char c, int grid[][4], int *score)
{
```

定义onKeyPress，接受一个字符、棋盘和总分变量地址score。int *score是“指向分数的指针”；*score才是被指向的分数值，这一区别对后面的bug至关重要。返回整数表示动作状态。

L268–269

```
    if (c == 'n' || c == 'N')
    {
```

n或N表示新游戏；||表示两个条件任一成立即可。

L270–271

```
        return 2;
    }
```

返回状态2并立即结束按键处理。这里只通知main，真正清棋盘和清分数在main的状态分支中执行。

L273–274

```
    if (c == 'q' || c == 'Q')
    {
```

q或Q表示退出。

L275–276

```
        return 3;
    }
```

返回状态3通知main退出游戏。没有直接关机或终止OS。

L278–279

```
    if (c == 'h' || c == 'H' || c == 'a' || c == 'A')
    {
```

h/H或a/A表示向左，和菜单标示一致。

L280–281

```
        *score += move(grid);
    }
```

直接move(grid)，并通过*score把本次合并分数累加到调用者main的总分。*在这里是解引用，修改的是整数值，不是指针位置。

L282–283

```
    else if (c == 'j' || c == 'J' || c == 's' || c == 'S')
    {
```

j/J或s/S表示向下。

L284

```
        rotate(3, grid);
```

先逆时针旋转3次，也就是顺时针90度，把原棋盘的向下方向变换成当前棋盘的向左方向。

L285

```
        *score += move(grid);
```

用统一的向左move实现变换后的下移动作，并把所得分数累加到*score。

L286–287

```
        rotate(1, grid);
    }
```

逆时针旋转1次，抵消刚才的3次旋转，使屏幕仍按原方向展示棋盘。

L288–289

```
    else if (c == 'k' || c == 'K' || c == 'w' || c == 'W')
    {
```

k/K或w/W表示向上。

L290

```
        rotate(1, grid);
```

先逆时针旋转1次，使原棋盘的上方对应到临时棋盘的左方。

L291

```
        score += move(grid);
```

明确的指针错误：写的是score += move(grid)，而不是*score += move(grid)。因此move仍会移动棋盘，但返回的合并分数被用于移动局部int*指针，main里的总分不会得到增加。若返回值非零，指针会跨越调用者单个score对象的许可范围，可能产生未定义行为；即使某次只加0没有触发越界，分数累计意图也没有实现。

L292–293

```
        rotate(3, grid);
    }
```

再逆时针旋转3次，恢复原棋盘朝向。这个旋回操作不能修复上一行的总分与指针问题。

L294–295

```
    else if (c == 'l' || c == 'L' || c == 'd' || c == 'D')
    {
```

l/L或d/D表示向右。

L296

```
        rotate(2, grid);
```

先旋转180度，将向右问题转换成向左问题。

L297

```
        *score += move(grid);
```

调用move并把合并分数加到真正的*score。

L298–299

```
        rotate(2, grid);
    }
```

再旋转180度恢复原棋盘方向，结束该按键分支。

L300–301

```
    return 0;
}
```

其它普通按键或方向键处理完后返回0。未识别按键不会调用move，但main仍会重画并做状态检查。它并不解析终端箭头键产生的ESC序列，只认菜单列出的字母。

L303–304

```
int check(int grid[][4])
{
```

定义check，扫描棋盘是否出现2048，以及是否还存在可能移动的位置。函数返回-1表示找到2048，0表示无可移动机会，正数表示还有机会。

L305

```
    int moveAvailable = 0;
```

moveAvailable先设0。它最终不是真正“多少种合法方向”的数量，代码只需发现至少一个机会，就停止增加它。

L306–307

```
    for (int i = 0; i < 4; i++)
    {
```

枚举4行。

L308–309

```
        for (int j = 0; j < 4; j++)
        {
```

枚举每行4列，逐个查看16个格子。

L310–311

```
            if (grid[i][j] == 2048)
                return -1;
```

任何格子恰好等于2048就立即返回-1，认定胜利；比较是==而不是>=，因此4096本身不满足这一分支。

L312–313

```
            if (moveAvailable == 0)
            {
```

只有尚未找到可移动机会时才继续检查空格或相等邻居。已经找到机会后，仍会遍历其余格子以寻找2048。

L314–315

```
                if (grid[i][j] == 0)
                    moveAvailable++;
```

当前格子为0就认为尚有空间，增加moveAvailable到1。它没有验证整个棋盘上是否存在可移动的非零块；例如全零棋盘也被判“可继续”。

L316–317

```
                else if (i < 3 && grid[i + 1][j] == grid[i][j])
                    moveAvailable++;
```

否则，若不是最下行且下方邻居与当前格相等，认为可以合并。i<3先检查边界，&&的短路求值使最后一行不会访问i+1。

L318–319

```
                else if (j < 3 && grid[i][j + 1] == grid[i][j])
                    moveAvailable++;
```

否则，若不是最右列且右侧邻居相等，也认为有合并机会。

L320–321

```
                else if (i > 0 && grid[i - 1][j] == grid[i][j])
                    moveAvailable++;
```

再检查上方相等邻居；i>0保证不会访问第-1行。这个函数的边界检查与countEmpty的错误形成对照。

L322–323

```
                else if (j > 0 && grid[i][j - 1] == grid[i][j])
                    moveAvailable++;
```

最后检查左方相等邻居；j>0保证左边索引合法。这四个方向检查存在重复，但不影响“至少一个机会”的意图。

L324–326

```
            }
        }
    }
```

依次结束机会检查、列循环与行循环。如果棋盘全满且没有任意相邻相等格，就会保持moveAvailable=0。

L327–328

```
    return moveAvailable;
}
```

返回0或正数并结束函数；-1已在发现2048时提前返回。它只根据棋盘形状判断，无法保证有缺陷的move函数确实正确执行相应移动。

L330–331

```
void initialize(int grid[][4])
{
```

定义initialize初始化一局游戏。它只接收棋盘，不处理main中的总分。

L332–334

```
    for (int i = 0; i < 4; i++)
        for (int j = 0; j < 4; j++)
            grid[i][j] = 0;
```

双重for遍历所有16格并赋值0。这里没有花括号，但缩进与紧接语句体现“每行执行整段内循环”，而内循环只执行一条grid[i][j]=0。

L335

```
    add(grid, 2);
```

意图是在空棋盘中放第一个值为2的方块；实际依赖有缺陷的add/countEmpty/rand，因此不能保证一定成功放入。

L336–337

```
    add(grid, 2);
}
```

再次调用add，意图再放一个2，结束初始化。正常设计是“两块2开局”，但当前源码的越界和随机问题使这一现象并非保证。

L339–340

```
void menu()
{
```

定义menu，负责输出帮助并等待用户确认。

L341

```
    bios_putstr(" 2048 - Game\n * Use h-j-k-l / w-a-s-d keys to move the tiles.\n * When two t
iles with the same number touch, they merge into one.\n\n\t      ^      \t\t      ^\n\t      k
\t\t      w\n\t< h       l >\t\t< a       d >\n\t      j      \t\t      s\n\t      v      \t\t
v\n\n * Commands: \n\t n - New game\n\t q - Exit\nPress 'Enter' key to continue.. ");
```

一次bios_putstr发送整份菜单字符串。\n表示换行，\t表示制表符；内容说明h/j/k/l或w/a/s/d移动，n新游戏、q退出，并请按Enter继续。菜单文字只是说明意图，不会自动实现键位或正确规则。

L343

```
    int ch;
```

声明临时字符变量ch。

L344–345

```
    while ((ch = getchar()) != '\n' && ch != '\r')
        ;
```

反复调用本文件getchar，直到读到换行\n或回车\r；赋值表达式先把字符存入ch，再比较。循环体是空分号，因此输入其它字符会被丢弃，不执行游戏动作。

L346

```
}
```

结束menu；只有按到回车／换行才会返回main继续初始化。

L348–349

```
void print_scores(long bestScore, long score)
{
```

定义print_scores，接受最佳分和当前分，统一负责分数区域输出。这里参数是long，调用时int分数会按值转换。

L350

```
    // printk("\n BEST SCORE: %d\n Score: %d\n You Win !\n",bestScore, score)
```

这是被//注释掉的printk示例，不参与编译，其中的You Win !不会真的显示。当前程序没有另外补上胜利／失败专用文案。

L351

```
    bios_putstr("\n BEST SCORE: ");
```

先输出换行和固定标签BEST SCORE:。

L352

```
    print_value(bestScore);
```

按十进制打印本次任务调用中记住的最佳分。

L353

```
    bios_putstr("\n SCORE: ");
```

换行并输出当前分数标签SCORE:。

L354

```
    print_value(score);
```

按十进制打印当前分数。

L355–356

```
    bios_putstr("\n ");
}
```

再换行并输出一个空格，然后结束分数打印。

L358–359

```
int main()
{
```

定义游戏主函数。进入它时由任务crt0保存返回地址，并完成本任务.bss初始化。

L360

```
    menu();
```

先显示菜单并等Enter。注意此时尚未生成棋盘。

L362

```
    int grid[4][4];
```

定义局部4×4 int棋盘grid，共16个整数；它通常在当前任务使用的栈中。局部数组本来没有自动清零保证，下一步initialize才显式清零。

L363

```
    int score = 0;
```

当前局分数score初始化为0。

L364

```
    int bestScore = 0;
```

最佳分bestScore也从0开始，只在这次main调用内保留；退出游戏再重新运行任务后会重新置0，不是磁盘存档。

L365

```
    initialize(grid);
```

调用initialize清棋盘并尝试加入两个2。由于辅助函数缺陷，正常开局只能视作设计意图。

L367–368

```
    while (1)
    {
```

进入游戏主循环，每一轮显示当前局面、等待一个按键、处理动作并检查结束条件。while(1)本身无限，后面用break退出。

L369

```
        clear();
```

发送清屏控制序列；它没有明确归位光标，终端表现还依赖ANSI处理方式。

L370

```
        print(grid);
```

绘制当前棋盘四行彩色格子。

L371

```
        print_scores(bestScore, score);
```

显示最佳分与当前分。画面显示的是上一轮处理完动作后的状态。

L373

```
        int status = onKeyPress(getchar(), grid, &score);
```

先getchar忙等一个有效字符，再把字符、棋盘和&score传给onKeyPress。&score是main分数变量的地址，允许正确写法*score修改它；返回状态保存在局部status。

L374–375

```
        if (status == 3)
        {
```

若status==3，说明用户按q/Q，进入退出分支。

L376

```
            bios_putstr("[2048]: Exit 2048 game ...\n");
```

打印[2048]: Exit 2048 game ...及换行。只有主动q退出走这里；赢或输的分支不会打印这句。

L377–378

```
            break;
        }
```

break跳出游戏while循环，随后将执行main最后的return。

L379–380

```
        else if (status == 2)
        {
```

若status==2，说明用户按n/N，开始新的一局。

L381

```
            score = 0;
```

把当前局分数清零，最佳分不在这里重置。

L382

```
            initialize(grid);
```

清棋盘并尝试放入两个2，重新开局；仍有前述初始化缺陷。

L383–384

```
            continue;
        }
```

continue立即进入下一轮while，直接重画新棋盘，跳过本轮后面的最佳分更新与check。

L386

```
        if(score > bestScore) bestScore = score;
```

正常按键处理后，若当前分数超过历史最佳分就更新bestScore。它没有补偿上移分支漏加的分数，所见最高分同样可能失真。

L387

```
        status = check(grid);
```

调用check，根据棋盘本身判断是否已有2048或没有继续移动机会；覆盖此前按键状态status。

L388–389

```
        if (status == -1)
        {
```

若check返回-1，即棋盘中发现值2048，进入胜利结束路径。

L390–391

```
            clear();
            print(grid);
```

清屏并重新绘制最终棋盘，让用户看到包含2048的局面。

L392–394

```
            print_scores(bestScore, score);
            break;
        }
```

显示最终分数后break退出游戏循环。没有You Win文本；因此“看见最终棋盘并返回装载器”才是这份代码写出的行为，而不是某种胜利动画。

L395–396

```
        else if (status == 0)
        {
```

否则若check返回0，即棋盘没有空格且未找到相等邻居，进入无路可走的结束路径。

L397–398

```
            clear();
            print(grid);
```

同样清屏并绘制最终棋盘。

L399–401

```
            print_scores(bestScore, score);
            break;
        }
```

显示最终分数并break。这里也没有Game Over文字；屏幕更新与胜利路径非常相似，只能结合棋盘和判定逻辑区别。

L402–403

```
        bios_putstr("\n");
    }
```

若仍可继续，则输出换行，然后结束这一轮并回到while开头等待下一次按键。

L404–405

```
    return 0;
}
```

main返回0，用户任务结束，crt0恢复保存的返回现场，内核接着打印任务完成并读取下一个任务名；这不是整个系统退出。

3 你能看到什么，什么代码保证它

加载2048任务后，应先出现2048 - Game帮助菜单；按Enter后才进入棋盘界面。

代码推导，未在课程平台实测

2048.c第358—360行先调用menu；339—346行打印帮助并循环等待\n或\r，之后main第362—365行才创建并初始化棋盘。该菜单路径不依赖countEmpty，因此比“正常两块2开局”的保证更强。

设计意图是显示4×4彩色棋盘、BEST SCORE与SCORE，用h/j/k/l或w/a/s/d操作，用n重开，用q返回内核。当前源码不能保证稳定实现完整游戏。

代码推导，未在课程平台实测

print在133—153行双层循环绘制16格，pickColor在103—131行发ANSI颜色；348—356行显示分数；266—301行分派字母按键，373—384行处理q/n。158行越界、80行有符号溢出、291行指针运算错误阻止把设计意图当成整体正确保证。

条件性局部行为：有效合并时，两个相等非零方块合成其两倍，并按新方块数值加分；例如两个2合成4应加4分。

代码推导，未在课程平台实测

move第203—209行对一般相等非零格执行左格*=2、右格=0、score+=左格。左/下/右按键分支280、285、297行通过*score累计；上移291行写错，因此并非所有方向都满足该分数关系。

按q/Q时，若程序尚在正常主循环，可看到[2048]: Exit 2048 game ...，随后返回内核。达到2048或无路可走时只重画棋盘和分数后返回，不显示专用胜败文字。

代码推导，未在课程平台实测

onKeyPress第273—275行返回3，main第374—378行输出退出信息并break，404行return；胜利388—394行和失败395—401行只有clear、print、print_scores与break。350行的You Win只存在于注释，不能当作实际输出。

可能出现开局方块不足、无效移动还新增方块、向上合并不加分、合并次序不符合标准2048等异常；其中越界和溢出属于C未定义行为，具体表现不固定。

代码推导，未在课程平台实测

countEmpty第158行i=-1导致越界，add第171行又依赖该错误计数；rand第79—80行既不推进状态又有int溢出；move第217—238行特殊分支对[2,0,0,0]误报moved，对[2,0,2,2]合并优先级错误；onKeyPress第291行错误移动score指针。

容易误解或需要修正的地方

- 边界错误须与“猜测会输出什么”分开：grid[-1][j]与有符号溢出都是C未定义行为。不能保证只多计4个空格、恒定随机值、卡住或崩溃中的任何一种具体结果。
- rand状态没有x=...：即使暂按某编译器的整数回绕结果理解，其输出也不随调用更新。这与“随机种子固定但随机序列仍会变化”不是同一个问题。
- countEmpty越界的直接后果还会传到add：错误编号可能大于所有合法空格编号，遍历完毕仍不插入方块；当取模除数为0则再触发除零问题。满棋盘时也没有先阻止add。
- move有独立算法问题，即使假设修正内存越界也不自动正确。[2,0,0,0]向左本应无变化，却可把moved置1；[2,0,2,2]向左标准结果是[4,2,0,0]，当前特殊分支会优先合并右侧得到[2,4,0,0]（不计后续新增方块）。这些只是解释原代码，文档不替换源程序。
- score是int*时，score+=n与*score+=n含义完全不同。上移漏掉*使分数不更新，并可能构造越界指针；在C里即使不解引用，超出许可对象范围的指针算术也不被保证。
- 箭头键不是这里支持的方向键。终端箭头通常发送多个字符组成的ESC序列，本文件没有解析这些序列，只认菜单给出的字母。
- ANSI清屏ESC[2J不等于归位光标；彩色背景、光标位置和回车换行表现都依赖终端。屏幕不呈现彩色时应先区分终端功能和程序逻辑。
- 最佳分只保存在当前main的局部变量里，n可保留该次调用的最佳分；退出后重新运行2048会重置为0，没有文件或磁盘持久化。

05 · 头文件和反汇编附录

18 BIOS 服务编号：给底层入口的操作菜单

原路径：arch/riscv/include/asm/biosdef.h · 10 行

> 供 C 和经过预处理的 .S 汇编共同包含的宏定义。

这些数字不是函数地址，而是传给固定 BIOS 入口的服务号。可以把 BIOS 想成一个窗口：入口地址相同，a7 中的编号决定是打印、输入还是读盘。

本文件的宏由 arch/riscv/boot/bootblock.S 直接使用，也由 arch/riscv/bios/common.c 的封装使用；不依赖 C 类型，因此适合与汇编共享。

1 完整代码，先看全貌

```
1  #ifndef _ASM_BIOS_DEF_H
2  #define _ASM_BIOS_DEF_H
3
4  #define BIOS_PUTCHAR 1
5  #define BIOS_GETCHAR 2
6  #define BIOS_PUTSTR 9
7  #define BIOS_SDWRITE 10
8  #define BIOS_SDREAD 11
9
10 #endif // _ASM_BIOS_DEF_H
```

2 按原始行号，一句一句理解

L1–2

```
#ifndef _ASM_BIOS_DEF_H
#define _ASM_BIOS_DEF_H
```

防止重复包含：第一次读到本头文件时尚未定义 _ASM_BIOS_DEF_H，于是进入并定义它；再次包含会跳到末尾。宏名本身不分配运行时内存。

L4

```
#define BIOS_PUTCHAR 1
```

将 BIOS_PUTCHAR 替换为数字 1，表示输出一个字符。汇编的 li a7, BIOS_PUTCHAR 最终装入的就是 1。

L5

```
#define BIOS_GETCHAR 2
```

输入一个字符的服务号是 2。没有可读字符时，本工程上层按返回 -1 的约定继续轮询。

L6

```
#define BIOS_PUTSTR 9
```

输出以零结尾的字符串使用服务号 9。字符串首地址作为第一个参数放入 a0。

L7

```
#define BIOS_SDWRITE 10
```

写 SD 卡块使用服务号 10；这里只声明编号，是否能成功还要求 BIOS 与存储设备确实支持。

L8

```
#define BIOS_SDREAD 11
```

读 SD 卡块使用服务号 11。bootblock 用它载入内核，loader 用封装后的 sd_read 载入任务。

L10

```
#endif // _ASM_BIOS_DEF_H
```

结束开头的条件包含区间；行尾注释只提示它对应哪个宏。

3 你能看到什么，什么代码保证它

启动阶段能够选择“打印字符串”和“读内核”两种服务。

代码推导，未在课程平台实测

arch/riscv/boot/bootblock.S:29、40 使用本文件第 6、8 行的 BIOS_PUTSTR=9、BIOS_SDREAD=11，并将编号送到 a7。

容易误解或需要修正的地方

> - 仅有服务号不能让通用 QEMU 自动具备课程 BIOS；还必须有地址 0x50150000 处的匹配实现。

05 • 头文件和反汇编附录

19 汇编辅助宏：标签、入口和符号大小

原路径：arch/riscv/include/asm.h • 35 行

> 把重复的汇编符号声明写成可读的宏。

这些内容在 .S 文件经过 C 预处理时展开。比如 ENTRY(_start) 不是 RISC-V 指令，而会变成 .globl、.balign 和 _start: 三条汇编器语句。

宏主要帮助链接器、反汇编器和调试器理解符号；它们通常不会直接产生“运行一条指令”的效果。

1 完整代码，先看全貌

```
/*
 * asm.h: Assembler macros to make things easier to read.
 *
 * Copyright (C) 1996 David S. Miller (dm@engr.sgi.com)
 * Copyright (C) 1998 Ralf Baechle
 */
#ifndef ASM_H
#define ASM_H

#define END(function)  \
       .size function, .- function

#define ENDPROC(name)                           \
        .type name, @function;                  \
        END(name)

#define EXPORT(symbol) \
        .globl symbol; \
        symbol:

#define FEXPORT(symbol)             \
        .globl symbol;              \
        .type symbol, @function;    \
        symbol:

#define ENTRY(name)                             \
        .globl name;                            \
        .balign 4;                              \
        name:

#define RISCV_PTR       .dword
#define RISCV_SZPTR     8
#define RISCV_LGPTR     3

#endif /* ASM_H */
```

2 按原始行号，一句一句理解

L1–6

```
/*
 * asm.h: Assembler macros to make things easier to read.
 *
 * Copyright (C) 1996 David S. Miller (dm@engr.sgi.com)
 * Copyright (C) 1998 Ralf Baechle
 */
```

文件说明和版权注释：表明这些宏用来提高汇编可读性，不是 CPU 要执行的代码。

L7–8

```
#ifndef ASM_H
#define ASM_H
```

使用 ASM_H 作为包含保护标记，避免宏被重复定义。

L10–11

```
#define END(function)  \
       .size function, .- function
```

END(function) 定义该符号的字节长度。汇编器中的点号 . 表示当前位置；“当前位置减函数起点”就是已经写出的函数长度。行尾反斜杠把两行连成一个预处理宏。

L13–15

```
#define ENDPROC(name)                          \
        .type name, @function;                 \
        END(name)
```

ENDPROC(name) 先把符号标为函数，再调用 END(name) 记录长度。分号在展开后分隔汇编器语句；此宏没有自动生成 ret。

L17–19

```
#define EXPORT(symbol) \
        .globl symbol; \
        symbol:
```

EXPORT(symbol) 先用 .globl 把符号公开给其他目标文件，再定义同名标签。标签像“当前位置的名字”，不等于占有一块变量。

L21–24

```
#define FEXPORT(symbol)             \
        .globl symbol;              \
        .type symbol, @function;    \
        symbol:
```

FEXPORT(symbol) 比 EXPORT 多一个 .type symbol,@function，明确告诉工具这是函数符号，然后定义入口标签。

L26–29

```
#define ENTRY(name)                             \
        .globl name;                            \
        .balign 4;                              \
        name:
```

ENTRY(name) 声明全局符号、用 .balign 4 将位置对齐到 4 字节边界，并定义入口标签。head.S 和 crt0.S 用 ENTRY(_start) 建立各自可供链接器寻找的入口。

L31

```
#define RISCV_PTR       .dword
```

RISCV_PTR 是 .dword 的别名；在这里用于汇编中写出一个 8 字节量，与 RV64 指针宽度匹配。

L32

```
#define RISCV_SZPTR     8
```

RISCV_SZPTR=8 表示指针占 8 字节，便于计算表项或保存区大小。

L33

```
#define RISCV_LGPTR     3
```

RISCV_LGPTR=3 是 log2(8)：如果采用对齐指数或左移计算，每个指针的字节数等于 1<<3。

L35

```
#endif /* ASM_H */
```

关闭 ASM_H 包含保护。

3 你能看到什么，什么代码保证它

在 ELF 中能找到 _start 符号，而 ENTRY 本身不会出现在反汇编指令列表里。

代码推导，未在课程平台实测

arch/riscv/kernel/head.S:10 和 arch/riscv/crt0/crt0.S:4 使用 ENTRY；本文件第 26—29 行将它展开为全局声明、对齐和标签。

容易误解或需要修正的地方

- .globl/.size/.type 是汇编器指令，不是 CPU 指令。
- END 和 ENDPROC 负责元信息，不负责返回；返回仍需 jr ra 或 ret。

05 · 头文件和反汇编附录

20 内核公共接口：串口、读盘和颜色常量

原路径：arch/riscv/include/common.h · 70 行

> 声明 BIOS 封装接口，以及遗留的设备寄存器偏移和终端颜色。

头文件声明“函数叫什么、参数是什么”；真正执行操作的是 arch/riscv/bios/common.c。看到一个以分号结束的函数声明，就称作实现是不对的：本文件的 sd_read 没有函数体。

REG_* 是相对设备基址的偏移；COLOR_* 是串口终端识别的转义字符串。本包的实际串口路径通过课程 BIOS，而非在这些宏处直接访问硬件。

1 完整代码，先看全貌

```
/* * * *
```

```
38 #define COLOR_RED      "\e[31m"
39 #define COLOR_GREEN    "\e[32m"
40 #define COLOR_YELLOW   "\e[33m"
41 #define COLOR_BLUE     "\e[34m"
42 #define COLOR_MAGENTA  "\e[35m"
43 #define COLOR_CYAN     "\e[36m"
44 #define COLOR_RESET    "\e[0m"
45
46 // enter a char into serial port
47 // use bios printch function
48 void port_write_ch(char ch);
49
50 // enter a message into seraial port
51 // use bios printstr function
52 void port_write(char *buf);
53
54 // get a char from serial port
55 // use bios getch function
56 int port_read_ch(void);
57
58 // read blocks from sd card
59 // use bios bios_sd_read function
60 int sd_read(unsigned mem_address, unsigned num_of_blocks, unsigned block_id);
61
62 // write blocks to sd card
63 // use bios bios_sdwrite function
64 int sd_write(unsigned mem_address, unsigned num_of_blocks, unsigned block_id);
65
66 /*************************************************************/
67 /* Do not touch this comment. Reserved for future projects. */
68 /*************************************************************/
69
70 #endif
```

2 按原始行号，一句一句理解

L1–22

（此处为连续注释或较长结构，请对照上方完整代码 L1–L22。）

版权与许可注释允许使用、修改和分发，但要求保留版权许可说明；它不改变程序执行流程。

L24–25

```
#ifndef INCLUDE_COMMON_H_
#define INCLUDE_COMMON_H_
```

定义 INCLUDE_COMMON_H_ 包含保护，避免同一编译单元重复处理。

L27

```
#define REG_DAT     0x00
```

REG_DAT=0x00：数据寄存器相对 UART 基地址的偏移。这里只给出偏移，没有给出设备基地址或读写指令。

L28

```
#define REG_IER     0x01
```

REG_IER=0x01：中断使能寄存器的偏移。宏本身不会开启中断。

L29

```
#define REG_IIR     0x02
```

REG_IIR=0x02：中断识别寄存器的偏移，用于解释设备中断状态。

L30

```
#define REG_FCR     0x02
```

REG_FCR 也为 0x02：FIFO 控制与中断识别可以共享地址偏移，由读写方向或设备约定区分。

L31

```
#define REG_LCR     0x03
```

REG_LCR=0x03：线路控制寄存器偏移，传统 UART 用它配置数据位等格式。

L32

```
#define REG_MCR     0x04
```

REG_MCR=0x04：调制解调器控制寄存器偏移。

L33

```
#define REG_LSR     0x05
```

REG_LSR=0x05：线路状态寄存器偏移，常见用途是判断收发缓冲是否就绪。

L34

```
#define REG_MSR     0x06
```

REG_MSR=0x06：调制解调器状态寄存器偏移。

L35

```
#define REG_CR      0x08
```

REG_CR=0x08：课程公共头中保留的控制寄存器偏移；本 P1 未用它实现具体设备动作。

L36

```
#define REG_MR      0x09
```

REG_MR=0x09：课程公共头中保留的模式寄存器偏移；不能仅凭定义就认定设备已被配置。

L38

```
#define COLOR_RED      "\e[31m"
```

COLOR_RED 是 ESC+[31m 字符序列，兼容 ANSI 的终端收到它后改变后续文字前景色为红色；它不显示成字面上的反斜杠 e。

L39

```
#define COLOR_GREEN    "\e[32m"
```

COLOR_GREEN 用 32m 选择绿色文字。

L40

```
#define COLOR_YELLOW   "\e[33m"
```

COLOR_YELLOW 用 33m 选择黄色文字。

L41

```
#define COLOR_BLUE     "\e[34m"
```

COLOR_BLUE 用 34m 选择蓝色文字。

L42

```
#define COLOR_MAGENTA  "\e[35m"
```

COLOR_MAGENTA 用 35m 选择品红文字。

L43

```
#define COLOR_CYAN     "\e[36m"
```

COLOR_CYAN 用 36m 选择青色文字。

L44

```
#define COLOR_RESET    "\e[0m"
```

COLOR_RESET 用 0m 恢复终端默认样式。定义颜色不代表当前程序一定发送这些串。

L46–48

```
// enter a char into serial port
// use bios printch function
void port_write_ch(char ch);
```

注释说明 port_write_ch 将一个字符送往串口，函数声明规定参数 ch 是 char、返回类型 void。真正包装 BIOS 服务号 1 的实现在 arch/riscv/bios/common.c:15—18。

L50–52

```
// enter a message into seraial port
// use bios printstr function
void port_write(char *buf);
```

port_write 接收字符指针 buf，约定指向以零字节结束的字符串，打印整串且不向调用者返回一个值。实现调用 BIOS 字符串输出服务。

L54–56

```
// get a char from serial port
// use bios getch function
int port_read_ch(void);
```

port_read_ch(void) 表示不接收参数、返回 int。使用 int 而非 char，是为了同时表达字符值与“没有字符”的 -1。

L58–60

```
// read blocks from sd card
// use bios bios_sd_read function
int sd_read(unsigned mem_address, unsigned num_of_blocks, unsigned block_id);
```

sd_read 声明接收目的内存地址、块数和起始块号，返回 int 状态。地址形参是 unsigned：在当前 ABI 通常仅 32 位，因此不是任意 RV64 地址都能无损传入。

L62–64

```
// write blocks to sd card
// use bios bios_sdwrite function
int sd_write(unsigned mem_address, unsigned num_of_blocks, unsigned block_id);
```

sd_write 的三个参数同读盘接口，但方向是从内存写到 SD 卡。本行只声明函数，不会在读盘流程中自动执行写盘。

L66–68

```
/*************************************************************/
/* Do not touch this comment. Reserved for future projects. */
/*************************************************************/
```

课程后续项目预留注释标记，不包含可执行语句。

L70

```
#endif
```

结束 INCLUDE_COMMON_H_ 包含保护。

3 你能看到什么，什么代码保证它

主循环可回显输入字符，loader 可以按扇区取数据。

代码推导，未在课程平台实测

init/main.c:33—36 把 port_write/port_write_ch/port_read_ch/sd_read 放入函数表；本文件第 48、52、56、60 行提供对应声明，实现在 arch/riscv/bios/common.c。

容易误解或需要修正的地方

- 这些寄存器常量并不代表已实现 UART 中断驱动。
- "\e" 是 GCC 接受的转义写法，不是所有严格 C 工具链都支持。
- 读写盘地址使用 unsigned，必须留意地址是否位于低 4 GiB；当前课程地址位于这个范围。

05 · 头文件和反汇编附录

21 CSR 常量：认识特权级，不等于实现了调度

原路径：arch/riscv/include/csr.h · 79 行

> 为汇编和未来异常处理代码提供 CSR 编号及位掩码。

CSR 是处理器内部的控制/状态寄存器，使用 csrw/csrr 等专用指令访问。要先区分“寄存器编号”“某寄存器中的位掩码”“陷阱原因编号”，三者都写成整数，但用途不同。

P1 的 head.S 只用到其中的 CSR_SIE 和 CSR_SIP 来关闭/清除相关中断设置。其余常量大量为后续项目预留；看见页故障、定时器宏，不表示当前内核已实现分页或抢占。

1 完整代码，先看全貌

```
/* SPDX-License-Identifier: GPL-2.0-only */
/*
 * Copyright (C) 2015 Regents of the University of California
 */

#ifndef CSR_H
#define CSR_H

/* Status register flags */
#define SR_SIE    0x00000002 /* Supervisor Interrupt Enable */
#define SR_SPIE   0x00000020 /* Previous Supervisor IE */
#define SR_SPP    0x00000100 /* Previously Supervisor */
#define SR_SUM    0x00040000 /* Supervisor User Memory Access */

#define SR_FS           0x00006000 /* Floating-point Status */
#define SR_FS_OFF       0x00000000
#define SR_FS_INITIAL   0x00002000
#define SR_FS_CLEAN     0x00004000
#define SR_FS_DIRTY     0x00006000

#define SR_XS           0x00018000 /* Extension Status */
#define SR_XS_OFF       0x00000000
#define SR_XS_INITIAL   0x00008000
#define SR_XS_CLEAN     0x00010000
#define SR_XS_DIRTY     0x00018000

#define SR_SD           0x8000000000000000 /* FS/XS dirty */

/* SATP flags */
#define SATP_PPN        0x00000FFFFFFFFFFF
#define SATP_MODE_39    0x8000000000000000
#define SATP_MODE       SATP_MODE_39

/* SCAUSE */
#define SCAUSE_IRQ_FLAG   (1UL << 63)

#define IRQ_U_SOFT      0
```

```
38 #define IRQ_S_SOFT      1
39 #define IRQ_M_SOFT      3
40 #define IRQ_U_TIMER     4
41 #define IRQ_S_TIMER     5
42 #define IRQ_M_TIMER     7
43 #define IRQ_U_EXT       8
44 #define IRQ_S_EXT       9
45 #define IRQ_M_EXT       11
46
47 #define EXC_INST_MISALIGNED 0
48 #define EXC_INST_ACCESS     1
49 #define EXC_BREAKPOINT      3
50 #define EXC_LOAD_ACCESS     5
51 #define EXC_STORE_ACCESS    7
52 #define EXC_SYSCALL     8
53 #define EXC_INST_PAGE_FAULT 12
54 #define EXC_LOAD_PAGE_FAULT 13
55 #define EXC_STORE_PAGE_FAULT    15
56
57 /* SIE (Interrupt Enable) and SIP (Interrupt Pending) flags */
58 #define SIE_SSIE    (0x1 << IRQ_S_SOFT)
59 #define SIE_STIE    (0x1 << IRQ_S_TIMER)
60 #define SIE_SEIE    (0x1 << IRQ_S_EXT)
61
62 #define CSR_CYCLE   0xc00
63 #define CSR_TIME    0xc01
64 #define CSR_INSTRET   0xc02
65 #define CSR_SSTATUS   0x100
66 #define CSR_SIE     0x104
67 #define CSR_STVEC   0x105
68 #define CSR_SCOUNTEREN    0x106
69 #define CSR_SSCRATCH    0x140
70 #define CSR_SEPC    0x141
71 #define CSR_SCAUSE    0x142
72 #define CSR_STVAL   0x143
73 #define CSR_SIP     0x144
74 #define CSR_SATP    0x180
75 #define CSR_CYCLEH    0xc80
76 #define CSR_TIMEH   0xc81
77 #define CSR_INSTRETH    0xc82
78
79 #endif /* CSR_H */
```

2 按原始行号，一句一句理解

L1–4

```
/* SPDX-License-Identifier: GPL-2.0-only */
/*
 * Copyright (C) 2015 Regents of the University of California
 */
```

SPDX 许可证和版权注释，不参与 CSR 读写。

L6–7

```
#ifndef CSR_H
#define CSR_H
```

以 CSR_H 做包含保护。

L9

```
/* Status register flags */
```

以下一组宏是状态寄存器的位掩码；掩码用来选择某些位，不是寄存器地址。

L10

```
#define SR_SIE    0x00000002 /* Supervisor Interrupt Enable */
```

SR_SIE=2，即第 1 位：S 模式总中断使能位。定义宏不会改变硬件，必须由指令实际修改状态寄存器。

L11

```
#define SR_SPIE   0x00000020 /* Previous Supervisor IE */
```

SR_SPIE=0x20，即第 5 位，保存进入陷阱前的 SIE 状态，供陷阱返回使用。

L12

```
#define SR_SPP    0x00000100 /* Previously Supervisor */
```

SR_SPP=0x100，即第 8 位，记录进入 S 模式陷阱之前的特权级别。

L13

```
#define SR_SUM    0x00040000 /* Supervisor User Memory Access */
```

SR_SUM=0x40000，即第 18 位：在适用的分页与权限条件下控制 S 模式能否访问用户页；单独定义它不意味着已开启分页。

L15

```
#define SR_FS           0x00006000 /* Floating-point Status */
```

SR_FS=0x6000 选出两位浮点状态字段。

L16

```
#define SR_FS_OFF       0x00000000
```

浮点状态 OFF 的编码为 0，表示浮点状态被关闭。

L17

```
#define SR_FS_INITIAL   0x00002000
```

浮点状态 INITIAL 的编码为 1（移到字段位置后为 0x2000），表示初始状态。

L18

```
#define SR_FS_CLEAN     0x00004000
```

浮点状态 CLEAN 的编码为 2（0x4000），表示状态未相对保存值改变。

L19

```
#define SR_FS_DIRTY     0x00006000
```

浮点状态 DIRTY 的编码为 3（0x6000），表示状态已修改，保存上下文时可能需要保存它。

L21

```
#define SR_XS           0x00018000 /* Extension Status */
```

SR_XS=0x18000 选择扩展状态字段，在此课程头文件里与浮点状态使用类似编码。

L22

```
#define SR_XS_OFF       0x00000000
```

扩展状态 OFF 编码为 0。

L23

```
#define SR_XS_INITIAL   0x00008000
```

扩展状态 INITIAL 编码为 1，位移后为 0x8000。

L24

```
#define SR_XS_CLEAN     0x00010000
```

扩展状态 CLEAN 编码为 2，位移后为 0x10000。

L25

```
#define SR_XS_DIRTY     0x00018000
```

扩展状态 DIRTY 编码为 3，位移后为 0x18000。

L27

```
#define SR_SD           0x8000000000000000 /* FS/XS dirty */
```

SR_SD 为 RV64 状态寄存器最高位的汇总脏标志。它表达扩展状态存在脏内容，不能把它当成一个普通的文件“保存”标志。

L29

```
/* SATP flags */
```

下面描述地址转换寄存器 satp 的字段。

L30

```
#define SATP_PPN        0x00000FFFFFFFFFFF
```

SATP_PPN 掩码保留 satp 的低 44 位物理页号字段，页号与字节地址不是一回事。

L31

```
#define SATP_MODE_39    0x8000000000000000
```

SATP_MODE_39 的最高位编码选择 RV64 的 Sv39 分页方式。写 satp 还需同时提供根页表物理页号；宏不会建立页表。

L32

```
#define SATP_MODE       SATP_MODE_39
```

SATP_MODE 使用上一行的 Sv39 编码作默认别名；P1 实际运行路径没有因此自动开启虚拟内存。

L34

```
/* SCAUSE */
```

下面是陷阱原因寄存器 scause 的中断标志和异常编号。

L35

```
#define SCAUSE_IRQ_FLAG   (1UL << 63)
```

1UL<<63 生成最高位掩码；scause 该位为 1 时表示中断，为 0 时表示同步异常。UL 让常量具有无符号 long 宽度。

L37

```
#define IRQ_U_SOFT      0
```

IRQ_U_SOFT=0 是历史用户软件中断的原因编号，也可用作中断位的位置；它不是处理函数或优先级。U 类编号属于该课程头文件保留的旧约定，不能据此认定现代平台实现了用户中断。

L38

```
#define IRQ_S_SOFT      1
```

IRQ_S_SOFT=1 是S 模式软件中断的原因编号，也可用作中断位的位置；它不是处理函数或优先级。U 类编号属于该课程头文件保留的旧约定，不能据此认定现代平台实现了用户中断。

L39

```
#define IRQ_M_SOFT      3
```

IRQ_M_SOFT=3 是M 模式软件中断的原因编号，也可用作中断位的位置；它不是处理函数或优先级。U 类编号属于该课程头文件保留的旧约定，不能据此认定现代平台实现了用户中断。

L40

```
#define IRQ_U_TIMER     4
```

IRQ_U_TIMER=4 是历史用户定时器中断的原因编号，也可用作中断位的位置；它不是处理函数或优先级。U 类编号属于该课程头文件保留的旧约定，不能据此认定现代平台实现了用户中断。

L41

```
#define IRQ_S_TIMER     5
```

IRQ_S_TIMER=5 是S 模式定时器中断的原因编号，也可用作中断位的位置；它不是处理函数或优先级。U 类编号属于该课程头文件保留的旧约定，不能据此认定现代平台实现了用户中断。

L42

```
#define IRQ_M_TIMER     7
```

IRQ_M_TIMER=7 是M 模式定时器中断的原因编号，也可用作中断位的位置；它不是处理函数或优先级。U 类编号属于该课程头文件保留的旧约定，不能据此认定现代平台实现了用户中断。

L43

```
#define IRQ_U_EXT       8
```

IRQ_U_EXT=8 是历史用户外部中断的原因编号，也可用作中断位的位置；它不是处理函数或优先级。U 类编号属于该课程头文件保留的旧约定，不能据此认定现代平台实现了用户中断。

L44

```
#define IRQ_S_EXT       9
```

IRQ_S_EXT=9 是S 模式外部中断的原因编号，也可用作中断位的位置；它不是处理函数或优先级。U 类编号属于该课程头文件保留的旧约定，不能据此认定现代平台实现了用户中断。

L45

```
#define IRQ_M_EXT       11
```

IRQ_M_EXT=11 是M 模式外部中断的原因编号，也可用作中断位的位置；它不是处理函数或优先级。U 类编号属于该课程头文件保留的旧约定，不能据此认定现代平台实现了用户中断。

L47

```
#define EXC_INST_MISALIGNED 0
```

EXC_INST_MISALIGNED=0 命名“取指地址未对齐”这个同步异常原因。这个编号方便未来陷阱分发；声明编号不等于本 P1 有处理函数。

L48

```
#define EXC_INST_ACCESS     1
```

EXC_INST_ACCESS=1 命名“取指访问故障”这个同步异常原因。这个编号方便未来陷阱分发；声明编号不等于本 P1 有处理函数。

L49

```
#define EXC_BREAKPOINT      3
```

EXC_BREAKPOINT=3 命名“断点”这个同步异常原因。这个编号方便未来陷阱分发；声明编号不等于本 P1 有处理函数。

L50

```
#define EXC_LOAD_ACCESS     5
```

EXC_LOAD_ACCESS=5 命名“读内存访问故障”这个同步异常原因。这个编号方便未来陷阱分发；声明编号不等于本 P1 有处理函数。

L51

```
#define EXC_STORE_ACCESS    7
```

EXC_STORE_ACCESS=7 命名“写内存/原子访问故障”这个同步异常原因。这个编号方便未来陷阱分发；声明编号不等于本 P1 有处理函数。

L52

```
#define EXC_SYSCALL     8
```

EXC_SYSCALL=8 命名“来自 U 模式的环境调用 ecall”这个同步异常原因。这个编号方便未来陷阱分发；声明编号不等于本 P1 有处理函数。

L53

```
#define EXC_INST_PAGE_FAULT 12
```

EXC_INST_PAGE_FAULT=12 命名“取指页故障”这个同步异常原因。这个编号方便未来陷阱分发；声明编号不等于本 P1 有处理函数。

L54

```
#define EXC_LOAD_PAGE_FAULT 13
```

EXC_LOAD_PAGE_FAULT=13 命名“读内存页故障”这个同步异常原因。这个编号方便未来陷阱分发；声明编号不等于本 P1 有处理函数。

L55

```
#define EXC_STORE_PAGE_FAULT    15
```

EXC_STORE_PAGE_FAULT=15 命名“写内存/原子操作页故障”这个同步异常原因。这个编号方便未来陷阱分发；声明编号不等于本 P1 有处理函数。

L57

```
/* SIE (Interrupt Enable) and SIP (Interrupt Pending) flags */
```

下面三个掩码可用于 sie/sip 等寄存器的对应中断位。

L58

```
#define SIE_SSIE    (0x1 << IRQ_S_SOFT)
```

SIE_SSIE 将 1 左移 IRQ_S_SOFT=1 位，得到 S 模式软件中断位掩码 0x2。

L59

```
#define SIE_STIE    (0x1 << IRQ_S_TIMER)
```

SIE_STIE 将 1 左移 IRQ_S_TIMER=5 位，得到 S 模式定时器中断位掩码 0x20。

L60

```
#define SIE_SEIE    (0x1 << IRQ_S_EXT)
```

SIE_SEIE 将 1 左移 IRQ_S_EXT=9 位，得到 S 模式外部中断位掩码 0x200。

L62

```
#define CSR_CYCLE   0xc00
```

CSR_CYCLE=0xc00 是“周期计数器”的 CSR 编号。CSR 指令通过编号选择 CPU 内部寄存器；不能用普通内存指针把它当作地址解引用。

L63

```
#define CSR_TIME    0xc01
```

CSR_TIME=0xc01 是“时间计数器”的 CSR 编号。CSR 指令通过编号选择 CPU 内部寄存器；不能用普通内存指针把它当作地址解引用。

L64

```
#define CSR_INSTRET   0xc02
```

CSR_INSTRET=0xc02 是“已退休指令数计数器”的 CSR 编号。CSR 指令通过编号选择 CPU 内部寄存器；不能用普通内存指针把它当作地址解引用。

L65

```
#define CSR_SSTATUS   0x100
```

CSR_SSTATUS=0x100 是“S 模式状态”的 CSR 编号。CSR 指令通过编号选择 CPU 内部寄存器；不能用普通内存指针把它当作地址解引用。

L66

```
#define CSR_SIE     0x104
```

CSR_SIE=0x104 是“S 模式中断使能”的 CSR 编号。CSR 指令通过编号选择 CPU 内部寄存器；不能用普通内存指针把它当作地址解引用。

L67

```
#define CSR_STVEC   0x105
```

CSR_STVEC=0x105 是“S 模式陷阱入口”的 CSR 编号。CSR 指令通过编号选择 CPU 内部寄存器；不能用普通内存指针把它当作地址解引用。

L68

```
#define CSR_SCOUNTEREN    0x106
```

CSR_SCOUNTEREN=0x106 是“S 模式对较低特权级的计数器访问控制”的 CSR 编号。CSR 指令通过编号选择 CPU 内部寄存器；不能用普通内存指针把它当作地址解引用。

L69

```
#define CSR_SSCRATCH    0x140
```

CSR_SSCRATCH=0x140 是“S 模式陷阱临时保存寄存器”的 CSR 编号。CSR 指令通过编号选择 CPU 内部寄存器；不能用普通内存指针把它当作地址解引用。

L70

```
#define CSR_SEPC    0x141
```

CSR_SEPC=0x141 是“S 模式陷阱返回地址”的 CSR 编号。CSR 指令通过编号选择 CPU 内部寄存器；不能用普通内存指针把它当作地址解引用。

L71

```
#define CSR_SCAUSE    0x142
```

CSR_SCAUSE=0x142 是“S 模式陷阱原因”的 CSR 编号。CSR 指令通过编号选择 CPU 内部寄存器；不能用普通内存指针把它当作地址解引用。

L72

```
#define CSR_STVAL   0x143
```

CSR_STVAL=0x143 是“S 模式陷阱附加信息”的 CSR 编号。CSR 指令通过编号选择 CPU 内部寄存器；不能用普通内存指针把它当作地址解引用。

L73

```
#define CSR_SIP     0x144
```

CSR_SIP=0x144 是“S 模式中断挂起状态”的 CSR 编号。CSR 指令通过编号选择 CPU 内部寄存器；不能用普通内存指针把它当作地址解引用。

L74

```
#define CSR_SATP    0x180
```

CSR_SATP=0x180 是“S 模式地址转换与保护”的 CSR 编号。CSR 指令通过编号选择 CPU 内部寄存器；不能用普通内存指针把它当作地址解引用。

L75

```
#define CSR_CYCLEH    0xc80
```

CSR_CYCLEH=0xc80 是 RV32 的周期计数高半部编号；RV64 已可用完整宽度计数器，不能假定读这个编号仍合法。

L76

```
#define CSR_TIMEH   0xc81
```

CSR_TIMEH=0xc81 是 RV32 的时间计数高半部，属于兼容定义。

L77

```
#define CSR_INSTRETH    0xc82
```

CSR_INSTRETH=0xc82 是 RV32 的退休指令数高半部，不能把这些宏误认为本项目执行了对应读指令。

L79

```
#endif /* CSR_H */
```

结束 CSR_H 包含保护。

3 你能看到什么，什么代码保证它

内核入口先关闭 S 模式中断来源并尝试清除可写挂起位。

代码推导，未在课程平台实测

arch/riscv/kernel/head.S:12—13 执行 csrw CSR_SIE,zero 和 csrw CSR_SIP,zero，编号由本文件第 66、73 行给出；清零并非对所有只读挂起位都有效。

容易误解或需要修正的地方

- 宏名保留了旧课程约定；EXC_SYSCALL=8 特指 U-mode ecall，S-mode ecall 原因不同。
- 仅定义 SATP_MODE 不会创建页表，也不会改变 CPU 地址转换状态。

05 • 头文件和反汇编附录

22 内核服务调用表：从下标找到函数地址

原路径：include/os/kernel.h • 59 行

> 为内核提供固定地址的服务表包装接口。

把跳转表看作电话簿：0 号记着打印整串函数的地址，1 号记着打印单字符函数地址。call_jmptab 先查号，再拨给对应函数。

init/main.c 的 init_jmptab 真正填表；本头文件只是读表和调用。所有任务共用地址空间，此路径不是现代 OS 那种通过陷阱实现的受保护系统调用。

1 完整代码，先看全貌

```
1  #ifndef __INCLUDE_KERNEL_H__
2  #define __INCLUDE_KERNEL_H__
3
4  #include <type.h>
5  #include <common.h>
6
7  #define KERNEL_JMPTAB_BASE 0x51ffff00
8  typedef enum {
9      CONSOLE_PUTSTR,
10     CONSOLE_PUTCHAR,
11     CONSOLE_GETCHAR,
12     SD_READ,
13     SD_WRITE,
14     NUM_ENTRIES
15 } jmptab_idx_t;
16
17 static inline long call_jmptab(long which, long arg0, long arg1, long arg2, long arg3, long
 >  arg4)
18 {
19     unsigned long val = \
20         *(unsigned long *)(KERNEL_JMPTAB_BASE + sizeof(unsigned long) * which);
21     long (*func)(long, long, long, long, long) = (long (*)(long, long, long, long, long))va
 > l;
22
23     return func(arg0, arg1, arg2, arg3, arg4);
24 }
25
26 static inline void bios_putstr(char *str)
27 {
28     call_jmptab(CONSOLE_PUTSTR, (long)str, 0, 0, 0, 0);
29 }
30
31 static inline void bios_putchar(int ch)
32 {
33     call_jmptab(CONSOLE_PUTCHAR, (long)ch, 0, 0, 0, 0);
34 }
35
```

```
36 static inline int bios_getchar(void)
37 {
38     return call_jmptab(CONSOLE_GETCHAR, 0, 0, 0, 0, 0);
39 }
40
41 static inline int bios_sd_read(unsigned mem_address, unsigned num_of_blocks, \
42                                unsigned block_id)
43 {
44     return call_jmptab(SD_READ, (long)mem_address, (long)num_of_blocks, \
45                        (long)block_id, 0, 0);
46 }
47
48 static inline int bios_sd_write(unsigned mem_address, unsigned num_of_blocks, \
49                                unsigned block_id)
50 {
51     return call_jmptab(SD_WRITE, (long)mem_address, (long)num_of_blocks, \
52                        (long)block_id, 0, 0);
53 }
54
55 /************************************************************/
56 /* Do not touch this comment. Reserved for future projects. */
57 /************************************************************/
58
59 #endif
```

2 按原始行号，一句一句理解

L1–2

```
#ifndef __INCLUDE_KERNEL_H__
#define __INCLUDE_KERNEL_H__
```

内核版 kernel.h 的包含保护；它与 tiny_libc 的同名宏一致，因此同一编译单元混用这两份头文件会由先包含者遮住后者。工程通过不同 -I 路径分别编译内核和用户程序。

L4

```
#include <type.h>
```

引入本项目的固定宽度整数类型。

L5

```
#include <common.h>
```

引入 common.h 中的串口和 SD 封装函数声明。

L7

```
#define KERNEL_JMPTAB_BASE 0x51ffff00
```

函数跳转表的固定内存起点为 0x51ffff00。这片内存保存的是“函数地址数组”，不是指令组成的 BIOS 本体。

L8

```
typedef enum {
```

定义一个枚举类型来给表项编号；首个未显式赋值的枚举项从 0 开始。

L9

```
    CONSOLE_PUTSTR,
```

CONSOLE_PUTSTR=0，函数表第 0 项负责打印字符串。

L10

```
    CONSOLE_PUTCHAR,
```

CONSOLE_PUTCHAR=1，第 1 项负责输出单字符。

L11

```
    CONSOLE_GETCHAR,
```

CONSOLE_GETCHAR=2，第 2 项负责输入字符。

L12

```
    SD_READ,
```

SD_READ=3，第 3 项负责读盘。

L13

```
    SD_WRITE,
```

SD_WRITE=4，预留第 4 项负责写盘；当前 init_jmptab 没有填写它，所以不能把这个接口当作可用功能。

L14–15

```
    NUM_ENTRIES
} jmptab_idx_t;
```

NUM_ENTRIES=5 是表项数量标记，再把整个枚举类型命名为 jmptab_idx_t；这不是第六个有效服务。

L17–18

```
static inline long call_jmptab(long which, long arg0, long arg1, long arg2, long arg3, long ar
g4)
{
```

定义只在包含它的编译单元内可见的内联辅助函数。which 选择表项，arg0—arg4 是最多五个以 long 携带的参数；static inline 不保证编译器一定消除函数调用。

L19–20

```
    unsigned long val = \
        *(unsigned long *)(KERNEL_JMPTAB_BASE + sizeof(unsigned long) * which);
```

先计算“表首地址 + 每项字节数×下标”，强制转换成 unsigned long 指针后解引用，取出保存的函数地址。RV64 LP64 中每项 8 字节，例如索引 2 对应 0x51ffff10。

L21

```
    long (*func)(long, long, long, long, long) = (long (*)(long, long, long, long, long))val;
```

将整数地址 val 转成“接收五个 long、返回 long 的函数指针”func，赋予地址可调用的 C 类型。指针地址与整数类型的转换依赖此课程 ABI。

L23–24

```
    return func(arg0, arg1, arg2, arg3, arg4);
}
```

间接调用所选函数，按调用约定传五个参数，并把其返回值交给上层。这里是普通函数调用，没有 ecall，也没有切换到受保护的用户/内核边界。

L26–27

```
static inline void bios_putstr(char *str)
{
```

定义 bios_putstr 包装器：调用者给出字符串地址，包装器没有返回值。

L28–29

```
    call_jmptab(CONSOLE_PUTSTR, (long)str, 0, 0, 0, 0);
}
```

选择第 0 项，把字符串指针转为 long 传作 arg0，其余参数置零；返回后包装器结束。

L31–32

```
static inline void bios_putchar(int ch)
{
```

定义 bios_putchar，参数为 int，方便承载字符值。

L33–34

```
    call_jmptab(CONSOLE_PUTCHAR, (long)ch, 0, 0, 0, 0);
}
```

选择第 1 项，把 ch 传入，再结束包装器。

L36–37

```
static inline int bios_getchar(void)
{
```

定义无参数、返回 int 的 bios_getchar；调用者需要用返回值判断是否有字符。

L38–39

```
    return call_jmptab(CONSOLE_GETCHAR, 0, 0, 0, 0, 0);
}
```

调用第 2 项，所有参数槽清零。底层返回 -1 时，此 int 返回值允许上层继续等待。

L41–43

```
static inline int bios_sd_read(unsigned mem_address, unsigned num_of_blocks, \
                               unsigned block_id)
{
```

定义读盘包装器，三项 unsigned 参数依次为目的内存地址、块数、起始块号；反斜杠连接行在 C 中等价于同一行。

L44–46

```
    return call_jmptab(SD_READ, (long)mem_address, (long)num_of_blocks, \
                       (long)block_id, 0, 0);
}
```

选择 SD_READ=3，把三项参数扩展为 long，另外两个槽填零，并返回底层状态；这些转换无法恢复已被 unsigned 参数截去的高位地址。

L48–50

```
static inline int bios_sd_write(unsigned mem_address, unsigned num_of_blocks, \
                                unsigned block_id)
{
```

定义写盘包装器，参数布局与读盘相同。声明存在不代表其表项已初始化。

L51–53

```
    return call_jmptab(SD_WRITE, (long)mem_address, (long)num_of_blocks, \
                       (long)block_id, 0, 0);
}
```

选择 SD_WRITE=4 并间接调用，但本包 init/main.c:33—36 没有写这个表项；若实际调用会跳到未定义内容，结果不能保证。

L55–57

```
/************************************************************/
/* Do not touch this comment. Reserved for future projects. */
/************************************************************/
```

为后续项目保留的注释边界，不执行任何操作。

L59

```
#endif
```

结束内核 kernel.h 的包含保护。

3 你能看到什么，什么代码保证它

bios_putstr 可最终通过 BIOS 在串口显示字符串。

代码推导，未在课程平台实测

本文件第 28 行选择 0 号表项；init/main.c:33 将该项设为 port_write；arch/riscv/bios/common.c:20—23 再调用 BIOS_PUTSTR。

声明了写盘包装器，但 P1 未保证它能使用。

代码推导，未在课程平台实测

本文件第 13、48—53 行读第 4 项；init/main.c:33—36 只填 0—3 项，缺少 SD_WRITE 的初始化。

容易误解或需要修正的地方

- call_jmptab 不检查 which 范围、函数地址或参数有效性；不能把它当作安全隔离层。
- generic long 函数类型与实际 void/int 包装函数并不完全匹配；课程 ABI 上可解释其机器调用路径，但这不是严格可移植的 C 接口设计。

05 • 头文件和反汇编附录

23 加载器声明：按字节读镜像、按名字找程序

原路径：include/os/loader.h • 10 行

> 让 main.c 能调用 loader.c 提供的两个函数。

声明告诉编译器如何传参和解释返回值。实现位于 kernel/loader/loader.c，本文件不会分配任务内存，也不会自己访问 SD 卡。

1 完整代码，先看全貌

```
1  #ifndef __INCLUDE_LOADER_H__
2  #define __INCLUDE_LOADER_H__
3
4  #include <type.h>
5  void load_img_bytes(uint32_t offset, uint32_t size, void *dst);
6  uint64_t load_task_img(char *taskname);
7
8
9
10 #endif
```

2 按原始行号，一句一句理解

L1–2

```
#ifndef __INCLUDE_LOADER_H__
#define __INCLUDE_LOADER_H__
```

建立 __INCLUDE_LOADER_H__ 包含保护。

L4

```
#include <type.h>
```

引入 uint32_t 和 uint64_t 等类型。

L5

```
void load_img_bytes(uint32_t offset, uint32_t size, void *dst);
```

load_img_bytes 的 offset 是镜像内的字节偏移，size 是字节数，dst 是目标内存首地址。void* 表示不限定目标数据类型，返回 void 表示不直接返回状态。

L6

```
uint64_t load_task_img(char *taskname);
```

load_task_img 接收以零结尾的任务名，返回 uint64_t 入口地址。当前实现用返回 0 表示没找到；找到时先把镜像内容装入该入口地址。

L10

```
#endif
```

结束包含保护。

3 你能看到什么，什么代码保证它

输入存在的任务名后，主循环可取得非零入口并调用。

代码推导，未在课程平台实测

本文件第 6 行声明返回地址接口，kernel/loader/loader.c:36—54 实现名字查找，init/main.c 的活动主循环检查 entry==0 后间接调用。

容易误解或需要修正的地方

- offset 不是扇区号；底层实现还要除以 512 换算。
- char* 说明接口类型，但不自动保证任务名合法或目标缓冲区安全。

05 · 头文件和反汇编附录

24 未来调度接口：必须分清“声明”与“已经实现”

原路径：include/os/sched.h · 115 行

> 遗留的调度结构模板，当前 P1 活动执行链不使用它。

本文件适合认识 PCB 和上下文的概念，但不能用来证明 P1 有进程调度。P1 的实际行为是内核根据任务名装入程序，像调用函数一样调用入口，等程序返回后再提示输入。

这份头文件依赖本压缩包缺失的 os/list.h，同时没有对应 switch_to、do_scheduler 等实现。默认 P1 源文件未包含它，所以这个缺口与“主项目一定无法编译”必须分开描述。

1 完整代码，先看全貌

```
/* * * *
```

```
36
37 /* used to save register infomation */
38 typedef struct regs_context
39 {
40     /* Saved main processor registers.*/
41     reg_t regs[32];
42
43     /* Saved special registers. */
44     reg_t sstatus;
45     reg_t sepc;
46     reg_t sbadaddr;
47     reg_t scause;
48 } regs_context_t;
49
50 /* used to save register infomation in switch_to */
51 typedef struct switchto_context
52 {
53     /* Callee saved registers.*/
54     reg_t regs[14];
55 } switchto_context_t;
56
57 typedef enum {
58     TASK_BLOCKED,
59     TASK_RUNNING,
60     TASK_READY,
61     TASK_EXITED,
62 } task_status_t;
63
64 /* Process Control Block */
65 typedef struct pcb
66 {
67     /* register context */
68     // NOTE: this order must be preserved, which is defined in regs.h!!
69     reg_t kernel_sp;
70     reg_t user_sp;
71
72     /* previous, next pointer */
73     list_node_t list;
74
75     /* process id */
76     pid_t pid;
77
78     /* BLOCK | READY | RUNNING */
79     task_status_t status;
80
81     /* cursor position */
82     int cursor_x;
83     int cursor_y;
84
85     /* time(seconds) to wake up sleeping PCB */
86     uint64_t wakeup_time;
87
88 } pcb_t;
89
90 /* ready queue to run */
91 extern list_head ready_queue;
92
93 /* sleep queue to be blocked in */
94 extern list_head sleep_queue;
```

```
 95
 96 /* current running task PCB */
 97 extern pcb_t * volatile current_running;
 98 extern pid_t process_id;
 99
100 extern pcb_t pcb[NUM_MAX_TASK];
101 extern pcb_t pid0_pcb;
102 extern const ptr_t pid0_stack;
103
104 extern void switch_to(pcb_t *prev, pcb_t *next);
105 void do_scheduler(void);
106 void do_sleep(uint32_t);
107
108 void do_block(list_node_t *, list_head *queue);
109 void do_unblock(list_node_t *);
110
111 /************************************************************/
112 /* Do not touch this comment. Reserved for future projects. */
113 /************************************************************/
114
115 #endif
```

2 按原始行号，一句一句理解

L1–27

（此处为连续注释或较长结构，请对照上方完整代码 L1–L27。）

课程版权、许可和模块说明注释。提到调度、阻塞、唤醒不等于这些机制已经在 P1 实现。

L29–30

```
#ifndef INCLUDE_SCHEDULER_H_
#define INCLUDE_SCHEDULER_H_
```

建立 INCLUDE_SCHEDULER_H_ 包含保护。

L32

```
#include <type.h>
```

引入 pid_t、reg_t 等基础类型。

L33

```
#include <os/list.h>
```

要求包含 os/list.h 来获得链表类型，但本压缩包没有此头文件。若新代码实际包含 sched.h 会遇到缺文件；当前 P1 有效源码没有因此启用调度。

L35

```
#define NUM_MAX_TASK 16
```

NUM_MAX_TASK=16 是这份未来调度设计支持的 PCB 槽数，与磁盘任务表 TASK_MAXNUM 虽同为 16，概念却不同。

L37–39

```
/* used to save register infomation */
typedef struct regs_context
{
```

开始定义保存陷阱现场的 regs_context 结构体；typedef 最后会给它简写类型名。

L40–41

```
    /* Saved main processor registers.*/
    reg_t regs[32];
```

regs[32] 为 32 个通用寄存器保留位置，每个 reg_t 在本项目是 64 位；数组只是存储布局，不会自己执行保存。

L43–44

```
    /* Saved special registers. */
    reg_t sstatus;
```

sstatus 字段预留保存状态寄存器的值。

L45

```
    reg_t sepc;
```

sepc 字段保存将来陷阱返回使用的程序计数器。

L46

```
    reg_t sbadaddr;
```

sbadaddr 是旧式命名，意图保存故障相关地址/附加信息，现代 CSR 名称通常为 stval。

L47–48

```
    reg_t scause;
} regs_context_t;
```

scause 字段记录陷阱原因，然后将结构体命名为 regs_context_t。

L50–52

```
/* used to save register infomation in switch_to */
typedef struct switchto_context
{
```

开始定义 switch_to 使用的较小现场结构；它不同于完整陷阱现场。

L53–55

```
    /* Callee saved registers.*/
    reg_t regs[14];
} switchto_context_t;
```

14 个 reg_t 槽位拟用于被调用者保存寄存器等上下文，并命名 switchto_context_t；仅有数组无法知道每个下标对应哪个寄存器，需要匹配的汇编实现，而本 P1 未提供。

L57

```
typedef enum {
```

开始给未来任务状态定义枚举。

L58

```
    TASK_BLOCKED,
```

TASK_BLOCKED=0：任务在等待条件，暂时不可运行。

L59

```
    TASK_RUNNING,
```

TASK_RUNNING=1：当前正在运行。

L60

```
    TASK_READY,
```

TASK_READY=2：具备运行条件、等待选择。

L61–62

```
    TASK_EXITED,
} task_status_t;
```

TASK_EXITED=3：任务已经退出，结束枚举并命名 task_status_t。

L64–66

```
/* Process Control Block */
typedef struct pcb
{
```

开始定义 PCB，即进程控制块。这里是后续调度需要的数据草图，不代表 P1 每个程序已经拥有 PCB。

L67–69

```
    /* register context */
    // NOTE: this order must be preserved, which is defined in regs.h!!
    reg_t kernel_sp;
```

kernel_sp 保存内核栈指针；注释提醒字段顺序必须匹配上下文切换汇编的固定偏移。被提及的 regs.h 也不是本包已提供的实现。

L70

```
    reg_t user_sp;
```

user_sp 保存用户栈指针，与 kernel_sp 分开，为后续用户/内核切换预留。

L72–73

```
    /* previous, next pointer */
    list_node_t list;
```

list 是嵌在 PCB 内的链表节点，用于将 PCB 串成就绪/等待队列；类型依赖缺失的 os/list.h。

L75–76

```
    /* process id */
    pid_t pid;
```

pid 是进程标识符，类型 pid_t 在 type.h 中为 int32_t。

L78–79

```
    /* BLOCK | READY | RUNNING */
    task_status_t status;
```

status 用前面的 task_status_t 记录任务状态，不能与 CPU 特权级混淆。

L81–82

```
    /* cursor position */
    int cursor_x;
```

cursor_x 预留该任务的横向屏幕光标位置。

L83

```
    int cursor_y;
```

cursor_y 预留纵向光标位置。

L85–86

```
    /* time(seconds) to wake up sleeping PCB */
    uint64_t wakeup_time;
```

wakeup_time 是 64 位唤醒时间；注释约定单位为秒，但实际语义仍需未来睡眠实现保证。

L88

```
} pcb_t;
```

结束 PCB 结构定义并将其命名为 pcb_t。

L90–91

```
/* ready queue to run */
extern list_head ready_queue;
```

extern 声明就绪队列 ready_queue：变量应在某个 .c 文件中真正定义，本行本身不分配队列。

L93–94

```
/* sleep queue to be blocked in */
extern list_head sleep_queue;
```

extern 声明睡眠队列 sleep_queue；仅有声明无法实现阻塞或唤醒。

L96–97

```
/* current running task PCB */
extern pcb_t * volatile current_running;
```

current_running 是指向当前 PCB 的指针；volatile 修饰指针对象，要求编译器按 volatile 规则访问它，不会自动提供锁或原子性。

L98

```
extern pid_t process_id;
```

process_id 是外部进程编号变量的声明。

L100

```
extern pcb_t pcb[NUM_MAX_TASK];
```

声明外部 PCB 数组 pcb，大小为 16；当前 P1 没有相应调度数组实现。

L101

```
extern pcb_t pid0_pcb;
```

pid0_pcb 是预留的 0 号任务控制块外部声明。

L102

```
extern const ptr_t pid0_stack;
```

pid0_stack 是 const ptr_t 的外部声明，表示其数值不可经此名字修改；不等于栈所指向的所有内存都只读。

L104

```
extern void switch_to(pcb_t *prev, pcb_t *next);
```

switch_to 接收旧 PCB 和新 PCB 的指针，意图切换上下文。这里只有函数声明，没有实际保存/恢复寄存器代码。

L105

```
void do_scheduler(void);
```

do_scheduler 声明未来的调度器，无参数、无返回值。

L106

```
void do_sleep(uint32_t);
```

do_sleep 接收 uint32_t 时间参数，但参数未命名，具体单位和行为需要实现说明；不能据此声称当前内核支持 sleep。

L108

```
void do_block(list_node_t *, list_head *queue);
```

do_block 意图把一个链表节点阻塞到指定队列；第二参数 queue 是队列头指针。

L109

```
void do_unblock(list_node_t *);
```

do_unblock 意图把一个节点解除阻塞。缺少实现时并不会产生可观察效果。

L111–113

```
/************************************************************/
/* Do not touch this comment. Reserved for future projects. */
/************************************************************/
```

后续项目预留注释标记。

L115

```
#endif
```

结束调度头文件包含保护。

3 你能看到什么，什么代码保证它

从当前活动代码不能观察到轮转、抢占、阻塞队列等调度效果。

代码推导，未在课程平台实测

本文件第 104—109 行只有声明；当前 init/main.c 的活动主循环直接调用任务入口，并在返回后提示 Task finished；没有调度器调用链。

容易误解或需要修正的地方

- 结构体定义只规定内存布局，不会主动保存寄存器、入队或切换任务。
- 不要因为看到了 ready_queue 和 TASK_READY，就把 P1 说成已有多进程隔离。
- 直接将本头文件接入新代码之前，应先补齐缺失依赖和对应实现，不能靠声明代替。

05 • 头文件和反汇编附录

25 字符串与内存函数声明

原路径：include/os/string.h • 16 行

> 为本工程自己的字符串库给出接口。

这里使用自己的 type.h 和 libs/string.c，不依赖宿主机的标准 libc。函数名看起来和标准 C 库相同，但某些参数/返回值不完全一致，要按本工程实现理解。

1 完整代码，先看全貌

```
1  #ifndef INCLUDE_STRING_H_
2  #define INCLUDE_STRING_H_
3
4  #include <type.h>
5
6  void memcpy(uint8_t *dest, const uint8_t *src, uint32_t len);
7  void memset(void *dest, uint8_t val, uint32_t len);
8  void bzero(void *dest, uint32_t len);
9  int strcmp(const char *str1, const char *str2);
10 int strncmp(const char *str1, const char *str2, uint32_t n);
11 char *strcpy(char *dest, const char *src);
12 char *strncpy(char *dest, const char *src, int n);
13 char *strcat(char *dest, const char *src);
14 int strlen(const char *src);
15
16 #endif
```

2 按原始行号，一句一句理解

L1–2

```
#ifndef INCLUDE_STRING_H_
#define INCLUDE_STRING_H_
```

建立 INCLUDE_STRING_H_ 包含保护。

L4

```
#include <type.h>
```

引入 uint8_t、uint32_t 等固定宽度类型。

L6

```
void memcpy(uint8_t *dest, const uint8_t *src, uint32_t len);
```

memcpy 将 src 的 len 个字节复制到 dest；这里返回 void，与标准库返回 void* 不同。src 上的 const 表示函数不应经此指针修改源数据。

L7

```
void memset(void *dest, uint8_t val, uint32_t len);
```

memset 把 dest 开始的 len 字节各设为 val；val 是单字节值，不能把它理解成按元素赋一个多字节整数。

L8

```
void bzero(void *dest, uint32_t len);
```

bzero 是把指定长度内存清零的快捷接口。

L9

```
int strcmp(const char *str1, const char *str2);
```

strcmp 比较两个以零结尾的字符串，约定返回 0 表示相等；loader 正是据此匹配任务名。

L10

```
int strncmp(const char *str1, const char *str2, uint32_t n);
```

strncmp 最多比较 n 个字符；长度限制有助于局部比较，但不会自动证明输入指针合法。

L11

```
char *strcpy(char *dest, const char *src);
```

strcpy 从 src 复制到 dest，包括字符串结束零；dest 必须足够大，接口本身不携带容量。

L12

```
char *strncpy(char *dest, const char *src, int n);
```

strncpy 最多处理 n 个字符；int n 可为负是接口设计上的隐患，具体遇到短串时是否补零要看 libs/string.c。

L13

```
char *strcat(char *dest, const char *src);
```

strcat 把 src 追加到 dest 原有字符串结尾，dest 容量必须足够；返回 char*。

L14

```
int strlen(const char *src);
```

strlen 返回结束零之前的字符字节数，返回类型 int，而不是标准库的 size_t。

L16

```
#endif
```

结束包含保护。

3 你能看到什么，什么代码保证它

按任务名查找时，名字完全一致才命中。

代码推导，未在课程平台实测

kernel/loader/loader.c 的 load_task_img 使用 strcmp(taskname,tasks[i].task_name)==0；本文件第 9 行声明其接口，比较实现在 libs/string.c。

容易误解或需要修正的地方

- 此库是教学实现，不应把标准库所有细节都自动套过来。
- 复制和拼接接口没有自动容量检查；调用者必须保证目标缓冲区足够大。

05 · 头文件和反汇编附录

26 任务目录格式：名字、偏移、大小和入口

原路径：include/os/task.h · 25 行

> 约定主机镜像工具与内核加载器共同理解的二进制任务记录。

最关键的区别是：offset 是磁盘镜像的字节位置，entrypoint 是运行时内存中的地址；两者属于不同空间，不能交换使用。

在本工程通常采用的 RV64 LP64 与 x86-64 LP64 ABI 下，task_name 占 0—31 字节，offset 位于 32—35，size 位于 36—39，entrypoint 位于 40—47，总计 48 字节。tools/createimage.c 另写了一份同布局结构体，因此主机和目标的类型宽度、对齐、字节序必须匹配。

1 完整代码，先看全貌

```
#ifndef __INCLUDE_TASK_H__
#define __INCLUDE_TASK_H__

#include <type.h>

#define TASK_MEM_BASE    0x52000000
#define TASK_MAXNUM      16
#define TASK_SIZE        0x10000
#define TASK_NAME_LEN    32


#define SECTOR_SIZE 512
#define NBYTES2SEC(nbytes) (((nbytes) / SECTOR_SIZE) + ((nbytes) % SECTOR_SIZE != 0))

/* TODO: [p1-task4] implement your own task_info_t! */
typedef struct {
    char task_name[TASK_NAME_LEN];
    uint32_t offset;
    uint32_t size;
    uint64_t entrypoint;
} task_info_t;

extern task_info_t tasks[TASK_MAXNUM];

#endif
```

2 按原始行号，一句一句理解

L1–2

```
#ifndef __INCLUDE_TASK_H__
#define __INCLUDE_TASK_H__
```

建立 __INCLUDE_TASK_H__ 包含保护。

L4

```
#include <type.h>
```

引入 uint32_t/uint64_t，避免把磁盘格式依赖在不明确的 int/long 宽度上。

L6

```
#define TASK_MEM_BASE    0x52000000
```

TASK_MEM_BASE=0x52000000 表示课程任务地址区域的起点；当前按名字加载时实际目的地址使用记录中的 entrypoint。

L7

```
#define TASK_MAXNUM      16
```

TASK_MAXNUM=16，tasks 数组最多预留 16 项。这个容量宏不会自动拒绝镜像里更大的 task_num，边界检查必须由代码另行完成。

L8

```
#define TASK_SIZE        0x10000
```

TASK_SIZE=0x10000，即 65536 字节，是任务地址槽的间距/约定大小；它不表示每个任务在磁盘上必须补齐 64 KiB。

L9

```
#define TASK_NAME_LEN    32
```

任务名缓冲区长度为 32 字节。若要以零结尾，最多可存 31 个可见字节；中文 UTF-8 的“字数”不能直接当成字节数。

L12

```
#define SECTOR_SIZE 512
```

扇区按 512 字节计算，加载器借此把字节偏移换算成扇区号和扇区内偏移。

L13

```
#define NBYTES2SEC(nbytes) (((nbytes) / SECTOR_SIZE) + ((nbytes) % SECTOR_SIZE != 0))
```

NBYTES2SEC 计算向上取整的扇区数：整除结果加“是否有余数”。例如 512→1、513→2。宏会重复求值 nbytes，不应传带自增等副作用的表达式。

L15–16

```
/* TODO: [p1-task4] implement your own task_info_t! */
typedef struct {
```

TODO 注释是课程题目痕迹；随后实际已定义任务信息结构体，不应把这里当作完全空白未实现。

L17

```
    char task_name[TASK_NAME_LEN];
```

task_name 是 32 字节字符数组，保存供用户输入匹配的程序名，不是一个单独的字符串指针。

L18

```
    uint32_t offset;
```

offset 是 uint32_t，记录该任务在 image 文件中的起始字节偏移。

L19

```
    uint32_t size;
```

size 是 uint32_t，记录从 image 中需要复制的文件字节数；未初始化数据 BSS 由 crt0 负责清零，不是靠这里把所有内存都存入镜像。

L20–21

```
    uint64_t entrypoint;
} task_info_t;
```

entrypoint 是 uint64_t 运行入口地址，结构体到此结束并命名为 task_info_t。在当前布局中第 40 字节起放这个 8 字节字段。

L23

```
extern task_info_t tasks[TASK_MAXNUM];
```

extern 告诉其他 .c 文件任务数组在别处定义。真正分配 tasks 存储的是 init/main.c:15，加载器通过此声明访问同一数组。

L25

```
#endif
```

结束包含保护。

3 你能看到什么，什么代码保证它

压缩排布在磁盘镜像中的任务仍能按名字精确装入。

代码推导，未在课程平台实测

tools/createimage.c:25—30 定义同布局记录并写出；init/main.c:42 按 tasknum*sizeof(task_info_t) 读取表；kernel/loader/loader.c 的 load_task_img 取 offset/size/entrypoint 完成装载。

容易误解或需要修正的地方

- 结构体直接 fwrite 到镜像是 ABI 约定，不是通用跨平台序列化协议；换到不同字节序/对齐的主机需重新校验。
- TASK_MAXNUM 与 32 字节名长都是容量约束，宏本身不实施边界检查。
- entrypoint 在当前加载方式中兼作段复制目的地址；这依赖程序镜像布局，不能泛化成任意 ELF 均支持。

05 · 头文件和反汇编附录

27 类型字典：整数宽度、指针与现有不一致

原路径：include/type.h · 78 行

> 在不使用标准库头文件时提供工程自定义类型。

typedef 是给已有类型起别名。uint32_t 的 32 意味着希望它准确占 32 位，便于磁盘格式和寄存器协议保持一致；这里用 GCC 特定的 __mode__ 属性实现。

必须照实际代码指出问题：int8_t 被写成 unsigned，且某些 INT64 常量用了无符号后缀。讲解不会悄悄把它们改成理想的标准 stdint.h。

1 完整代码，先看全貌

```
/* * * *
```

```
38
39 typedef int bool;
40 #ifndef true
41     #define true   1
42     #define false  0
43 #endif
44
45 typedef unsigned __attribute__((__mode__(QI))) int8_t;
46 typedef unsigned __attribute__((__mode__(QI))) uint8_t;
47 typedef int      __attribute__((__mode__(HI))) int16_t;
48 typedef unsigned __attribute__((__mode__(HI))) uint16_t;
49 typedef int      __attribute__((__mode__(SI))) int32_t;
50 typedef unsigned __attribute__((__mode__(SI))) uint32_t;
51 typedef int      __attribute__((__mode__(DI))) int64_t;
52 typedef unsigned __attribute__((__mode__(DI))) uint64_t;
53
54 typedef int32_t pid_t;
55 typedef uint64_t reg_t;
56 typedef uint64_t ptr_t;
57 typedef uint64_t uintptr_t;
58 typedef int64_t intptr_t;
59 typedef uint64_t size_t;
60
61 /* Minimum of signed integral types.  */
62 # define INT8_MIN               (-128)
63 # define INT16_MIN              (-32767-1)
64 # define INT32_MIN              (-2147483647-1)
65 # define INT64_MIN              (-9223372036854775807lu-1)
66 /* Maximum of signed integral types.  */
67 # define INT8_MAX               (127)
68 # define INT16_MAX              (32767)
69 # define INT32_MAX              (2147483647)
70 # define INT64_MAX              (9223372036854775807lu)
71
72 /* Maximum of unsigned integral types.  */
73 # define UINT8_MAX              (255)
74 # define UINT16_MAX             (65535)
75 # define UINT32_MAX             (4294967295U)
76 # define UINT64_MAX             (18446744073709551615lu)
77
78 #endif
```

2 按原始行号，一句一句理解

L1–24

（此处为连续注释或较长结构，请对照上方完整代码 L1–L24。）

版权许可注释，要求分发时保留相应说明；不定义任何机器类型。

L26–27

```
#ifndef INCLUDE_TYPE_H_
#define INCLUDE_TYPE_H_
```

建立 INCLUDE_TYPE_H_ 包含保护。

L29–31

```
#ifndef NULL
#define NULL    ((void*)0)
#endif
```

仅在 NULL 尚未定义时，将其定义为 (void*)0。NULL 是空指针常量的常用名字，不是“指向一块全零内存”。

L33

```
typedef int BOOL;
```

BOOL 是 int 的类型别名；没有创建一个叫 BOOL 的变量，也没有新建独立的机器表示。

L34–37

```
#ifndef TRUE
    #define TRUE    1
    #define FALSE   0
#endif
```

若 TRUE 未定义，则将 TRUE/FALSE 宏设为 1/0。C 条件判断仍把所有非零值视为真，而不只接受 1。

L39

```
typedef int bool;
```

小写 bool 也被定义成 int；这里不是 C99 的 _Bool，也没有包含 stdbool.h。

L40–43

```
#ifndef true
    #define true   1
    #define false  0
#endif
```

若 true 未定义，则给 true/false 宏值 1/0。预处理宏按大小写区分名字。

L45

```
typedef unsigned __attribute__((__mode__(QI))) int8_t;
```

注意实际代码：int8_t 也用了 unsigned，再用 GCC 的 mode(QI) 选择 1 字节整数模式，因此它是无符号 8 位类型！名字像有符号类型但实现不是；不能把它讲成 -128..127。

L46

```
typedef unsigned __attribute__((__mode__(QI))) uint8_t;
```

uint8_t 用 unsigned 与 mode(QI) 定义无符号 8 位整数，范围通常为 0..255。它与上一行实际同类。

L47

```
typedef int      __attribute__((__mode__(HI))) int16_t;
```

int16_t 用有符号 int 加 mode(HI) 指定 16 位整数；HI 在此为 half-integer 模式，不是高半部地址。

L48

```
typedef unsigned __attribute__((__mode__(HI))) uint16_t;
```

uint16_t 是无符号 16 位整数，范围 0..65535。

L49

```
typedef int      __attribute__((__mode__(SI))) int32_t;
```

int32_t 用 mode(SI) 选择有符号 32 位整数。

L50

```
typedef unsigned __attribute__((__mode__(SI))) uint32_t;
```

uint32_t 用 mode(SI) 选择无符号 32 位整数，适合本镜像格式的字节偏移与大小。

L51

```
typedef int      __attribute__((__mode__(DI))) int64_t;
```

int64_t 用 mode(DI) 选择有符号 64 位整数；这些 __attribute__ 是 GCC 扩展。

L52

```
typedef unsigned __attribute__((__mode__(DI))) uint64_t;
```

uint64_t 为无符号 64 位整数，用于 RV64 的入口地址、寄存器值等。

L54

```
typedef int32_t pid_t;
```

pid_t 是 int32_t 的别名，意图表示进程/任务编号；类型存在不表示已实现进程管理。

L55

```
typedef uint64_t reg_t;
```

reg_t 是 uint64_t 的别名，用 8 字节保存一个 RV64 通用寄存器。

L56

```
typedef uint64_t ptr_t;
```

ptr_t 是 uint64_t 的别名，课程用它承载指针数值；真正 C 指针仍有其自己的类型。

L57

```
typedef uint64_t uintptr_t;
```

uintptr_t 是足以承载目标指针数值的无符号整数别名；这里直接假定 64 位目标。

L58

```
typedef int64_t intptr_t;
```

intptr_t 是承载指针数值的有符号 64 位别名。

L59

```
typedef uint64_t size_t;
```

size_t 定义成 uint64_t，表达大小/长度。它与宿主系统标准头可能重名，因而项目采用独立头文件和 freestanding 编译环境。

L61–62

```
/* Minimum of signed integral types.  */
# define INT8_MIN               (-128)
```

注释宣称下列为有符号最小值；INT8_MIN=-128 是数学上的有符号 8 位下限，但本文件 int8_t 实际为无符号，因此这组名字与第 45 行不一致。

L63

```
# define INT16_MIN              (-32767-1)
```

INT16_MIN 写成 -32767-1，结果为 -32768；拆写是常见的避免直接给出超范围正整数后再取负的技巧。

L64

```
# define INT32_MIN              (-2147483647-1)
```

INT32_MIN 同理得到 -2147483648。

L65

```
# define INT64_MIN              (-9223372036854775807lu-1)
```

INT64_MIN 写成带 lu 的常量表达式；在 LP64 下 lu 是无符号 long，所以表达式的类型为无符号，最终数值按无符号运算得到 0x8000000000000000，不能简单说其类型就是有符号 int64_t。

L66–67

```
/* Maximum of signed integral types.  */
# define INT8_MAX               (127)
```

INT8_MAX=127 是有符号 8 位上限，仍与实际 unsigned int8_t 的取值范围不匹配。

L68

```
# define INT16_MAX              (32767)
```

INT16_MAX=32767，是有符号 16 位上限。

L69

```
# define INT32_MAX              (2147483647)
```

INT32_MAX=2147483647，是有符号 32 位上限。

L70

```
# define INT64_MAX              (9223372036854775807lu)
```

INT64_MAX 的数学值为 2^63-1，但 lu 后缀使本表达式在 LP64 中仍是 unsigned long 类型；比较和类型转换时要留意。

L72–73

```
/* Maximum of unsigned integral types.  */
# define UINT8_MAX              (255)
```

UINT8_MAX=255，是无符号 8 位上限。

L74

```
# define UINT16_MAX             (65535)
```

UINT16_MAX=65535，是无符号 16 位上限。

L75

```
# define UINT32_MAX             (4294967295U)
```

UINT32_MAX=4294967295U，即 2^32-1；U 指定无符号常量。

L76

```
# define UINT64_MAX             (18446744073709551615lu)
```

UINT64_MAX=18446744073709551615lu，即 2^64-1；依赖 LP64 的 unsigned long 足够宽。

L78

```
#endif
```

结束类型头文件包含保护。

3 你能看到什么，什么代码保证它

任务表中 offset/size 各占 4 字节，入口占 8 字节。

代码推导，未在课程平台实测

本文件第 50、52 行固定 uint32_t/uint64_t 宽度，include/os/task.h:18—20 使用这些类型，形成主机工具与内核共同约定的 48 字节记录。

容易误解或需要修正的地方

- 自定义类型并非完整标准库替代品；int8_t 的 signedness 有明确笔误。
- 整数的数学值、位模式和表达式的 C 类型是三个层次；INT64_MIN 的 lu 后缀尤其值得区分。
- 本头文件按 64 位课程平台编写，不适合直接混入宿主标准头或改成 32 位平台而不检查。

05 • 头文件和反汇编附录

28 用户程序怎样调用打印与输入

原路径：tiny_libc/include/kernel.h • 36 行

> 向简单用户程序提供三项服务表包装器。

这份 kernel.h 是编译 test/test_project1/*.c 时通过 -Itiny_libc/include 找到的；内核则使用 include/os/kernel.h。两者前 3 个服务编号与表首地址一致，所以用户程序无需知道 port_write 函数具体被放到了哪里。

“用户程序”是课程命名。当前调用是共享地址空间中的函数间接跳转，没有 ecall、页表保护或特权级切换。

1 完整代码，先看全貌

```
#ifndef __INCLUDE_KERNEL_H__
#define __INCLUDE_KERNEL_H__

#define KERNEL_JMPTAB_BASE 0x51ffff00
typedef enum {
    CONSOLE_PUTSTR,
    CONSOLE_PUTCHAR,
    CONSOLE_GETCHAR,
    NUM_ENTRIES
} jmptab_idx_t;

static inline long call_jmptab(long which, long arg0, long arg1, long arg2, long arg3, long
 arg4)
{
    unsigned long val = \
        *(unsigned long *)(KERNEL_JMPTAB_BASE + sizeof(unsigned long) * which);
    long (*func)(long, long, long, long, long) = (long (*)(long, long, long, long, long))va
l;

    return func(arg0, arg1, arg2, arg3, arg4);
}

static inline void bios_putstr(char *str)
{
    call_jmptab(CONSOLE_PUTSTR, (long)str, 0, 0, 0, 0);
}

static inline void bios_putchar(int ch)
{
    call_jmptab(CONSOLE_PUTCHAR, (long)ch, 0, 0, 0, 0);
}

static inline int bios_getchar(void)
{
    return call_jmptab(CONSOLE_GETCHAR, 0, 0, 0, 0, 0);
}

#endif
```

2 按原始行号，一句一句理解

L1–2

```
#ifndef __INCLUDE_KERNEL_H__
#define __INCLUDE_KERNEL_H__
```

建立与内核版同名的包含保护宏；不要在同一编译单元随意混合两份 kernel.h，否则后包含的一份会被整个跳过。

L4

```
#define KERNEL_JMPTAB_BASE 0x51ffff00
```

固定服务表基地址 0x51ffff00，必须与内核初始化时使用的地址完全一致。

L5

```
typedef enum {
```

开始定义用户程序可见的服务编号枚举。

L6

```
    CONSOLE_PUTSTR,
```

CONSOLE_PUTSTR=0，对应打印字符串。

L7

```
    CONSOLE_PUTCHAR,
```

CONSOLE_PUTCHAR=1，对应打印单字符。

L8

```
    CONSOLE_GETCHAR,
```

CONSOLE_GETCHAR=2，对应读取字符。

L9–10

```
    NUM_ENTRIES
} jmptab_idx_t;
```

NUM_ENTRIES=3 表示用户头公开三项服务，再命名枚举类型 jmptab_idx_t。它不会重新分配或初始化内核的表。

L12–13

```
static inline long call_jmptab(long which, long arg0, long arg1, long arg2, long arg3, long ar
g4)
{
```

定义 static inline 的查表调用助手；which 是下标，其余五项为要传递的参数。

L14–15

```
    unsigned long val = \
        *(unsigned long *)(KERNEL_JMPTAB_BASE + sizeof(unsigned long) * which);
```

按基址加 which×sizeof(unsigned long) 定位表项，解引用取得函数地址。RV64 LP64 中表项步长为 8 字节。

L16

```
    long (*func)(long, long, long, long, long) = (long (*)(long, long, long, long, long))val;
```

把取得的整数地址转换为五参数、返回 long 的函数指针。它依赖本课程的地址和 ABI 约定。

L18–19

```
    return func(arg0, arg1, arg2, arg3, arg4);
}
```

通过函数指针发起普通调用并返回结果；没有引发陷阱。

L21–22

```
static inline void bios_putstr(char *str)
{
```

定义 bios_putstr，接收一个以零结尾的字符串地址。

L23–24

```
    call_jmptab(CONSOLE_PUTSTR, (long)str, 0, 0, 0, 0);
}
```

把字符串指针作为第一个 long 参数送给 0 号服务，其他参数槽清零。

L26–27

```
static inline void bios_putchar(int ch)
{
```

定义 bios_putchar，接收要输出的字符整数。

L28–29

```
    call_jmptab(CONSOLE_PUTCHAR, (long)ch, 0, 0, 0, 0);
}
```

调用 1 号服务输出该字符。

L31–32

```
static inline int bios_getchar(void)
{
```

定义 bios_getchar，不带参数，返回 int。

L33–34

```
    return call_jmptab(CONSOLE_GETCHAR, 0, 0, 0, 0, 0);
}
```

调用 2 号服务，返回字符或 -1；2048 等程序可以据此轮询键盘。

L36

```
#endif
```

结束用户版 kernel.h 包含保护。

3 你能看到什么，什么代码保证它

data、bss、auipc 等任务能在共享串口上输出文字；2048 能读到键盘字符。

代码推导，未在课程平台实测

本文件第 23、28、33 行读 0—2 号表项；init/main.c:33—35 将相同编号填为实际串口封装函数，用户程序调用 bios_* 就沿此路径到达 BIOS。

容易误解或需要修正的地方

- 包装器名字带 bios，不代表用户程序直接知道 BIOS 实际入口地址；它先经内核写好的函数表。
- 该设计不隔离不可信用户程序；任意地址读写和错误函数指针依然可能破坏整个内核。

05 • 头文件和反汇编附录

29 原包 README：目前只有待写说明

原路径：README.md • 1 行

> 说明文件而非程序源码。

上传包里的 README 只有一行 TODO，所以不能从中引用具体运行承诺。本讲解的流程和现象分析以实际 Makefile、汇编和 C 源码为依据。

1 完整代码，先看全貌

```
1 TODO：请在此完成你自己项目的“说明书”。
```

2 按原始行号，一句一句理解

L1

`TODO：请在此完成你自己项目的“说明书”。`

这句话要求项目作者补充自己的使用说明，没有可执行指令、编译选项或测试结果。它本身不能说明项目已通过某项验收。

3 你能看到什么，什么代码保证它

阅读原始 README 不会得到完整启动步骤。

代码推导，未在课程平台实测

原文件第 1 行是 TODO，占位内容并未提供命令或预期输出；应结合本讲解中的实际代码路径阅读。

容易误解或需要修正的地方

- 不要把本讲解补充的分析误当成原包已经写出的说明或已有运行记录。

05 · 头文件和反汇编附录

30 附录：如何把 prime.S 与反汇编对上

原路径：zhangxiyuan24_prime.txt · 44 行

> 包内现成反汇编文本，用于对照源代码与已链接机器指令。

每条指令左边依次是地址、机器码、助记符和操作数。地址变化 2 或 4，是因为 RISC-V 压缩指令与普通 32 位指令可混用，不是漏掉了源码行。

这份文本展示的质数算法与 prime.S 对应，但它是包内预存产物，不能单凭文件存在就断言“当前源码已在你的课程平台运行成功”。地址以此文本显示的 ELF 为准。

1 完整代码，先看全貌

```

./zhangxiyuan24_prime:     file format elf64-littleriscv


Disassembly of section .text:

00000000000100e8 <main>:
   100e8:   00001417            auipc   s0,0x1
   100ec:   05040413            addi    s0,s0,80 # 11138 <__DATA_BEGIN__>
   100f0:   4485                    li  s1,1
   100f2:   0c800913            li  s2,200

00000000000100f6 <check_loop>:
   100f6:   8526                    mv  a0,s1
   100f8:   012000ef            jal 1010a <is_prime>
   100fc:   00a40023            sb  a0,0(s0)
   10100:   0405                    addi    s0,s0,1
   10102:   0485                    addi    s1,s1,1
   10104:   fe9959e3            bge s2,s1,100f6 <check_loop>

0000000000010108 <finish>:
   10108:   a001                    j   10108 <finish>

000000000001010a <is_prime>:
   1010a:   4289                    li  t0,2
   1010c:   02554163            blt a0,t0,1012e <not_prime>
   10110:   00550d63            beq a0,t0,1012a <prime>
   10114:   4289                    li  t0,2

0000000000010116 <divisor_loop>:
   10116:   02528333            mul t1,t0,t0
   1011a:   00654863            blt a0,t1,1012a <prime>
   1011e:   025563b3            rem t2,a0,t0
   10122:   00038663            beqz    t2,1012e <not_prime>
   10126:   0285                    addi    t0,t0,1
   10128:   b7fd                    j   10116 <divisor_loop>

```

```
000000000001012a <prime>:
    1012a:   4505                    li  a0,1
    1012c:   8082                    ret

000000000001012e <not_prime>:
    1012e:   4501                    li  a0,0
    10130:   8082                    ret
```

2 按原始行号，一句一句理解

L2

```
./zhangxiyuan24_prime:     file format elf64-littleriscv
```

反汇编工具指出原文件是小端 RISC-V ELF64。ELF 是带头部和段信息的可执行文件格式，不是把这些打印出来的文字直接交给 CPU 执行。

L5

```
Disassembly of section .text:
```

下面列的是 .text 段的反汇编，不含 prime_result 的完整数据内容。

L7

```
00000000000100e8 <main>:
```

main 标签地址是 0x100e8。这属于该份已链接 ELF 的地址，不能套用 test.S/prelab.lds 中 0x50200000 的布局。

L8

```
    100e8:   00001417            auipc   s0,0x1
```

左列 100e8 是本条指令地址；00001417 是 32 位机器码；auipc s0,0x1 把当前 PC 加 0x1000，先形成全局数据地址的高部。

L9

```
    100ec:   05040413            addi    s0,s0,80 # 11138 <__DATA_BEGIN__>
```

在上一步结果上加 80（0x50），得到 0x11138；这两条是 la s0,prime_result 的展开。注释显示链接器给这个位置的符号名 __DATA_BEGIN__，不能据此认定 C 变量名变了。

L10

```
    100f0:   4485                    li  s1,1
```

4485 只有 4 个十六进制数，代表 16 位压缩指令，反汇编仍用易读别名 li s1,1。它初始化被测试数字 n=1，下一地址只增加 2。

L11

```
    100f2:   0c800913            li  s2,200
```

用 32 位指令把上界 200 放入 s2，所以下一条地址加 4。

L13

```
00000000000100f6 <check_loop>:
```

check_loop 的入口地址为 0x100f6，main 每处理一个整数会回到这里。

L14

```
   100f6:   8526                    mv  a0,s1
```

mv a0,s1 把当前 n 复制到函数第一个参数寄存器 a0。

L15

```
   100f8:   012000ef            jal 1010a <is_prime>
```

jal 0x1010a 跳到 is_prime，并把返回地址保存在 ra；它是 prime.S 中 call is_prime 的链接后形式。

L16

```
   100fc:   00a40023            sb  a0,0(s0)
```

sb 把返回值 a0 的最低字节存到 s0 指向的位置，即一项质数标记。

L17

```
   10100:   0405                    addi    s0,s0,1
```

s0 加 1，移到下一个字节槽。

L18

```
   10102:   0485                    addi    s1,s1,1
```

s1 加 1，把待测数字递增。

L19

```
   10104:   fe9959e3            bge s2,s1,100f6 <check_loop>
```

bge s2,s1,check_loop 等价于“若 s1<=s2 则继续”。源程序 ble s1,s2 是伪指令，汇编器交换两比较数后使用实际 bge。

L21–22

```
0000000000010108 <finish>:
   10108:   a001                    j   10108 <finish>
```

finish 位于 0x10108；j 跳回自己，程序在计算完 200 项后停留在死循环，不会自动 ret 或向串口打印全部质数。

L24

```
000000000001010a <is_prime>:
```

is_prime 函数入口在 0x1010a，a0 既作为输入 n，也作为返回 0/1 的寄存器。

L25

```
   1010a:   4289                    li  t0,2
```

把除数下界 2 放到 t0。

L26

```
    1010c:   02554163           blt a0,t0,1012e <not_prime>
```

当 n<2 跳到 not_prime，所以 1 的标记为 0。

L27

```
    10110:   00550d63           beq a0,t0,1012a <prime>
```

n==2 直接跳到 prime，所以 2 的标记为 1。

L28

```
    10114:   4289                   li  t0,2
```

对 n>2 的情况重新把候选除数置为 2。

L30

```
0000000000010116 <divisor_loop>:
```

divisor_loop 地址 0x10116，开始试除循环。

L31

```
    10116:   02528333           mul t1,t0,t0
```

t1=t0*t0，形成除数平方。

L32

```
    1011a:   00654863           blt a0,t1,1012a <prime>
```

若 $n<t0^2$，则没有必要继续试除，跳到 prime；这是只需检查到平方根的依据。

L33

```
    1011e:   025563b3           rem t2,a0,t0
```

t2=n%t0，取余数判断整除关系。

L34

```
    10122:   00038663           beqz    t2,1012e <not_prime>
```

若余数等于 0，跳到 not_prime；beqz 是“与 zero 比较相等”的易读别名。

L35

```
    10126:   0285                   addi    t0,t0,1
```

候选除数加 1。

L36

```
    10128:   b7fd                   j   10116 <divisor_loop>
```

无条件跳回 divisor_loop，继续下一次试除。

L38–40

```
000000000001012a <prime>:
   1012a:   4505                    li  a0,1
   1012c:   8082                    ret
```

prime 分支将返回值 a0 设为 1，再用 ret 经 ra 返回调用点。ret 通常是 jalr zero,0(ra) 的伪指令写法；这里使用了压缩编码。

L42–44

```
000000000001012e <not_prime>:
   1012e:   4501                    li  a0,0
   10130:   8082                    ret
```

not_prime 分支把 a0 设为 0 再返回。机器码列用于理解 CPU 真正执行的编码，不需要初学者逐位背诵。

3 你能看到什么，什么代码保证它

源文件一行 la 在反汇编里可能变成两条指令，ble 也可能显示成操作数交换后的 bge。

代码推导，未在课程平台实测

prime.S 的 la s0,prime_result 与 ble s1,s2,check_loop，在本文件第 8—9 行及第 19 行具体展开；伪指令只是写法，CPU 执行展开后的编码。

素数结果储存在内存，没有预设“打印所有素数”的输出。

代码推导，未在课程平台实测

本文件第 16 行只有 sb 写内存，第 21—22 行进入 finish 自循环，未出现调用输出服务的指令。

容易误解或需要修正的地方

- 这是反汇编文本，不是另一个独立算法，也不是可以直接运行的程序。
- 文本地址 0x100e8 并非 prelab.ld 中的 RAM 装载地址；要运行这份 ELF 必须有与其布局一致的装载环境。

原文件清单与完整性说明

上传压缩包共35个文件。其中30份UTF-8文本全部展示并解释，其他5个文件是已生成的二进制或编辑器交换数据；这些文件没有可直接逐句解释的C/汇编源码。附带反汇编作为文本单独解释，并明确它不是另一份源程序。

文件	字节数	本书处理
prelab.lds	348	全文与逐句讲解
riscv.lds	6806	全文与逐句讲解
tiny_libc/include/kernel.h	866	全文与逐句讲解
.test.S.swp	12288	二进制／交换文件，仅登记；不执行上传二进制
zhangxiyuan24_prime	1912	二进制／交换文件，仅登记；不执行上传二进制
libs/string.c	1487	全文与逐句讲解
test.S	2347	全文与逐句讲解
test/test_project1/data.c	5398	全文与逐句讲解
test/test_project1/2048.c	7738	全文与逐句讲解
test/test_project1/auipc.c	1457	全文与逐句讲解
test/test_project1/bss.c	437	全文与逐句讲解
tools/createimage.c	9562	全文与逐句讲解
init/main.c	3901	全文与逐句讲解
prime.S	989	全文与逐句讲解
zhangxiyuan24.bin	312	二进制／交换文件，仅登记；不执行上传二进制
zhangxiyuan24_prime.txt	1437	全文与逐句讲解
README.md	58	全文与逐句讲解
include/type.h	3024	全文与逐句讲解
include/os/sched.h	3557	全文与逐句讲解
include/os/kernel.h	1632	全文与逐句讲解
include/os/loader.h	190	全文与逐句讲解

文件	字节数	本书处理
include/os/string.h	507	全文与逐句讲解
include/os/task.h	546	全文与逐句讲解
zhangxiyuan24	6680	二进制／交换文件，仅登记；不执行上传二进制
arch/riscv/boot/bootblock.S	1334	全文与逐句讲解
arch/riscv/crt0/crt0.S	890	全文与逐句讲解
arch/riscv/bios/common.c	1201	全文与逐句讲解
arch/riscv/include/common.h	2761	全文与逐句讲解
arch/riscv/include/csr.h	2170	全文与逐句讲解
arch/riscv/include/asm/biosdef.h	191	全文与逐句讲解
arch/riscv/include/asm.h	826	全文与逐句讲解
arch/riscv/kernel/head.S	479	全文与逐句讲解
Makefile	9728	全文与逐句讲解
kernel/loader/loader.c	1678	全文与逐句讲解
createimage	23208	二进制／交换文件，仅登记；不执行上传二进制

每个文本文件的解释区间经过程序校验：所有非空行恰好覆盖一次。source-manifest.json记录原始文件大小与SHA-256，coverage.json记录覆盖统计。此机械检查保证没有漏页漏行，不替代语义审读。

最后：我应该看到什么，凭哪几句代码？

先把“现象”说清楚，再追到写内存、打印、分支或返回的具体语句。以下是整份实验的验收索引，不是把一个成功截图当作所有功能都正确。

操作／场景	应观察的现象	保证或限制它的代码
Prelab test.S执行至finish	sum_result的8字节值为1275(0x4fb)；质数标志200字节中46个1；终端可以没有文字	test.S L35–52累加并sd；L60–94逐个call并sb；L119–174判素数；L102自旋；无BIOS打印调用
prime.S执行至finish	只有200字节质数标志；没有求和步骤，也没有自动打印	prime.S L12–26遍历并sb；L29保持结果；L39–67判素数
启动进入小内核	先看到It's a bootloader...，再Hello OS!、bss check: t version: 2和名字输入提示	bootblock.S末尾msg决定真实字符串；head.S清BSS；main.c bss_check与version=2、输出占位替换及输入循环
在任务提示输入bss并回车	[bss] Info: passed bss check!，然后回到loader提示	crt0.S每次入口清BSS；bss.c L10–16扫描50字节，L19打印；return0→crt0恢复ra→内核打印Task finished
输入data	checking data!，随后checking data ---- [OK]!并返回	data.c数组0..1023；sizeof计算1024；每项data[i]!=i都会进错误分支，否则完成后才打印OK
输入auipc	[auipc] Info: testpc = 0x…；地址受链接和编译结果影响，不是固定模板数	auipc.c L59 AUIPC取该指令的PC；simple_itoa以16进制转字符串；程序没有自己比较“预期PC”
输入不存在的名字	Task not found，然后继续让你输入	loader.c strcmp必须完全相等；没有匹配返回0；main.c检测entry==0后continue
输入空行／过长名字	空行直接重试；最多保存31个字符，额外字符不再存入也不回显	main.c pos==0分支；pos<TASK_NAME_LEN-1；回车位置写字符串结束符（反斜线0）。这里没有退格编辑实现
输入2048	设计意图是彩色棋盘、WASD方向控制与Q退出；不能保证当前版本完整正确运行	pickColor输出ANSI颜色；main处理按键；但countEmpty的i=-1越界、rand溢出且不更新状态、up指针误用破坏可依赖的行为
任务中出现错误分支的死循环	程序可能不再回到名字提示	bss/data失败分支while(1)；内核是同步直接调用，没有另一个调度器自动接管

本次实际验证记录

QEMU内存实测（明确做了地址适配）

test.S：重编译退出0，GDB退出0；已逐字节比对200个质数标志，共46个1。读回sum_result=1275。 仅临时链接脚本把0x50200000改成0x80200000，并显式选用RV64IMAC/LP64与关闭松弛；汇编源码逐字节未变。这验证算法结果，不等于原地址在课程环境实测。

QEMU内存实测（明确做了地址适配）

prime.S：重编译退出0，GDB退出0；已逐字节比对200个质数标志，共46个1。 仅临时链接脚本把0x50200000改成0x80200000，并显式选用RV64IMAC/LP64与关闭松弛；汇编源码逐字节未变。这验证算法结果，不等于原地址在课程环境实测。

小内核串口实测（测试BIOS与地址移植）

临时源码副本中的课程地址整体增加0x30000000，并使用单独的测试BIOS提供串口与镜像字节读取服务；原上传源未改。以下是从该副本实际运行得到的输出，不是原课程环境验收。验证后已停止自行启动的QEMU进程。

实际输入／阶段	实际输出（均已检查返回提示）
启动	It's a bootloader... → Hello OS! → bss check: t version: 2
bss	[bss] Info: passed bss check! → Task finished, back to loader
data	[data] Info: checking data! → [data] Info: checking data --- [OK]! → Task finished
auipc	[auipc] Info: testpc = 0x82010042；与该次ELF的AUIPC反汇编地址一致
missing	Task not found → Please input task name:

为了真正检验清零代码，测试固件在启动前先把内核和bss任务的BSS区域填成0xa5，然后才启动原逻辑；两处BSS检查仍通过。这排除了“模拟器内存碰巧一开始就是零”的假阳性。2048不在本次整机运行范围内。

宿主机与构建检查共13项达到各自预期：9项正常构建或行为观察，4项是预期拒绝或错误检测。后者不是游戏功能通过。具体命令、退出码与日志保存在validation/results.json。

检查	实测结论	判读
host-createimage-build	从上传C源码重新编译宿主工具；没有执行上传的createimage二进制。	正常／构建观察

检查	实测结论	判读
image-normal	合成ELF实测：boot补到512字节，内核元数据写600字节而非2扇区，task_info_t按48字节布局；app的memsz=12但镜像只存3字节。	正常／构建观察
image-vaddr-gap	两个PT_LOAD虚拟地址相距0x100仍紧接打包为ABC；工具不保存虚拟地址间隙，所以只能配合连续布局，不能保证任意ELF正确。	正常／构建观察
image-large-kernel	内核65536字节被写元数据前的上限检查拒绝。	预期错误被检出
host-createimage-asan-build	构建AddressSanitizer版本以观察真实边界缺陷，不修改源码。	正常／构建观察
image-17-tasks	传入17个任务触发taskinfo[16]写越界；当前生效实现缺少任务数量上限检查。	预期错误被检出
game-sanitizer-build	原始2048.c经include复用，仅把BIOS调用替换为空桩并重命名入口；目标检查是C函数，不是课程整机。	正常／构建观察
game-countempty-bounds	全0的4x4数组传给countEmpty，i=-1导致真实越界读；不是正常可依赖的第五行。	预期错误被检出
game-rand-overflow	第一次rand调用中1103515245*42就溢出32位signed int；C标准行为未定义。	预期错误被检出
game-score-harness-build	仅在此隔离实验使用-fwrapv和额外前置缓冲行，避免另外两处已确认缺陷掩盖score问题；没有改原始程序。	正常／构建观察
game-up-score	向上合并2+2后棋盘出现4，但调用者score仍为0；291行score += move(grid)移动局部指针，未更新*score。	正常／构建观察
project-original-build	在临时完整副本按原Makefile顺序构建（仅覆盖交叉工具前缀），真实结果见日志；不改上传源。	正常／构建观察
project-adapted-build	在另一临时源码副本使用显式静态非PIE、RV64IMAC及lp64 ABI、关闭小数据寻址等Makefile变量覆盖，未修改源码。此为工具链适配构建。	正常／构建观察

一个很重要的区别：能make不等于能启动

在当前GCC工具链上，仅把CROSS_PREFIX覆盖为riscv64-linux-gnu-时，make dirs elf image确实返回0，ELF类型仍为ET_EXEC。但默认工具链生成的附加节和首个PT_LOAD把ELF头也映射进去，段起点比入口低0x1000；当前createimage顺序复制段、不按p_vaddr放置，再从入口调用，因而无法保证执行到正确指令。仅凭“编译没报错”或“类型是ET_EXEC”不能宣布实验成功，也不能把问题简化成ELF类型或单一PIE开关。

独立验证还做了显式静态、非PIE、RV64IMAC/LP64和关闭小数据寻址/松弛的构建覆盖，未修改上传源码。该次镜像内核为2160字节、任务数4、任务表字节偏移12708；这些是该次构建值，不是源码永久规定的常量。完整覆盖参数保留在validation/results.json，便于复现。

课程专用QEMU、U-Boot、真实SD卡、USB串口与板端没有在本次云环境中实测。若要按课程验收，应使用对应课程运行环境，观察相同逻辑现象，同时核对BIOS地址、链接地址和任务元数据的约定。

新手可以照着做的观察步骤

1. 先只读test.S，在纸上跟踪t0/t1第一轮与最后一轮，解释为什么最后写1275而不是1225或1326。
2. 在正确课程环境或明确地址适配的调试构建中，在finish下断点。GDB：break finish；continue；x/1gd &sum_result；x/200ub &prime_result。必须等到finish，不能把未计算完的零值当结果。
3. 对prime_result第0、1、2、3、198、199项解释它们对应数字1、2、3、4、199、200；应为0、1、1、0、1、0。
4. 启动小内核后先读真实输出中的t与2，分别追踪到head.S清零和main.c的version，而不是把它们归因于printf。
5. 先运行bss、data、auipc三项。每项返回后都看到Task finished与下一个提示，再输入一个不存在的名字对照错误分支。
6. 最后再读2048。先理解棋盘存实际的2/4/8，再看转置/旋转/合并。当前缺陷需要另行修复后才能把它当作完整游戏验收，不用反复按键猜测未定义行为。

> **为什么“保证”要写出前提**
>
> 一条打印语句只能保证控制流到达它且BIOS可用时会请求输出；它不能保证此前的镜像布局、栈、地址和数组访问都正确。完整因果链应当是：合法镜像 → 正确入口/栈/BSS → BIOS与跳转表 → 装载字节 → 正确分支/指令 → 实际可见现象。